FreeSWITCH 1.2

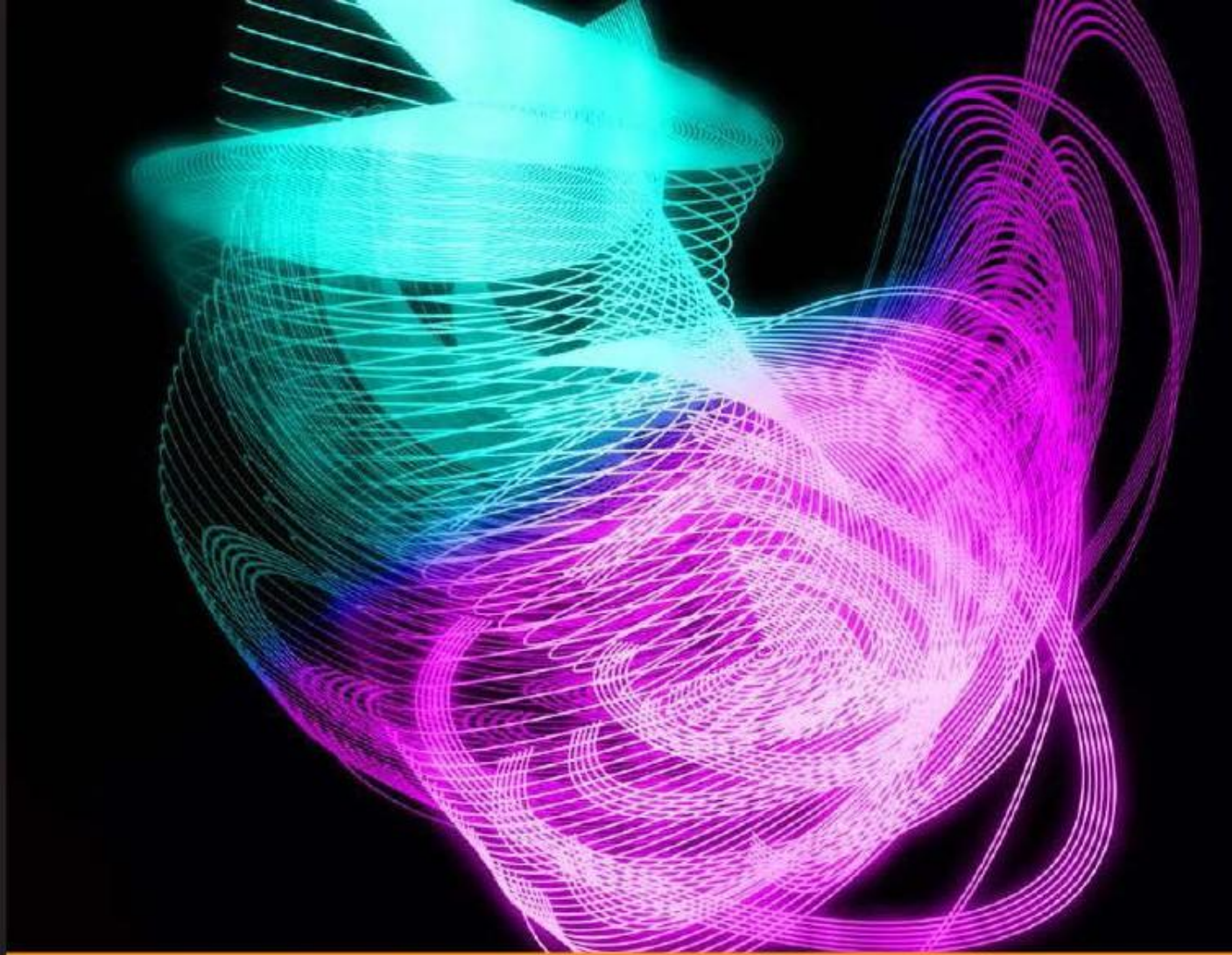
使用 FreeSWITCH 构建强大的开源高性能电话系统

James.zhu， QQ：522137361, email：james.zhu@hiastar.com
www.hiastar.com www.sangoma.cn

2013-9-2

Hiastar.com
星昊通科技





Community Experience Distilled

FreeSWITCH 1.2

Build robust, high-performance telephony systems using FreeSWITCH

Anthony Minessale
Michael S Collins
Darren Schreiber
Raymond Chandler

[PACKT] PUBLISHING open source* community experience distilled

FreeSWITCH 1.2

第二版

使用FreeSWITCH构建强大的高性能电话通信系统

Anthony Minessale

Michael S Collins

Darren Schreiber

Raymond Chandler

FreeSWITCH 1.2

第二版

Copyright © 2013 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the authors, nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First edition: July 2010

Second edition: May 2013

Production Reference: 1170513

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78216-100-4

www.packtpub.com

Cover Image by Suresh Mogre (suresh.mogre.99@gmail.com)

本书贡献者

Authors

Anthony Minessale

Michael S Collins

Darren Schreiber

Raymond Chandler

Reviewers

Norm Brandinger

Kristian Kielhofner

Jeff Leung

Brian Wiese

Acquisition Editor

Usha Iyer

Lead Technical Editor

Ankita Shashi

Technical Editors

Chirag Jani

Soumya Kanti

Ankita Meshram

Dheera Meril Paul

Zafeer Rais

Project Coordinator

Arshad Sopariwala

Proofreader

Paul Hindle

Indexer

Monica Ajmera Mehta

Graphics

Abhinash Sahu

Production Coordinator

Pooja Chipulunkar

Cover Work

Pooja Chipulunkar


为开源代言
靠谱的开源SS7/ISDN解决方案
FreeSWITCH
Asterisk
SANGOMA 8E1/16E1
FreeTDM/ISDN
chan_ss7

关于作者

Anthony Minessale 在计算机领域有着30年的工作经验. 他是FreeSWITCH项目的发起人和核心技术人，目前担任梭子鱼旗下公司CudaTEL的CTO。

他创建了并主持ClueCon 通信开发人员大会，此大会每年八月份在美国芝加哥举行。,

他有着非常丰富的网络和VOIP经验。在创建FreeSWITCH之前，他一直为Asterisk开源项目贡献代码，并且为Asterisk开发了许多功能，这些功能今天仍然在使用。在公司梭子鱼网络, Anthony负责总体CudaTEL PBX 设备的开发，此设备的核心电话系统引擎使用FreeSWITCH平台。

以下是作者-技术牛人，好男人对家庭的感谢：

I would like to thank my awesome family: my wife, Jill, son, Eric, and daughter, Abbi, for putting up with the long hours and supporting me on my cause to revolutionize the telephony industry. Thanks to my dogs, Chewie and Gypsy, for staying up with me at 2 a.m. on long nights. I would also like to thank the open source community at large, especially those involved in the FreeSWITCH project, and I hope to see you all every summer at ClueCon!

Michael S Collins 是一位开源通信软件的狂热爱好者. 他对PBX有着非常深入的了解，从事PBX技术5年，呼叫中心开发长达9 年以上。他是活跃的FreeSWITCH社区成员。

另外他是FreeSWITCH CookBook核心作者. 目前在梭子鱼网络工作，和妻子孩子居住在加利福尼亚州中部。以下是作者的感谢语：

I would like to thank first and foremost my wife, Lisa, my daughter, Katherine, and my son, Sean, who keep me going each day. Without them I could not contribute to open source projects like FreeSWITCH, much less co-author a book.

I would also like to thank the many FreeSWITCH experts around the world who are so willing to answer technical questions: Anthony Minessale, Michael Jerris, Moises Silva, Raymond Chandler, Ken Rice, Travis Cross, and many more. I would especially like to thank Brian K. West for patiently educating me in the ways of VoIP.

I am also grateful to the many people who devote themselves to answering questions on the FreeSWITCH mailing list and IRC channel: Steven Ayre, Avi Marcus, Steve Underwood, and all the rest. The FreeSWITCH worldwide community is a great example of what an open source community should be. Well done!

Darren Schreiber 是2600 HZ项目的CEO。他对一直对开源的FreeSWITCH非常关心，
投入了大量的时间和精力，并且和Brian, Mike 和Anthony一起推动FreeSWITCH。
他的项目支持了企业级的VOIP平台，支持客户做多方面的拓展语音开发，短信和视频服务。

他有着15 年的IT经验，部署了企业级的SaaS 环境。他同时也是一个大学的讲师，
负责国际性的VOIP培训。从年轻时开始，他一直专注于VOIP技术。他毕业于
伦斯勒理工学院，获得电脑科学和经济管理本科学位。

他也是FreeSWITCH Cookbook的核心作者，以下是他的感谢语：

> I'd like to thank, first and foremost, the FreeSWITCH team. Without them, I wouldn't have been challenged with some of the most intriguing technology and people I've ever worked with. It has been a gift working with them.
>
> I'd also like to thank my family and friends who have put up with my crazy work schedule and constant tardiness, and have helped provide funds and moral support for our work. Thanks for everything.
>
> Finally, I'd like to thank the open source community. Their tireless patience and countless selfless contributions are a constant reminder that the world is not an evil place, and that people are generally out for the greater good of society.

Raymond Chandler(@intralanman) 在过去10 年一直专注于开源项目。
Raymond 的VOIP经验开始于一个SER路由和Asterisk语音邮箱集成的项目。通过此项目，他发现一些asterisk的局限性，和SER本身功能的缺乏，他后来开始使用OpenSER和CallWeaver。这样的组合比以前的方式好一点，但是他还是没有找到他喜欢的完美解决方案。

在2006的时候，朋友给他介绍了FreeSWITCH。从那时开始他一直使用FreeSWITCH并且定期贡献一些代码。他是 mod_lcr 和其他PHP 脚本工具的作者。现在在CudaTel 专门从事软件开发。
从2011 春天开始，他是OSTAG 小组成员)，他的工作是推动开源通信项目，使得开源项目能够达到一个新高度。此项目的共享一些OSTAG计划，并且获得了其他组织在的资金援助。以下是感谢语：

I'd like to thank my loving wife, Samantha, our daughters Makenzie, Trinity, Alecsys, and Kristian, and our sons Kaiden and Casper for their support while they get less time with me than any of us would like. I'd also like to thank the countless volunteers who step up to help out in the FreeSWITCH and other open source project communities. It would be impossible to keep any project running without them.

关于审稿人

Norm Brandinger has worked in the computer industry focusing in the areas of systems programming and communications for many years. He graduated with an Engineering Degree in Computer Science from the University of Florida just before the industry began experiencing the exponential growth that continues today. He has worked on the communication challenges that Fortune 100, mid-sized, and small companies all face as an owner, employee, and consultant. Today he balances his high-tech career with a love of the outdoors. Hiking in the woods, climbing mountains, and biking are just some of the ways he finds some balance.

He would not be where he is today without the support, love, guidance, and good genes from his parents. His mom is a retired Special Education professor from Trenton State College. Most recently, his dad was the Executive Director of the New Jersey Commission on Science and Technology. His brother Paul also inherited some good genes as he now works for NASA's Goddard Space Flight Center.

I would like to acknowledge and thank my partner, Donna. She works full time as a nurse, yet manages to find the time to take care of much of the home and social responsibilities. Her tireless efforts are the reason I have had the time to work on this project.

Kristian Kielhofner is the Co-founder and Chief Technology Officer of Star2Star Communications, The World's Most Reliable Business Grade Communications Solution. He is the inventor and architect of Star2Star's patent-pending StarPath and Constellation technologies. Prior to joining Star2Star, he was most well known as the creator and maintainer of AstLinux, the first Linux distribution to target embedded devices for open source telephony applications.

Jeff Leung is a Small Business Networking Consultant and a Telecommunications Technician. He has seven years of experience in various networking protocols and network architectures including Microsoft Active Directory Networks, Legacy Nortel PBX systems, and FreeSWITCH IP-PBX systems.

He started learning networking protocols at the age of 14 when he stumbled across a book about Active Directory networks at his local library, and the experiences and self-teaching of networking protocols started to snowball from there, eventually turning a hobby into a career that he is very passionate about.

I extend my thanks to the Richmond Public Library located in Richmond, British Columbia for jumpstarting my career with the selection of printed material, and my family for the support they have given me during my tenure at British Columbia Institute of Technology for further training in the Telecommunications and Networking career.

Brian Wiese maintains IT systems for school districts in Central Wisconsin, USA. For more than 12 years he has brought cost-effective and sustainable solutions to education, including recommending open source software such as FreeSWITCH. Brian has also authored many programs and scripts to increase productivity and streamline daily operations for both education and the business sector. In his spare time he enjoys experimenting with new technology, tinkering with his home network, and giving back to his community.

First, I want to thank Anthony for creating FreeSWITCH and giving it to the world, as well as the community for being so willing to help others. I want to express my sincere gratitude to my parents for always pushing me to achieve success. Finally, I want to thank everyone who gave me unprecedented opportunities and inspiration to pursue my passion for technology.

www.PacktPub.com

Support files, eBooks, discount offers, and more

You might want to visit www.PacktPub.com for support files and downloads related to your book.

Did you know that Packt Publishing offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.PacktPub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at service@packtpub.com for more details.

At www.PacktPub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt Publishing books and eBooks.



http://PacktLib.PacktPub.com

Do you need instant solutions to your IT questions? PacktLib is Packt Publishing's online digital book library. Here, you can access, read, and search across Packt Publishing's entire library of books.

Why Subscribe?

- Fully searchable across every book published by Packt Publishing
- Copy and paste, print, and bookmark content
- On demand and accessible via web browser

Free Access for Packt Publishing account holders

If you have an account with Packt at www.PacktPub.com, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

目录

前言 1

第一章: FreeSWITCH系统架构 7

通信革命的开始和揭露的秘密 7
FreeSWITCH 设计 –模块化, 可拓展, 稳定性 8
几个重要的模块 – 终端和拨号规则 11
复杂应用变得如此简单 15
语音邮箱 15
多方会议 18
FreeSWITCH API (FSAPI) 接口 19
XML 注册 21
语言模块 22
演示配置 22
总结 24

第二章: 构建和安装 25

设置FreeSWITCH 环境 26
操作系统 26
操作系统的支持安装包 27
Linux/Unix 27
Mac OS X 27
Windows 28
文本编辑器和XML 28
下载源代码 29
从最新的源代码安装 30
在Linux/Unix/Mac OS X环境下编译FreeSWITCH 30
编译 FreeSWITCH 30

第一步 – 编辑 modules.conf 模块 31
第二步 – 执行 configure 配置脚本 31
第三步 – 执行命令 make 和 make install 32
第四步 – 编辑 modules.conf.xml 34

第五步 – 安装语音文件和音乐文件 35
在Windows环境下编译FreeSWITCH 35
Windows 用户需要注意的几个问题 36
搭建MSVC/MSVCEE的编译环境 36
启动FreeSWITCH 41
在后台运行FreeSWITCH 42
总结 44
第三章: 启动测试举例配置 45
需要理解的几个重要概念 45
让FreeSWITCH 工作 50
通过CLI 控制FreeSWITCH 50
在FreeSWITCH环境下配置SIP 电话 53
SIP 设置 53
X-Lite 软电话 54
物理电话 57
Aastra phones 58
Polycom phones 59
Snom phones 60
测试举例拨号规则 61
从单个电话测试呼叫 61
呼叫 Tetris extension 61
回声测试 62
音乐等待 62
演示语音IVR 62
测试 info 打印信息 63
两台电话或多台呼叫测试 63
呼叫其他分机 63
驻留通话 63
呼叫会议 63
举例拨号规则快速参考 64
总结 65
第四章: SIP和用户目录 67
了解FreeSWITCH 用户目录 67
如何配合FreeSWITCH 用户目录工作 70
用户功能 70
添加用户 72
测试语音邮箱 75
用户组 77
通过网关连接外部呼叫 79
设置一个新gateway 79
发起呼叫 82
接收呼叫 82

不通过gateway发起呼叫 83
SIP profiles 和用户代理 84
总结 85

第五章: 了解XML Dialplan 87

FreeSWITCH XML Dialplan 核心要素 88
Contexts 88
Default 89
Public 89
Features 89
Extensions 89
Conditions 90
Call legs 和通道变量 92
访问通道变量 94
正则表达式 95
Actions和anti-actions 98
拨号规则如何工作 98
创建分机 103
几个重要的Dialplan应用 105
bridge 106
playback 106
say 106
play_and_get_digits 108
ivr 109
sleep 109
answer 109
pre_answer 110
hangup 110
set 110
transfer 110
Dialstring 格式 111
总结 114

第六章: 使用XML IVRs和Phrase Macros 115

IVR 引擎概览 116
IVR XML配置文件 116
IVR 引擎概览 116
IVR菜单定义 118
greet-long 118
greet-short 119
invalid-sound 119
exit-sound 119
timeout 120

inter-digit-timeout 120
max-failures 120
max-timeouts 121
digit-len 121
tts-voice 121
tts-engine 121
confirm-key 122
confirm-macro 122

IVR菜单定义 122
menu-exec-app 124
menu-play-sound 124
menu-back 124
menu-top 124

路由呼叫到IVR 125
内嵌IVRs 125
在IVR中使用phrases 126
呼叫Phrase Macros 126
Phrase Macro举例– voicemail 127
高级路由 132
总结 134

第七章: 使用Lua开发Dialplan Scripting 135

开始使用Lua语言 136
从Dialplan中运行Lua脚本语言 136
基本Lua 语法 137
创建语音应用 138
一个简单的IVR –和呼叫方交互 141
条件和循环语句 142
更多的条件和循环语句 146
高级IVR概念 152
通过LuaSQL连接数据库 152
通过curl发起web点击呼叫 158
Lua表达式和正则表达式对比 163
脚本语言技巧 164
总结 165

第八章: 高级拨号规则概念 167

Dialplan总览 168
基本Diaplan概念 171
Contexts 172
Conditions 173
Actions 174
所有模块聚合在一起 175
XML Dialplan模块回顾 176

Extensions 178
Conditions 179
特别condition变量 182
内嵌的execution 183
Actions和anti-actions 185
regex operator 186
内置的conditions 188
避免一些技术陷阱 190
XML Dialplan应用 191
mod_dptools 191
mod_sofia 194
mod_commands 196
调用变量 197
通过正式表达式测试变量 197
呼叫方profile值 197
通道变量 197
通道变量和呼叫建立 198
全局变量 199
Dialplan 函数 200
Real-time条件测试 201
String条件 202
Database查询 202
SIP contact头域 203
Set, export, 和legs 205
Set 对比export 205
通过呼叫头域传递变量 206
XML Dialplan私房菜 206
通过IP 地址匹配，呼叫一个号码 206
匹配IP 地址和呼叫方号码 208
匹配一个号码并且编辑数字号码 208
匹配一个号码，编辑此号码，添加一个前缀 209
呼叫一个注册的设备 209
尝试呼叫 party A, 然后party B 210
路由DIDs 到extensions 210
可选的呼出网关 211
企业应用环境中，对多终端设备发起呼叫 211
总结 213

第九章: 比静态XML 配置文件更加灵活的配置方法 215
mod_xml_curl 基础 216
mod_xml_curl 拨号规则 219
mod_xml_curl 文件夹 220

mod_xml_curl配置 222
mod_xml_curl总结 225
通过语言绑定，动态生成配置文件 225
从命令行发起呼叫 227
使用ESL执行命令 229
总结 232

第十章: 外部程序控制FreeSWITCH 233

总览 234
事件系统架构 234
基于事件的模块 235
mod_event_socket 235
配置事件 套接字设置 236
读取套接字事件 237
最少套接字信息 239
发送事件 240
从拨号规则发起的事件 241
mod_event_multicast 242
FreeSWITCH 事件系统命令 243
auth <password> 243
api 243
bgapi 244
event 245
noevents 246
divert_events 246
filter 246
filter delete 247
nixevents 248
sendevent 248
sendmsg <uuid> 248
execute 249
hangup 250
nomedia 250
log <level> 250
nolog 250
linger 251
nolinger 251
FreeSWITCH后台应用 251
Event Socket Library 251
支持的库 252
ESLObject 252

eslSetLogLevel($loglevel) 252

ESLevent对象 253
serialize([$format]) 253
setPriority([$number]) 253
getHeader($header_name) 253
getBody() 253
getType() 253
addBody($value) 253
addHeader($header_name, $value) 253
delHeader($header_name) 253
firstHeader() 254
nextHeader() 254

ESLconnection对象 254
new ($host, $port, $password) 254
new ($fd) 254
socketDescriptor() 254
connected() 254
getInfo() 255
send($command) 255
sendRecv($command) 255
api($command[, $arguments]) 255
bgapi($command[, $arguments]) 255
sendEvent($send_me) 256
recvEvent() 256
recvEventTimed($milliseconds) 256
filter($header, $value) 256
events($event_type,$value) 256
execute($app[, $arg][, $uuid]) 256
executeAsync($app[, $arg][, $uuid]) 257
setAsyncExecute($value) 257
setEventLock($value) 257
disconnect() 257

事件实战练习 257
Event Socket Library举例 – 运行一个命令 258
发送套接字到FreeSWITCH举例 258
点亮电话指示灯 258
重新启动电话 260
发送配置请求，重新配置电话 260
自定义提示信息 261
总结 262
第十一章: 使用mod_httpapi模块实现基于页面的点击呼叫 263
HTTAPI语法 264
如何工作 265
playback 265
vmname 266
record 267
pause 267
speak 268
say 269

execute 270
sms 271
dial 271
recordCall 272
conference 272
hangup 273
break 273
log 273
continue 274
getVar 274
voicemail 275

mod_httapi配置文件 275
执行权限 277
退出呼叫控制 280
通过保存的数据访问以前的请求 280
请求中丢失的数据 281
更加简单的方法 281
使用HTTAPI开发简单的IVRdemo 282
总结 286

第十二章: NAT处理 287

NAT简单介绍 288
了解NAT发展历程 289
NAT带来的四个问题 290
理解FreeSWITCH中的NAT设置 292
标记媒体流流程 296
高级选项和配置 297
FreeSWITCH在客户端 299
NAT环境下使用多个FreeSWITCH 300
结论 301
总结 302

第十三章: VoIP安全 303

不同层次的网络防范 303
分离接口和严格控制流量 304
举例设置–简单设置 305
举例设置–复杂设置 307
VLANs 308
网络入侵检测 309
注册监测 309
Fail2Ban 310
加密 312
SIP信令保护 313
选择加密选项 313
SSL加密 314

TLS加密 315
语音保护 316
SRTP加密 317
ZRTP加密 318
密码保护 320
注册密码 320
语音邮箱密码 321
总结 322
第十四章: 高级功能和进一步阅读 323
多方会议 324
配置 324
会议profiles 324
呼叫方控制 329
Advertise 会议启动 330
发送和接收XMPP 事件 330
连接呼叫方到会议 331
控制活动的会议 332
Nibblebill计费模块 332
用户用例 332
计费预付方式 332
计费后付方式 333
按照每路通话计费 333
最大额度设置和防止盗打 333
设计目标 333
安装配置 334
数据库表 335
在PostgreSQL环境下创建数据库表 336
在MySQL环境下创建数据库表 336
对呼叫计费 336
nibble 方式计费 (默认) 336
可选计费方式 337
举例 338
每个用户不同费率 338
所有用户实行单费率 339
一个地区代码不同费率 340
一种服务不同费率 341
无话费余额时结束通话 342
应用模块/CLI/API 命令 343
添加和扣除话费 345
启用会话心跳机制 345
仅对 B Leg计费 345
可选终端接入方式 345
Skype和GSM终端 346
通过mod_skypopen模块使用SKYPE 347

通过mod_gsmopen模块使用GSM 348
通过FreeTDM模块接入模拟线路和E1线路 349
配置工具和相关开源项目 349
图形界面 350
FusionPBX 350
FreePyBX 350
blue.box 351
Kazoo 351
支持的语言包 351
Liverpie (Ruby) 352
FreeSWITCHeR (Ruby) 352
Librevox (Ruby) 352
EventSocket (Python/Twisted) 353
FSSocket (Perl) 353
Vestec自动语音识别 353
总结 354

附录 A: FreeSWITCH在线社区 355
FreeSWITCH邮件列表 355
通过IRC实时交流 357
FreeSWITCH 主要网站和技术wiki 359
FreeSWITCH主页 – www.freeswitch.org 360
FreeSWITCH wiki主页– wiki.freeswitch.org 360
每年一次的ClueCon开源开发者大会 361

附录 B: 从Asterisk到FreeSWITCH切换 363
开始 364
启动或停止Asterisk或FreeSWITCH 364
基本排查 365
Asterisk 365
FreeSWITCH 365
配置文件 366
两个SIP 电话 366
Asterisk配置 367
FreeSWITCH配置 368
分析 372
Voicemail 373
Asterisk 373
FreeSWITCH 374
访问voicemail邮箱 375
Asterisk 375
FreeSWITCH 375
总结 376

附录 C: FreeSWITCH历史 377
把一件事情做的更好 378
新创意和新项目 380
第一届ClueCon会议 382
介绍FreeSWITCH 383
索引 387

前言

自从FreeSWITCH 1.0.6发布以后，在过去三年来发生了很多事情。FreeSWITCH 1.0 仅仅两岁多。一些早期的FreeSWITCH 用户变成了FreeSWITCH 忠实的用户。很多新的功能模块被加入到了FreeSWITCH 新版本里面，为发布新版本的FreeSWITCH 增加了不少亮点。短短几年，FreeSWITCH 私房菜也发布了。还有很多关于FreeSWITCH事情正在发生。

与此同时，FreeSWITCH 团队继续活跃在他们的开发过程中。一些新的功能和一些已经存在的功能被优化和提示。甚至于在FreeSWITCH 1.2.0发布之前，我们还在重新审查此书的，可以称之为一本承上起下的书。

当开始写这本书的时候，FreeSWITCH 1.2.1 已经开发完毕。在随后的几个月内，发布了几个小版本。目前，FreeSWITCH 项目已经转移到版本1.2.8 的稳定版本状态，在此版本上不添加任何功能，仅仅对bug进行修复。新的开发分支最后将在FreeSWITCH 1.4 版本上进行，可能在2016 年，你将看到FreeSWITCH 1.4 发布？我们仅仅告诉你一个时间，但是我们不敢和你打赌。

就像上次在此书中我们做的一样，我们回答几个非常重要的问题。

FreeSWITCH 适合我吗？ 正确的答案总是一样的，具体使用看情况。FreeSWITCH 开发团队和长期用户经常被问到使用哪个通信平台做开发。答案也总是一样的：适合于你的就是最合适的。当然我们可能认为FreeSWITCH 可以满足多种使用环境和应用场景。 如果Asterisk 或者Yate 可以满足你的需求，你当然可以使用他们。我们对开源软件满怀希望。

FreeSWITCH是什么? FreeSWITCH 是一款可拓展的软交换. 比较简单的理解，它可以实现任何传统PBX的功能，而且不仅仅于此。它完全可以胜任商业版本的软交换的需求。 通过拓展，它可以处理上千并发的呼叫。同样，通过裁剪，它可以作为一个简单的软电话运行在个人电脑或者笔记本电脑。当然，它也可以实现服务器集群。FreeSWITCH是来自于Barracuda Networks 的电话引擎，驱动强大的CudaTel 通信服务器电话系统。

FreeSWITCH 不是一个代理服务器。如果你需要实现代理服务器的功能，你可以考虑OpenSIPS, Kamailio, Repro 或者类似的软件. FreeSWITCH是一款背靠背用户代理或B2BUA。从这个角度说，类似于Asterisk或其他类似的IPPBX 软件。

FreeSWITCH采用的是哪种开源许可证? FreeSWITCH 发布遵循 Mozilla Public License (MPL) Version 1.1. 因为FreeSWITCH 是一个软件包，它可以在其他的软件项目中使用，开发人员认为我们需要做一个平衡，保持和这个生态系统中的非常自由的BSD 许可证，GPL许可证平衡。MPL非常好的满足了这样的模式，并且允许其他厂家利用FreeSWITCH 开发商业产品，无需担心许可证问题。

反过来，如果使用FreeSWITCH中基于GPL的软件怎么办？ 如果开发人员可以保证一些优质客户和使用GPL 许可证软件的用户能够使用FreeSWITCH，强大的事件套接字接口提供了丰富的功能，允许通过一个简单的TCP socket 外部编程接口可以控制FreeSWITCH. 无论你使用什么许可证，你仍然可以连接FreeSWITCH 服务器端，无需担心可许可证的问题。

本书涵盖的内容

第一章, FreeSWITCH 架构，通过介绍系统架构，给用户一个简单的介绍。

第二章, 搭建和安装, 提示用户如何在Windows 和Linux 环境下，如何下载安装FreeSWITCH。

第三章, 测试举例配置，提供一个可上手，强大，功能丰富的FreeSWITCH举例测试。

第四章, SIP和用户目录, 提供一个简单介绍，解释用户和用户目录的概念和初步了解SIP 用户代理。

第五章, 了解XML拨号规则，解释如何在FreeSWITCH环境下，创建和编辑拨号规则的extensions，添加高级功能。

第六章，使用XML IVRs 和Phrase Macros，讨论如何创建语音菜单和语音phrases，以便实现和用户进行IVR语音交互，并且讨论非常有用的Phrase Macro system。

第七章，Dialplan 脚本语言Lua，介绍高级呼叫处理。如何使用轻量级的语言Lua对呼叫进行处理。

第八章，高级拨号规则概念，在第五章知识的基础上添加高级功能来实现不同的路由策略。

第九章，讨论范围将超出静态的 XML配置文件，为了动态配置控制FreeSWITCH，必须解释一些必要的概念，例如数据库系统.

第十章，通过外部命令控制FreeSWITCH，介绍如何通过强大的 Event Socket 和 Event Socket 包访问控制FreeSWITCH 服务器端.

第十一章，通过mod_httapi 开发页面点击呼叫功能，给读者演示如何使用 mod_httapi 模块通过HTTP开发创建一个电话应用模块。

第十二章，NAT处理， 提供丰富的介绍，解释在VOIP 环境下，NAT问题的已经如何工作。

第十三章，VoIP 安全，提供一些建议如何使得 VoIP 通信更加安全，和如何对FreeSWITCH进行安全策略的配置，防止各种攻击。

第十四章，高级功能和进一步阅读，重点介绍一些强大的FreeSWITCH 功能，例如电话会议功能，并且提供一些思路和建议，让读者如何学习此功能的更多资源。

附录 A，FreeSWITCH 在线社区，简单介绍世界范围的在线社区和沟通工具。

附录 B，如何从Asterisk迁移到FreeSWITCH平台，帮助熟悉Asterisk的用户可以快速使用FreeSWITCH。

附录 C，FreeSWITCH历史，简单介绍Anthony Minessale-核心架构师，如何启动FreeSWICTH项目。

阅读此书需要的工具

至少你应该有一台自己的测试电脑来运行FreeSWITCH. 这是一台服务器，但是不是一个必要的要求。
你需要至少一台SIP终端设备，可以是软电话，桌面电话机或者一个ATA。
没有这些必要的工具，你的FreeSWITCH系统不能做任何简单的呼叫测试。

尽管VOIP帐号不是必要的条件，但是如果有这样的帐号，你可以通过这些服务呼叫到PSTN线路.

谁应该阅读此书

本书读者可能是FreeSWITCH 系统管理员，或者一些FreeSWITCH爱好者，希望了解
如何搭建，设置和运行FreeSWITCH。
如果你已经使用了FreeSWITCH，你可以发现更多的关于FreeSWITCH的技术细节。

阅读此书时，当然希望读者对基本的网络概念有一个了解。VOIP知识不是一个基本要求。如果你了解VOIP知识，有助于快速学习FreeSWITCH。

常规标注

本书中，你可能看到一些特别的注释风格和正文有所不同。以下是例子和解释

代码显示风格."

一段代码:

```
<extension name="get voicemail">
  <condition field="destination_number" expression="^\*98$">
    <action application="answer"/>
    <action application="voicemail"
    data="check auth default ${domain_name}"/>
  </condition>
</extension>
```

如果我们作为重点强调的部分，字体加粗：

```
<extension name="get voicemail">
  <condition field="destination_number" expression="^\*98$">
    <action application="answer"/>

  <action application="voicemail"
  data="check auth default ${domain_name}"/>

  </condition>
</extension>
```

任何命令行输入和输出显示：

```
# /usr/local/freeswitch/bin/fs_cli –x version
```

新名词和重点字都加粗。一些提醒和技巧提示将以输出框的形式显示：



警告和重要提示。



小技巧

读者反馈

我们非常欢迎读者给我们及时的反馈信息，让我们知道你是否喜欢此书。读者的反馈是非常重要的，有助于于我们继续提高出版的质量。

发送反馈信息到feedback@packtpub.com,
描述关于你对此书的感受和内容。

如果你对一些其他的讨论比较感兴趣，或者计划贡献此内容，请参考www.packtpub.com/authors.

客户支持

非常高兴你现在获得了此书。如果购买以后有什么问题，我们可以及时帮助你。

下载例子代码

你可以通过注册到http://www.packtpub.com/support ，下载相关代码。

本书错误

尽管我们已经非常小心地对此书做了完善，到底正确准确。但是还有可能发生错误。
如果你发现我们书中的错误，或者文本，代码错误，我们非常希望你告诉我们
通过这种办法，可以提高本书的质量，我们在下一个版本中做修订。
如果你发现错误，可以访问到http://www.packtpub. com/submit-errata, 现在这本书，检查错误链接，
然后提交。如果错误被修订，我们将更新错误列表。任何目前存在的错误都可以通过访问
http://www.packtpub.com/support. 获得。

版权

Piracy of copyrighted material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works, in any form, on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at copyright@packtpub.com with a link to the suspected pirated material.

We appreciate your help in protecting our authors and our ability to bring you valuable content.

问题

You can contact us at questions@packtpub.com if you are having a problem with any aspect of the book, and we will do our best to address it.

1 FreeSWITCH系统架构

欢迎使用FreeSWITCH! 如果你正在阅读此书，毫无疑问，你已经对通信和VoIP感兴趣。FreeSWITCH 是一款在通信革命时期开发的革命性的通信软件。我们了解软件架构之前，让我们了解一下通信世界的五光十色。这样有助于对FreeSWITCH完整认识。

在这一章节，我们将介绍:

- 一个通信革命
- FreeSWITCH的优势
- 终端和拨号规则模块
- 利用FreeSWITCH简化复杂模块的开发例如语音邮箱的开发

技术革命已经开始，通信的秘密被揭开

对大多数人来说，电话系统如何工作是一个谜。这个谜延续了很多年。我们只是把电话线插到墙上的电话插口，我们就可以打电话了。通信的技术革命已经开始，我们开始慢慢揭开传统通信工业的谜团。现在，和你一样，每个人可以搭建一个比传统PBX更加强大，功能更加丰富的相对低成本的通信系统。一些人使用FreeSWITCH作为运营平台，通过提供语音服务，公司可以获得利润。FreeSWITCH 通过本身强大的设计，已经使通信系统变得更加简单，所以我们在这里简单介绍一下FreeSWITCH 的架构，使得读者对系统有一个全面的了解。

不用关心太多这些看起来非常抽象的概念。学习通信技术学院时间，特别是VOIP技术。
事实上，我们可以你多次阅读本章节。第一次阅读时，阅读时吸收更多的知识，然后完成第五章
理解XML 拨号规则，不断了解这些内容。你将非常惊讶你多VOIP知识了解如此之多，并且对
FreeSWITCH有了丰富的理解。 完成阅读第十章，通过外部接口控制FreeSWITCH以后，到这里
你将对VOIP知识和FreeSWITCH有一个全面的理解。让自己花更多时间消化这些奇怪的VOIP概念，
我们相信，不久，你将成为一个非常熟练的FreeSWITCH管理员。 如果你不断学习中心知识，你将
非常了解这些通信技术的神奇和精彩之处。

通过多年的发展，电话和通信系统是非常复杂的系统，支持很多复杂的技术变革。在英国和美国
最受欢迎的电话是传统的模拟电话，这些电话使用在 POTS 和老式电话服务。 从传统的固定电
话到我们现在使用的无绳电话，一直讨论的一个技术就是通信。在过去的10-15年中间，一个介于
电脑和通信的趋同技术产生出来一对相对于传统模拟线路非常便宜的-移动电话和VOIP 电话。

FreeSWITCH 通过技术的整合，把这些相关的通信技术结合在了一起，所以以前的技术都可以
相互融合在FreeSWITCH平台，否则他们是完全不兼容的技术。FreeSWITCH 同样给用户带来了
一个新的技术，用户可以通过自己的程序来控制呼叫，就像传统的系统一样。FreeSWITCH是一个
软件可以运行在Windows 平台和一些Unix 变种的操作系统，例如Mac OS X, Linux, Solaris 和 BSD.
你可以安装FreeSIWTCH在普通的PC机，或者安装在相对高端的服务器，通过服务器来处理语音呼叫。
第二章我们介绍如何搭建和安装FreeSWITCH。现在我们讨论FreeSWITCH的基础架构。

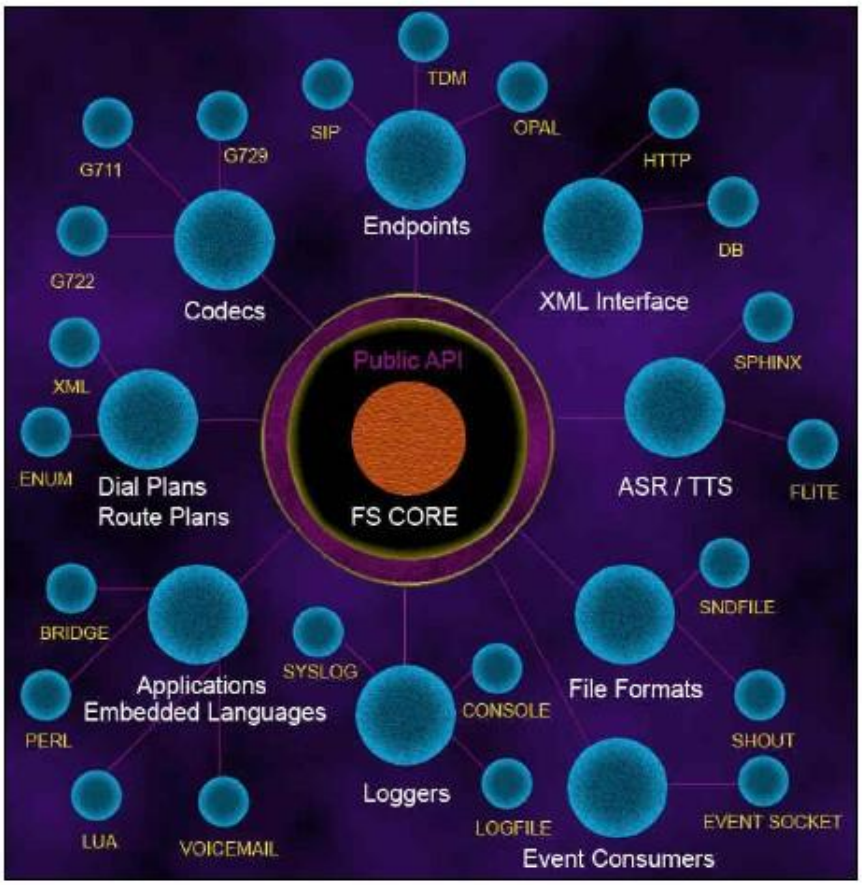
FreeSWITCH设计–模块化，可拓展，稳定性

FreeSWITCH 的设计目标是提供一款模块化，可拓展，稳定的软交换核心，并且提供强劲的
接口应用，能够让开发人员添加和控制系统。在FreeSWITCH中，每个模块都是独立的，并且更多
没有涉及模块之间如何工作。在FreeSWITCH中，每个功能的拓展仅仅通过添加模块，并且
加载模块。

FreeSWITCH 提供多种不同的模块，这些模块都围绕FreeSWITCH设计，就像卫星围绕卫星轨道运转。列表包括以下模块:

模块类型	目的:
Endpoint	通信协议包括 SIP/H.323 和 POTS 模拟线路
Application	执行一个任务，例如播放语音和设置数据
Application Programming Interface (API)	导出一个功能，输入文本，然后输出文本,用来和模块交互或者实现外部连接
Automated Speech Recognition (ASR)	提供语音识别系统接口
Chat	桥接和交换各种聊天协议
Codec	提供语音编码格式转换
Dialplan	解析呼叫信息，决定呼叫路由
Directory	连接目录信息服务，例如 LDAP, 或者标准的lookup API
Event handlers	支持外部程序接口控制FreeSWITCH
File	对外提供一个语音播放文件接口，播放各种语音格式的文件。
Formats	支持各种语音格式的文件播放
Languages	提供各种程序接口对呼叫实现控制
Loggers	控制控制台的日志，系统日志和日志文件
Say	系统对用户播放各种语言的电话号码，时间，日期和一些组词的拼写
Text-To-Speech (TTS)	语音识别引擎接口
Timers	POSIX 或者 Linux kernel 时间
XML Interfaces	使用 XML，实现 Call Detail Records (CDRs), RADIUS, CURL, LDAP, RPC, 或SCGI

以下图例说明了FreeSWITCH架构，其他模块如何围绕FreeSWITCH工作:



通过组合各种模块接口，FreeSWITCH 配置以后，可以连接IP 电话，或者基于IP的电话服务，POTS 线路等等。它也可以实现语音编码的转换，并且接入到用户自己创建的语音菜单系统，或者IVR系统。甚至于用户可以通过外部的程序控制FreeSWITCH运行。让我们现在近距离看看一对非常重要的模块。

重要的模块–Endpoint和Dialplan

Endpoint 模块是非常重要的模块，添加了很多核心的功能，才使得FreeSWITCH今天变得如此强大。这些模块的主要角色是控制某些标准的通信协议，然后经过标准化处理以后，进入到一个抽象实体，我们称之为session或者会话。在FreeSWITCH中，一个session 代表FreeSWITCH和一个协议的连接。在FreeSWITCH 环境下，系统支持多个 Endpoint 模块，这些模块支持了多个通信协议，例如 SIP, H.323, Jingle (Google Talk), 和其他的模块。我们花更多时间解释非常受欢迎的模块mod_sofia.

Sofia-SIP (http://sofia-sip.sourceforge.net) 是一款开源的SIP 协议栈，项目发起人是诺基亚。提供一个对SIP的语言接口。我们使用这个软件包作为FreeSWITCH mod_sofia SIP 接口模块。这个模块注册到所有FreeSWITCH 钩子，然后形成一个Endpoint 模块，把所有的FreeSWITCH结构转换成SIP结构，然后返回到FreeSWITCH。配置参数来自于FreeSWITCH的中心配置文件，支持用户配置定义的参数和连接参数。通过这样的方式，FreeSWITCH 支持来自SIP 电话或者设备的注册，然后注册到其他的SIP终端服务，例如运营商服务，发送提示信息，对终端电话提供语音邮箱服务。



SIP 协议包含多种 RFC的定义说明。基本的 RFC资料可以在 http://www.ietf.org/rfc/rfc3261.txt. 找到。

当在FreeSWITCH和SIP设备之间建立一个SIP呼叫以后，在FreeSWITCH中，这个呼叫被看作是一个活动的会话。如果此通话是呼入的呼叫，这个通话可以被转接或者桥接到IVR，语音等待或者通过其他的选项转到其他分机。让他们来看一个典型的呼叫流程，2000作为一个注册的分机，2001分机希望呼叫2000分机。

首先, SIP电话通过网络发送一个呼叫 setup 信息到mod_sofia (mod_sofia 一直监听这些信息). 收到setup信息以后，mod_sofia然后解析相关信息，然后传递这些信息到FreeSWITCH核心状态机。这个状态机发送这个呼叫到 ROUTING 状态。

下一步是通过配置数据来定位拨号规则模块。通常默认使用的 Dialplan拨号规则模块是XML拨号规则模块。这个模块查询在FreeSWITCH中心XML注册的指令。XML Dialplan模块将通过正规表达式匹配模式来解析一系列的XML分机对象。

当我们呼叫2001时，希望XML分机对比测试destination_number 值，可以通过相应的匹配关系找到2001．拨号规则不仅仅局限于匹配一个分机。事实上在第五章，深入了解XML 拨号规则中，你将了解更多的关于exension的定义。 在每个呼叫中，XML 拨号规则模块加载了很多的任务。每个匹配的extension，将添加一些呼叫任务逻辑。

假设FreeSWITCH 至少找到一个extension, XML Dialplan将在这个session对象中插入一个连接指令。这些指令包含呼叫2001的必要信息。一旦这些指令给添加到session里面， 呼叫session的状态将从ROUTING 转换到EXECUTE状态, 下一个处理机制将查找命令列表，执行从 ROUTING 状态获得的指令。这里就是应用接口进入的地方。

在每个指令通过应用模块名称和参数添加到一个session中。这里，我们使用的bridge 应用就是一个例子。此模块的目的是为了呼出，建立另外一个session，然后连接两个session，通过两个session实现直接语音的交互。对于bridge 来说，提供的参数是 user/2001, 这是最简单的对分机2001 呼叫的办法. 一个2001 呼叫入口类似于这样：

```
<extension name="example">
  <condition field="destination_number"
             expression="^2001$">
    <action application="bridge" data="user/2001"/>
  </condition>
</extension>
```

这里，extension 命名为example, 并且有一个条件匹配。如果可以匹配这个条件，一个应用模块开始执行。在普通的程序中，这个extension 可以这样表示: 如果呼叫方拨打2001, 此时，呼叫方和在2001 终端之间建立了一个连接。让我们看看这是怎么发生的.

一旦我们在session 插入指令，session的状态就会修改到EXECUTE状态，FreeSWITCH 核心模块将通过获得的数据来执行相应的命令。首先，默认的 execute 状态处理机将解析命令，对user/2001执行bridge命令，然后系统查询 bridge应用模块，然后传递 user/2001 数据进入到session。这样，FreeSWITCH core 创建一个系统需要的呼出session类型。User 2001 也是一个SIP 电话，所以user/2001将引入一个 SIP拨号字符串，这个字符串将被彻底到mod_sofia 模块，建立一个呼出的session。

如果这个新的session成功建立，在FreeSWITCH core 中将有两个session。Bridge 应用模块将负责桥接呼叫方和被呼叫方。这样，语音流将可以在双方进行交互。如果被呼叫方不能应答此通话或者忙音，系统生成一个超时信息，并且发送这个测试信息到呼叫方。如果出现未应答或者分机忙音状态，可以实现多种路由，包括呼叫前转或者进入到语音邮箱。.

用户拿起电话，拨2001的时候，所有这些流程就开始执行。FreeSWITCH 负责所有的SIP复杂流程，降低了流程的复杂程度，并且使得简单。从分机2000 呼叫分机2001的流程中，通过拨号规则的配置，降低了流程的复杂性。如果我们想从2001 拨打2000，我们可以添加另外一个入口:

```
<extension name="example 2">
  <condition field="destination_number" expression="^2000$">
    <action application="bridge" data="user/2000"/>
  </condition>
</extension>
```

在这个流程中，Endpoint 模块把SIP转到 FreeSWITCH session 中，Dialplan 模块转XML到一个extension. bridge 应用接口创建一个外呼，并且连接简单的语音附带彻底需要的数据。Dialplan 模块和应用模块接口支持默认的的FreeSWITCH sessions。 因为在实际呼叫过程中，可能有一些其他的语音接口协议，这些抽象接口在用户层面为呼叫流程做了简单化处理，同时简化了应用模块和拨号规则的设计。因为这些抽象接口的设计，为我们未来新接口的设计提供了方便，例如Skype 接口，我们可以重复使用这些应用模块和Dialplan 模块。这些设计原理应用在了Say, Automatic Speech Recognition (ASR), Text-to-Speech (TTS), 和其他模块.

用户完全可以获得终Endpoint认协议支持的数据。例如，在SIP协议中，用户可以获得SIP数据包中SIP头域的数据或者其他用户需要的数据。我们可以通过在通道中添加变量来解决这个问题。通过使用这些通道变量，用户可以在拨号规则和应用模块中，通过通道名称在获得一些数据，这些数据就是我们在mod_sofia中设置的。我们通过特别定义的变量来共享SIP Endpoint数据。但是，FreeSWITCH core模块把这些看作是一个任意的通道变量，并且可以被核心模块忽略。在FreeSWITCH环境中，有一些系统特别预留的通道变量，通过非常有趣的方法影响到FreeSWITCH。通道变量的概念和这些默认的变量使用非常相似，如果你一直使用脚本语言或者配置引擎使用了这些变量（属性-值对应），你已经拥有了这个优势。 通过变量名称和变量值设定就可以传递数据到通道变量。

这里是一个应用模块接口的例子，例如set, 可以让我们在拨号规则中设置自己的变量值:

```
<extension name="example 3">
  <condition field="destination_number" expression="^2000$">
    <action application="set" data="foo=bar"/>
    <action application="bridge" data="user/2000"/>
  </condition>
</extension>
```

这个例子和上面的例子几乎一样，但是除了发起一个呼叫，我们首先设置了变量foo等于bar值. 这个变量将一直保留直到完成整个呼叫，甚至可以作为一个呼叫结束日志的参考值。

我们添加了越来越多的小功能，越来越多的功能就可以重复使用，这样我们的系统使用起来就更加方便。例如，编码接口根本不知道什么是系统的核心模块，编码模块之间完全独立地实行对语音文件编码压缩和解码。新的编码模块一旦完成，对终端接口来说，它可以变成一个可用的模块来承载语音流的编码。意味着，如果我们开发的语音合成Text-To-Speech模块可以工作，我们可以对任何终端播放一个同步的语音。哪个模块先开发，哪个模块后开发都没有直接的关系。但是如果添加更多的功能模块，那么其他模块就具有更强大的功能。TTS模块变得更加有用，因为编码模块支持了更多的编码。模块之间的互相交互使得两者更加强大。同样的道理，其他模块也是如此。如果我们开发了新的模块，目前的终端可以马上运行这些模块。

复杂的应用变得简单

FreeSWITCH移除了一些高级应用模块，简化了这些模块。让我们看两个比较复杂的应用模块例子。

Voicemail

我们第一个讨论的例子就是语音邮箱。语音邮箱功能比较容易解释我们的讨论。语音邮箱功能就是提供语音邮箱服务。这个应用比较有用，万一呼叫不能成功的时候，在 bridge应用以后添加第二个选项。添加这个选项的组合要特别小心。后面我们将讨论这些问题。让我们看看如何对呼叫方进行语音邮箱留言:

```
<extension name="example 4">
  <condition field="destination_number" expression="^2000$">
    <action application="set"
            data="hangup_after_bridge=true"/>
    <action application="bridge" data="user/2000"/>
    <action application="voicemail"
            data="default ${domain} 2000"/>
  </condition>
</extension>
```

这里我们看到，我们使用了两个通道变量. 首先我们设置了hangup_after_bridge=true通知系统一旦和对端的电话终端成功桥接通话，让系统停止后续的命令指令。我们还使用了带dollar标志的，带括号的变量 domain, ${domain}. 这是一个特别的变量，默认支持所有配置文件中的所有电话自动配置域名。

在这个例子中，我们检查是否有人正在拨打2000，然后桥接这个通话到注册的分机2000. 如果呼叫失败或者无应答，我们将继续执行下一个指令，这个指令就是执行voicemail应用模块。我们提供相应的应用模块必须了解的，邮箱邮箱对应的分机，和如何对此环境进行控制。下一步，语音邮箱模块播放预设的问候语或者通过Say 接口模块生成的语音。这个应用模块通过合并几个语音文件来播放一些特定的语音，例如，"分机2000 目前无法应答，请留言.' 然后，mod_voicemail 模块提示用户留言。然后用户可以语音留言。系统支持一个附加功能，如果你不满意你自己的录音留言，你可以重复录音，直到满意为止。一旦录音留言完成，系统将发起一个 FreeSWITCH MESSAGE_WAITING 事件。这个事件被mod_sofia模块引用，事件信息将被传递到SIP的 NOTIFY 消息。 这个消息通知SIP电话，系统正在有一个消息等待的通知。如果在正常情况下，在分机2000 注册的电话终端的MWI 指示灯启动。

在这个例子中，不仅仅只我们看到如何播放和对留言进行录音，如何对用户发送录音，我们还没有涉及FreeSWITCH core 的幕后英雄 - 事件系统。不像其他的例子， 事件系统本身不是一个模块接口，它是一个核心的引擎，你可以使用它来绑定一个命名的事件，并且当接收事件以后做出相应的回应。换句话说，通过FreeSWITCH core 的事件系统，有很多事件被发送或者接收。模块可以绑定任何事件。模块可以监听任何事件。模块可以在事件引擎中获得五个事件；其他模块可以监听这些事件。就像我们以前讨论的，Sofia SIP 模块绑定或者订阅这个事件，这个事件设计目的就是为了获得MESSAGE_WAITING信息。这样支持了 our mod_voicemail 模块和 mod_sofia交互，无需让系统知道对方是否存在。mod_voicemail 模块发起这个事件, mod_sofia模块接收这个事件， 然后转换成正确的SIP 信息-看起来非常不可思议，事件系统功不可没。

当实施支持多语言复杂的语音交互系统时，有几个非常与挑战性的问题需要注意，还有什么文件需要播放，如何把播放文件完美组合起来。Say 模块是一个比较好的解决办法，可以非常完美地把这些文件组合在一起，但是还是有一定的局限性，例如拼写一个单词，计数，或者播报一些日期。解决这些问题的方法是在Say 模块上再定义一个比较复杂的应用层，称之为Phrase Macros。 Phrase Macros 是一个XML 表达式的集合， 可以支持一个参数组。传递的参数组可以匹配不同的正式表达式或者执行一个字符串命令。这个工作流程和XML 拨号规则是一样的，只有用户匹配正确的IVR流程，系统才能按照正确的流程工作。例如，当 mod_voicemail 模块要求用户对你的信息进行录音，而不是文件的字符来让系统播放你需要的文件, 输入的命令将调用一个 Phrase Macro，那就是voicemail_record_message。 mod_voicemail 和 Phrase Macro 在配置文件中将共享这个字符串，允许用户轻松编辑这个文件，无需复杂的程序变动。

```
<macro name="voicemail_record_message">
  <input pattern="^(.*)$">
    <match>
      <action function= "play-file"
          data="voicemail/vm-record_message.wav"/>
    </match>
  </input>
</macro>
```

当mod_voicemail 执行voicemail_record_message macro时, 它首先执行这个模式匹配，万一匹配了所有的参数，但是这个macro 无任何输入。如果macro输入了相应的值，模式匹配输入的不同来执行不同的指令。一旦有一个匹配成功，匹配的标签解析XML的action 标签。这个macro 将播放 vm-record_message.wav 文件，但是比较复杂的macro，例如你更新留言信息其中的一条留言或者目前有多少留言匹配，你可以使用各种Say模块和播放各种语音文件。在第六章中，我们将讨论Phrase Macros 一些技术细节，在第七章我们将讨论配合Lua使用XML IVRs和Phrase Macros。

一旦完成这些语音文件和Say 模块加载，core 核心引擎就将这些模块连接在一起，通过这样的组合，就可以搭建一个强大的语音系统。这个Say模块就可以特别支持指定的语言和指定的语音邮箱。通过输入的变量的不同程序来实现一个指定的时间，然后转换成基于Say 开发的模块。Phrase Macro系统具有强大的技术优势，可以根据变量的不同，经过适当调整支持用户的要求，例如用户的每天的业务流程需求。例如，如果我们计划搭建一个小的IVR系统，允许用户拨打4位数字的号码，然后读这4位数字，然后挂机，我们可以执行一个macro myapp_ask_for_digits，其他的指令呼叫myapp_read_digits 。在我们的代码中，当系统要求输入这些数字时，我们将执行这些前面定义的macro，然后根据我们输入的指令，回放这些数字。一旦以上工作完成，即使是很少经验的开放人员也可以调用这个XML文件播放设置的语音。他或她可以使用Say模块读出回放这些号码，这个小功能应该可以支持多国语言，并且无需修改代码。在FreeSWITCH中的生成服务器环境下，Voicemail仅仅是其中的一个例子。在FreeSWITCH的环境下，有很多种可能性，可以使用电话配合电脑进行呼叫。

多方会议

在FreeSWITCH环境下，另外一个比较受欢迎的功能就是对方会议，通过模块mod_conference会议模块来实现。mod_conference模块提供了一个动态的会议室。这些会议室把所有的语音媒体通道进行桥接混音。这个功能用来主持一个电话会议，几个会议的通话人可以实现互动。每一个连接到同一个会议室新的会话可以加入到会议中，可以马上和其他的成员同时进行通话。使用以下拨号规则，我们可以添加另外一个extension 进入到一个会议室：

```
<extension name="example 4">
  <condition field="destination_number" expression="^3000$">
    <action application="conference" data="3000@default"/>
  </condition>
</extension>
```

这个例子非常简单，就像桥接一个电话一样， 但是特别之处在于许多呼叫方可以呼叫extension 3000加入同样的会议室。如果有三个人在会议室，其中一个人离开会议室，其他两个人还可以继续进行他们的谈话。

会议模块支持一些特别的功能，例如即使只有一个会议室成员的情况下，还可以播放语音文件或者进行语音合成。你可能猜想，我们可以使用TTS 和语音文件接口支持他们想要的功能。还是以前说的，一些小的功能模块结合在一起就可以拓展出一个很强的功能，而且无需了解系统中的其他部分。

Conference 模块也同样用特别的方法使用了事件系统，我们称之为custom 事件。当这个事件首先加载时mod_conference可以预留一个特别的事件空间，我们称之为一个subclass. 当一些有趣的事情发生时，例如一个呼叫方加入或者离开一个会议，它首先在core中发起一个CUSTOM 事件通道. 当我们希望接收事件时，我们必须先对 CUSTOM 事件做一个订阅，提供额外的subclass 字符。这个字符就是我们感兴趣的事件字符。 在这个例子中就是 conference::maintenance。通过这样的订阅，我们可以监控到一些非常重要的事件，例如有人加入或者离开会议，甚至于他们开始或者停止会议。关于会议的功能，我们将在第十四章的高级功能和进一步阅读中讨论。

FreeSWITCH API (FSAPI)接口

在FreeSWITCH环境下，另外一个强大的模块就是 FSAPI模块。它的设计原则非常简单-把用户输入的字符串或者文本作为一个输入结果，这个结果可能没有见过解析，然后按照输入的结果执行一个命令。根据功能的不同，返回的结果可以是任何不同的结果，可能是当个的字符，字母也有可能是几页的文本。FSAPI功能主要优点在于如果模块使用这些接口命令做呼叫路由，模块中调用的其他模块无需再次直接链接这些实际的功能代码。FreeSWITCH的命令行或者CLI使用了FSAPI从系统的命令提示传递FreeSWITCH API命令。

以下是一个简单的例子，我们从 FreeSWITCH CLI 执行 FSAPI的 status 命令:

```
freeswitch@internal> status
UP 0 years, 3 days, 2 hours, 25 minutes, 5 seconds, 49 milliseconds, 603 microseconds
FreeSWITCH (Version 1.3.13b git 1a3a11f 2013-02-15 11:16:49Z) is ready
0 session(s) since startup
0 session(s) - 0 out of max 30 per sec
1000 session(s) max
min idle cpu 0.00/98.00
Current Stack Size/Max 240K/8192K
```

当我们输入status 然后摁Enter 键的时候，让我们看看这里到底发生了什么，
输入的 "status" 是用来从加载的模块中查找 status FSAPI 函数。 调用后续的函数。
然后core 查询Status 信息。一旦获得status 数据，结果被写成数据流返回，然后打印出
命令结果。

我们已经了解一个模块可以创建或导出FSAPI功能函数。这些函数可以从任何地方执行，
例如CLI 命令。但是，大家等等，有更多的功能远远不止这些。模块可以通过指定的协议
推送到FSAPI接口，并且可以发送结果。在FreeSWITCH中有两个模块可以实现类似的功能，他们是
mod_xml_rpc和mod_event_socket (在第九章和第十章将讨论这些细节)。现在来看一下mod_xml_rpc模块
的例子. 在FreeSWITCH 模块中，这个模块实施了标准的 XML-RPC 协议。XML-RPC 接口可以连接
FreeSWITCH，执行选择的任何FSAPI 命令。所以，一个远端用户可以执行PRC 指令，可以获得
Status 结果，这个结果和我们刚才看到的是一样的。同样的模块可以支持标准的web 服务器端，
用户可以通过URL 链接，访问FSAPI命令。例如，我们可以通过浏览器访问
http://example.freeswitch.box:8080/api/status，获得status结果。通过这一技术，完全可以创建FSAPI
命令相似的CGI动态应用界面，这个界面可以直接访问FreeSWITCH的内部模块数据。

因为我们已经了解FSAPI 是一个功能非常丰富的接口。它可以提供一个CLI 接口，从一个模块
调用另外模块的数据，并且可以通过www或 XML-RPC 功能函数导出必要的数据。仍然有很多我们没
有涉及的功能函数。早一点，我们简单介绍了通道变量的概念，我们使用了这个表达式 ${myvariable}
获得某些变量的值。FSAPI 功能函数可以通过这样的方法获得 ${myfunction()}. 这个标志符说明系统
调用了myfunction，然后生存表达式的结果。因此，同样我们可以使用 ${status()} 来访问status命令。
例如:

```
<action application="set" data="my_status=${status()}"/>
```

my_status 变量的值就是 status 输出结果。

单一模块提供如此多的功能是有一定的缺点的。为了提高多方面的访问功能，我们不得不调整支持的功能类型。简单来说，在很多情况下，就像我们刚才讨论的，单个的FSAPI命令可以通过很多方式轻松访问。另外，还有一些指定的的功能是专门为一些特别的访问函数设计的。例如，如果我们开发一个FSAPI 功能，这个功能支持HTML界面，可以通过浏览器访问，我们就不希望通过系统的CLI命令或者其他的变量访问FSAPI功能模块。同样的道理，如果我们计划开发的FSAPI模块希望通过拨号规则或者呼叫记录来实现，那么对于CLI或者浏览器的方法就不是很有用。因此，强大的功能产生强大的实现可能性，这是一个简单的情况，我们需要利用我们的常识来判断什么时候，什么地方使用恰当的FSAPI 功能。

XML注册

我们已经讨论了许多FreeSWITCH的基础模块和组件，了解了这些模块之间是如何互动的。我们看到了事件系统如何通过core 来传递信息，XML拨号规则如何查询XML 注册信息获得数据。这是一个恰当的时间来解释一点关于XML注册。XML registry是一个可管理的XML文件，这个文件控制FreeSWITCH所需的所有关键数据，确保正确工作。初始文件从硬盘加载，然后传递到一个特别的予处理器。这个予处理器可以支持其他的XML文件和其他的指定的运行操作，例如设置全局变量，这些变量可以设置予处理器设置变量来解决。

一旦整个文件和其他的包含文件解析，替换，然后生成一个静态XML文件，这个文件将被加载到系统内存中。XML registry分为几个部分—configuration, odbc, dialplan, directory, locations, chatplan, languages, 和phrases. Core 和模块将从他们的配置文件中获得相应的配置选项。XML拨号规则模块将从拨号规则的部分获得拨号规则数据。SIP认证, 用户查找, 和voicemail模块将从directory 获得相应的配置数据。Phrase Macros将从phrases部分获得相应的数据。如果我们对硬盘中的文件做相应的修改，我们可以在CLI环境下，通过重新加载命令reloadxml把修改的数据重新加载到系统内存中。 (这是一个简单的使用FSAPI接口和FreeSWITCH core 通信的例子。)

语言模块

语言模块是FreeSWITCH模块中非常清晰的模块，不像文件或者终端模块，这个模块不支持直接和FreeSWITCH通信的接口，但是语言模块仍然提供一个非常强大的连接，支持目前的技术。 语言模块支持内置目前流行的几个主要程序语言，例如Lua, JavaScript, Perl, 甚至于 C# (使用 mod_managed)，可以实现core和程序runtime之间的功能转换. 这样的话， 一些应用功能就可以通过嵌入式程序来实现（例如IVR），通过简单的接口和FreeSWITCH进行交互，FreeSWITCH负责处理高负载工作。
通常语言模块是和FSAPI和其他的接口模块一起注册到Core，在拨号规则中执行。语言模块提供了很多机会开发非常强大的语言应用。使用语言模块在可以搭建强大的语音应用程序，支持标准的程序语言。在一些方面，因客户的要求，你可以通过其他的程序语言来控制你的电话系统。

演示配置文件

让用户迅速正确理解所有的概念不是一个简单的事情。作为一个软件维护者，我们不希望用户仅仅靠点击来解决所有问题。这是主要原因，当创建新的接口时，我们必须基于最顶层的core来做设计，尽量使其变得简单化，让用户非常容易的学习了解这些模块。 系统中的演示配置文件是介于新用户和那些复杂应用的一道最后的防线。我们尽量让用户轻松进入通信的学习旅程，让用户首先轻松了解FreeSWITCH。

FreeSWITCH 演示配置文件的主要目的就是演示如何配合系统的上百个参数工作。
我们为了防止用户使用一些工作的模块时漏掉一些必要的参数。用户可以把FreeSWITCH 看作是乐高玩具，可以进行各种不同的拼凑，通过不同的拼凑组合，我们可以组成不同的图案。
演示文件向一个太空飞船的样本包含了各种你需要的安装说明。它包含了各种一步步的安装指导，告诉你如何搭建你想实现的任何应用。你一旦有了经验，就可以修改你现在的玩具飞船，也许可以把一些模块加载到你自己设计的汽车上或者其他新的创意产品。幸运的是，FreeSWITCH本身就像一个集成的大盒子。如果你对一些模块不敢兴趣或者不想安装最新模块，你可以根据自己的喜好或者需求删除这些模块。如果将来需要这些模块，你可以重新安装删除的模块。

一旦成功安装以后，你可以无需修改配置文件的任何参数就可以启动你的FreeSWITCH。你可以使用通过电脑安装的SIP 软电话开始通话测试。当然，如果你有足够的好奇心，你可以连接传统的电话机。当然你必须购买硬件的语音卡或者ATA来实现你需要的功能。

如果你有多个电话机，系统默认支持了20个分机号码，范围是1000-1019, 直接使用这些号码，注册你的分机就可以实现通话。当然你也可以使用这些分机呼入到会议室。会议室的号码范围是3000-3399。如果你呼叫了一个没有注册的号码，或者呼叫超时的号码，系统将启动语音邮箱，提示你进行语音邮箱留言。如果你拨打号码 5000，你将进入一个IVR 演示系统。系统将引导你输入相应的菜单。

例如，使用我们刚才提到的预处理路径，演示文件将加载到最后的XML 配置文件。
这里，最重要的两点需要我们提及是用户帐号和拨号规则中的分机。20个分机帐号是系统默认配置的分机，都单独保存到了自己的配置文件。我们可以轻松添加一个新的文件和分机帐号，然后执行reloadxml 就实现了分机的添加。同样道理，我们可以轻松添加拨号规则例子。

总结

FreeSWITCH是一个非常复杂的系统，通过添加新的模块，可以开发出坚固，稳定的核心引擎，同时支持了灵活的，可拓展的第三方模块。核心模块通过接口支持了更多的模块。对用户来说，这些模块则通过模块的方式实现了简单化安装使用。这些模块将一些外部的通信接口需要引入了FreeSWITCH，使其成为一个标准的格式。我们查看了各种类型的模块， 演示了如何实现核心模块和其他模块的互通和如何实现抽象层到高级功能的互通。我们简单介绍了比较热门的几个模块，例如会议和语音邮箱功能，反过来，我们介绍了如何使用其他模块来调用这些模块，并且无需了解这些模块。这个“不可知黑盒子”是通过FreeSWITCH的事件系统来完成的。我们也看到了系统提供了几个演示文件，帮助用户了解系统的基本功能和相关的业务逻辑，以便实现一个功能完善的软交换系统。

现在我们基本具备了必要的知识-什么技术使得FreeSWITCH 开始运行。我们将更加深入地了解这些概念和实现一些现实生活中的功能。 首先，我们必须通过网络获得代码，然后我们才能编译软件包，安装这些软件。从那里开始，我们将测试中心配置文件，确认你至少安装了一个软电话。一旦我们完成这些基本的实验，我们将了解这些功能是如何实现的。所以，我们做一个深呼吸，做好准备，开始进入FreeSWITCH的通信世界！

在下一章节，我们将开始我们的第一步，搭建启动我们的FreeSWITCH，让我们的FreeSWITCH跑起来。

2

构建和安装

FreeSWITCH 是一套开源的软件。通常来说，任何人可以获得，阅读，编译或者修订这些源代码。对于许多用户来说，特别是新手认为和代码大交道是一个非常痛苦的事情，好在我们尽最大努力让这些工作变得尽轻松一些。未来，我们将添加二进制的软件包支持各种不同的linux发行版本，但是现在我们将解释如何通过手动的方式在Linux和windows 环境下安装FreeSWITCH。 这里的Unix 包括Unix的一些变种版本和Linux的一些变种版本，例如FreeBSD 和Mac OSX。 千万让这些安装的问题让你浮躁。通过你的耐心和小运气，安装过程可以变得相当的顺畅。当然也不完全像做一个可怕的拔牙的手术。不使用拔牙的钩子就把许多牙拔出来，那样只能是越来越可怕。这些都是我们听到的噩梦。

在这一章，我们将讨论如何从源代码在linux和Windows 环境下下载和安装FreeSWITCH。我们将涉及一些必要的不同操作系统的安装支持包。最后，我们将解释如何启动FreeSWITCH和如何在后台运行FreeSWITCH。

在这一章，我们将覆盖以下内容:

- 设置FreeSWITCH 运行环境
- 为 FreeSWITCH 安装做一些准备工作
- 下载安装FreeSWITCH
- 启动FreeSWITCH 和后天运行设置

设置FreeSWITCH环境

FreeSWITCH 像其他软件一样，需要一个合适的工作环境。基本来说，就是为你的系统选择合适的硬件系统和保证有一个成功的网络连接和网络环境。.

操作系统

第一个问题需要我们考虑的是: 我们需要什么样的操作系统?
通常来说，用户根据自己的使用习惯选择合适的系统，选择你自己熟悉的，使用比较舒服的操作系统就可以。另外，需要说明的是考虑使用32位的还是64位的系统。一些用户向我们报告在64位的硬件平台使用32位的操作系统。我们强烈建议如果你有64位的硬件，最好使用64位的操作系统。

如果用户愿意使用Windows 环境，可以使用 XP, Vista, Windows 7, Server 2003, Server 2008 R2, 或者Server 2012. 有一些用户报告说他们已经成功在Windows Server 2008环境下运行FreeSWITCH的生产系统。

另外，市场上有很多免费的各种Linux的操作系统。大部分人有 CentOS, Debian, Ubuntu等等开源的系统。我们建议使用这些开源的系统。FreeSWITCH 不鼓吹任何指定的操作系统或者发行版本。

一些用户询问我们哪个平台是支持FreeSWITCH最好的平台。当开发一个通信系统时，这里有几个因素需要考虑。FreeSWITCH是一个跨平台的软件，因此可以运行在很多的操作系统。通过血的教训和实际经验，我们了解什么样的操作系统可以支持实时的通信软件。当然，底线是这个系统是稳定的和具有可靠性的。FreeSWITCH 社区的很多用户使用 CentOS 5和Debain 6作为生产系统。



写到这里，很多用户关心FreeSWITCH在CentOS6 环境下的的性能。社区用户还没有报告在CentOS6.3 环境下有什么问题。如果你安装了 以前的版本，并且发现一些问题的话，请更新到 6.3 版本测试。

注意，非常前沿的发布版本不适合用于实时的通信系统。建议使用通用的，可控的系统。

操作系统的必要安装包

每个操作系统都有独特的安装支持包。确认你安装最新支持包。在以下部分，我们将讨论这些内容。

Linux/Unix

通常，以下这些软件包已经在系统安装时安装。注意不需要Git客户端:

- Git: A Git client also gives you access to the current code repository (recommended especially for developers and those who want the latest code)
- GNUMAKE: The GNU version of Make
- AUTOCONF: Version 2.60 or higher
- AUTOMAKE: Version 1.9 or higher
- LIBTOOL: Version 1.5.14 or higher
- GCC: Version 3.3 or higher
- WGET: Any recent version
- LIBNCURSES: Any recent version
- BZIP2: Any recent version

Mac OS X

我们强烈建议 Mac 用户必须安装最新的OS 版本，例如 10.4. 在 Mac 环境下编译FreeSWITCH 需要安装 Apple XCode 开发工具。你可以从这里下载 http://connect.apple.com. 要求免费注册。



Apple 已经对开发工具做了修改. FreeSWITCH 社区尽最大努力更新一些关于 OS X的消息。访问http://wiki.freeswitch.org/wiki/Installation_and_Setup_on_OS_X. 很多最新消息。

Windows

在Windows 环境下安装FreeSWITCH需要满足两个基本要求. 他们是:

1. Microsoft Visual C++ 2008 or 2010 (or 2008 or 2010 Express Edition).
2. 解压工具.

在Windows 环境编译FreeSWITCH， 必须安装Microsoft Visual C++ (MSVC) 或者Visual C++ Express Edition (MSVCEE). Express Edition 版本可以免费下载，不过需要注册才能下载。访问： http://www.microsoft.com/Express/VC. 另外，还要限制压缩工具WinZip (www.winzip.com), 或者 (www.rarlab.com). 另外一个可选的免费工具是 7-Zip (www.7-zip.org).



Visual C++ 的 Express Editions 默认不支持 64位 压缩文件。如果要编译64位的FreeSWITCH版本，建议你使用 Visual Studio 的Professional Editions 版本。

文本编辑工具和XML

在FreeSWITCH 环境下工作，你需要安装一个自己感觉好用的文本编辑工具。不管你选择什么样的编辑工具，确定此工具可以支持XML语法高亮，这样方便阅读。

如果你已经有自己喜欢的编辑工具，我们建议使用其中一个比较工具。另外，你现在使用的linux平台没有带GUI，你的选择的工具还是比较有限。幸运的是，在linux 环境下，还是有几个不错的编辑工具:

- Emacs: 很多Linux 环境支持的编辑工具。支持XML，HTML等等代码高亮显示。FreeSWITCH开发团队也使用这个编辑器作为开发工具。
  Vi/Vim: 比较受欢迎的文本编辑工具。同样支持代码高亮显示。
  Notepad++: 带界面的Windows文本工具。
-
-

- Microsoft Visual Studio/Visual C++ Express: 微软的IDE 工具，当然同样支持XML语法。/

下载代码

大部分的开源项目把代码分成两个下载目录：稳定版本和最新版本。 FreeSWITCH
最近重新命名了两个分支。Version 1.2.x 是稳定版分支，Version 1.3.x 是最新版本分支。
未来的版本将按照奇数和偶数的命名方法，例如 1.4.x 是稳定版本，1.5.x 是未稳定版本或者
开发分支版本。你可以使用Git 随时更新到最新的分支版本。另外，请注意，二进制的发布版本
不一定支持你的平台。当然，根据我们的经验，最容易排查问题，更新或者定制的办法还是通过
代码安装。.

确认你的机器可以连接外部网络，有时候需要从FreeSWITCH官方网站下载其他的代码。

可以从FreeSWITCH 官方网站获得源代码：:

http://files.freeswitch.org

找到文件名freeswitch-1.2.x.tar.gz (x 代表相应的最新版本数),
下载到本地机器，然后解压这个文件，linux 的下载方式是这样的:

```
#>cd /usr/src
#>wget http://files.freeswitch.org/freeswitch-1.2.1.tar.bz2
#>tar jxvf freeswitch-1.2.1.tar.bz2
```

以上命令将解压文件，并且创建一个FreeSWITCH 文件路径，包含所有你下载的代码。后续的举例中，
我们将按照这个文件路径来讲解。

Windows 用户需要创建一个新的路径来安装源代码。参考本章后续部分-在Windows平台编译的内容。

从最新代码配置

如果你希望通过最新代码安装的FreeSWITCH，你需要安装 一个Git 客户端。通过yum，apt 或者其他的软件包管理工具来安装Git. 在Windows 环境下，可以使用 TortoiseGit.

在linux 环境下，典型的安装步骤应该是这样的:

```
#>cd /usr/src
#>git clone git://git.freeswitch.org/freeswitch.git
#>cd freeswitch
#>./bootstrap.sh
#>./configure -C
#>make install
#>make cd-sounds-install
#>make cd-moh-install
```

以上步骤需要话一点时间完成。你可以通过命令带&& 来一次性自动完成所有的命令。在稍后的内容中我们将讨论这些。

在Linux/Unix/Mac OS X平台编译FreeSWITCH

在Linux, Unix, 或者 Mac OS X安装的流程是一样的，但是必须确认你已经成功安装了所有的支持包。

编译 FreeSWITCH

编译FreeSWITCH 需要经过步骤，这个过程需要一定的时间，安装的速度快慢取决于你的系统速度和下载的网速。基本的安装流程如下:

1. 运行 the bootstrap.sh 脚本。
2. 编辑模块文件，添加定制的模块。大部分已经默认添加。.
3. 运行configure 脚本。
4. 运行 make 和 make install 编译安装。
5. 编辑modules.conf.xml 使得系统启动时加载指定的模块。
6. 安装系统语音和音乐文件。

以下是详细的FreeSWITCH编译安装指导.

第一步 – 编辑 modules.conf文件

modules.conf 文件包含了各种FreeSWITCH支持编译的的模块列表。默认环境下，这些模块已经被包含，编译时会自动编译这些列表中的模块。但是，还有一些模块需要我们在编译前开启，否则不能被编译。你现在应该可以看到一个新的文件路径，名称为 freeswitch 1.2.x，这里，1.2.x 代表版本. 例如，如果是最新的稳定版本是 1.2.1 ，你可以进入这个文件目录下，然后执行以下步骤:

1. 进入到FreeSWITCH源代码目录:

   ```
   #>cd /usr/src/freeswitch-1.2.x
   ```

2. 使用文本编辑器打开文件modules.conf，找到这一行:

   ```
   #asr_tts/mod_flite
   ```

3. 删除这一行前面的#号字符，然后保存文件，退出。
   mod_flite模块启用了FreeSWITCH，调用开源的语音合成Festival Lite text-to-speech (TTS) 引擎. (这个模块不能支持高质量的语音合成，但是方便我们测试语音合成测试。



更多信息，访问
http://www.speech.cs.cmu.edu/flite/.

编辑文件以后，我们准备开始真正的编译过程.



删除 # 字符，系统将编译此模块。添加此字符，系统将不编译此模块。

第二步 – 运行配置脚本

像其他开源软件一样，FreeSWITCH 支持linux 环境下的脚本配置。在FreeSWITCH 源代码路径下，执行以下步骤:

```
#>./configure -C
```

configure 脚本执行很多任务，包括确认你的支持包是否安装。如果缺少支持包，脚本将退出，提示你哪一个依赖包没有安装。如果发生这样的情况，你必须解决这个问题，然后进行执行configure脚本，直到完成所有的步骤。 -C 参数将通知 configure模块创建一个新的缓存文件 config文件. 继续执行的configure将所有这个文件来，并且包含各种支持包的source tree。



configure 脚本是标准的开源软件的配置工具。它支持很多选项来执行不同的命令参数。执行help 命令查询完整的支持选项。

在安装过程中，你可以看到脚本会执行多次。FreeSWITCH 使用了许多的支持包，例如Apache Portable Runtime(APR) 和Perl Compatible Regular Expressions (PCRE). 每个小的部分包含了自己的配置脚本。

通过一定时间的运行，configure 完成以后，返回一个系统提示符。屏幕上会显示很多输出的结果，并且没有任何错误。只有这样，你才可以继续进行下一步的安装流程。

第三步 – 执行 make 和 make install 安装命令

以前的价格步骤实际上是为freeSWITCH，软件包和各种模块创建一个Makefile。通过make 工具绑定编译和安装FreeSWITCH的过程。首先，执行make 命令，然后执行make install 命令。许多用户也使用组合的形式来完成这些命令：

```
#>make && make install
```

像 configure 脚本一样，make 过程也需要一定的时间来完成。如果有错误的话，它将自动停止。 通常情况下是没有问题的。成功执行以后，屏幕出现以下信息:

```
+-------- FreeSWITCH install Complete --------+
+ FreeSWITCH has been successfully installed.                    +
+                                                               +
+          Install sounds:                                      +
+          (uhd-sounds includes hd-sounds, sounds) +
+          (hd-sounds includes sounds)                          +
```

```
+                 ----------------------                 +
+                         make cd-sounds-install         +
+                         make cd-moh-install            +
+                                                        +
+                         make uhd-sounds-install        +
+                         make uhd-moh-install           +
+                                                        +
+                         make hd-sounds-install         +
+                         make hd-moh-install            +
+                                                        +
+                         make sounds-install            +
+                         make moh-install               +
+                                                        +
+                 Install non english sounds:            +
+                 replace XX with language               +
+                 (ru: Russian)                          +
+                 ----------------------                 +
+                         make cd-sounds-XX-install      +
+                         make uhd-sounds-XX-install     +
+                         make hd-sounds-XX-install      +
+                         make sounds-XX-install         +
+                                                        +
+                 Upgrade to latest:                     +
+                 ----------------------                 +
+                         make current                   +
+                                                        +
+                 Rebuild all:                           +
+                 ----------------------                 +
+                         make sure                      +
+                                                        +
+                 Install/Re-install default config:     +
+                 ----------------------                 +
+                         make samples                   +
+                                                        +
+                                                        +
+                 Additional resources:                  +
```

```
+          ----------------------                    +
+          http://www.freeswitch.org                 +
+          http://wiki.freeswitch.org                +
+          http://jira.freeswitch.org                +
+          http://lists.freeswitch.org               +
+                                                    +
+          irc.freenode.net / #freeswitch            +
+                                                    +
+--------------------------------------+
```

如果你看到的信息就像以上那样的，说明你已经成功安装了FreeSWITCH，你可以执行下一步的命令。如果出现错误的话，以上编译过程将会停止。你需要解决这些出现的问题。如果你不熟悉那些出现的错误，请访问技术支持的邮件列表获得帮助。所有的免费技术支持的资源在附录A中列出。

第四步 – 编辑modules.conf.xml文件

modules.conf.xml 文件包含了所有FreeSWITCH启动时需要加载的模块。默认的 modules.conf.xml 文件对应默认的 modules.conf 文件。在默认模块配置文件 modules.conf
配置的文件在默认的modules.conf.xml启用. 我们启用了mod_flite 模块，同样也必须在modules.conf模块中加载这些模块，当FreeSWITCH启动时，mod_flite 模块就会被自动加载。这里的经验法则是，如果想让一些指定的模块自动加载，你必须在 modules.conf.xml中启用这些模块.

modules.conf.xml 文件保存在 conf/autoload_configs 子目录. 默认的文件路径是 /usr/local/freeswitch/conf/autoload_configs/modules.conf.xml. 使用编辑器打开这个文件，找到以下这一行:

```
<!-- <load module="mod_flite"/> -->
```

删除 <!-- and -->标签，变成这样:

```
<load module="mod_flite"/>
```

保存退出文件。你现在已经准备好了，可以确定FreeSWITCH。



modules.conf 和modules.conf.xml files有什么不同?
modules.conf 文件是保存在源代码路径的文件，它原来管理
任何需要被即将编译的模块。它是一个标准的文本文件，每一行以
#开头来表示一个注释。modules.conf.xml文件是XML样本配置文件，安装在
FreeSWITCH autoload_configs 的子路径。它是一个标准的XML文件，使用
<!-- and --> 来表示一个注释。当FreeSWITCH启动时，它负责控制需要加载的
模块。

第五步 – 安装语音文件和音乐文件

语音文件和音乐文件对FreeSWITCH来说不是必要的条件。但是，这些文件是我们推荐安装的。
没有这些文件，你的系统没有音乐等待，和语音邮箱等类似的功能，并且系统的IVR的例子将不能
工作。FreeSWITCH配备了四种不同采样率的语音文件和音乐文件。 我们建议安装这些文件，充分
利用这些文件的高品质语音质量获得更好的体验。

安装声音文件只需要执行以下命令:

```
#>make cd-sounds-install
```

安装音乐文件，只需要执行以下命令:

```
#>make cd-moh-install
```

以上命令将下载安装所有的语音文件和音乐文件，支持分别支持这四种格式 8 kHz, 16 kHz, 32 kHz, 和
48 kHz 采样率. 当FreeSWITCH播放语音文件或者音乐文件时，将调用其中一种文件格式。

现在，你已经准备好了，可以启动FreeSWITCH. 下一个部分的内容是解释如何在Windows 环境
下安装FreeSWITCH, 可以跳到启动FreeSWITCH 部分。

在Windows环境下编译FreeSWITCH

刚才我们已经提到，在Windows 环境下需要安装必要的支持包MSVC 或者MSVCEE.
这里我们使用的是MSVCEE 2010; 但是所有的 MSVC 版本功能应该是一样的。

Windows 用户需要注意的地方

除非你是一个开发人员，如果一般用户，我们建议你安装二进制的FreeSWITCH 二进制包，这样的安装方式相对简单，而且完全可以满足你的需求。直接从http://files.freeswitch.org/windows/installer/ 下载 freeswitch.msi X86的安装包。 这是最简单的安装办法。更多信息可以访问 http://wiki.freeswitch.org/wiki/Installation_for_Windows#Precompiled_Binaries.

Microsoft's Visual Studio 2010支持了更多的功能，我们建议用户应该使用此开发环境，不要使用 Visual Studio 2008. 请留意，这个提醒也同样适用于Express Editions. 从 FreeSWITCH 1.0.6 以后添加了一些非常有趣的模块，这些模块可能不支持Visual Studio 2008 工程文件, 因为很多开发人员主要利用Visual Studio 2010来开发.

到写作此书时为止，我们不建议用户从Visual Studio 2010 环境导入到 Visual Studio 2012 编译环境，因为FreeSWITCH 当前还没有正确地在2012 环境下安装。

通过MSVC/MSVCEE安装FreeSWITCH

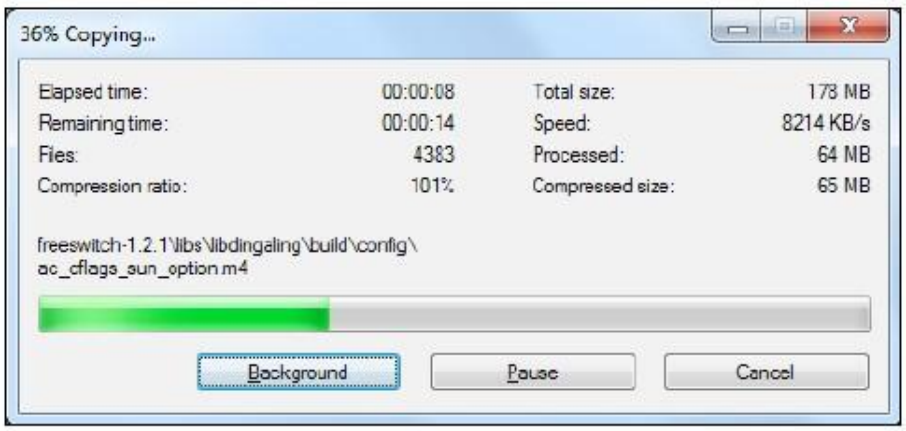
这里有几个步骤通过MSVCEE来搭建你自己的FreeSWITCH。 这些步骤是:

1. 创建一个新文件夹，拷贝这些压缩文件到从文件夹，这里我们使用:

   C:\FreeSWITCH\freeswitch-1.2.1.tar.bz2

2. 使用鼠标右点压缩文件freeswitch-1.2.1.tar.bz2 ， 解压文件到此路径. 你将获得一个文件freeswitch-1.2.1.tar.

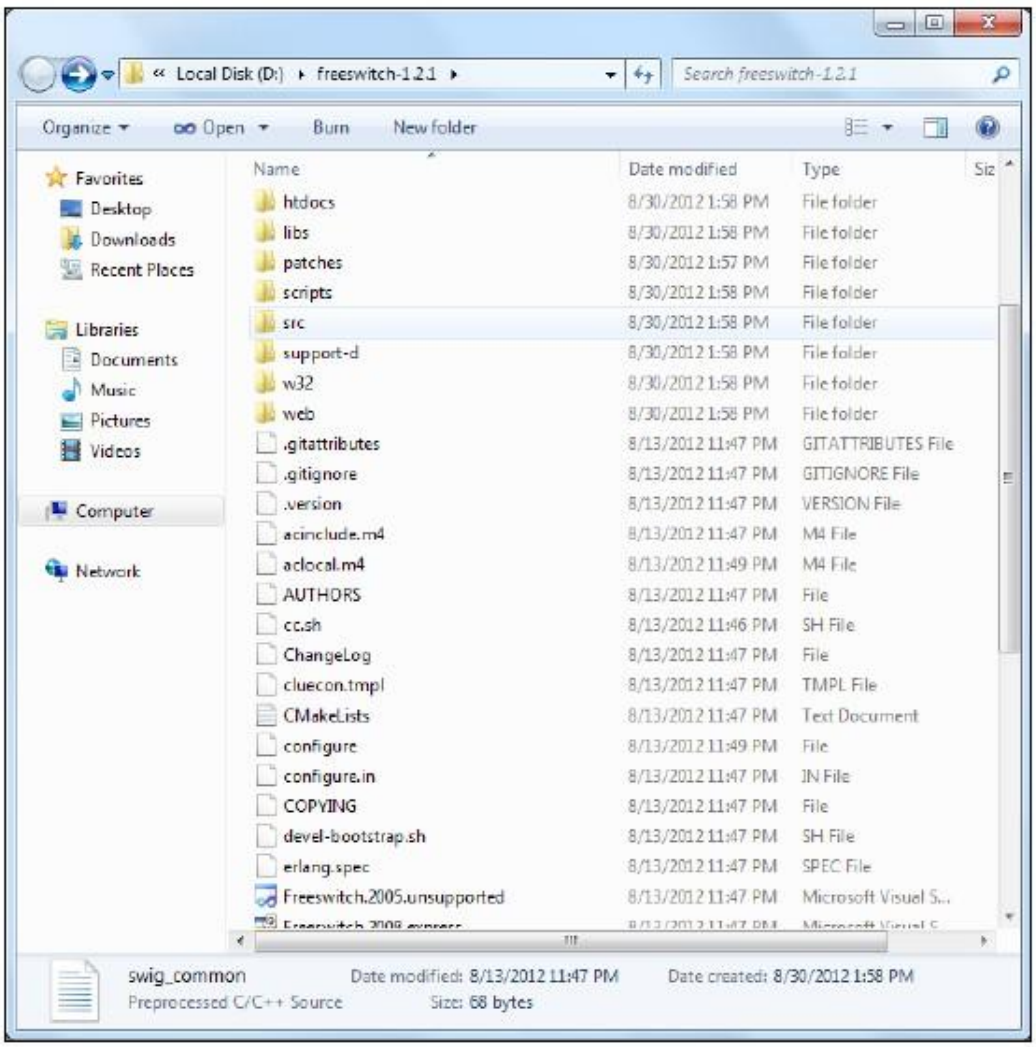
3. 右点文件on freeswitch-1.2.1.tar ， 解压所有的文件. 此步骤将需要一点时间，这是使用7-zip的截图:





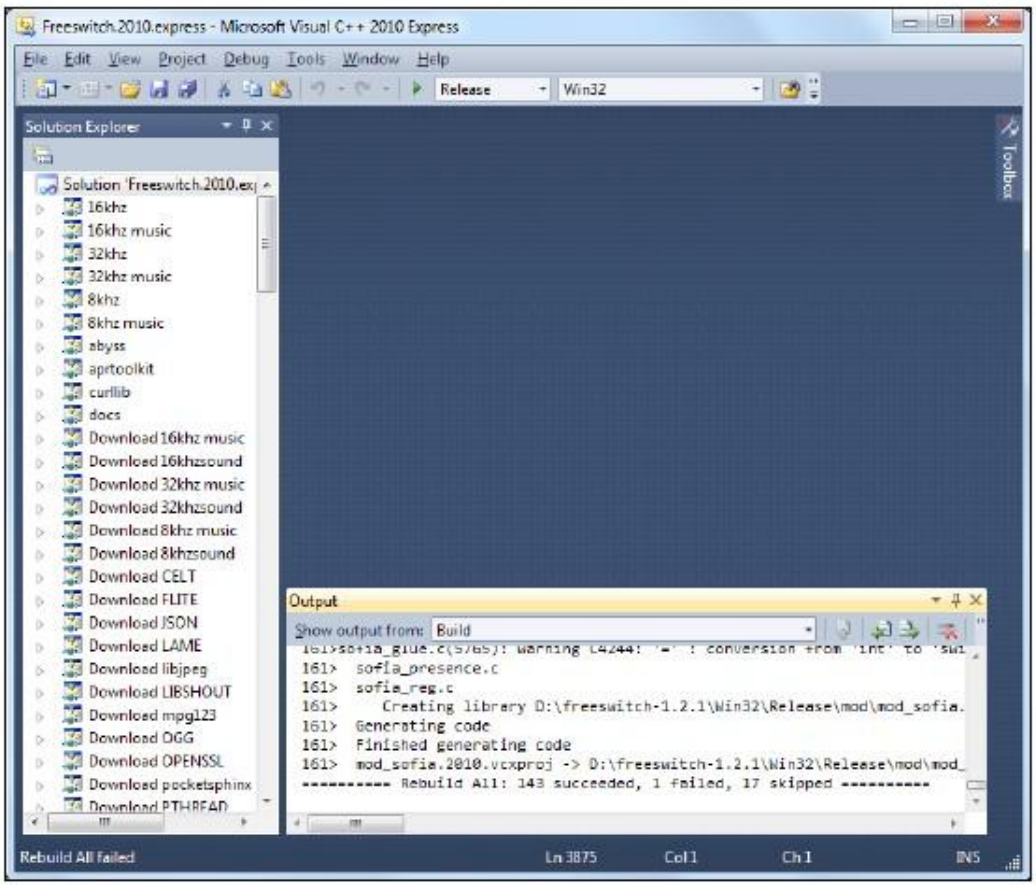
WinRAR 解压 .gz 和 .tar 文件仅一步。

4. 解压以后，系统将生成一个新的子文件夹，文件夹名称是FreeSWITCH的最新版本号。在我们的例子中，我们有一个子文件夹是 freeswitch-1.2.1. 双击文件，可以看到所有的代码树内容，如截图:



5. 这里有很多文件，我们只关心和MSVC相关的文件Freeswitch.2010.sln, 和MSVCEE修改的Freeswitch.2010.express.sln. 双击相应的文件版本。出现如下截图，这里的举例是MS Visual C++ Express Edition. 但是Professional 和 Ultimate editions 也是类似的显示内容.

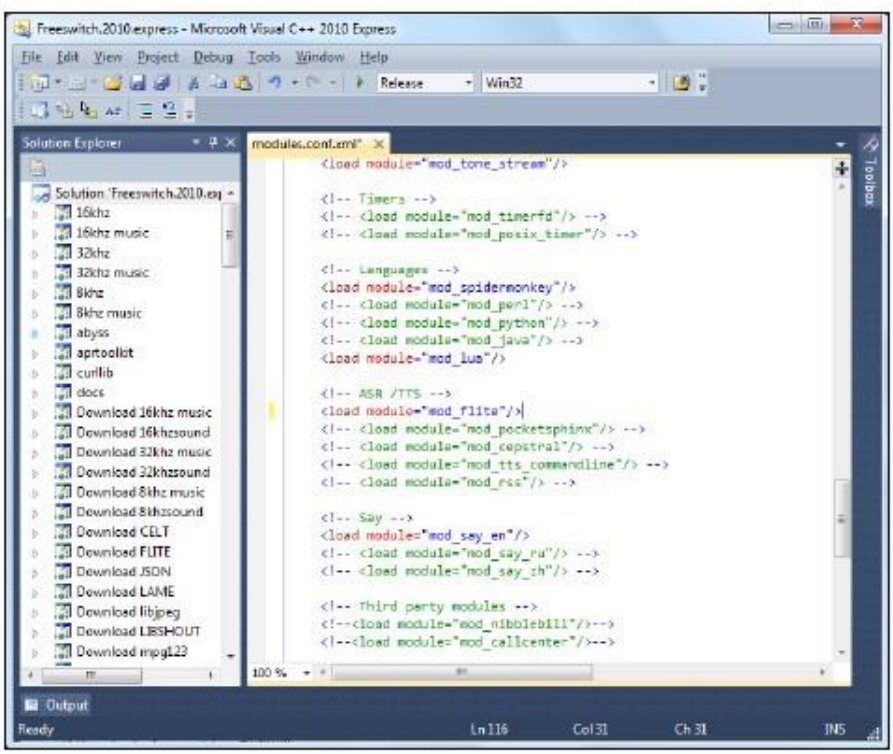
6. 所有的文件加载以后，点击下拉菜单，从 Debug 修改到Release, 然后点击Build | Build Solution 或者 摁 F7. 如果你使用Visual Studio 2010 IDE, 可以去 Build | Build Solution 或者输入Ctrl + Shift + B 键. 这个工程就开始构建. 窗口将出现很多信息。如果成功构建以后，界面将显示如下截图:





MSVC/MSVCEE 文件自动执行几个步骤，不像linux 环境需要手动执行。这些步骤包括下载声音文件和语音等待的文件，和安装语音合成模块和语音识别模块 但是如果你希望FreeSWITCH 自动加载最新模块，你需要编辑模块配置文件 modules.conf.xml。 更多信息访问 http://cmusphinx.sourceforge.net/wiki/start.
在构建过程中，你将看到创建了一个新文件，名称为Release. 这是一个FreeSWITCH安装目录. 启动之前的最后一步就是启用mod_flite ，编辑配置文件，确保启用这个模块，我们将在后续的章节中使用这个模块 mod_flite text-to-speech (TTS)引擎。

7. 双击conf 文件夹, 双击autoload_configs 文件夹，打开文件 modules.conf.xml ，例如如下截图，我们将使用MSVCEE 来编辑这个文件:



8. 找到这一行:

   <!-- <load module="mod_flite"/> -->

   删除 <!-- and --> 标签，例如以下格式:

   <load module="mod_flite"/>

9. 保存退出文件。现在FreeSWITCH 安装已经大功告成。开始第一次启动。

启动FreeSWITCH

一旦完成编译和安装过程，你可以启动FreeSWITCH:

- Linux/Unix/OS X: 运行 /usr/local/freeswitch/bin/freeswitch
- Windows: 运行Release路径下的freeswitchconsole.exe 文件

执行以上命令，FreeSWITCH将会启动，启动过程中，出现很多带颜色的代码段。
成功启动以后，你可以看到一个FreeSWITCH后台窗口，通常称作
CLI 或者命令行界面，类似以下截图:

freeswitch@localhost>

让我们输入几个命令，确认系统是可以工作的。首先输入 version 命令，系统将显示FreeSWITCH的安装版本.
你可以看到类似的结果:

FreeSWITCH Version 1.2.1

然后在输入 status 命令，显示系统的统计结果和基本的运行状态。你可以看到这样的输出:

freeswitch@localhost> status

UP 0 years, 0 days, 0 hours, 0 minutes, 16 seconds, 808 milliseconds, 260 microseconds

FreeSWITCH is ready

0 session(s) since startup

0 session(s) 0/30

1000 session(s) max

min idle cpu 0.00/100.00

Current Stack Size/Max 240K/8192K

FreeSWITCH 支持很多命令，可以通过简单命令检查这些内容。简单输入 help然后摁 Enter键.
最后，如果执行关机命令的话，可以执行: fsctl shutdown. 当系统关机时，将显示很多信息。
(如果启动的是 freeswitchconsole.exe ，Windows 系统将自动关闭.)。

在后台运行FreeSWITCH

大多数情况下，你想让FreeSWITCH在后台运行。在linux 环境下，通常称作为daemon. 在 Windows 环境下，称之为运行一个服务。

如果作为一个linux FreeSWITCH 作为一个 daemon服务来运行，执行以下命令:

```
#>/usr/local/freeswitch/bin/freeswitch –nc
```

各种linux 发布版本都支持类似的运行模式。在FreeSWITCH wiki 我们提供了几个默认启动的脚本文件举例， 可以访问这个链接: wiki.freeswitch.org/wiki/Freeswitch_init. 查阅你使用的相应的linux 版本文档，配置如何支持系统默认启动。

Windows 环境下，需要执行机构步骤把 FreeSWITCH 设置为一个服务。他们设置为以下几步:

1. 打开 Windows 命令行 (点击 Start | Run, 输入 cmd, 然后点击OK button).
2. 修改到FreeSWITCH 安装路径, 例如:

   ```
   cd FreeSWITCH\freeswitch-1.2.1\Release
   ```

3. 运行freeswitchconsole.exe 必须加参数 –install , 如下方式:

   ```
   freeswitchconsole –install FreeSWITCH
   ```

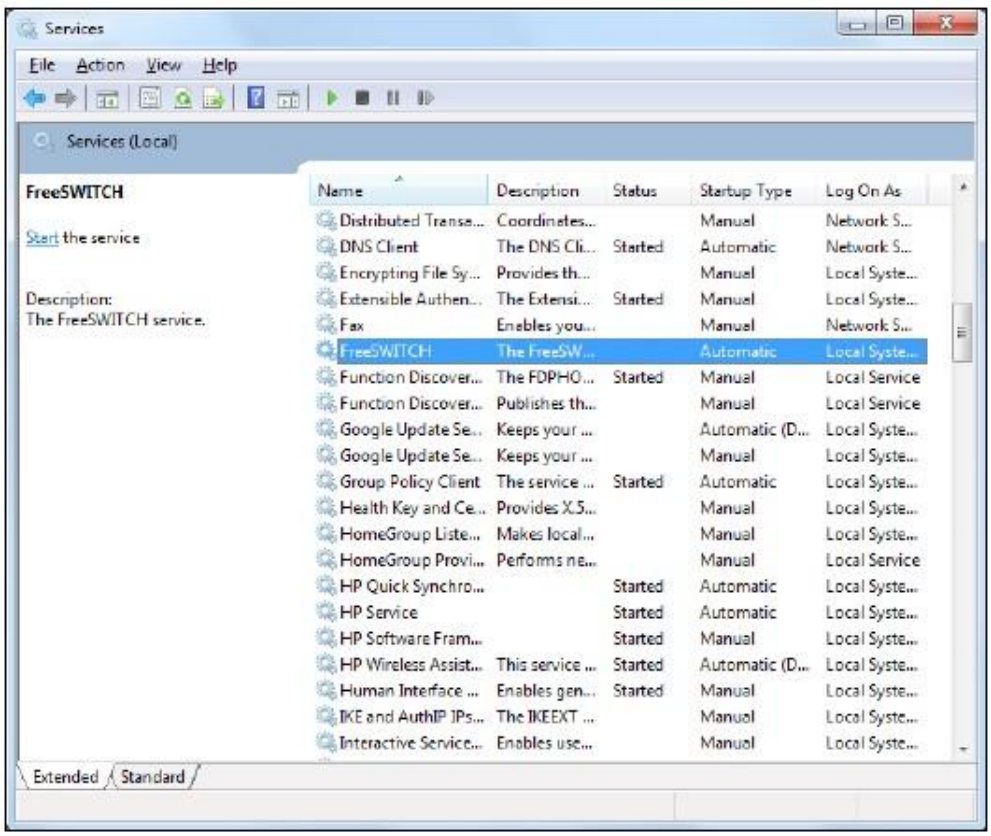
4. 最后一步，配置服务设置.

   如果你正在运行Windows XP 或者Server 2003, 打开 services tool 点击 Start | Control Panel | Administrative Tools | Services.

   否则，如果你正在运行旧一点的版本，可能需要查找 Services 工具菜单，然后点击配置。

   另外一种办法使用 MMC 界面操作输入

Windows Key + R 然后输入services.msc, 点击OK.
FreeSWITCH 将设置成为一个系统的服务:



5. 右点FreeSWITCH，然后点击Start. 系统将花费一点时间启动这个服务.

6. 在Release 文件夹下执行fs_cli.exe 工具集，确认系统启动.

7. 你将看到一个欢迎界面和一个命令提示符. 输入 status 命令将显示系统结果，说明了系统已经运行。

8. 输入 /exit 退出 fs_cli.exe 程序.

现在你的FreeSWITCH 作为一个系统的服务正在运行了。

fs_cli 工具集将在第十章进行详细的讨论-通过外部程序控制 FreeSWITCH。

总结

在这一章，我们完成了几个主要的讨论话题，他们是:

- 下载安装FreeSWITCH
- 修改modules.conf 编译mod_flite TTS 模块 (Linux/Unix/Mac OS X only)
- 通过修改 modules.conf.xml来自定义FreeSWITCH的配置文件, 当FreeSWITCH启动时自动加载 mod_flite 模块。
- 启动FreeSWITCH 和执行几个简单命令来确认系统的工作状态
- 如何设置FreeSWITCH为daemon (Linux/Unix) 或者为一个服务 (Windows)

在下一个章节，我们将把这个安装的环境投入到实际的FreeSWITCH 演示环境。

3
测试强大的实例配置

现在你已经安装好了FreeeSWITCH，是开始了解更多本书举例配置的时刻了（例如书中的举例）。书中的实例是按照用户配置文件提前配置好的，包括了拨号规则，安全设置和更多内容。实例的目的是让用户能够尽可能多地掌握第一手的FreeSWITCH配置经验，了解FreeSWITCH可以做实现什么。

在本章节中，我们将介绍以下内容:

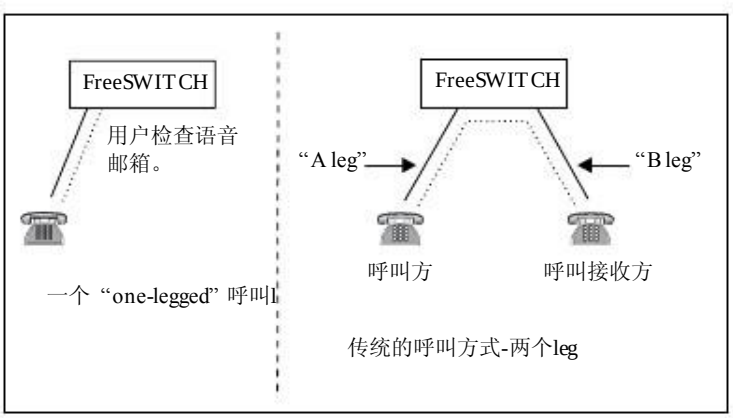
- VoIP和 FreeSWITCH的重要概念
- 使用FreeSWITCH 命令接口 (fs_cli)
- 配置一部话机，测试FreeSWITCH
- 呼叫系统中不同的分机

需要了解的重要概念

FreeSWITCH 是一款功能丰富的软件。一个最主要的原因是，它强大是因为通信世界是动态的.作为一个软件开发人员，当我们做一个决定的时候，我们会经常碰到类似艰难的决定，FreeSWITCH应该如何实现各种需求和复杂灵活的环境。我们常常会碰到一个难题，一些潜在的用户经常会提一些特别的需求，但是相反的，其他用户有希望另外一种相反的需求。我们可以轻松地添加一些设备功能，并且确保设备正常工作，但是同时我们必须调整一些需求，适当灵活地支持非常特别的设备功能。FreeSWITCH设计的目的是就是支持扩展，所以我们也需要设计一些特别的地方，用户可以从这个地方开始进行独立静态配置，并且可以拓展出动态的配置，并且不会可以跟上开发的节奏。 这是比较痛苦的，但是它也不是一个非常烦躁的事情。当你从上一个章节安装FreeSWITCH时，你已经安装了一个功能完整的实例配置文件，这些配置文件将贯穿本书的大部分章节，只需修改小部分就可以工作。

就像我们在第一章讨论FreeSWITCH架构，FreeSWITCH是基于一个核心内核，借助了XML 注册模块，所有的应用接口模块围绕中心模块来运行。我们使用几个默认注册的用户来做一些简单的呼叫测试。当你发起一个呼叫时，SIP模块将推送一个请求到XML 拨号规则，拨号规则通过正式表达式做一些逻辑匹配，然后发送到相应的接口。一旦找到匹配条件，XML 分机数据将被拷贝到本地的通道，然后按照一个指令表来执行下一个呼叫动作。可能在拨号规则中同时匹配了几个分机，这依赖于配置关键词的选择。对于第一次测试我们使用一个分机做呼叫测试，当呼叫的通道进入路由状态时，你可以看到相关的呼叫数据（例如，通道状态等等，参考第八章 高级路由概念）。

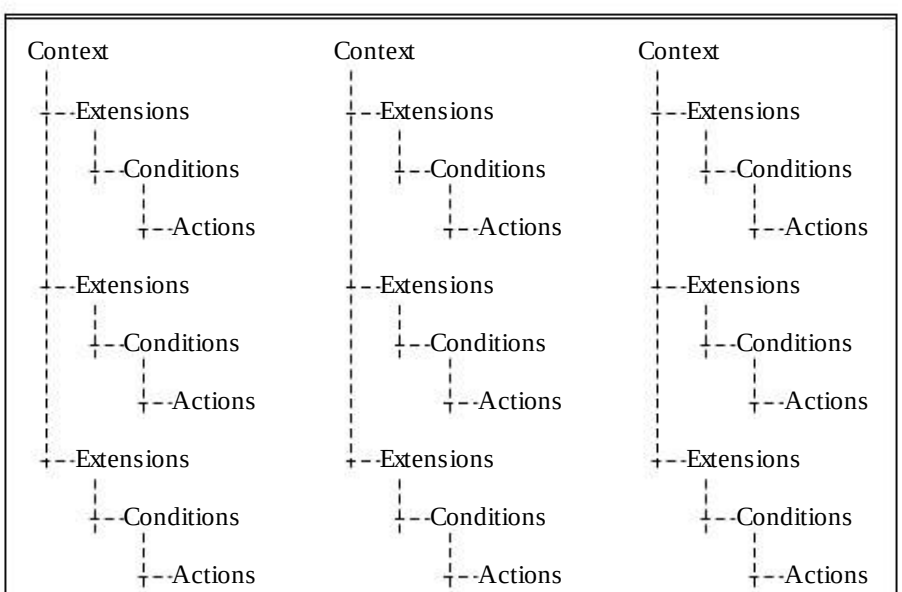
在通信的专有名词中，我们把两个设备之间的连接称之为一个呼叫leg。A leg 用来描述呼叫方和FreeSWITCH的通信路径。B leg 用来描述接听方和FreeSWITCH的通信路径。
让我们看看以下的图例解释:



如果你使用一个分机电话发起呼叫，听一个系统的一个demo，那么这时只有一个leg，只有你的分机电话机和FreeSWITCH之间通信。如果你拨打一个号码注册在FreeSWITCH平台的号码，或者通过中继呼出到运营商那里，运营商接通你的手机，你们这里，系统有两个leg。第一个就是A leg，你的电话呼叫到FreeSWITCH，另外一个就是B leg，这个leg 就是通过FreeSWITCH 连接另外一个电话或者运营商服务设备。在每个呼叫中，每个leg 都有自己独有的属性并且通过特别的关系和对应的leg 关联。当呼叫中的legs 进行媒体交互时，我们称之为一个bridge 或者桥接. 在桥接的呼叫中，任何一方都可以对另外一方进行某些操作或者控制，例如可以将对方置于语音等待，转接到另外的分机，或者加入到一个三方通话中。

有一些呼叫只有一个leg，那就是一个电话到FreeSWITCH连接，FreeSWITCH直接和呼叫方交互。常见的有IVR 语音交互或者IVR 语音菜单。另外一个 单leg的呼叫就是语音邮箱或者会议室。IVR 的功能非常强大，你肯定使用了IVR来引导用户按照语音指定进入客户需要的服务类别或者服务流程。如果你使用过电话卡的话，你了解这个过程。那也是IVR的一种形式，要求你输入帐号，密码和你需要拨打的目的地号码，通过这个流程来完成电话卡的整个呼叫流程。一些IVR可以通过语音识别来和呼叫方进行互动，例如你可以说当前的时间，系统转换成你的语音成为一个文字形式的时间。在FreeSWITCH呼叫下，开发一个IVR系统是非常简单的工作，我们将通过几张方法来实现语音IVR，这些方法将在第六章的使用XML IVRs 和Phrase Macros和第七章 使用Lua 脚本实现拨号规则。

XML Dialplan 把 extensions 划分为几个特别的contexts组. 在XML 标签中，一个context包含几个extensions元素。一个extension是一个拨号模式的集合，匹配已拨号码，并且通过匹配结果不同来来执行不同的命令。参考以下列表:



XML拨号规则继承列表

每个新进入FreeSWITCH core 的呼叫必须含有预设的属性，包括 context, Dialplan, 和 extension来指示这个呼叫应该走什么路由。在我们的举例中，我们将使用XML 拨号规则和 默认的default Dialplan. extension 号码当你发起呼叫时，你拨打的号码。一旦你拨打了一个extension 号码，SIP Endpoint 模块将把从sip 话机解析出来的数据插入到这个呼叫中，然后在XML中设置， 设置 context为默认值，最后把这个呼叫推送ROUTING路由. core中的默认ROUTING将查找XML dialplan 模块，然后推送这个呼叫进入到呼叫查询处理路由接口。路由接口连接到XML 注册，并且查询context 列表。一旦查询到 context，然后解析context的每个extension，对condition的标签进行检测，直到找到匹配的条件。 在condition 标签中的action 并且都包含一个application 和可选的传递参数。这些applications 是通过 Application 模块提供。Application 模块我们在第一章的FreeSWITCH 架构中已经有所介绍。 这些应用模块按照顺序执行，直到执行完成最后一个模块或者呼叫结束。.

应用模块中传递的参数可以支持通道变量，特定的一对值name/value 来决定通道的行为，并且 提供一种存储呼叫数据的方法。他们看起来和我们最近讨论的特别的预处理变量非常相似，仅仅使用一个 单美元符号，而不是双符号。例如，${destination_number} 告诉你说拨打的号码，此号码作为基本的路由号码。 condition 标签将通过此值来匹配设定的号码。如果在呼叫方的属性列表中包含一个指定的号码，为了操作 简单，你可以忽略 ${}。

以下是一个指定的呼叫属性文件，其中一些变量看起来不是相当常见，但是你可能使用其他比较常用的 一些变量:

- username
- dialplan
- caller_id_name
- caller_id_number
- callee_id_name
- callee_id_number
- network_addr
- ani
- aniii
- rdnis
- destination_number
- source
- uuid
- context

呼叫属性文件是每个呼叫的基本信息集合，在呼叫过程中从一个leg 发送到另外一个leg。 这些信息可以通过类似于获得变量的方法来获得，这些信息应该是只读信息，因为这些数据是呼叫创建时的数据。以下是一个实际的系统默认的举例，通过core的语音工具来播放一首80年代比较流行的游戏主题曲, Tetris:

```
<extension name="tone_stream">
  <condition field="destination_number"
    expression="^9198$">
    <action application="answer"/>
    <action application="playback"
      data="tone_stream://path=${base_dir}
      /conf/tetris.ttml;loops=10"/>
  </condition>
</extension>
```

如你看到的，系统使用了field="destination_number" 来检查你是否拨打了 9198,
如果你拨打了9198，系统应答这个呼叫，然后播放语音流。它使用 ${base_dir}
来代表配置文件的存放路径，同时可以使用比较短的系统路径。 (无需担心，这里看起来有一些简短.
我们将做更多的介绍.) 你已经有一个可以进行呼叫测试的FreeSWITCH 实战环境，并且包括了
实战举例的样本文件和配置举例。通道变量是非常有用的，在你的应用软件中，如果需要对呼叫进行控制或者你想设置或获得通道数据，都可以通过通道变量获得呼叫的控制权。通道变量可能包括的信息很多，例如呼叫方账户号码，这个信息非常有用 如果用户呼入以后对帐号信息进行管理。FreeSWITCH 本身提供了一个应用模块接口，可以通过Set 实现. 来看下面的例子:

```
<action application="set" data="customer_id=1234"/>
```

从这里看到，通道变量customer_id 将被赋值1234. 如果这个值在接下来的呼叫中没有被修改，这个值将保存到呼叫详细记录中，也就是通常说的(CDR) 。

因为XML注册可以是非常大的，并且是可拓展的，我们通过系统设计把这些需要加载的配置文件分割成一些小的文件存放在配置路径，这些文件按照一定的逻辑来执行。通过这样的设计，我们无需使用一个庞大的文件来配置所有的环境，我们可以创建更加小，简单的文件来按照我们的需求来设置。



FreeSWITCH wiki 包含了一张演示图，来说明举例文件是如何配置的:
http://wiki.freeswitch.
org/wiki/Default_config#Overview_Diagram_of_
the_Demo_Configuration.

一旦FreeSWITCH 加载了这个主要的注册文件freeswitch.xml, 这个文件运行一个预处理，来查找一些默认的注册文件和一些这些注册文件里面的包含文件。在一些环境下，预处理可以设置一些全局变量来支持顶层的配置文件，同时可以对一些注册的功能提供全局的支持。我们拿IP地址和域名作为一个例子。假设你有一个IP地址是74.112.132.98. 如果你想在你的配置环境中多次使用这个地址，你可以把这个以下语句放在第一个加载的文件的顶部:

```
<X-PRE-PROCESS cmd="set" data="my_ip=74.112.132.98"/>
```

现在你可以把$${my_ip}的设置添加到你想添加的配置文件中。预处理帮助你完成了这个IP地址的全局设置，FreeSWITCH 将把这一设置加载到配置文件中，如果你想这个IP 地址出现到什么地方，你可以使用$${my_ip}来调用这个IP地址。

另外一个XML 预处理的强大功能就是可以通过单行设置来包含其他的配置文件。以下是一个简单的例子来说明如何实现包含其他加载文件的功能:

```
<X-PRE-PROCESS cmd="include" data="default/*.xml"/>
```

这里的意思是，每个在default文件夹下的带.xml 后缀文件都被包含并且加载文件到前一行.
通过这样的方式，可以创建新的属于自己的分机号码配置文件，并且将此文件自动包含到上层文件中，无需对 default.xml 文件进行修改 (此文件包含默认的拨号规则).

让FreeSWITCH工作

现在我们简单介绍了FreeSWITCH的基本流程。现在是开始让FreeSWITCH真正运行起来的时候了。我们需要再学习一点对FreeSWITCH 进行控制的主要工具，FreeSWITCH 命令接口(通常所说的CLI)，然后我们配置两个电话做呼叫测试。

通过CLI 控制FreeSWITCH

在第二章，搭建和安装FreeSWITCH部分，我们已经简单讨论了FreeSWITCH的一个工具fs_cli。因为，通常情况下，我们在Linux 环境下运行FreeSWITCH 作为一个daemon，或者在Windows 环境下作为一个服务来运行，作为FreeSWITCH的用户，了解 fs_cli是非常重要的.
为了使用方便，可以在Windows 环境下添加 fs_cli.exe 到系统的路径，也可以在Linux/Unix环境下，创建一个软链接，例如:

```
#>ln –s /usr/local/freeswitch/bin/fs_cli /usr/local/bin/fs_cli
```

一些Linux 用户喜欢添加 /usr/local/freeswitch/bin 到Linux 系统路径。

现在，如果你在系统命令行，简单输入fs_cli，系统为你自动启动fs_cli 命令。



通常来说, Windows 可执行文件带有.exe 后缀。
在Windows 系统中，fs_cli 被命名为fs_cli.exe. 用户可以输入 fs_cli.exe 或者 fs_cli, 同样Linux/Unix用户输入fs_cli 来启动的FreeSWITCH CLI 工具.

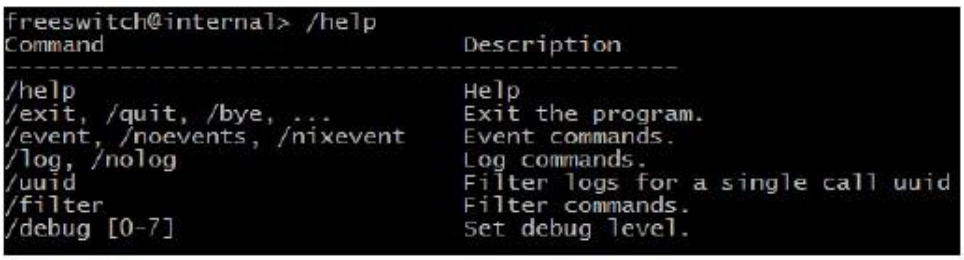
启动命令行:

#>fs_cli

你将看到以下登录FreeSWITCH后台的欢迎信息，如下截图:



一旦成功连接，除了起始符 斜线/ 以外，你输入的所有命令都将发送到FreeSWITCH 服务器端。斜线命令控制fs_cli 本身的运行状态。输入命令/help 可以显示所有fs_cli 支持的命令，如下截图:



注意，在FreeSWITCH 环境下，有几个带斜杠的命令: /exit, /
quit，和/bye. 同时注意, 还有一个省略号快捷键 (…) 退出 fs_cli程序。
所有这4个命令都可以退出 fs_cli，返回到系统提示符。他们的作用是相同的。
但是必须注意，当从控制台运行FreeSWITCH的时候，FreeSWITCH不是一个linux
daemon 或者服务，如果输入省略号… FreeSWITCH 系统关闭!

另外一个支持斜杠的是命令 /log. 默认环境下，fs_cli 启动时，debug 是开启的.
(启动欢迎界面提示了debug的级别 是level [7].) /log 命令可以允许用户控制在 fs_cli
登录以后控制debug 级别. 用户可以在任何时候修改debug级别。当用户退出或者重新启动fs_cli,
Debug 日志级别将被重新设置到7. (用户可以通过启动时添加参数 -d 或者 –-debug来设置
debug级别。) 除非用户想获得更多的debug 信息，最好是设置debug 级别为6, 就像以下设置:

```
freeswitch@internal>/log 6
+OK log level 6 [6]
```

在以下的列表中，列出了从0到7的不同级别的日志错误:

Debug 级别:	名称:	文本显示颜色:
0	Console	White
1	Alert	Red
2	Critical (Crit)	Red
3	Error (Err)	Red
4	Warning	Violet
5	Notify	Light Blue
6	Information (Info)	Green
7	Debug	Yellow

用户可以使用名称或者debug 级别的数字来指定debug 级别，例如:

```
freeswitch@internal>/log info
+OK log level info [6]
```

所有用户输入的命令将发送到FreeSWITCH 服务器端。这里有一些非常必要的命令，用户应该有所了解。它们是:

- help: 显示所有支持的CLI命令; 他们称之为 FSAPI 命令或者简单称之为APIs
- version: 显示用户正在运行的FreeSWITCH 版本
- status: 显示当前正在运行的FreeSWITCH对象的数据统计
- showchannels: 显示活动通道的列表
- showcalls: 显示已经桥接的通话列表

一个通道上一个呼叫的leg。一个legged的话机举例就是用户检查他或她的语音邮箱。另一方面，一个话机是两个独立的呼叫leg的桥接 (连接在一起). 用户必须确定了解显示通道和显示呼叫的不同。

在接下来的部分，我们将学习命令来帮助我们设置电话，以便在FreeSWITCH环境下可以正常工作。

在FreeSWITCH环境下配置SIP电话

大部分连接FreeSWITCH的终端设备是基于SIP的终端设备。SIP 或者Session Initiation Protocol是非常普通的支持电话呼叫的信令协议。 (SIP 不仅仅局限于语音，可以处理聊天，视频和其他的会话类型。)
SIP 电话的形式有两种: 物理电话和软电话. 物理电话是一种标准的设备带有听筒，输入键盘和数字显示屏. 软电话是一种软件形式的电话，运行在电脑终端，通过电脑的麦克风或者外部的耳麦和喇叭采集语音。我们将介绍如何设置软电话X-Lite的过程, 和基本的物理终端SIP配置包括物理电话Aastra, Polycom, 和Snom.

SIP 设置

所有SIP设备必须进行最小的配置才能进行话机测试。像所有复杂的协议一样，SIP 也同样具有一些隐藏的，比较神秘的设置选项。但是这些选项超过了本书讨论的范围。我们将限定我们讨论的范围来确保配置一下基本的选项成功注册到FreeSWITCH来实现标准PBX 的功能: 发起呼叫和接收呼叫，转接呼叫，设置到语音等待等等。

在我们的SIP配置文件中，我们将把我们的SIP终端设备配置注册到
FreeSWITCH 上。当一个SIP终端设备带有注册消息时，注册消息知道如何路由把呼叫路由到
设备终端设备。FreeSWITCH 工作原理类似于一个 SIP 注册. SIP将对SIP终端进行注册认证。
同样，也允许无签权的SIP终端设备注册，但是我们不推荐这种方法。 (类似的好办法就是开
一个匿名转发SMTP 服务器。如果用户仅仅是为了让某人发送邮件或者呼打电话，当然也会
带来风险，请不要这样做！）

SIP 用户工作的原理表面上看和发送邮件类似。一个SIP URI 包含 user@domain 这样的
信息和邮件地址非常相似。对于用户名称或者域名来说，SIP用户支持了一个真实名称和显示名称。另外，也包含
了签权用户名称和密码。签权用户名称不一定和用户名称完全一样，但是在大部分情况下，这两个名称是相同的。

FreeSWITCH 安装之后本身已经设置了20 个SIP 帐号。 (在第四章, SIP 和 User Directory, 我们将
讨论更多的细节包括如何配置更多的用户。) 用户名称是从1000到1019。用户可以使用任何一个帐号来做
测试。

以下是用户名 1000 的SIP设置信息:

- Username: 1000
- Authorization Username: 1000
- Password: 1234
- Domain: [FreeSWITCH 服务器IP地址]

在手边保存这些SIP设备的设置. 让我们看看几个SIP 电话机的设置过程。
如果用户手头的SIP终端不是我们举例中所使用的终端，用户仍然可以使用这些
基本的配置，应该无需做太多的配置就可以成功连接FreeSWITCH。
在每个下面的举例中，我们在内网使用几个不同厂商的SIP话机注册到
FreeSWITCH 服务器。

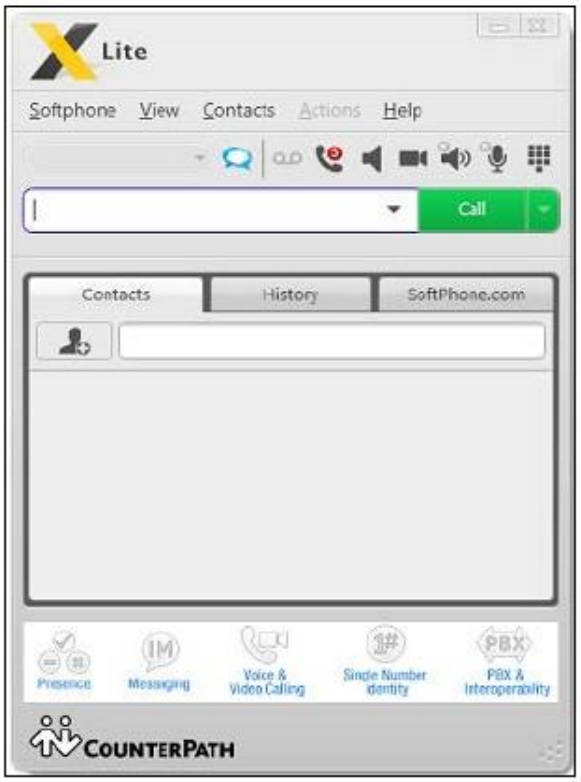
X-Lite soft phone

X-Lite (http://www.counterpath.com) 是一个免费的软电话。 (X-Lite 不是开源的软电话。)
在内网中下载X-Lite到你的电脑，确保和FreeSWITCH是同一内网。



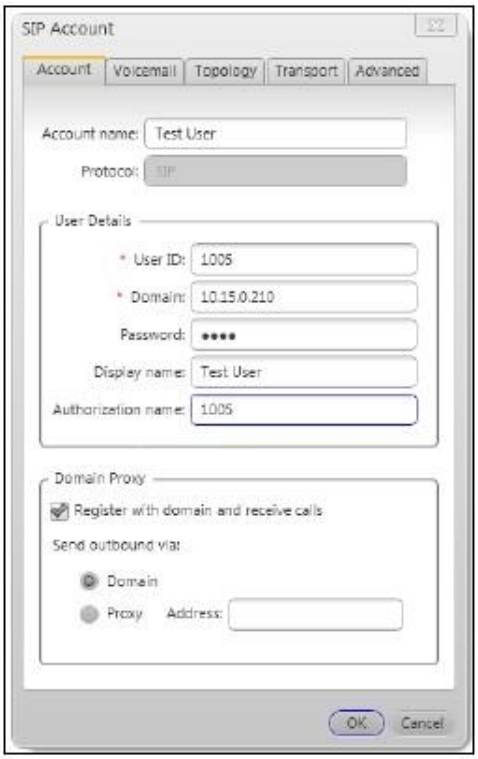
技术上讲，软电话可以和FreeSWITCH在同一台服务器上运行。
我们不建议也不支持用户这样操作。

启动X-Lite 软电话。用户将看到电话的截图:

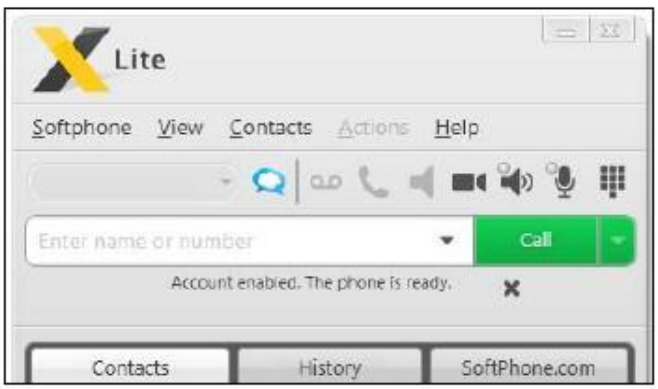


点击Softphone标签，然后点击SIP Account Settings，创建帐号。 X-Lite 支持单个帐号。点击添加 SIP Account 帐号。填入用户信息。

帐号1005 分机设置为如下截图:



点击OK ，保存配置. 这个帐号将注册到 FreeSWITCH服务器端。如果注册成功，话机状态如下截图:



电话已经注册。如果用户接收到一些不同的回复信息，大部分情况下是因为配置错误。常见错误信息是403，404和408信息:

- 403—Forbidden: 表示签权用户名称或者密码不正确.
- 404—Not Found: 表示没有在FreeSWITCH 服务器端发现此用户.
- 408—Timeout: 通常因为域名不对或者网络问题。检查防火墙设置，确认端口5060没有被过滤。

现在用户分机已经注册，可以开始发起测试呼叫。可以跳到Testing the example Dialplan 部分.

物理电话

我们将稍微花一点时间来看看如何设置不同厂家的物理电话机。熟悉了设置 Aastra, Polycom, 和 Snom 物理话机例子后，用户基本了解了话机设置的基本原理，用户应该可以设置任何SIP电话机。

用户开始之前，为了成功配置SIP话机，必须支持有以下信息:

- IP address: 用户必须了解SIP电话的IP地址以便登录电话配置界面。大部分的SIP话机支持一个界面配置菜单，用户可以配置话机的基本信息。
- Admin name: 大部分电话支持一个管理员用户。
- Admin password: 大部分话机需要管理员密码。

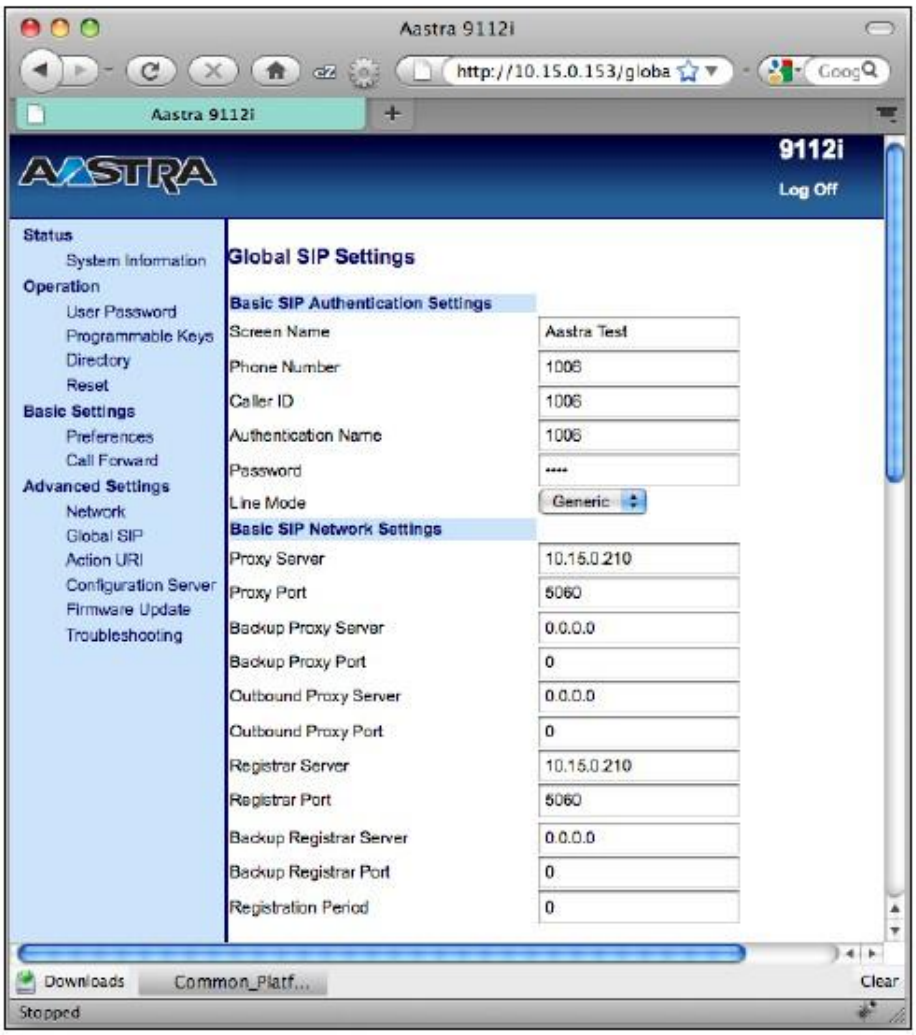
如果用户不熟悉电话机默认的用户名称和密码，请登录厂家官方网站获得详情。



大部分的电话机配置是可以支持升级固件，厂家提供这些挂机并且可以在官方网站下载。访问电话机官方网站检查用户话机是否需要升级来支持用户想要的功能。

Aastra 电话

Aastra 电话支持标准的界面。在浏览器中输入电话的IP地址，然后登录电话机界面。
在导航页面点击 Global SIP 设置。以下是Aastra 9112i 界面配置截图:

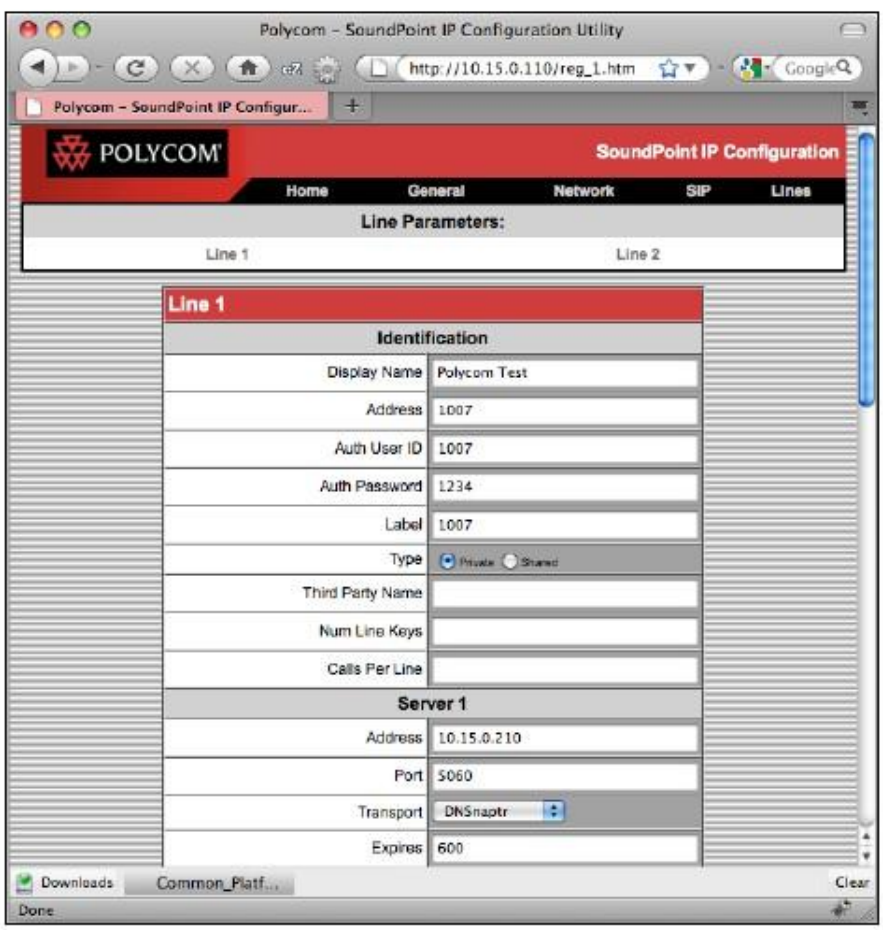


输入必要的签权设置。在 Basic SIP Network Settings，填写 Proxy Server, Proxy Port, Registrar Server, 和 Registrar Port. 填写所有必要信息后， 点击 Save Settings 保
点击导航模板的Reset link 重新启动电话机。电话终端将连接到FreeSWITCH 服务器。

如果成功注册，显示名称Aastra Test，电话号码（1006）将在电话显示界面显示。
现在用户话机已经注册，用户可以开始发起呼叫测试。可以跳到Testing the example Dialplan 部分。

Polycom 电话

宝丽通支持配置界面和电话配置菜单. 用户可以使用其中任意一个方式来配置电话机。
使用浏览器输入电话机IP地址，然后登录电话设置页面。点击 Lines link.
像其他电话机一样，SoundPoint IP 330 可以支持注册到多个服务器. 在这个例子中，我们使用Line 1 连接到FreeSWITCH. 以下是Polycom SoundPoint IP 330 页面截图:

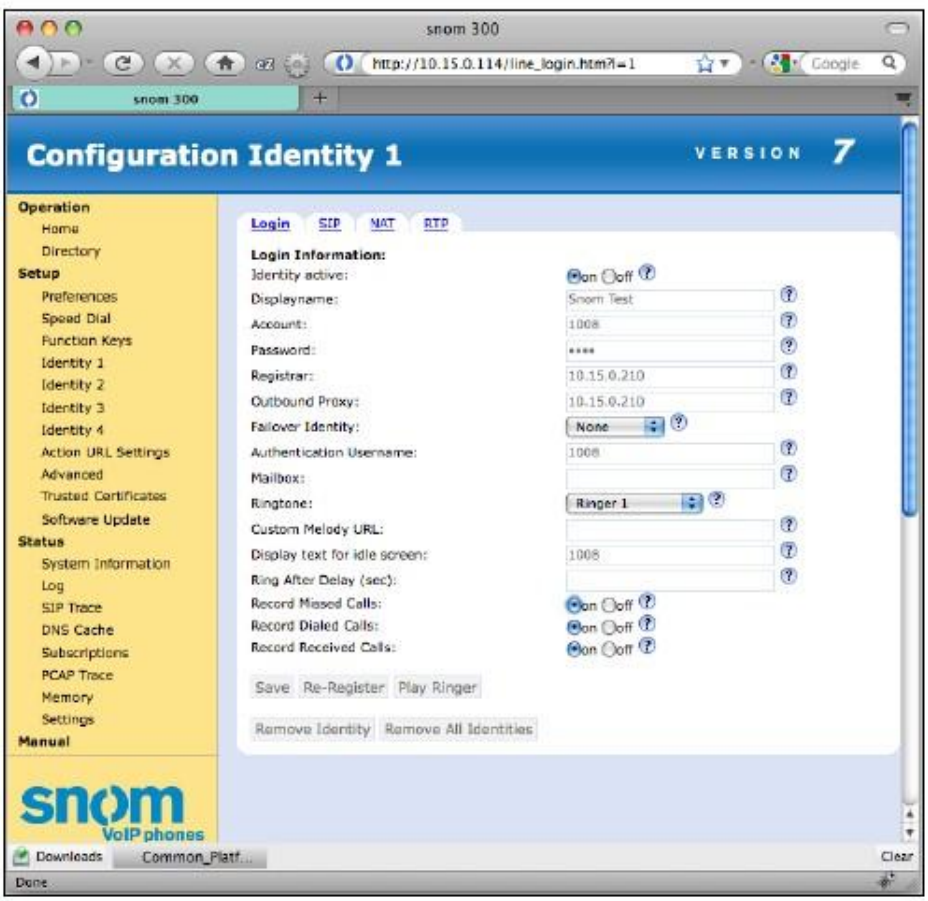


如果使用Line 1，填写以下必要的帐号信息: Display Name, Address, Auth User ID, Auth Password，和Label. 在Server 1下，填写地址和端口。然后点击Submit 保存. 电话将重新启动，然后连接到FreeSWITCH 服务器。

现在电话已经注册，用户可以发起测试呼叫。跳到 Testing the example Dialplan 部分。

Snom 电话

Snom 支持全功能的页面配置. 使用浏览器登录电话机的IP地址。注意，Snom 电话机有 Identities 的用法，相当于其他话机帐号。可以支持注册到多个服务器。点击导航页面的Identity 1 link，输入必要的帐号信息。Snom 300 页面如下截图:



为了配置帐号Identity 1，用户需要填写以下信息: Displayname, Account, Password, Registrar, Outbound Proxy, Authentication Username, 和 Display test. 点击 Save，然后点击Re-Register. 电话将马上注册FreeSWITCH。

现在用户电话已经注册到FreeSWITCH, 用户可以发起呼叫测试。跳到Testing the example Dialplan 部分。



FreeSWITCH 支持通过本地声卡或者外部的耳机连接到FreeSWITCH。
可以编译可选模块PortAudio module (mod_portaudio) 启用这个功能。
访问 http://wiki.freeswitch.org/wiki/Mod_portaudio
获得如何配置使用 PortAudio 模块。

测试拨号规则实例

现在用户已经配置好了电话, 可以进行各种呼叫测试.
如果用户配置了两个不同的话机，可以多一些测试实例. 拨号之前，用户必须确认系统已经安装了默认的语音文件和语音等待文件。 (Windows 环境以及默认安装了这些文件. Linux/ Unix 用户应该参考第五部-第二章的安装语音和语音等待文件部分, 搭建和安装 获得更多信息)。

对单一电话进行呼叫测试

以下测试是比较简单的办法来确认FreeSWITCH 可以正常工作，了解更多的关于FreeSWITCH 可以胜任的工作。在每个例子中，用户仅需要拨打一个4位数的号码，然后点击话机的Send按钮.

Tetris分机号

拨打9198. 用户应该可以听到系统播放的一个语音文件，这是一首Tetris. 系统仅语音生成工具就生成这个声音。具体使用参考链接(http://wiki.freeswitch.org/wiki/TGML for more information on TGML。)

回音测试

拨打9196. 对着电话机说话，语音流将以回声的形式回放给用户. 通过这样的测试，可以说明语音流是双向交互的。 (注意一点，如果用户话机和FreeSWITCH在同一内网，返回的声音将会非常快。拨打9195, 系统将播放一个带5秒钟延迟的回音.)

音乐等待

拨打9664. 系统播放默认的音乐等待. 如果用户可以听到音乐，说明音乐等待文件正确安装，FreeSWITCH 可以成功播放这些文件。

IVR演示

拨打 5000. 系统默认的IVR 演示实例会播放IVR 菜单。系统将会用户提供多种演示菜单选项，如下描述:

- 呼叫FreeSWITCH 电话会议
- 回音测试
- 语音等待
- 注册到 ClueCon
- 媒体流播放
- 实例IVR 子菜单

FreeSWITCH 公开会议系统是一个相当相当自由的公开会议室，任何人可以呼入到这个会议室。注意，用户的FreeSWITCH需要配置支持外网访问，防火墙必须设置支持NAT 穿越，允许SIP和RTP的数据交互。

回音测试和音乐等待是可选的，用户可以通过拨打 9196 和9664来做进一步测试。

ClueCon 是一个每年举行的开发者会议. 拨打 4 ，系统将用户转接到一个接线员，接线员非常高兴获知用户已经注册了这个会议。

实例中的子菜单是一个非常简单的举例— 摁 * 键将返回主菜单。

IVR 演示菜单保存在 conf/ivr_menus/demo_ivr.xml。

系统信息应用

拨打9192. 这个分机号使用非常简单。拨打此号码之前，必须确认已经打开了fs_cli 工具. 这个info 应用模块将输出当前呼叫的时的系统的debug 数据。 拨号之前，把debug 的级别修改到6 (INFO) 。用户可以在第三章看到这些输出实例。

不用担心这些FreeSWITCH输出的信息。仅记住，FreeSWITCH保存了每个活动呼叫的leg的所有信息。Info 模块是一个非常有用的模块，可以帮助用户来排查用户自定义的呼叫规则入口。

测试两个或者两个以上的呼叫

FreeSWITCH 真正强大的能力在于可以处理多个终端发起的多个呼叫。
以下呼叫测试让用户了解FreeSWITCH 支持的多种呼叫功能。这些测试需要至少配置两个不同的电话。

呼叫其他电话

拨打 1000, 1001, 或者其他号码. 用户仅需要拨打其他电话分机，对端设备就可以振铃。大部分的SIP 电话机像其他2一样，需要一个听筒来应答这个呼叫。如果电话没有配置一个指定的号码，呼叫方将被连接到语音邮箱，用户将根据系统的提示，对呼叫的分机留言。

驻留呼叫

呼叫另外一个电话，等待应答时。点击终端的Transfer 按钮，然后拨打6001，然后挂机。
另外一方的通话将被设置为驻留，此时将听到系统播放的音乐等待。通过以下任何一种方法再次接听电话驻留的通话:

1. 拨打6001. 被驻留的电话将被自动解锁。
2. 拨打6000 ，等待系统应答。系统将提示拨打分机号.

    拨打 6001. 被驻留的电话将被自动解锁。

加入会议室

从不同的分机终端拨打3000. 所有的分机可以相互听到对方。

系统Dialplan实例快速参考

系统默认的环境支持一些默认的拨号规则实例。通过拨打以下号码实现相应的功能:

号码	功能
1000 - 1019	本地分机
** + extension number	抢接电话
2000	实例呼叫组:销售部
2001	实例呼叫组:技术支持部
2002	实例呼叫组: 计费
3000 - 3399	实例会议室
4000 or *98	获取语音邮件
5000	Demo IVR
5900	FIFO 队列驻留
5901	FIFO 队列获取
6000	带泊/获取, 手动
6001-6099	带泊/获取, 自动
7243	RTP 多播
0911	组内对接实例 #1
0912	组内对接实例 #2
0913	紧急呼出会议实例
9178	传真接收实例
9179	传真发送实例
9180	振铃测试, 远端生成振铃音
9181	振铃测试, 发送 U.K. 振铃音
9182	振铃测试, 发送语音作为振铃音
9183	应答后发送U.K. 振铃音
9184	应答后发送语音作为振铃音
9191	ClueCon 注册
9192	系统信息显示
9195	延迟的回音测试
9196	回音测试
9197	Milliwatt tone(测试信号质量)
9198	Tetris
9664	Music on hold

在 conf/dialplan/default.xml文件路径定义了很多实例代码。

总结

在本章中，我们介绍了FreeSWITCH的一些默认的配置文件. 具体图例的内容如下:

- 当用户发起呼叫时，一些非常FreeSWITCH 背后的非常重要的概念需要了解
- 基本的fs_cli使用方法
- 如何在SIP终端设备上使用系统已默认配置的SIP帐号来连接FreeSWITCH
- 通过拨打不同的分机号码来测试默认的拨号规则

现在我们需要花一点时间来了解FreeSWITCH的另外一个重要的概念: 用户目录.

在下一个章节，我们将更加详细了解地 FreeSWITCH 用户目录.

4
SIP和用户目录

在上章节中，我们简单介绍了SIP, Session Initiation Protocol, 讨论了如何注册在FreeSWITCH 平台上注册电话。在此章节中，我们夯实SIP的基础，并且学习如何使用连接用户SIP，包括本地的和外网的。在VOIP世界中，SIP是比较特殊的一种协议。在这个章节中，我们将讨论以下内容:

- 学习FreeSWITCH 用户目录背后的原理
- 首次探讨和配置FreeSWITCH 用户目录
- 学习如何对接FreeSWITCH 和服务提供商
- 对拨号规则和XML目录配置文件进行修改
- 简单介绍 SIP profiles 和用户代理

了解FreeSWITCH用户目录

FreeSWITCH 用户目录是基于一个中央XML控制文件,
由一个或者多个<domain> 要素组成。每个 <domain> 可以包含<users> 要素或者<groups> 要素. 一个 <groups> 要素包含一个或者多个<group> 要素, 每个group包含一个或者多个<users> 要素. 同理, 一个<users> 包含一个或者多个 <user> 要素. 以下是一个简单举例:

```
<section name="directory">
  <domain name="example.com">
    <groups>
      <group name= "default">
        <users>
          <user id="1001">
```

```
            <params>
              <param name="password" value="1234"/>
            </params>
            </users>
          </user>
        </group>
      </groups>
    </domain>
  </section>
```



下载实例代码

如果用户是从 packtpub 购买的书，用户可以下载本书使用的实例代码。如果用户从其他地方购买的本书，可以访问 http://www.packtpub.com/support，然后注册，我们将把代码的文件通过邮件发送给用户。

对于users 来说，一些基本的配置文件可以不包括groups，因此用户可以忽略<groups> 这个要素，在顶部的<domain>要素中，仅添加几个<user> 要素。

这里必须注意，每个user@domain 都来自于这个用户目录，这个目录对系统的所有模块有效。这个目录是一个单一的中央控制目录，保存所有的FreeSWITCH 用户信息。如果使用一个SIP电话作为一个用户注册或者其他用户执行语音留言，FreeSWITCH将在同一地方查询用户的数据。这一点非常重要，通过这样的方法，可以限定系统的操作，避免数据的重复，并且对每个模块对非常有效率，因为每个独立的模块可以独立跟踪用户信息。

这个系统在用户量非常少的时候可以工作的非常好，但是如果用户量上千的大型系统应该怎么办呢？如果用户想通过访问FreeSWITCH数据库来获得用户目录信息，我们如何实现？答案是肯定的。我们在第一章FreeSWITCH 系统架构中讨论了如何使用mod_xml_curl模块，我们可以创建web 服务通过一个请求对全部用户目录进行访问，获得数据，方法类似于一个HTML的页面环境下的表格提交方式。反过来，web 服务可以无需担心用户数据的格式，或者XML文件格式来查询用户，获得希望得到的用户信息。mod_xml_curl 返回查询结果到从查询模块。这样做的目的是系统可以实现用户设置和中央控制的用户数据实现无缝集成，并且保持用户原始数据的格式。

在FreeSWITHC 下，可以通过任何子系统来访问用户目录。这些子系统包括模块，脚本和FSAPI 接口等等。在本章中，我们将学习Sofia SIP模块如调用用户目录对用户的软电话和物理电话进行认证。如果用户是开发人员，你可以理解更多关于用户目录的一些不错的方法，例如添加一个 <domain>, the <group>或者 <user>下添加 <variables> 的要素。在这个要素中，用户可以设置许多 <variable>, 支持系统对认证的用户的呼叫进行通道变量设置。这样，在拨号规则中可以非常方便地设置基于某个用户的呼叫路由. 同样，认证的用户可以使用CIDR 标志符定义一个IP地址访问范围来，通过这样的设置来决定路由规则。远端认证的用户可以通过用户名称和密码来访问系统做相应的呼叫路由。



Authentication（认证）是一个确定用户身份的过程。Authorization （签权）是决定用户访问级别的过程。Authentication 回答这个问题,"这个人是确定他自己吗?" 签权回答这个问题,"这个用户可以允许做什么?" 当用户看到一下表达方式 IP Auth 或者Digest Auth, 请记住，他们表达的是两种基本的身份确认方法来确认用户 (authenticating) . IP 认证是基于用户的IP地址的认证方法. Digest 认证是基于用户帐号和密码的认证方法。SIP (和FreeSWITCH) 可以使用任何一种来认证用户. 访问 http://en.wikipedia.org/wiki/Digest_access_authentication 获得关于认证工作机制的讨论。.

用户目录是基于纯XML文件. 使用XML文件有很多的优点，当然也不是最小的XML中的X: 可拓展. 因为XML被定义为一种可拓展的文件，目录结构当然也是可拓展的。如果需要在这个目录下添加新的要素，我们可以在目前的XML文件中简单添加这个要素就可以。

配合FreeSWITCH 用户目录工作

实例配置中，一个域名支持一个包含20 个用户的目录. 用户可以可以非常方便添加或者删除。在系统环境下，用户添加没有任何上限。一个组的用户可以看作一个目录. 用户可以属于一个或者多个组。最终，所有用户属于一个域。默认环境下，这个域就是FreeSWITCH的IP地址.



FreeSWITCH 可以支持多个域名. 用户可以访问 http://wiki.freeswitch.org/wiki/Multi-tenant 获得更多信息。

在接下来的章节，我们将讨论以下内容:

- 用户功能
- 添加用户
- 测试语音邮箱
- 用户组

用户功能

让我们看看定义用户的XML文件. 找到文件 conf/directory/default/1000.xml，然后使用编辑器打开这个文件，用户可以看到以下信息:

```
<include>
  <user id="1000">
    <params>
      <param name="password" value="$${default_password}"/>
      <param name="vm-password" value="1000"/>
    </params>
    <variables>
      <variable name="toll_allow"
        value="domestic,international,local"/>
      <variable name="accountcode" value="1000"/>
      <variable name="user_context" value="default"/>
      <variable name="effective_caller_id_name"
      value="Extension 1000"/>
      <variable name="effective_caller_id_number"
      value="1000"/>
```

```
            <variable name="outbound_caller_id_name"
            value="$${outbound_caller_name}"/>
            <variable name="outbound_caller_id_number"
            value="$${outbound_caller_id}"/>
            <variable name="callgroup" value="techsupport"/>
          </variables>
        </user>
      </include>
```

用户XML文件结构非常简单. 在<include>标签中，用户支持以下几个值:

- user 要素带有id 属性
- params 要素指定用户参数
- variables 要素定义通道相关变量

在我们了解这些定义的真正含义之前，我们从 用户ID 1000 中查找一下 password 和 vm-password. 在这个实例中，password 参数是SIP 认证密码.
(我们已经在第三章中讨论了这个话题) 表达式 $${default_password} 就是在全局变量中的 default_password, 这个值在定义在conf/vars.xml 文件中. 如果你猜测vm-password可能是语音邮箱密码，那说明你猜对了。用户可以拨打用户邮箱访问号码，然后输入这个密码，访问他或她的语音留言。id 的值是签权用户名称和SIP 用户名称。

另外，系统对用户支持了多个通道变量。大部分的变量直接和默认的拨号规则修改。
以下这个列表就是通道变量名称和如何使用这些通道变量:

变量名称	作用
toll_allow	指定用户可以发起何种类型的呼叫
accountcode	在CDR显示的指定的任意值
user_context	用户发起呼叫时需要路由的拨号规则中的context。
effective_caller_id_name	Caller ID name，在已注册的被呼叫方显示呼叫方的透传的以上信息。
effective_caller_id_number	Caller ID number，在已注册的被呼叫方显示呼叫方透传的以上信息。

变量	作用
outbound_caller_id_name	Caller ID name，对出局电话发送以上信息
outbound_caller_id_number	Caller ID number，对出局电话发送以上信息
callgroup	定义的任意值，用在拨号规则和CDR

一般来说，默认配置文件中的用户包含以下设置的值:

- 用户SIP帐号的签权名称
- 一个语音邮箱密码
- 一种允许/限制拨号呼叫的设置
- 一种处理 caller ID 发送机制
- 其他几个任意值，这些值需要时可以添加或者忽略。

让我们添加一个新用户帐号.

添加用户

添加一个或多个用户，仅需要2个简单步骤:

1. 通常通过拷贝目前的用户文件，可以创建一个新用户。
2. 修改Local_Extension 拨号规则入口。

在这个实例中，我们将创建一个用户名称为 Gwen 和 1100 的用户名称.
执行以下几个步骤:

1. 打开终端窗口，修改执行路径到默认的路径 conf/directory/default.
2. 拷贝文件 1000.xml，重命名为1100.xml. 在Linux/Unix 环境下是这样的执行的:

```
#>cd /usr/local/freeswitch/conf/directory/default
#>cp 1000.xml 1100.xml
```

3. 使用编辑器打开文件1100.xml，修改配置，如下:
    - 使用1100 替换所有的1000
    - 修改 effective_caller_id_name 的值为Gwen

4. 新文件最后应该是这样的:

```
<include>
  <user id="1100">
    <params>
      <param name="password" value="$${default_password}"/>
      <param name="vm-password" value="1100"/>
    </params>
    <variables>
      <variable name="toll_allow"
        value="domestic,international,local"/>
      <variable name="accountcode" value="1100"/>
      <variable name="user_context" value="default"/>
      <variable name="effective_caller_id_name" value="Gwen"/>
      <variable name="effective_caller_id_number"
        value="1100"/>
      <variable name="outbound_caller_id_name"
        value="$${outbound_caller_name}"/>
      <variable name="outbound_caller_id_number"
        value="$${outbound_caller_id}"/>
      <variable name="callgroup" value="techsupport"/>
    </variables>
  </user>
</include>
```

5. 保存文件。下一步，我们为Local_Extension修改拨号规则的入口.
   打开文件conf/dialplan/default.xml，找到以下几行:

```
<extension name="Local_Extension">
  <condition field="destination_number"
    expression="^(10[01][0-9])$">
```

就像命名的名称一样，拨号规则中的extension作为一个本地分机之间的路由.
在我们的实例中, 本地分机就是一个在目录下的电话终端注册用户。回忆一下，
FreeSWITCH 本身默认支持预设的20 用户目录，分别从1000 到1019. 这些分机相对应
那20个用户。默认环境下，任何介于1000, 1001, …, 1019 之间的呼叫都进入到Local_Extension
拨号入口进行处理。我们需要添加1100 在这个正式表达式中。按照以下例子编辑表达式:

```
^(10[01][0-9]|1100)$
```

保存文件。(我们将在第五章，理解XML 拨号规则中的正式表达式部分做详细介绍。)

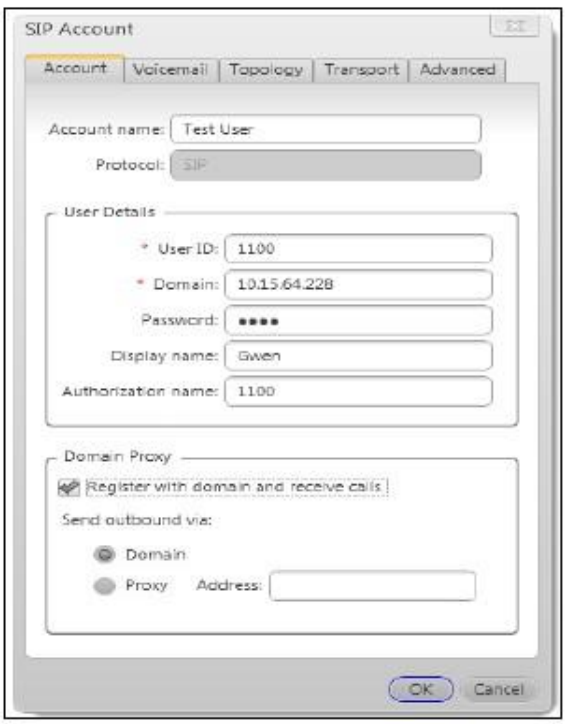
最后我们重新加载XML 配置文件。启动 fs_cli 工具，执行reloadxml 命令:

```
freeswitch@internal> reloadxml
+OK [Success]
2012-09-14 14:27:32.942464 [INFO] mod_enum.c:871 ENUM Reloaded
2012-09-14 14:27:32.942464 [INFO] switch_time.c:1163 Timezone reloaded
530 definitions
```



Linux/Unix 使用两个终端窗口操作可以节省一点时间。
一个窗口运行fs_cli in，另外一个窗口编辑文件。更多窗口控制的高级选项，请参考http://www.gnu.org/software/screen.

我们的新分机帐号已经创建加载，我们可以通过软电话注册用户1100。查看第三章的SIP 电话注册部分的方法可以注册1100. 以下是X-lite 配置文件截图：



现在注册的话机可以接收或者呼叫从1100 拨打的电话。



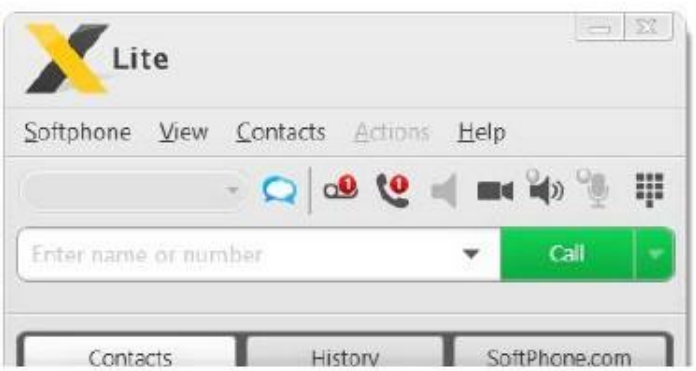
检查SIP电话注册信息，在FreeSWITCH 命令行执行命令:
sofia status profile internal reg.

现在我们已经成功添加了用户，让我们测试一个普通的功能：语音邮箱.

测试语音邮箱

每一个在目录下的用户都有一个语音邮箱，可以用于其他用户对其的语音留言。
在我们的实例中，无应答电话30秒后就将进入语音邮箱. 通过终端发起呼叫，确认
所有功能正常。拨打一个本地的分机号码，让电话振铃30秒。30秒后，语音留言系统将
应答这个通话；开始进行语音留言录音（最短3秒钟录音），然后挂机。

用户终端出现一个消息等待指示。以下是 X-Lite 软电话支持的一个消息等待指示状态截图:



注意，录音图标和听筒图标。每个图标上有一个红色圆圈，表示有语音留言和无应答呼叫。



Save time when leaving a voice message by pressing #, to skip
past the user's outbound greeting.

访问留言信息也相当简单；拨打 *98 或者 4000。语音留言系统将引导用户访问语音留言

，然后播放留言信息或保存信息。一个典型的会话场景听起来像如下描述:

4000

"欢迎访问语音留言。请输入用户ID， 然后输入#号键."

1100#

"请输入密码，然后输入#号键"

1100#

"有一条新的语音留言."

当用户有一条新的语音留言，系统自动播放这条录音，以及留言日期和时间。默认语音留言菜单配置如下:

主菜单:

1—收听新语音信息

2—收听已保存的语音信息

5—选项菜单 (留言名称, 问候，等等)

#—退出语音留言

在收听信息时，用户摁以下按键:

1—重新播放语音留言

2—保存留言

4—回放

6—快进

收听语音留言后:

1—重新播放留言

2—保存留言

4—发送语音留言到邮箱 (要求配置)

7—删除语音留言

欢迎用户测试这些选项。访问用户语音留言，保存一个呼出留言。默认环境下，系统可以支持10种不同的问候语；实际上，大部分用户仅录制一种问候语。通常，我们使用第一个问候语。



所有语音留言的选项是可定制的。查看文件 conf/autoload_configs/voicemail.conf.xml. 用户可以编辑默认的语音留言文件属性或创建自己自定义的语音留言配置文件。

现在，语音留言功能可以工作了，我们可以重点讨论一下另外一个非常有用的功能：用户组。

用户组

稍微大型的系统通常需要支持拨打多个电话的功能. 例如，一个公司的部门，部门的所有工作人员都必须应答对这个部门呼入的呼叫。同样的时间内，他们的分机又可以互相接收通话。FreeSWITCH 支持这样一个目录功能，所有在此目录下的用户都可以组成一个用户组。一个用户可以属于多个用户组。



一些PBX系统支持了一个高级呼入路由功能，称之为ACD自动呼叫分配。呼叫组不是这样一种应用。这个话题超出了我们本书讨论的范围，FreeSWITCH需要类似的高级功能，建议研究一下 FIFO 队列. 请访问：http://wiki.freeswitch.org/wiki/Mod_fifo for more information.

在conf/directory/default.xml文件定义了组的功能. 打开文件，找到groups 的节点。注意。系统已经设置了4个组. 它们是:

- Default—All users in the directory
- Sales—1000 to 1004
- Billing—1005 to 1009
- Support—1010 to 1014

后面的三个组是系统任意定义的组，可以修改或者删除这三个组。默认的组需要注意一下。它包含了目录下的每个用户。(小心使用!) 让他们添加一个新的组，然后看看这个新的组是如何工作的。执行以下步骤:

1. 打开conf/directory/default.xml文件. 在groups 节点上添加以下几行:

```
<group name="custom">
  <users>
    <user id="1000" type="pointer"/>
    <user id="1100" type="pointer"/>
  </users>
</group>
```

2. 如果系统有多个注册的号码使用，不使用1000和1100，可以修改为用户需要的号码。退出保存文件。
3. 启动 fs_cli ，然后摁F6 或者执行命令reloadxml 重新加载文件。

   通过命令group_call 命令检查新的组是否成功添加. CLI 显示结果如下截图:

```
freeswitch@internal> group_call custom
[sip_invite_domain=10.15.64.229,presence_
id=1000@10.15.64.229]error/user_not_registered,[sip_invite_
domain=10.15.64.229,presence_id=1100@10.15.64.229]sofia/internal/
sip:1100@10.15.129.38:5060;rinstance=8eecf059256b51f1;fs_
nat=yes;fs_path=sip%3A1100%4010.15.129.38%3A5060%3Brinstance%3D8ee
cf059256b51f1
```

系统显示的这些看起来乱七八糟的数据相当重要吗？ group_call 命令用来创建一个SIP 的拨号符来呼叫多个电话。在这个实例中，用户1000没有注册，因此这个用户不能接收来电。(因此出现user_not_registered错误信息.) 这里1100 是注册的。如果在组中的用户没有注册，当来电呼叫进入这个组的时候，这个用户就被完全忽略。在呼入这个组之前，我们需要在拨号规则中添加这个新的组。具体操作如下:

1. 打开 conf/dialplan/default.xml，然后找到 group_dial_billing extension:

```
<extension name="group_dial_billing">
  <condition field="destination_number" expression="^2002$">
    <action application="bridge"
      data="group/billing@${domain_name}"/>
  </condition>
</extension>
```

2. 在</extension> 标签中的group_dial_billing extension后插入以下几行:

```
<extension name="group_dial_custom">
  <condition field="destination_number" expression="^2003$">
    <action application="bridge"
      data="group/custom@${domain_name}"/>
  </condition>
</extension>
```

3. 保存文件.
4. 启动fs_cli 和执行reloadxml 命令重新加载。
5. 拨打号码2003测试用户组. 所有组内用户分机都将振铃。

当振铃组内所有电话振铃时，一旦第一个分机应答了来电。所有其他分机将停止振铃。

我们已经了解了通过电话连接FreeSWITCH和一些比较重要的功能。现在让我们讨论一下如何让FreeSWITCH服务器端的内部分机呼出到外部世界。

通过网关连接外部世界

上面，我们介绍了用户注册到你的FreeSWITCH 服务器，与之对应的是你的FreeSWITCH 注册作为一个用户注册到远端的服务器。通过网关可以完成这个工作。网关是一种比较简单的方法来配合SIP 服务器认证。对SIP服务器认证需要通过SIP REGISTER 和INVITE 消息来进行。运营商使用了非常强大的服务器端 (包括一些真正运行的FreeSWITCH!) 对用户提供SIP中继订阅服务。在FreeSWITCH环境中，我们可以使用一个网关来连接SIP运营商。当然我们也可以通过网关连接另外一个SIP服务器，例如FreeSWITCH 服务器或者任何注册标准SIP协议的IP-PBX。

设置一个新网关

使用网关连接SIP服务器端就像SIP电话连接FreeSWITCH 一样简单。网关配置和SIP终端配置基本相似。像SIP 电话注册一样，网关配置同样也有一些最少配置的要求。这些参数是：

- 用户名称和密码
- 服务器IP地址和端口

这些参数需要运营商提供。有时候，他们也支持其他的参数，例如 proxy server和端口。
如果用户已经有了运营商提供的这些帐号信息，用户就可以配置网关参数。在我们的实例中，我们使用了iptel.org的帐号。



访问 http://www.iptel.org/service 获得免费SIP帐号。

添加新的网关，执行以下步骤:

1. 在conf/sip_profiles/external文件夹下创建一个新XML文件. 在此实例中，我们使用iptel.org.xml. 添加以下几行，插入运营商提供的帐号信息:

```
<include>
 <gateway name="iptel">
   <param name="username" value="MY_USER_NAME"/>
   <param name="password" value="MY_PASSWORD"/>
   <param name="realm" value="iptel.org"/>
   <!-- iptel.org requires a 'proxy' parameter -->
   <param name="proxy" value="sip.iptel.org"/>
 </gateway></include>
```

2. 保存文件，退出，然后启动fs_cli.
3. 执行命令 /log 6 降低 debug 信息级别。
4. 简单重新加载XML 文件不能加载新的网关配置文件。需要执行以下命令: sofia profile external restart reloadxml.

   结果如下:

```
freeswitch@internal> sofia profile external restart reloadxml

Reload XML [Success]

restarting: external

freeswitch@internal> 2012-09-14 16:29:23.509986 [INFO] mod_
enum.c:808 ENUM Reloaded

2012-09-14 16:29:23.511578 [INFO] switch_time.c:661 Timezone
reloaded 530 definitions

2012-09-14 16:29:24.118566 [NOTICE] sofia_reg.c:85 UN-Registering
iptel

2012-09-14 16:29:24.713768 [NOTICE] sofia.c:1218 Waiting for
worker thread

2012-09-14 16:29:24.713768 [NOTICE] sofia_glue.c:3690 deleted
gateway example.com
```

```
2012-09-14 16:29:24.713768 [NOTICE] sofia_reg.c:2237 Added gateway
'iptel' to profile 'external'

2012-09-14 16:29:24.713768 [NOTICE] sofia_reg.c:2237 Added gateway
'example.com' to profile 'external'

2012-09-14 16:29:24.713768 [NOTICE] sofia.c:3149 Started Profile
external [sofia_reg_external]

2012-09-14 16:29:25.736445 [NOTICE] sofia_reg.c:333 Registering
iptel
```

5. 确认网关成功注册。执行命令 sofia status. 结果如下:

```
freeswitch@internal> sofia status
                                Name                Type
Data            State
================================================================
============================

                    external                profile             sip:mod_
sofia@10.15.0.91:5080               RUNNING(0)

                 example.com                gateway                 sip:joeuser@
example.com           NOREG

                       iptel                gateway               sip:MY_USER@sip.
iptel.org             REGED

                    internal                profile             sip:mod_
sofia@10.15.0.91:5060               RUNNING(0)

                internal-ipv6                profile                 sip:mod_sofia@
[::1]:5060            RUNNING(0)

                  10.15.0.91                 alias
internal              ALIASED
================================================================
============================

3 profiles 1 alias
```

现在网关状态应该是REGED，说明网关已经注册成功. 如果用户看到的结果是问题回复信息，例如FAIL_WAIT，大部分情况下，可能是配置文件的错误。检查配置文件，再次测试。



警告: 重新启动此配置文件将断掉所有的和此文件相关的通话。
另外一种添加网关配置文件，无需启动整个文件的办法是，通过此命令:
sofia profile <profile name> rescan reloadxml.

现在网关已经添加，我们需要修改拨号规则来发起呼叫和接收呼叫。

发起呼叫

我们将添加一个简单的变化规则入口。通过这个入口，可以发送呼叫到网关端。
我们新的extension 将接受数字9，然后1，最后10位以上的电话号码. (在生成环境下
有很多种可能性发送，有可能是字母开头的拨号规则。这些内容将在第五章理解
XML 拨号规则中讨论.)

为了系统可以通过网关呼出，我们在拨号规则中添加新的 extension 标签，具体步骤如下:

1. 在conf/dialplan/default 目录下，创建文件01_custom.xml.
2. 在文件中添加以下文本:

```
<include>
  <extension name=" iptel-outbound">
    <condition field="destination_number"
      expression="^9(1\d{10})$">
      <action application="bridge"
        data="sofia/gateway/iptel/$1"/>
    </condition>
  </extension>
</include>
```

3. 保存文件，启动 fs_cli and，摁F6 或者执行命令reloadxml 重新加载。

新的extension 加载成功. 从FreeSWITCH平台注册的电话拨打9 和10位数的
电话号码.例如，拨9，1-800-555-1212. 系统需要一点时间建立呼叫连接。确认
双方可以听到对方的语音。如果仅是一方可以听到语音，可能是一个单项的语音流，
很可能是NAT问题。NAT 穿越的问题我们将在第十四章的如何处理NAT中进行讨论。

接听来电

一般来说，当用户的系统注册到运营商服务器时，运营商可以支持用户服务器接听来电。
(电话注册到FreeSWITCH中就是一个简单的例子.) 在这个实例中，FreeSWITCH 对待呼入
的通话都是不可信的，甚至于这个通话来自于已经注册的网关端。这些呼入的通话会
进入public拨号规则. 在这里，这些通话可能被丢弃或者路由到正确的路由目的地。让我们建立
一个简单的拨号规则控制从iptel.org 帐号呼入的呼叫. 这个方法可以对任何SIP中继或用户创建的
网关连接有效。

1. 在conf/dialplan/public 文件夹下，创建文件名称为01_iptel.xml的文件.

2. 在文件中文件这几行，注意使用自己的帐号信息:

```
<include>
  <extension name="iptel-inbound">
    <condition field="destination_number"
      expression="^(MY_IPTEL_USERNAME)$">)$">
      <action application="set"
        data="domain_name=${domain}"/>
      <action application="bridge" data="1000 XML default"/>
    </condition>
  </extension>
</include>
```

3. 保存文件，退出。启动fs_cli，摁 F6 或者执行命令reloadxml 重新加载模块。

确认使用真实的iptel 用户名称. 呼入的来电将路由到分机1000. 用户可以路由呼叫到任何有效的分机，包括我们在第三章测试的分机号码。

无网关呼叫（电话会议）

有时，系统呼叫可以不使用网关. 例如，不是所有的服务都要求digest 签权. FreeSWITCH的公众电话会议服务器就是这样的一个例子。事实上，默认的拨号规则包含了可以拨打会议的分机号码:9888. (实际上，FreeSWITCH 服务器支持了几个不同的会议室.) 让我们看看这个extension. 打开文件conf/dialplan/default.xml，找到Extension freeswitch_public_conf_via_sip。 注意bridge 这一行:

```
<action application="bridge"
  data="sofia/${use_profile}/$1@conference.freeswitch.org"/>
```

在这个实例中，${use_profile} 的值设置为internal (在conf/vars.xml中定义). 当用户拨打9888，拨号符将发送出去，显示如下结果:

```
sofia/internal/888@conference.freeswitch.org
```

注意，这里没有提到网关配置信息. 并且, FreeSWITCH 对internal SIP 发送呼出信息。换句话说，本地的FreeSWITCH服务器没有经过任何签权认证，直接呼叫conference.freeswitch.org。这样做是可以的，因为在 conference.freeswitch.org的服务器端无需任何呼入的验证。(这就是需要部署网关的地方—如果目的地网关要求安全认证信息，网关必须发送相关的认证用户姓名和密码，以便通过认证，完成呼叫.)

不是所有的SIP运营商明确要求对呼叫进行digest 认证；一些运营商要求IP 认证. 在那种情况下，用户无需创建网关。仅对SIP发送呼叫。通常情况下，internal SIP已经满足了这个要求。

SIP profiles和user agents

在完成SIP和用户目录讨论之前，比较好的方法是再讨论另外一个技术话题。这个话题对于一些用户来说一开始就有一点望而却步，那就是关于: SIP profiles. 用一个相当严谨的词来说，freeSWITCH 中的一个SIP profile就是一个user agent. 在实际环境中，每个SIP profile 监听特别的地址和端口号。在实例配置文件中, internal profile 监听端口 5060, external profile 监听端口5080. 不仅仅监听端口，还要回复消息。例如，当一个电话发送一个 SIP REGISTER 包到FreeSWITCH (端口5060), internal profile听到这个注册请求，然后做相应的回复。在conf/sip_profiles/ 文件将决定如何回复这些信息。在这些profiles 文件中支持很多参数，通过自定义这些参数来让FreeSWITCH处理各种SIP数据流程。大部分情况下，默认的配置是可行的，应该可以正常工作的。还有一些环境，因为话机厂商或者内网环境的古怪设计或者部署不同，用户需要做适当的参数调整满足环境的要求。

最后，用户不要让profile名称 internal和external迷惑。每个文件都是一个简单user agent。这个user agent用来满足不同的要求。internal profile用来控制注册和注册分机之间的呼叫，甚至于这些电话不在内网。external profile 用来管理呼出网关的连接和一些NAT穿越的环境。

更多关于用户代理和背靠背用户代理(B2BUA) 介绍，请访问 http://en.wikipedia.org/wiki/Back-to-back_user_agent.

总结

在本章节，我们讨论了以下内容:

- FreeSWITCH 如何管理目录下的用户
- FreeSWITCH 任何使用SIP协议连接每个用户和系统外部世界。
- SIP和e-mail 非常相似，也包含用户和域名
- 使用各种用户功能，例如语音邮箱
- 添加新用户，对拨号规则做相应修改
- 通过网关连接系统以外的世界
- 关于SIP profiles和user agents介绍

在本章节中，我们对默认的 XML 拨号规则做了一点细小的修改，我们学习了如何在XML用户目录下创建用户和域名。现在我们对修改配置文件有了一个基本的了解，我们将继续在这个基础上进行讨论。在下一章，因为我们将探寻freeSWITCHXML拨号模块，默认的和常用的呼叫路由引擎。

5

理解XML拨号规则

在任何FreeSWITCH安装中，拨号规则是一个非常主要的部分. 确实，任何PBX必须有一个拨号规则，有时称之为号码计划，来处理呼叫路由规则。简单地说，拨号规则就是控制呼叫的列表。例如，当用户拿起电话，拨打1000，系统如何知道怎么处理这个呼叫? 在实例的拨号规则中，系统将连接呼叫方和注册的用户1000。但是, 拨号规则可以做更多工作，不仅仅是连接呼叫方和被呼叫方。拨号规则包含了很多命令来控制呼叫应该怎么进行和如何进行。

在上一个章节中，我们对拨号规则做了一点细小的修改。在本章节中，我们基于以上的基础继续介绍路由基础和路由控制，因此我们将讨论以下内容:

- XML拨号规则概要
- Contexts, extensions和actions
- Conditions, patterns和regular expressions
- 通道变量
- 创建测试新分机
- 重要拨号规则应用模块
- 编写拨号符

FreeSWITCH XML 拨号规则要素

FreeSWITCH XML 拨号规则实例是一个比较好的学习XML 拨号规则概念的切入点。在 conf/dialplan文件夹下，配置文件包含三个主要文件和两个文件路径:

- default.xml: 包含FreeSWITCH 拨号规则的基本配置文件。
- public.xml: 包含从FreeSWITCH 以外呼入的处理配置文件
- features.xml: 包含一些特别的context 配置，负责处理特定的拨号功能。
- default/: 所有default 目录下的文件，都包含在default context
- public/: 所有public 目录下的文件，包含在 public context

XML 实例配置文件包含了多个呼叫路由指令，这些指令构成了基本的拨号规则，它们是: contexts, extensions, conditions和actions. 一个context 是一个或多个extensions的逻辑组. 一个extension 包含一个或多个必须满足的条件. Conditions 包含一些执行命令，根据不同的条件设置或者判断结果，执行这些命令。在进一步讨论这些结构体之前，我们有必要回忆一下我们第三章学习的一些概念.

Contexts

Contexts 是extensions逻辑组. 系统把contexts 作为拨号规则的一部分。每个部分有一个特定的目的，每个部分包含extensions，这些extensions和这些目的相关联. 这样做的目的就是使得extensions 相互独立。一个典型的例子就是“多租户”。一个FreeSWITCH 服务器可以支持一个或者多个业务入口或者多用户使用，每个租户都有自己的context，防止和其他业务体号码发生冲突。例如，每个租户可以支持这样的服务 "拨0转前台"的extension. 在每个租户的平台上，用户可以拨0 呼叫到自己的前台分机, 不同的租户平台的用户可以拨0拨打完全不同的extensions. Contexts的安全也是一个应该考虑的因素。从一个context 呼出的电话都必须在指定的路由规则中进行，例如可能长途电话，国际长途，或者占用系统的其他资源，例如多方会议的会议室等等。

系统没有对Extensions的号码数进行定义。. 实例中的XML拨号规则定义了三个不同的contexts, 让我们继续做更多了解。

Default

default context 包含所有extension 支持注册的用户。当我们在conf/dialplan/default.xml添加了分机1000时, 实际上我们修改了默认context下的extension. 大部分实例拨号规则中的功能都定义在这里。

Public

public context 是一个非常好的例子说明如何使用contexts 来保证系统的安全。在FreeSWITCH环境下，所有未签权的呼入来电都由public context负责处理。"public" 意味着"本身不可信". 默认环境下，未签权的呼叫方可以通过呼叫FreeSWITCH，系统系统执行一些命令，这对FreeSWITCH 用户来说也是一件不可思议的事情。通常来说，public context 是对呼入的DID (Direct Inward Dial) 号码做路由，然后转接到一个指定的内部分机 (查看conf/dialplan/public/00_inbound_did.xml 文件中的实例). 用户也可以通过使用public context 来控制用户认为正确的呼叫，让这些正确的来电进入到用户的电话系统。

Features

features context是一个好的功能实例，通过功能的形式把extensions 归类划分。这些extensions 可以非常轻松地添加到default context；存放到它们自己的Context 更加便于用户管理。在features context定义的extensions 大部分不是Endpoints 本身，是一些系统帮助extensions, 这些extensions 执行一个功能，然后负责转接呼叫方来电到其他的目的地。例如please_hold extension，这个extension的作用是对呼叫方播放音乐等待，通知呼叫方, "请不要挂机，电话正在转接中。", 然后转接通话到最终目的地extension 号码。

Extensions

extension 这个这个词误导了很多人。在传统的PBX 环境中，一个extension就是一个连接到电话系统的电话机，而且带有一个三位数或四位数的分机号码。在FreeSWITCH环境下，一个extension 实际上是一系列的系统指令，这些指令来控制一个呼叫。它可能简单认为是拨打的三位数或四位数的号码，拨打这个号码桌面的电话会振铃。准确地说，我们已经在第四章的用户目录讨论中，提到了添加，当我们添加一个新用户时，我们添加了一个这个帐号的相应的一个号码，其他用户可以通过拨打这个号码来呼叫这个添加的新用户。

系统中，以一个 <extension> 开始标签和一个结束标签</extension> 来定义一个Extension。Extension 定义中包含了conditions, actions, 和anti-actions。Extensions 还支持两个可选属性: name 和continue. name 属性用来管理用户的拨号规则，使之具有可读性，用户查阅方便。可以想像一下，如果<extension> 标签没有名称，如果有很多extension的话，这个拨号规则就无法进行管理。continue 属性来决定如果拨号规则的解析器发现了匹配的extension 应该如何执行下一个指令。如果所有的conditions 成功匹配，extension被告知判断正确. 默认环境下，如果拨号规则发现第一个匹配成功，extensions 将不在继续做进一步的检查。如果设置continue="true"，系统将匹配更多的extensions。 (参考本章节后续部分关于拨号规则处理的工作机制).

Conditions

Extension的Conditions 来通过判断的结果决定列出的指令动作. 例如, 用户可以设置这样的一个condition "当拨999以后" ，然后执行extension块列出的指令动作。提醒一下，condition不仅仅局限于拨打一个号码。Conditions 的判断可以基于日期，时间，caller ID，发送服务器的IP地址甚至于通道变量。查看拨号规则的实例文件，就会发现类似这样的conditions:

```
<condition field="destination_number" expression="^(1234)$">
```

因为大部分情况下，我们需要根据呼入的号码来做呼叫的路由，destination_number field是大家通常使用的一个判断语句。

很多extensions 仅仅有一个condition:

```
<extension name="Simple example">
  <condition field="destination_number" expression="^(1234)$">
    <!-- actions performed here -->
  </condition>
</extension>
```

但是，用户也可以叠加更多的conditions 来创建一个逻辑与或AND，如下举例:

```
<extension name="Two condition tags example">
  <condition field="ani" expression="^(1111)$"></condition>
  <condition field="destination_number" expression="^(1234)$">
    <!-- actions performed here but only if both of the above
      conditions are true -->
  </condition>
</extension>
```

在上面的例子中， 两个conditions 的判断结果都为真，那么第二个
<condition> 标签中的action 才可以执行。ani 域 (呼叫方号码) 必须是 "1111",
拨打号码必须是 "1234"，这样，actions 就可以执行。简单的语言描述
就是,"如果呼叫方号码是1111，并且拨打的号码是1234，那么开始执行actions".
在一个extension 标签下，添加多个 <condition> 标签，我们称之为堆叠的conditions.
注意，第一个XML condition 的节点仍然需要一个关闭标签，语法举例如下:

```
<condition field="ani" expression="^(1111)$"/>
```

使用反斜杆符来关闭<condition> 标签。

<condition> 标签的另外一个功能可以是一个break 属性.(有时候用户听到的例如
中断标志符或中断参数.) 当叠加了多个conditions时, break 属性可以控制
解析器解析结果以后的处理机制，可以对每一个condition 进行灵活控制。break 属性
可以支持以下四个值:

- on-true: 如果当前的condition为真，停止继续检查其他的condition。
- on-false: 如果当前的condition为假（默认环境），则停止检查其他的condition。
- never: 无论当前的condition真或假，继续检查其他的conditions。
- always: 无论当前的condition真或假（很少使用），停止检查其余的condition。

通常环境下，只要第一个condition判断为假，跳出condition的检查。
让我们看看添加了break 属性的extension 是如何执行的:

```
<extension name="Two condition tags example">
  <condition field="ani" expression="^(1111)$" break="never">
    <!-- actions performed here if caller is 1111 -->
  </condition>
  <condition field="destination_number" expression="^(1234)$">
    <!-- actions performed here if caller dialed 1234 even if caller
      was not 1111 -->
  </condition>
</extension>
```

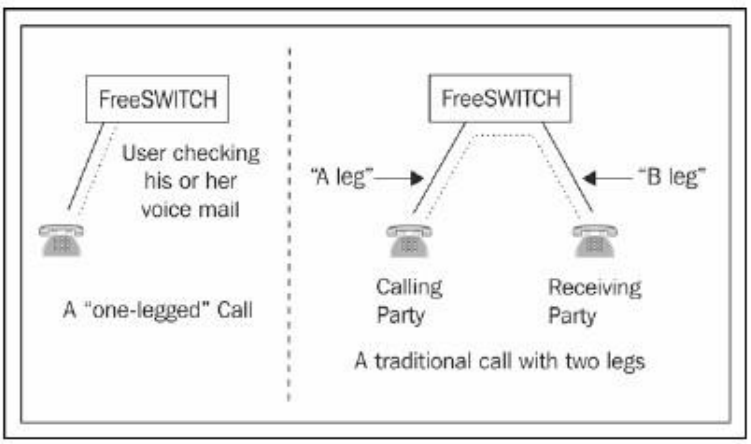
我们已经对第一个condition添加了break="never". 当解析器执行到这个Condition的时候，可能工作的流程就发生不同的变化。无论第一个condition结果是真或假，解析器将继续检查这个extension里面后续的condition。当没有添加break="never" 属性时，解析器将停止检查这个extension中的其他condition，然后进入到这个拨号规则的其他流程。但是，如果这个condition为真，需要会发生什么？第一个condition中的action 将被保存到任务列表，然后解析器将继续检查这个extension中的下一个condition。最后的结果是，我们将设置一系列的执行指令，如果呼叫方拨打1234，同时我们还有另外一些指令来判定呼叫方是否是1111。



更多高级的conditions 应用讨论，请参考第八章, 高级拨号规则概念。

Call legs和通道变量

才FreeSWITCH 呼出或者呼出的通话通常由一个或多个call legs组成. 一个one-legged的呼叫连接通常类似于拨打他或她的语音邮箱. 传统的呼叫方式是双方必须建立一个连接，这样的方式就需要两个call legs. 让我们回忆一下第三章中提到的拓扑图:



两个电话终端之间的呼叫由一个A leg (呼叫方，发起呼叫端) 和一个B leg (接收呼叫端).
每个呼叫的leg 也称之为一个通道，或者一个语音通道。每个通道支持一系列的逻辑属性，
用户可以对呼叫的leg 指定一些特别的设置。每个属性会保存在相应的通道变量中。
在上一个章节中，我们已经学习过，一个注册的用户可以支持对对该通道变量进行定义，
这些变量将被包含在这个用户的呼叫中。为了获得有效的呼叫信息，用户可以呼叫通过以下步骤
呼叫9192:

1. 启动fs_cli ，执行命令 /log 6.
2. 从注册的用户端拨打号码 9192.

用户将看到很多信息显示出来。以下是一些摘抄的信息:

```
2012-09-28 12:26:27.862464 [INFO] mod_dptools.c:1504 CHANNEL_DATA:
Channel-State: [CS_EXECUTE]
Channel-Call-State: [ACTIVE]
Channel-State-Number: [4]
Channel-Name: [sofia/internal/1010@10.15.64.229]
Unique-ID: [660e8db4-09a2-11e2-98a1-a759c95c7090]
Call-Direction: [inbound]
Presence-Call-Direction: [inbound]
Channel-HIT-Dialplan: [true]
Channel-Presence-ID: [1010@10.15.64.229]
Channel-Call-UUID: [660e8db4-09a2-11e2-98a1-a759c95c7090]
Answer-State: [answered]
Channel-Read-Codec-Name: [PCMU]
Channel-Read-Codec-Rate: [8000]
Channel-Read-Codec-Bit-Rate: [64000]
Channel-Write-Codec-Name: [PCMU]
Channel-Write-Codec-Rate: [8000]
Channel-Write-Codec-Bit-Rate: [64000]
Caller-Direction: [inbound]
Caller-Username: [1010]
Caller-Dialplan: [XML]
Caller-Caller-ID-Name: [1010]
Caller-Caller-ID-Number: [1010]
Caller-Network-Addr: [10.15.64.123]
Caller-ANI: [1010]
Caller-Destination-Number: [9192]
…
```

```
variable_sip_from_user: [1010]
variable_sip_from_uri: [1010@10.15.64.229]
variable_sip_from_host: [10.15.64.229]
variable_channel_name: [sofia/internal/1010@10.15.64.229]
variable_sip_call_id: [3c27e62339dd-652d94imggcb]
variable_sip_local_network_addr: [10.15.64.229]
variable_sip_network_ip: [10.15.64.123]
variable_sip_network_port: [2048]
variable_sip_received_ip: [10.15.64.123]
variable_sip_received_port: [2048]
variable_sip_via_protocol: [udp]
variable_sip_authorized: [true]
variable_sip_number_alias: [1010]
variable_sip_auth_username: [1010]
variable_sip_auth_realm: [10.15.64.229]
…
variable_toll_allow: [domestic,international,local]
variable_accountcode: [1010]
variable_user_context: [default]
variable_effective_caller_id_name: [Extension 1010]
variable_effective_caller_id_number: [1010]
variable_outbound_caller_id_name: [FreeSWITCH]
variable_outbound_caller_id_number: [0000000000]
…
```

以variable_ 开头的每一行信息显示了各个通道相应的变量值。
例如，variable_sip_authorized: [true] 显示是sip_authorized 通道变量设置为true. 用户可能注意到还有很多数据显示出来，例如 Unique-ID 和Call-Direction. 这些是info 应用模块的变量值. 大部分的（不是所有）值都是只读的值，都是可以访问的，就像通道变量。

访问通道变量

在拨号规则中，通道变量是可以通过特别标志符来访问: ${variable_name}. 让我们看看以下举例:

```
<action application="log" data="INFO The value in sip_authorized
  is '${sip_authorized}'"/>
```

这个命令的结果，会在FreeSWITCH 打印日志中显示:

```
2009-12-09 14:32:48.904383 [INFO] mod_dptools.c:897 The value in sip_
authorized is 'true'
```

访问只读变量的方式也是一样的。每个变量都有相应的变量名称。例如:

```
<action application="log" data="INFO The value of Unique-ID is
  '${uuid}'"/>
```

在FreeSWITCH环境下，打印以下信息:

```
2009-12-09 14:46:31.695458 [INFO] mod_dptools.c:897 The value of Unique-
ID is '169ae42e-29f5-4e1c-9505-8ee6ef643081'
```



用户可以通过以下链接访问完整的info应用模块变量列表和相应的通道变量名称:
http://wiki.freeswitch.org/wiki/Channel_Variables#Info_Application_Variable_Names_.28variable_xxxx.29

我们将在第八章 高级拨号规则概念来进一步讨论通道变量。

正则表达式

FreeSWITCH XML 拨号规则非常依赖于Perl的正则表达式 (PCRE). 一个正则表达式的表示对一串字符串执行一个真或假的判断处理。通常称之为规则匹配。当我们使用正则表达式对字符串进行判断时，我们通常回答一个简单的问题: 它匹配这个规则吗？如果回答是yes, 通常来说，说明满足了一个condition ，然后Extension执行。在一些环境中，我们可能想执行以下其他的指令，例如，如果匹配不能成功。(查看本章节的Actions 和 anti-actions部分.)

Perl的正则表达式必须遵守非常严格的语法. 第一次接触感觉有一点很难驾驭。但是一旦掌握这些基础，用户一定会喜欢上这种语言，因为用户感觉到了它们的强大。以下是正则表达式的举例和他们各自的含义:

格式	含义
123	匹配任何包含顺序为 "123"的字符
^123	匹配任何以顺序 "123"开头的字符
123$	匹配任何以顺序 "123"结束的字符
^123$	匹配以顺序"123"的字符
\d	匹配(0-9)中的任何一位数字
\d\d	匹配连续的两位数
^\d\d\d$	匹配任何一个三位数长度的字符串
^\d{7}$	匹配任何一个七位数长度的字符串
^(\d{7})$	匹配任何一个七位数长度的字符串，并且保存一个匹配的$1 命名特殊变量
^1?(\d{10})$	匹配任何可选以数字1开头的字符，并且$1包含另外十个数字，这十个数字保存在$1中
^(3\d\d\d)$	匹配一个四位数的字符串，起始数字为 "3", 并且在$1中保存了匹配的值。

用户无需对正则表达式有什么疑问，正则表达式可以实现任何用户需要的拨号匹配模式。它们可以匹配任何的字母和标点符号。查看conf/dialplan/default.xml 文件用户可以发现很多系统使用了很多不同的正则表达式。

如果用户希望了解更多特殊匹配的方式和匹配模式，可以通过FreeSWITCH命令行，输入regex 命令来执行需要的匹配模式.(词汇regex 是正则表达式的缩写形式，通常念作"REJ-ex".) regex 命令至少需要两个参数: 需要测试的数据和需要匹配的表达式. 参数通过| 字符分开。如果模式匹配，则 regex 命令返回真，否则返回假. 用户通过 fs_cli 根据如下实例来测试:

```
freeswitch@internal> regex 1234|\d
true
freeswitch@internal> regex 1234|\d\d\d\d
true
freeswitch@internal> regex 1234|\d{4}
true
freeswitch@internal> regex 1234|\d{5}
```

```
false
freeswitch@internal> regex 1234|^1234$
true
freeswitch@internal> regex 1234|234
true
freeswitch@internal> regex 1234|^234
false
```

regex 命令支持捕捉语法来判断返回结果，而不是仅仅返回一个 true 或者false。expression %0 包含一个整体匹配的值， %1 包含第一个捕捉的值，%2 包含第二个捕捉的值，依此类推。
用户可以测试以下命令:

```
freeswitch@internal> regex 18005551212|1?(\d\d\d)(\d\d\d)(\d\d\d\d)
true
freeswitch@internal> regex 18005551212|1?(\d\d\d)(\d\d\d)(\d\d\d\d)|%0
18005551212
freeswitch@internal> regex 18005551212|1?(\d\d\d)(\d\d\d)(\d\d\d\d)|%1
800
freeswitch@internal> regex 18005551212|1?(\d\d\d)(\d\d\d)(\d\d\d\d)|%2
555
freeswitch@internal> regex 18005551212|1?(\d\d\d)(\d\d\d)(\d\d\d\d)|%3
1212
freeswitch@internal> regex 18005551212|1?(\d\d\d)(\d\d\d)(\d\d\
d\d)|%1%2%3
8005551212
freeswitch@internal> regex 18005551212|1?(\d\d\d)(\d\d\d)(\d\d\
d\d)|%1-%2-%3
800-555-1212
freeswitch@internal> regex 18005551212|^7|%0
18005551212
```

使用regex 命令可以快速检查字符串和匹配模式的结果。



正则表达式是非常有用的系统工具语言，可以使用在计算机领域的很多方面，用户可以方便了解更多复杂的匹配模式的语法结构。
FreeSWITCH wiki 包含了很多学习资源:

http://wiki.freeswitch.org/wiki/Regular_Expression

Actions和anti-actions

Actions 代表当拨号规则发现了匹配的结果，则需要执行的步骤。
actions 总是出现在一个extension 和condition 块中。

Actions和anti-actions 都是通知FreeSWITCH 如何对呼叫做出响应。不同之处在于: 如果条件匹配，执行actions 指令；如果条件不匹配，则执行anti-actions 指令。来看看以下的举例:

```
<extension name="Action vs. anti-action example">
  <condition field="destination_number" expression="^(9101)$">
    <action application="log" data="INFO You dialed 9101"/>
    <anti-action application="log"
      data="INFO You did NOT dial 9101"/>
  </condition>
</extension>
```

在上面的例子中, <extension> 将根据用户拨打号码的不同，FreeSWITCH CLI命令显示不同的Log日志信息。如果用户拨打了9101，action 执行，如何显示日志信息, "You dialed 9101". 如果拨打的号码不是9101，是其他号码，则the anti-action执行，显示日志, "You did NOT dial 9101".

用户创建的大部分extensions (实例中确实有) 都有多个actions 和几个anti-actions。
大部分的实例中，action 调用了很多拨号规则中的应用模块，并且设置了传递的参数。
在上面的举例中，log 应用模块被调用，同时data 属性包含了传递的参数值。

拨号规则流程是如何工作的

中国有句古话，耳听为虚，眼见为实。同样的道理，如果用户可以看到电话呼入以后的工作流程，那么用户就非常容易理解拨号规则. 我们经常听到很多用户这样表达 "呼叫已经转到了拨号规则" 或者“呼叫进入了拨号规则”. 他们到底说的是什么意思？让我们一步步查看一下呼叫处理的流程，我们就可以理解XML 拨号规则正在执行的流程。

拨号规则实际上分为两个阶段: 解析和执行. 拨号规则的解析器查找extensions，最后执行它们。当解析器发现一个匹配的extension, 系统会把相应的actions(或anti-actions) 添加到任务执行列表。当对所有的extension 解析完成以后，执行阶段开始启动，所有列表中的actions 开始依次执行。

在FreeSWITCH 模式下，发起呼叫，用户调整到debug模式，这样比较容易看到所有的流程。启动fs_cli, 然后对9196发起呼叫(回声测试), 然后挂机. 访问终端窗口，鼠标移回，会看到类似于这样的信息:

```
2012-09-28 20:10:21.930188 [INFO] mod_dialplan_xml.c:485 Processing Test
User <1010>->9196 in context default
```

这是一个拨号规则流程的起始处理. 用户名为Test User 拨打了号码 9196。
(用户系统终端会显示这个用户名称和关联的话机。) 拨号规则开始部分是一些debug信息，显示已经匹配的和没有匹配的extension。首先解析的是unloop. 这是一个非常重要的extension, 但是我们现在对它不是特别有兴趣，我们主要讨论拨号规则。看看下一个解析的extension。
在我们的实例中，系统显示的debug 输出结果如下:

```
Dialplan: sofia/internal/1010@10.15.64.229 parsing [default->unloop]
continue=false

Dialplan: sofia/internal/1010@10.15.64.229 Regex (PASS) [unloop]
${unroll_loops}(true) =~ /^true$/ break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 Regex (FAIL) [unloop] ${sip_
looped_call}() =~ /^true$/ break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 parsing [default->tod_example]
continue=true

Dialplan: sofia/internal/1010@10.15.64.229 Date/TimeMatch (FAIL) [tod_
example] break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 parsing [default->holiday_
example] continue=true

Dialplan: sofia/internal/1010@10.15.64.229 Date/TimeMatch (FAIL)
[holiday_example] break=on-false
```

现在extension tod_example (举例，一天的时间) 已经解析. 这些Debug信息相应的tod_example extension 可以在conf/dialplan/default.xml文件中找到:

```
<extension name="tod_example" continue="true">
  <condition wday="2-6" hour="9-18">
    <action application="set" data="open=true"/>
  </condition>
</extension>
```

这个extension 简单检查一天的时间和这一天是星期几. 如果这个用户的时间段在工作时间段(星期一到星期五) 时间介于 (9:00 AM 到6:59PM)，那么设置通道变量为true. 这个呼叫的时间是发生在星期五晚上10:10 。因此，这个呼叫通过了wday (day of week) 验证，但是没有通过 hour (hour ofday) 验证. 所有上午9点到下午6点的呼叫都可以通过验证, 并且通过set 应用模块，添加到任务列表中。
注意，tod_example extension 设置了continue="true". 意思是如果tod_example 匹配成功，拨号规则将继续解析extensions.

解析器将继续匹配extensions，大部分已经匹配失败:

Dialplan: sofia/internal/1010@10.15.64.229 Regex (FAIL) [Check IVR-based CF] ${cf_target}() =~ /^\d+$/ break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 parsing [default->global-intercept] continue=false

Dialplan: sofia/internal/1010@10.15.64.229 Regex (FAIL) [global-intercept] destination_number(9196) =~ /^886$/ break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 parsing [default->group-intercept] continue=false

Dialplan: sofia/internal/1010@10.15.64.229 Regex (FAIL) [group-intercept] destination_number(9196) =~ /^*8$/ break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 parsing [default->intercept-ext] continue=false

Dialplan: sofia/internal/1010@10.15.64.229 Regex (FAIL) [intercept-ext] destination_number(9196) =~ /^**(\d+)$/ break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 parsing [default->redial] continue=false

Dialplan: sofia/internal/1010@10.15.64.229 Regex (FAIL) [redial] destination_number(9196) =~ /^(redial|870)$/ break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 parsing [default->global] continue=true

Dialplan: sofia/internal/1010@10.15.64.229 Regex (FAIL) [global] ${call_debug}(false) =~ /^true$/ break=never

Dialplan: sofia/internal/1010@10.15.64.229 Regex (FAIL) [global] ${sip_has_crypto}() =~ /^(AES_CM_128_HMAC_SHA1_32|AES_CM_128_HMAC_SHA1_80)$/ break=never

上面debug信息显示匹配失败，这是完全正常的结果。接下来，让我们查看一些比较有兴趣的输出结果:

Dialplan: sofia/internal/1010@10.15.64.229 Absolute Condition [global]

Dialplan: sofia/internal/1010@10.15.64.229 Action hash(insert/${domain_name}-spymap/${caller_id_number}/${uuid})

Dialplan: sofia/internal/1010@10.15.64.229 Action hash(insert/${domain_name}-last_dial/${caller_id_number}/${destination_number})

Dialplan: sofia/internal/1010@10.15.64.229 Action hash(insert/${domain_name}-last_dial/global/${uuid})

Dialplan: sofia/internal/1010@10.15.64.229 Action set(RFC2822_DATE=${strftime(%a, %d %b %Y %T %z)})

注意debug 信息中提到的Absolute Condition. absolute condition简单来说
这个条件匹配总是为true, 总是执行包含的actions. 这个condition 标签被配置到default.xml
中的global extension 中。列表如下:

```
<extension name="global" continue="true">
  <condition field="${call_debug}"
        expression="^true$"
        break="never">
    <action application="info"/>
  </condition>
  <!--
        This is an example of how to auto detect if telephone-event
        is missing and activate inband detection
  -->
  <!--
  <condition field="${switch_r_sdp}"
        expression="a=rtpmap:(\d+)\stelephone-event/8000"
        break="never">
    <action application="set" data="rtp_payload_number=$1"/>
    <anti-action application="start_dtmf"/>
  </condition>
  -->
  <condition field="${sip_has_crypto}"
    expression="^(AES_CM_128_HMAC_SHA1_32|AES_CM_128_HMAC_SHA1_80)$"
        break="never">
    <action application="set" data="sip_secure_media=true"/>
    <!-- Offer SRTP on outbound legs if we have it on inbound. -->
    <!-- <action application="export" data="sip_secure_media=true"/>
-->
  </condition>




  <condition>
    <action application="hash" data="insert/${domain_name}-
      spymap/${caller_id_number}/${uuid}"/>
    <action application="hash" data="insert/${domain_name}-
      last_dial/${caller_id_number}/${destination_number}"/>
    <action application="hash" data="insert/${domain_name}-
      last_dial/global/${uuid}"/>
    <action application="set" data="RFC2822_DATE=${
      strftime(%a, %d %b %Y %T %z)}"/>
  </condition>
</extension>
```

在globla extension 的第三个<condition> 标签（加深的）没有支持任何域或表达式，因此总是检测为真, 所以包含的actions 会添加到任务列表。如果这样的话，一个问题就出来了：正常情况下，如果解析器发现condition匹配不成功，将停止继续解析，为什么这次失败以后不跳过global extensions？答案的关键在 break 属性这里。注意，在这个extension 中，我们在前两个<condition>标签中指定了 break="never"。这个设置告诉解析器不管当前的condition是匹配成功或者失败，继续解析后面的condition。(查看本章中Conditions 部分介绍)。

大部分的conditions将匹配失败，直到解析器解析到music on hold extension，发现了成功匹配的condition:

Dialplan: sofia/internal/1010@10.15.64.229 Regex (PASS) [echo]
destination_number(9196) =~ /^9196$/ break=on-false

Dialplan: sofia/internal/1010@10.15.64.229 Action answer()

Dialplan: sofia/internal/1010@10.15.64.229 Action echo()

解析器在任务列表中添加了answer 和echo 拨号规则应用模块。

到这里为止，解析完成，现在开始启动执行阶段。用户将看到以EXECUTE 开始的debug 信息，以下是启动执行阶段的实例显示:

EXECUTE sofia/internal/1010@10.15.64.229 hash(insert/10.15.64.229-spymap/1010/1e67fc46-09e5-11e2-8893-b106a33054f8)

EXECUTE sofia/internal/1010@10.15.64.229 hash(insert/10.15.64.229-last_dial/1010/9196)

EXECUTE sofia/internal/1010@10.15.64.229 hash(insert/10.15.64.229-last_dial/global/1e67fc46-09e5-11e2-8893-b106a33054f8)

EXECUTE sofia/internal/1010@10.15.64.229 set(RFC2822_DATE=Fri, 28 Sep 2012 20:24:03 -0700)

EXECUTE sofia/internal/1010@10.15.64.229 answer()

EXECUTE sofia/internal/1010@10.15.64.229 echo()

上面的输出结果就是一个完整的例子，这个例子包括了从呼叫进入拨号规则到这个呼叫被最后处理。让我们花几分钟时间讨论一下这个处理的流程。"首先解析，然后执行"的处理策略使得XML拨号规则的效率相对更高。 作为一个练习，让我们看看从一个注册用户呼叫另外一个用户时拨打9664（音乐等待）时的debug 信息。

现在用户已经看到拨号规则的解析，让我们创建一个新的extension.

创建一个新的extension

让我们现在创建一个新的extension. 打开在第四章SIP和用户目录创建的文件conf/dialplan/default/01_custom.xml。我们现在定义的新的extensions 将包含在这个文件中。



总是以数字顺序来定义拨号规则文件名称。原因是XML解析器是按照ASCII名称来按照顺序读取XML文件。在conf/dialplan/default/ 中的最后一个被解析的是99999_enum.xml. 这个文件包含ENUM extension，当拨打号码不能匹配所有的extension时，这个ENUM作为一个最后的处理策略。查看http://wiki.freeswitch.org/wiki/Mod_enum 获得更多信息。

一个Dialplan XML文件可以包含一个或多个extension 定义. 唯一的限定就是以XML标签<include>开始，以</include>结束。

新的extension 非常简单, 但是同样也可以演示出FreeSWITCH拨号规则的的强大和它的灵活性。这个extension 具有以下特性:

- 应答呼叫
- 通过两种格式读出呼叫的用户分机号码，名称读取以后等待一秒钟
- 休眠两秒钟
- 对呼叫方说再见
- 挂机
- 当拨打9101时，extension 开始执行

通过以下几步来创建新的extension:

1. 在01_custom.xml 中添加以下内容:

```
<include>
  <extension name="simple test">
    <condition field="destination_number"
     expression="^(9101)$">
      <action application="answer"/>
      <action application="say"
       data="en number iterated ${ani}"/>
      <action application="sleep" data="1000"/>
      <action application="say"
       data="en number pronounced ${ani}"/>
```

```
            <action application="sleep" data="1000"/>
            <action application="playback"
             data="voicemail/vm-goodbye.wav"/>
            <action application="sleep" data="2000"/>
            <action application="hangup"/>
          </condition>
        </extension>
      </include>
```

2. 保存文件. 启动fs_cli ， 然后执行reload_xml 命令或者摁按键F6.

现在系统已经添加了extension is. 拨打9101来测试这个extension. (用户可以观察FreeSWITCH处理extension时的系统反馈.) 系统应该执行应答，播放两个语音提示，然后挂机。让我们讨论一下extension 中的每一行命令，看看做了什么:

```
<extension name="simple test">
```

这个标签表示此extension定义的开始. 实际上name属性是可选的。但是可以使得拨号规则具有可读性。同时，这个extension 名称会出现在FreeSWITCH日志中, 这样对排查问题会更加简单。extension 定义以</extension> 标签为结束符.

```
<condition field="destination_number" expression="^(9101)$">
```

condition 标签定义这个extension的匹配参数. 这里我们设置了匹配destination_number对拨打的号码正则表达式 ^(9101)$来进行匹配对比. 用通俗的语言来说，这个condition的意思就是, "如果用户拨打了数字9101, 执行condition块中的actions." 所有标签中的actions都将执行.

```
<action application="answer"/>
```

answer 模块就像自己本身表明的: 应答此呼叫。

```
<action application="say" data="en number iterated ${ani}"/>
```

这里开始执行say 应用模块, 调用预先录制的提示音来播放号码，字母，货币数额等等。
在say 应用模块中，前三个设置的参数是: say 引擎(通常是一种语言), 播放类型, 播放策略.
在这个例子中，我们设置say 应用模块以使用英文引擎(使用了英文语音文件)，并且播放号码播放方式为重复播放。号码1234 播放出来就是, "One-two-three-four". say应用模块将在本章的重要拨号规则应用部分做进一步讨论。

```
<action application="sleep" data="1000"/>
```

以上的action 简单系统休眠，系统会暂停执行1000 毫米 (1秒钟).

```
<action application="say" data="en number pronounced ${ani}"/>
```

再次执行say 一次，这次使用的策略是 pronounced 策略。使用pronounced 策略，say 应用模块，系统读出号码位数，而不是读出几个数字。数字1234将读为 "One thousand, two hundred thirty-four".

```
<action application="sleep" data="1000"/>
```

这个action再次暂停执行1000 毫秒。

```
<action application="playback" data="voicemail/vm-goodbye.wav"/>
```

以上的action 执行playback 应用模块。playback 对呼叫方播放一个语音文件。语音文件的路径可以作为一个传递的参数. 在这个实例中，我们使用了FreeSWITCH 语音邮箱的文件。更多playback 讨论，我们将在本章后续章节介绍。



FreeSWITCH 包含很多已经录制好的语音文件。英文文件名和内容都保存在FreeSWITCH源代码路径下的docs/phrase/phrase_en.xml。

```
<action application="sleep" data="2000"/>
```

暂停执行2000毫秒 (2 秒钟).

```
<action application="hangup"/>
```

执行extension的最后的action挂机，对呼叫方挂机。

这就是我们需要讨论的内容，到现在为止，用户已经完整了解了基本过程和添加的新的extensions。我们创建的大部分extension都保存在 default context, 因为 extension 是专门为我们系统用户设计的。极少环境下，我们在其他的context 下创建 extensions。例如，为了处理呼入的DID呼叫，用户需要更新 public中的context. 我们也创建了可定制的 context 来处理一下特别的需求，例如在单个FreeSWITCH 服务器来处理不同部门或者不同公司的呼叫流程。

重要的拨号规则应用模块

FreeSWITCH 支持多大140 拨号规则的应用。因为其中一些是经常使用的，所以显得非常重要。这个部分我们将讨论一些最重要的和被广泛使用的应用模块.

bridge

bridge 用来连接两个终端.

参数语法:

<target_endpoint>[,<target_endpoint>][|<target_endpoint>]

以逗号分开的终端被同时呼叫。以竖线分开的终端被依次连续呼叫。如果第一个终端应答了呼叫，其他的终端将不再被继续呼叫。

举例:

```
<action application="bridge" data="user/1000"/>
<action application="bridge"
   data="sofia/gateway/my_gateway_name/$1"/>
```

参考本章后续部分的拨号字符。

playback

Playback用来对呼叫方播放一个语音文件。支持多种语音文件格式。在FreeSWITCH包含的声音文件和音乐文件都是 .wav 文件。

参数语法: 安装文件路径可以是相对路径和绝对路径.

举例:

```
<action application="playback"
   data="/absolute/path/to/sound.wav"/>
<action application="playback" data="voicemail/vm-goodbye.wav"/>
```

用户可以从流媒体路径下，使用playback 播放语音文件(如果是mp3 文件，用户需要加载mod_shout). 举例如下:

```
<action application="playback"
   data="http://sounds.com/path/to/sound.mp3"/>
```

say

say 应用模块使用了内置的播报引擎，可以对呼叫方播放一些设置的系统信息。这种方式不是语音合成。查看speak 应用模块。

参数语法:

<module_name> <say_type> <say_method> <text>

module_name 名称通常是语言设置, 例如en或者es.

say_type 支持以下传递的参数:

- number
- items
- persons
- messages
- currency
- time_measurement
- current_date
- current_time
- current_date_time
- telephone_number
- telephone_extension
- url
- ip_address
- e-mail_address
- postal_address
- account_number
- name_spelled
- name_phonetic
- short_date_time

say_method 支持的参数可以是以下任何一种:

- n/a (这个方法不可用)
- pronounced ("one hundred, twenty-three")
- iterated ("one two three")
- counted (特殊环境下，类似于 "pronounced")

举例:

```
<action    application="say"    data="en    number pronounced 1234"/>
<action    application="say"    data="en    number iterated 1234"/>
<action    application="say"    data="en    currency pronounced 1234"/>
<action    application="say"    data="en    items pronounced 1234"/>
```

play_and_get_digits

play_and_get_digits 对呼叫方播放一个语音文件，同时可以听到呼叫方拨打的数字后面。
通过这样的方式，用户可以参加一个交互的拨号规则，无需创建一个完整的IVR。

参数语法:

<min> <max> <tries> <timeout> <terminators> <file> <invalid_file>
<var_name> <regex> [<digit_timeout>] ['<failure_ext> [failure_dp
[failure_context]]']

参数:

- min: 最少采集的号码
- max: 最大采集的数字号码
- tries: 播放次数
- timeout: 完整输入超时时间（毫秒）
- terminators: 如果少于最小采集号码，用户输入的结束符
  (通常使用 #)
- file: 播放的文件
- invalid_file: 当输入的数字没有匹配 regex 参数发生错误时，系统播放的文件
- var_name: 包含输入数字的通道变量
- regex: 正则表达式来匹配输入的数字
- digit_timeout (可选): 号码输入间隔时间超时（输入号码时中间的等待时间）
  (默认的是超时值)
- failure_ext (可选): 转接号码输入错误
- failure_dp (可选，配合failure_ext使用): 当调用failure_ext时的目的地拨号规则类型
- failure_context (可选，配合failure_dp使用): 调用failure_dp时的目的地拨号规则context

举例:

```
<action application="play_and_get_digits"
   data="2 5 3 8000 # /path/to/sound_file.wav
      /path/to/invalid_sound.wav my_digits \d+ "/>
<action application="log" data="User entered these digits:
   ${my_digits}"/>
```

执行play_and_get_digits 思路中配置的参数:

- 查找包含最小两个数字
- 查找包含最大五个数字
- 重复三次
- 等待八秒中，确认用户在8秒中之内完成输入
- 使用 # 键作为结束符
- 采集号码时播放指定路径的语音文件
- 输入号码错误时，播放指定路径的语音文件
- 在通道变量my_digits中保存拨打的号码
  匹配正则表达式 \d+

ivr

ivr 发送呼叫方到预设的 IVR.

参数语法: Name of IVR to execute.

举例:

```
<action application="ivr" data="ivr_demo"/>
```

参考第六章，使用XML IVRs 和Phrase Macros, 另外，请参考 conf/dialplan/default.xml的ivr_demo extensions.

sleep

sleep 支持在指定时间内暂停执行.

参数语法: 暂停休眠时间（毫秒）。

举例:

```
<action application="sleep" data="1000"/>
```

answer

answer 被呼叫方接听电话，建立一个语音流路径. 在SIP 中，FreeSWITCH 将发送一个 "200 OK" 消息，选择使用的编码 (如果没有被选), 然后开始RTP包的数据传输.

举例:

```
<action application="answer"/>
```

pre_answer

pre_answer 使用方法类似于answer, 但是它会建立一个早期媒体流。
在SIP 协议中，它会触发FreeSWITCH 发送一个"183 Session Progress with SDP"
消息。

举例:

```
<action application="pre_answer"/>
```

hangup

hangup 将断开媒体路径，结束通话。

参数语法: Optional hang up cause.

举例:

```
<action application="hangup"/>
<action application="hangup" data="USER_BUSY"/>
```

set

set 设置通道变量或从拨号规则中处理API命令。(这个功能我们将在第八章，高级拨号概念中做演示。)

参数语法:

```
<variable_name=value>
```

举例:

```
<action application="set" data="my_chan_var=example value"/>
<action application="log"
   data="INFO my_chan_var contains ${my_chan_var}"/>
```

transfer

transfer 发送呼叫重新返回到拨号规则中. Transfer 会产生一个完全新的解析阶段和
执行阶段。

参数语法:

```
<destination number> [destination dialplan [destination context]]
```

举例:

```
<action application="transfer" data="9664"/>
<action application="transfer"
   data="12345 XML custom"/>
```

拨号字符串格式

在我们结束拨号规则讨论时，让我们再讨论一下另外一个话题: 拨号字符串。一段拨号字符串听起来像一 一串字符，是一个FreeSWITCH 拨打的目的地号码。所有拨号字符串都有特定的语法。根据拨打的终端类型的不同，这个语法会有所不同。在FreeSWITCH中，最重要的拨号字符串都基本上和Sofia相关，因为它们引导我们如何拨打SIP终端。但是，就像我们看到的一样，FreeSWITCH中有价格不同的拨号字符串，通常，他们都用在以下两个地方，他们是:

- 通过brdge，在拨号规则中桥接一个当前的call leg
- 在CLI 后台命令环境下，通过originate命令创建一个新的call leg



无论使用拨号规则中的bridge 或 originate API 命令，拨号字符串的语法都是相同的。

让他们通过Sofia举例来了解更多拨号字符串的使用方法。基本的Sofia拨号字符串有两种格式:

- sofia/<profile name>/<user@domain>
- sofia/gateway/<gateway name>/<user>

就像我们在第四章, SIP 和用户目录学习的那样，如果我们拨打另外一个终端，可以通过网关或者不提供网关。如果我们通过一个 SIP profile呼叫时，指定user和domain是一个必要的手动。但是，如果通过网关拨打，拨号规则不需要包含domain，因为它已经被定义在网关配置文件。因此，不能通过以下方法来呼叫:

```
<!-- Wrong -->
<action application="bridge"
   data="sofia/gateway/my_gateway/user@1.2.3.4"/>
```

正确的语法是:

```
<!-- Correct -->
<action application="bridge"
   data="sofia/gateway/my_gateway/user">
```

同样的例子，如果通过internal profile 呼叫的话，需要这样设置:

```
<!--Also correct -->
<action application="bridge"
   data="sofia/internal/user@1.2.3.4/">
```

以上两种语法支持了绝大部分SIP拨号的呼叫方法。
都是，有时候也有一些特殊情况. 完整的关于拨号字符串讨论，请查看
http://wiki.freeswitch.org/wiki/Dialplan_XML#SIP-Specific_Dialstrings.

当呼叫一个在FreeSWITCH系统中已注册的用户，有一个比较简单的方法:

```
user/<user id>[@domain]
```

通过以上语法，可以轻松拨打另外一个已注册的电话。事实上，
conf/dialplan/default.xml文件中的Local_Extension 通过这样的方式和注册用户建立呼叫:

```
<action application="bridge"
   data="user/${dialed_extension}@${domain_name}"/>
```

这里，如果用户仅有一个定义的FreeSWITCH domain，那么@domain 参数就是可选的。

以下是几个不同类型的拨号字符串:

- loopback/<destination number>: 创建一个call leg，置入到拨号规则中的<destination_number>
- freetdm//<channel>/<phone number>: 通过语音卡接口创建一个call leg (查看 http://wiki.freeswitch.org/wiki/ 获得FreeTDM资料)

- error/<error code>: 模拟一个错误环境; 对测试非常有用
- group/<group name>[@domain]: 呼叫用户组 (参考第四章，SIP和用户目录)

现在可以测试这些设置。如果已经注册了电话分机，通过fs_CLI 执行 originate 以下命令:

originate <dialstring> <destination number>

测试以下命令，看看会发生什么事情。用测试号码替换1000:

originate loopback/9664 1000

originate user/1000 9664

originate error/USER_BUSY 1000

originate loopback/9192 1000

originate loopback/4000 1000

就像现在看到的一样，FreeSWITCH 支持很多工具来创建呼叫。通常情况下，可以处理或模拟任何虚拟呼叫流程。

总结

对于一个初学者来说，完成此章节的学习是一个重要的里程碑。这些概念的理解是非常重要的一个环节，对于配置维护FreeSWITCH会起到非常重要的作用。
我们讨论了几个方面的内容:

- FreeSWITCH XML 拨号规则的基本结构:
  - 一个或多个contexts组成的拨号规则
  - 一个或多个extensions组成的contexts
  - Extensions 包含一个或多个conditions
  - Conditions 通常支持一个或多个actions 或anti-actions
- 正则表达式和模式匹配
- 通道变量概念的介绍
- 拨号规则解析和处理是如何工作的
- 创建自定义的extension
- 通常使用的拨号规则的应用列表

掌握了以上这些基本的技巧，用户可以创建一个真正，有用的extensions，
通过extension可以连接多个电话用户。虽然不会规则是非常强大，和非常灵活的，
但是它包含在IVR引擎中，当然也不是真正的程序。FreeSWITCH 支持很多模块，可以
配合拨号规则一起使用，通过调用这些模块，可以使用户的系统变得更加强大。

在下一个章节中，我们将探讨FreeSWITCH非常重要的两个方面，帮助呼叫方
和系统交互: XML IVR 子系统和 FreeSWITCH phrase macros.

6

使用XML IVRs 和 Phrase Macros

FreeSWITCH内置的IVR引擎是一个非常强大的模块。它支持通过播放的语音信息和用户进行互动 (通常是通过按键音) ，最后发生呼叫到特别的目的地。IVR 引擎最后完成对呼叫方播放一段语音提示（无需真人接听），呼叫方通过语音提示选项，输入用户帐号数据，计费信息等等，进行其他的操作。

很多人熟悉的电话系统的IVR，IVR可以作为提高自动接线员来应答呼入的电话，提供多个语音导航让呼叫方来选择。(例如，"销售部请摁1，技术支持请摁2"). 这样就可以避免打扰电话接听人员，可以减少或无需设置公司前台人员。通过IVR的高级功能，可以采集呼叫方的帐号信息，输入的会议密码等等数据。在这一章节中，我们探讨FreeSWITCH自带的内置IVR引擎。用户可以通过在第五章学习的技巧，通过呼叫路由设置，使得呼入的通话进入到IVR模块，我们将使用内置的XML配置文件来创建一个IVR菜单。

我们将讨论以下内容:

- IVR 引擎总览
- IVR XML 配置文件
- IVR 菜单定义
- IVR 菜单目的地
- 路由呼叫到你的IVR
- 多级IVR菜单
- 在IVR中使用phrases
  高级路由

IVR 引擎总览

不像许多模块，这些模块是FreeSWITCH应用模块，IVR 被看作为FreeSWITCH 的核心功能。它可以用来播放语音文件，采集输入的数字。如果用户不在拨号规则中使用IVR本身，也使用了各种IVR相关的功能。举例，语音邮箱 voicemail 就采用了IVR的功能，通过语音邮箱，可以播放留言信息，删除，保存或管理其他的语音邮箱。

在这个部分，我们查看IVR在拨号规则中的应用功能。这个功能的典型实例是创建一个自动接线员语音菜单，当然实现其他的功能也是可能的。

IVR XML配置文件

FreeSWITCH 默认支持了一个简单的 IVR菜单，通过拨打5000 可以触发这个IVR菜单。当用户拨打5000时，就可以听到一个欢迎进入到FreeSWITCH的PBX的语音，有多个语音菜单让用户选择。语音菜单包含多个选择项，例如呼叫FreeSWITCH 会议系统，呼叫回音测试分机，音乐等待，或到子菜单。现在我们通过讨论XML文件来看看实例的强大之处。

IVR引擎总览

打开 conf/ivr_menus/demo_ivr.xml文件，包含以下XML内容:

```
<!-- demo IVR setup -->
<!-- demo IVR, Main Menu -->
<menu name="demo_ivr"
      greet-long="phrase:demo_ivr_main_menu"
      greet-short="phrase:demo_ivr_main_menu_short"
      invalid-sound="ivr/ivr-that_was_an_invalid_entry.wav"
      exit-sound="voicemail/vm-goodbye.wav"
      confirm-macro=""
      confirm-key=""
      tts-engine="flite"
      tts-voice="rms"
      confirm-attempts="3"
      timeout="10000"
      inter-digit-timeout="2000"
      max-failures="3"
      max-timeouts="3"
```

```
        digit-len="4">

    <!-- The following are the definitions for the digits the user dials
-->
    <!-- Digit 1 transfer caller to the public FreeSWITCH conference -->
    <entry action="menu-exec-app" digits="1" param="bridge
    sofia/$${domain}/888@conference.freeswitch.org"/>
    <entry action="menu-exec-app" digits="2" param="transfer 9196
    XML default"/> <!-- FS echo -->
    <entry action="menu-exec-app" digits="3" param="transfer 9664
    XML default"/> <!-- MOH -->
    <entry action="menu-exec-app" digits="4" param="transfer 9191
    XML default"/> <!-- ClueCon -->
    <entry action="menu-exec-app" digits="5" param="transfer
    1234*256 enum"/> <!-- Screaming monkeys -->
    <entry action="menu-sub" digits="6" param="demo_ivr_submenu"/>
    <!-- demo sub menu -->
    <!-- Using a regex in the digits tag lets you define a dial
    pattern for the caller
            You may define multiple regexes if you need a different
            pattern for some reason -->
    <entry action="menu-exec-app" digits="/^(10[01][0-9])$/"
    param="transfer $1 XML features"/>
    <entry action="menu-top" digits="9"/> <!-- Repeat this menu -->
</menu>




<!-- Demo IVR, Sub Menu -->
<menu name="demo_ivr_submenu"
      greet-long="phrase:demo_ivr_sub_menu"
      greet-short="phrase:demo_ivr_sub_menu_short"
      invalid-sound="ivr/ivr-that_was_an_invalid_entry.wav"
      exit-sound="voicemail/vm-goodbye.wav"
      timeout="15000"
      max-failures="3"
      max-timeouts="3">



    <!-- The demo IVR sub menu prompt basically just says, "press
    star to return to previous menu..." -->
    <entry action="menu-top" digits="*"/>
</menu>
```

在上面的例子中，包含两个IVR 菜单: demo_ivr 和demo_ivr_submenu. 让我们从IVR菜单定义来先看看第一个菜单例子。

IVR菜单定义

以下是XML定义的一个IVR菜单，名称为demo_ivr:

```
<menu name="demo_ivr"
      greet-long="phrase:demo_ivr_main_menu"
      greet-short="phrase:demo_ivr_main_menu_short"
      invalid-sound="ivr/ivr-that_was_an_invalid_entry.wav"
      exit-sound="voicemail/vm-goodbye.wav"
      confirm-macro=""
      confirm-key=""
      tts-engine="flite"
      tts-voice="rms"
      confirm-attempts="3"
      timeout="10000"
      inter-digit-timeout="2000"
      max-failures="3"
      max-timeouts="3"
      digit-len="4">
```

我们从拨号规则中的IVR路由呼叫时，我们将使用上面的例子。紧接着IVR名称，我们定义了多个XML属性，这些属性来决定IVR如何工作。以下的选项定义了IVR的工作方式。

greet-long

greet-long 属性来定义一个初始化的问候语，当呼叫方呼入执行到IVR时，播放此文件。和greet-short 有所不同，greet-short 仅仅播放一个公司介绍，例如 "Thank you for calling XYZ Company". 在 demo IVR, greet-long属性是一个Phrase Macro，对呼叫方播放一个介绍性的信息 ("Welcome to FreeSWITCH...") 紧接着一些可选择的菜单。

参数语法: 语音文件(或者路径 + 名称), TTS, 或者Phrase Macro.

举例:

```
greet-long="my_greeting.wav"
greet-long="phrase:my_greeting_phrase"
greet-long="say:Welcome to our company. Press 1 for sales, 2 for
support."
```

greet-short

如果呼叫方输入无效信息或无任何信息输入，greet-short属性用来指定重复播放一个语音信息。这是一个典型的无初始介绍的 greet-long 语音文件。在这个举例IVR中, Phrase Macro 中的greet-short 对呼叫方简单播放一个菜单选项，没有像greet-long那样播放一段介绍。

参数语法: 语音文件名称(或路径+ 名称), TTS, 或Phrase Macro.

举例:

```
greet-short="my_greeting_retry.wav"
greet-short="phrase:my_greeting_retry_phrase"
greet-short="say:Press 1 for sales, 2 for support."
```

invalid-sound

当呼叫方输入一个无效的输入时， invalid-sound 属性指定播放一个语音文件。

参数语法: 语音文件名称 (或路径 + 名称), TTS, 或Phrase Macro.

举例:

```
invalid-sound="invalid_entry.wav"
invalid-sound="phrase:my_invalid_entry_phrase"
invalid-sound="say:That was not a valid entry"
```

exit-sound

当呼叫方多次输入错误或多次错误超时， exit-sound 属性指定播放文件。IVR退出时播放此文件。(呼叫仍然在拨号规则中继续。)

参数语法: 语音文件名称 (或路径+文件名称), TTS, 或Phrase Macro.

举例:

```
exit-sound="too_many_bad_entries.wav"
exit-sound="phrase:my_too_many_bad_entries_phrase"
exit-sound="say:Hasta la vista, baby."
```

timeout

timeout 属性指定最长等待时间，这个时间为欢迎信息播放以后，系统等待用户输入数字的时间。
如果在此设置时间内，用户没有输入完整的数字，菜单将重复，直到max-timeouts 时间结束。

参数语法: 任何数字，以毫米为单位。

举例:

```
timeout="10000"
timeout="20000"
```

inter-digit-timeout

inter-digit-timeout 属性用来定义用户输入数字时，数字输入之间的间隔时间。
这个和总的超时是不同的。这个设置非常有用，允许用户有足够的时间输入
多个数字，完成最后输入以后，无需为太长的暂停时间困扰。例如，如果IVR
输入选项中包含1000和1两种选项，如果用户输入1以后，系统将根据inter-digit-timeout
设置，继续等待用户最后输入。如果未超时，则继续等待；否则根据最后的输入结果
进入到下一个流程。

参数语法: 任何数字, 以毫米为单位.

举例:

```
inter-digit-timeout="2000"
```

max-failures

在退出IVR之前，max-failures 属性指定了一个最多失败次数。
(呼叫将继续在拨号规则中执行。)

参数语法: 任何数字.

举例:

```
max-failures="3"
```

max-timeouts

max-timeouts 最多超时次数-指定超时次数。 (呼叫继续在拨号规则中执行.)

参数语法: 任何数字.

举例:

```
max-timeouts="3"
```

digit-len

digit-len 属性指定最长允许用户输入的数字.

参数语法: 任何大于1或等于1的数字.

举例:

```
digit-len="4"
```

tts-voice

tts-voice 指定一个特别的语音合成.

参数语法: 任何有效的语音合成.

举例:

```
tts-voice="Mary"
```

tts-engine

tts-engine 指定一个语音合成引擎.

参数语法: 任何有效的语音合成引擎.

举例:

```
tts-engine="flite"
```

confirm-key

confirm-key 属性设置一个确认键，当用户完成分机号码输入以后输入此键做最后的确认。它配合confirm-macro 属性一起使用。

参数语法: 任何有效的DTMF 数字.

举例:

```
confirm-key="#"
```

confirm-macro

confirm-macro 属性使用Phrase Macro播放一个等待提示，来用户输入等待设置的confirm-key键. 当设置了这个参数以后，用户输入分机号码以后，播放macro.

参数语法: 任何有效的Phrase Macro.

举例:

```
confirm-macro="my_confirm_macro"
```

这些属性控制IVR的一般工作流程。

IVR菜单目的地

定义了IVR的全局属性以后，用户需要提供特别的目的地或者目的地选项，以便让呼叫方选择。用户可以通过 <entry> XML 要素设置. 让我们再看看IVR中的前六个XML选项:

```
<entry action="menu-exec-app" digits="1" param="bridge
sofia/$${domain}/888@conference.freeswitch.org"/>
<entry action="menu-exec-app" digits="2" param="transfer 9196 XML
default"/> <!-- FS echo -->
<entry action="menu-exec-app" digits="3" param="transfer 9664 XML
default"/> <!-- MOH -->
<entry action="menu-exec-app" digits="4" param="transfer 9191 XML
default"/> <!-- ClueCon -->
<entry action="menu-exec-app" digits="5" param="transfer 1234*256
enum"/> <!-- Screaming monkeys -->
<entry action="menu-sub" digits="6" param="demo_ivr_submenu"/>
<entry action="menu-exec-app" digits="/^(10[01][0-9])$/"
param="transfer $1 XML features"/>
```

呼叫方可以根据每一个entry 定义的特别指令来进入的相应的流程或者相应的呼叫目的地。每个entry 支持三个参数—一个执行的action流程, digits，用户输入的数字来激活action, 和一个param 属性（传递给action）. 大部分情况下，用户可能使用menu-exec-app, 这个app 使用拨号规则参数对呼叫控制的转接进行管理，就像正常的拨号规则一样(bridge, transfer, hangup，等等.).

在上面的例子中，可选项非常简单—定义了一个单一数字 (例如 digits="3")，IVR可以桥接这个通话到一个终端（按数字1）或转接这个通话到指定的分机号码（例如，按数字2，3，4和5）. 最后，数字6和9是和IVR的子菜单相关联(在接下来的章节中介绍).

这里有一个entry 和其他的有所不同。大家可以仔细检查一下.

```
<entry action="menu-exec-app" digits="/^(10[01][0-9])$/"
param="transfer $1 XML features"/>
```

这个entry 入口定义了一个正则表达式. 这个正则表达式和用户在拨号规则中的是完全一致的。在这个IVR实例中，IVR 将从分机1000到1009中查找一个四位数的号码(这些号码是系统默认以配置的号码). 注意，正则表达式解析匹配输入的号码。匹配的号码会被赋值到特别变量 $1 ，同时作为一个参数传递到transfer应用模块中。通过这样的设置，IVR可以非常有效率地接受数字1000-1019之间的任何一个号码，当呼叫方在IVR中输入相应的号码时，IVR可以灵活地转接到以上号码。

其余的IVR entry action 有所不同. 它们引入了一个menu-sub 作为一个action, 转接呼叫方进入一个IVR 的二级菜单和menu-top, 这个菜单重新启动当前的IVR，重复菜单选项。

```
<entry action="menu-sub" digits="6"
 param="demo_ivr_submenu"/>
<entry action="menu-top" digits="9"/>
```

这两个入口控制了IVR的转接过程，进入到了新的IVR流程。注意，通过通过Transferring 的方式进入到二级菜单 sub-menu，用户创建了一个IVR的历史记录。所以，用户可以按返回按键，使得用户返回前面一个菜单或者历史菜单。在一个IVR中，可以使用多个actions。完整的action列表将在后续章节介绍。

menu-exec-app

menu-exec-app 结合param域一起使用, 执行指定的应用模块，并且给应用模块传递参数列表。这个方法等同于拨号规则中的使用的<action application="app" data="data">。最常用的是，拨号规则menu-exec-app 来转接呼叫方的通话到另外一个分机。

参数语法: application <params>

举例:

```
<entry digits="1" action="menu-exec-app"
param="application param1 param2 param3 ..."/>
<entry digits="2" action="menu-exec-app"
param="transfer 9664 XML default"/>
```

menu-play-sound

menu-play-sound 结合参数param 使用，用来播放一个指定的语音文件.

参数语法:有效的语音文件.

举例:

```
<entry digits="1" action="menu-play-sound" param="screaming_monkeys.
wav"/>
```

menu-back

menu-back 如果有上一个IVR菜单，则返回到上一个IVR菜单.

参数语法: 无.

举例:

```
<entry digits="1" action="menu-back"/>
```

menu-top

menu-top 重新启动这个IVR菜单.

语法参数: 无.

举例:

```
<entry digits="1" action="menu-top"/>
```

路由呼叫到IVR

现在IVR已经创建好了， 用户需要一些方法管理呼叫，并且可以保证让呼叫到达这个IVR菜单. 通过拨号规则可以路由呼叫到用户的IVR菜单。
在拨号规则的extension中添加以下XML 应用模块，可以触发一个IVR:

```
<action application="ivr" data="demo_ivr"/>
```

通过以上方法，可以让FreeSWITCH查找一个命名为demo_ivr的IVR，并且启动这个IVR。

这个demo_ivr拨号规则的XML入口已经添加到了FreeSWITCH实例配置文件中，如下所示:

```
<!-- a sample IVR -->
<extension name="ivr_demo">
  <condition field="destination_number" expression="^5000$">
    <action application="answer"/>
    <action application="sleep" data="2000"/>
    <action application="ivr" data="demo_ivr"/>
  </condition>
</extension>
```



注意，在以上这个实例中，IVR执行之前有一个暂停时间。
这一点是非常重要的，这样做的目的是在IVR开始播放之前，先让媒体流开始在呼叫方和FreeSWITCH的实体中传输。如果拨号规则没有这样设置，可能用户会听到一些打电话的人的抱怨，他们感觉到当IVR语音开始播放时，听到一些断断续续的声音。

多级菜单

有两种办法创建多级菜单或组合IVR. 第一种办法就是像我们以前的提到的使用子菜单列表的方式。只是简单创建两个或多个菜单，他们之间互相独立，并且具有唯一的名称。然后从主菜单利用menu-sub的action 创建一个子菜单入口，然后传递一个参数，这个参数包含子菜单名称。例如:

```
<entry digits="1" action="menu-sub" param="child_ivr"/>
```

这样做的优点是用户可以使用 menu-back action 返回到上级菜单。如果有多个父级菜单调用同样的子菜单时，这样的方式就非常方便。

另外一种办法是利用子菜单指定一个唯一的extension 号码给每一个IVR，仅仅把呼叫方电话转接到另外一个分机号码，通过这样的方式实现父级菜单和子级菜单的关联。无论如何触发IVR，通过这样的方式，可以保证从指定的IVR转接到另外一个IVR，或者返回IVR。这样的创建方法可以保证拨号规则的一致性，可以在任何时间测试个体的IVR（通过直接拨打分机号码），而不需要查询整个IVR树结构。

在IVR中使用phrases

用户可能注意到了，在实例的选项中，greet-long和greet-short都使用了demo_ivr_main_menu 而不是使用了指定的语音文件名称和路径。IVRs 可以支持用户通过phras和Text-To-Speech macro指定语音文件。这样做非常方便，其中有几个原因；比较重要的一个功能就是在一个phrase中关联多个语音文件，还有就是根据呼叫方信息的不同来设置不同的服务语言。

呼叫Phrase Macros

我们可以从拨号规则，IVR 或者拨号规则脚本语言（下一个章节讨论）来呼叫Phrase Macros。后面的这个方法我们将在下一个章节讨论。Phrase Macros几乎可以使用在任何使用语音文件名称的地方。Phrase Macros仅仅用来起到回放的目的，如果这个文件是执行一个录音操作时，不能使用PhraseMacros来指定一个文件名称。我们已经看到了实例XML IVR配置使用Phrase。以下是一些举例来说明如何从拨号规则中使用Phrase Macros:

```
<action application="playback"
data="phrase:myphrase:arg1:arg2:arg3"/>
<action application="play_and_get_digits"
data="2 5 3 7000 # phrase:myphrase:arg1 /invalid.wav my_var
\d+"/>
```

注意，这里没有要求添加任何参数. 以下举例同样有效:

```
<action application="playback" data="phrase:myphrase"/>
```

现在，让我们看看phrases 是如何完成我们的工作的.

Phrase Macro 举例– 语音邮箱

大家还记得，FreeSWITCH的IVR举例中，语音邮箱是使用频繁的一个功能。
它也是一个比较典型的Phrase Macros环境，简化了预录语音文件的管理，并且
可以重复使用。通过了解Phrase Macros在邮件系统中的使用实例，我们实际上了解
了所有必要的工具。

使用文本编辑器打开文件 conf/lang/en/vm/sounds.xml，浏览一下这个文件。用户可以发现
熟悉的<include>标签，还有附随的 <macro> 标签.
简单看一下macros的定义，用户可以大致了解macros可以完成的任务.

Phrase Macro基本语法看起来是这样的:

```
<macro name="<macro name>">
  <input pattern="<pattern>">
    <match>
      <action/>
    </match>
    <nomatch>
      <action/>
    </nomatch>
  </input>
  <input pattern="<pattern>">
    <match>
      <action/>
    </match>
    <nomatch>
      <action/>
    </nomatch>
  </input>
</macro>
```

在<macro> 和</macro> 标签中定义了macro的内容。
input pattern 是一个正则表达式，传递任何的参数，这个参数最终和Phrase Macro的
参数匹配。如果匹配为真，则执行<match> 和</match> 标签中的actions，否则
执行<nomatch> 和</nomatch> 中的action。如果发现匹配结果，特别的表达式赋值变量
($1, $2, 等等) 会自动挂载到<match> 内部的节点. 注意，用户可以设置多个输入
模式。这个功能的工作方式非常类似于XML 拨号规则的功能. 以下代码就是一
个使用多个输入模式节点实例。

让我们看看简单的macros 实例。voicemail_goodbye macro 是这样的:

```
<macro name="voicemail_goodbye">
  <input pattern="(.*)">
    <match>
      <action function="play-file"
      data="voicemail/vm-goodbye.wav"/>
    </match>
  </input>
</macro>
```

当呼叫方退出时，语音邮箱系统调用了这个macro。在这个例子中，输入的匹配模式（.*）永远是匹配的，甚至于phrase 调用时可以不传递任何参数。这样的匹配在Phrase Macros中是经常出现的。第一眼看上去，这样的方法似乎没有任何优点，仅是简单的几行代码来播放一个语音文件。但是，如果当用户退出语音邮箱时，可以使用Phrase Macro 允许这个流程，用户无需修改任何代码。当然还有其他方面的优势.

找到voicemail_enter_pass macro:

```
<macro name="voicemail_enter_pass">
    <input pattern="(.*)">
      <match>
        <action function="play-file" data="voicemail/vm-
            enter_pass.wav"/>
        <action function="say" data="$1" method="pronounced"
        type="name_spelled"/>
      </match>
    </input>
  </macro>
```

注意，这个macro 采集了传递的参数，然后对特别变量$1 赋值。在这个举例配置文件中，语音邮箱模块发送了 # 作为一个参数。这个macro控制用户退出语音邮箱时的对话。系统播放语音文件，播报 "Please enter your password, followed by…" 然后使用 say 应用模块来播报单词 "pound". 表面上看，呼叫方听到这样的播报"Please enter your password, followed by… pound". 当用户听到提示音输入密码时，这个macro 可以让用户自定义对用户播报的语音。



当登录语音邮箱系统，检查信息时，观察fs_cli 程序。用户可以看到phrases被解析，然后执行。

到这里，我们可以看到，用户可以非常方便定制Phase Macros，它们可以集成各种语音提示来创建一个完整的播放语音，对呼叫方播放此语音。一个经典的例子就是IVR 语音菜单。语音邮箱就是一个真正的IVR语音菜单，只是为了特殊的用途。它是对呼叫方播放的对话，"按1，收听新语音留言。按2 收听保存的语音留言。按5，选择高级选项。退出，请按#号键".
让我们看看以下macro.

找到macro voicemail_menu, 如下所示:

```
<macro name="voicemail_menu">
  <input pattern="^([0-9#*]):([0-9#*]):([0-9#*]):([0-9#*])$">
    <match>
      <!-- To listen to new messages -->
      <action function="play-file"
      data="voicemail/vm-
      listen_new.wav"/>
      <action function="play-file"
      data="voicemail/vm-press.wav"/>
      <action function="say"
      data="$1" method="pronounced"
          type="name_spelled"/>
      <action function="execute" data="sleep(100)"/>
      <!-- To listen to saved messages -->
      <action function="play-file"
      data="voicemail/vm-listen_saved.wav"/>
      <action function="play-file"
      data="voicemail/vm-press.wav"/>
      <action function="say" data="$2" method="pronounced"
      type="name_spelled"/>
      <action function="execute" data="sleep(100)"/>
      <!-- For advanced options -->
      <action function="play-file"
      data="voicemail/vm-advanced.wav"/>
      <action function="play-file"
      data="voicemail/vm-press.wav"/>
      <action function="say" data="$3" method="pronounced"
      type="name_spelled"/>
      <action function="execute" data="sleep(100)"/>
      <!-- To exit -->
      <action function="play-file"
      data="voicemail/vm-to_exit.wav"/>
      <action function="play-file"
      data="voicemail/vm-press.wav"/>
      <action function="say" data="$4" method="pronounced"
      type="name_phonetic"/>
    </match>
  </input>
</macro>
```

大部分phrase 无需过多解释. 在匹配模式中（强调的）发现有一些关键的信息.
语音邮箱调用这个macro 时携带了一个参数列表，例如这样: 1:2:5:#.
这个输入模式是一个简单的正则表达式，解析出这些值，结果是$1 包含1，$2 包含2，
$3 包含5，最后$4包含#号。



用户可能觉得奇怪，在哪里定义了按键事件，从什么地方开始，用户通知FreeSWITCH 语音模块，引导呼叫方应该按1 收听新留言信息，按2 收听保存的信息，等等选项。
答案就是在conf/autoload_configs/voicemail.conf.xml文件。查看<profiles> 节点上的default voicemail profile。注意，许多参数的名称以-来结束。这些是可以自定义的。
参数 play-new-messages-key 定义了用户按键来收听的新留言信息。参数config-menu-key 定义的用户按键收听高级选项菜单。用户可以随意选择自己的自定义方式。
FreeSWITCH 开发人员推荐，如果在生产系统中定义自profile，请首先备份默认的语音留言profie。

让我们看看更多使用Phrase Macros的例子来解决较为复杂的IVR流程。voicemail_message_count macro 解决了两个完全不同的问题。首先，我们有两个类型的语音邮箱留言，命名为新留言和已保存留言。其次，我们面对一个挑战，什么时候使用留言的复数还是单数，什么时候通知呼叫方当前系统有多少留言信息。注意，看看我们的voicemail_message_count macro 是如何轻松地同时解决这两个问题的:

```
<macro name="voicemail_message_count">
  <input pattern="^(1):(.*)$" break_on_match="true">
    <match>
      <action function="play-file"
      data="voicemail/vm-you_have.wav"/>
      <action function="say" data="$1" method="pronounced"
      type="items"/>
      <action function="play-file"
      data="voicemail/vm-$2.wav"/>
      <action function="play-file"
      data="voicemail/vm-message.wav"/>
    </match>
  </input>
  <input pattern="^(\d+):(.*)$">
    <match>
      <action function="play-file"
      data="voicemail/vm-you_have.wav"/>
```

```
            <action function="say" data="$1" method="pronounced"
            type="items"/>
            <action function="play-file"
            data="voicemail/vm-$2.wav"/>
            <action function="play-file"
            data="voicemail/vm-messages.wav"/>
          </match>
        </input>
      </macro>
```

再次说明，这些语法的语法非常明了，和上一个举例一样，关键是了解输入模式匹配（强调的部分）的作用。Voicemail 语音模块调用了macro，并且macro 支持了两个传递的参数 x:new 或 x:saved, 各自代表新留言的数量和已保存的留言数量。留言数量被赋值到了变量$1, 留言类型（新留言或已保存留言）被赋值到了$2. macro 调用 $2 来决定是否播放语音留言voicemail/vm-new.wav 或 voicemail/vm-saved.wav, 通过这样的方式轻松解决了问题。但是，如何除了判别出留言类型，如何播放这些留言呢？

注意，第一个输入参数多带来一个属性break_on_match. 如果设置此属性为true, 命令macro停止检查macro 输入模式的剩余部分。如果用户只有一个新的语音留言，语音留言模块调用这个带传递参数1:new 的phrase。 (同样，如果用户有一个已保存的留言信息，它可能调用传递的参数1:saved.) 第一个模式如果成功匹配的话，将停止查找剩余的匹配模式。在第一个<match>标签中的actions 会开始执行. 在这个举例中，Phrase Macro 将关联一个phrase，这个phrase 播报，"您有 … 一个 … 新 … 留言". 但是，如果有多个语音留言(或零个语音留言), 那么传递的参数类似于这样的2:new. 在这个举例中，第一个输入模式匹配就会失败，继续查找第二个匹配模式，在第二个模式执行中成功匹配。在<match>节点中的actions将生成一个phrase，这个phrase开始播报，"您有 … 二个 … 新 … 留言信息". 首先，使用特别指定的输入模式匹配，设置break_on_match为true, 其次，使用标准的输入模式匹配，我们可以通过这样简单的方式轻松处理了多语言使用过程中普遍存在的复数留言的问题。

记住这些处理方法，我们将在第七章, 在拨号规则脚本语言Lua中使用。

高级路由

IVRs 不仅仅局限在菜单功能。用户喜欢使用程序语言来开发复杂的IVR语音功能，也可能使用其他方法来利用内置的XML IVRs做开发。

例如，公司的系统要求呼叫方输入验证密码，以便通过相应的应答服务。用户可以创建一个IVR，如果输入的验证密码有效，接入到相应的入口，然后把欢迎致辞替换成相应的语音文件，如果输入的验证密码无效，则播放无效入口的密码错误语音文件。这个语音菜单会相当简单,例如:

```
<menu name="enter_pin"
greet-long="phrase:enter_your_pin"
invalid-sound="phrase:invalid_pin"
exit-sound="phrase:invalid_pin"
timeout="15000"
max-failures="3"
max-timeouts="3">
   <entry digits="1828" action="menu-exec-app"
   param="transfer after_hours XML default"/>
</menu>
```

这样非常有效的创建了一个语音提示，密码验证要求，如果输入3次失败则挂机。

另外一种选择是，用户可以创建一个IVR 来获得用户输入的数字，在后续的执行流程中调用这些数据。举例，用户想让呼叫方输入caller ID，在下一个呼出的呼叫中显示这个caller ID号码。用户应该创建一个IVR 获得这个10位数的号码，传递这个号码到另外一个分机号码，把这个10位数号码设置为当前的caller ID，然后发送到目的地。这个IVR 可能就是这样的：

```
<menu name="set_callerid"
greet-long="phrase:enter_your_callerid"
invalid-sound="phrase:invalid_callerid"
exit-sound="phrase:invalid_callerid"
timeout="15000"
digit_len="10"
max-failures="99"
max-timeouts="99">
   <entry digits="/^(\d{10})$/" action="menu-exec-app"
   param="transfer $1 XML set_callerid"/>
</menu>
```

用户应该在拨号规则中创建一个特别的context 标签，例如:

```
<context name="set_callerid">
   <extension name="SetIt">
      <condition field="destination_number"
      expression="^(\d{10})$">
         <action application="set"
       data="effective_caller_id_number=$1"/>
         <action application="bridge"
         data="sofia/external/18005551212"/>
      </condition>
   </extension>
</context>
```

通过上一个IVR实例和这个辅助性的context 可以支持呼叫方输入他们的Caller ID
然后设置 来电显示号码以后桥接到呼叫目的地，对目的地发起呼叫.

总结

FreeSWITCH 的IVR 系统是一个强大，灵活的工具，可以用来收集用户输入数据信息。当配合其他的应用模块一起工作时，使用了动态的呼叫路由和灵活的呼叫流程，各种需求可能性可以无限拓展。

现在我们研究了XML IVR系统和Phrase Macro系统，让我们把注意力转向另外一种方法，通过这种方法控制复杂的呼叫方和系统之间的互动，那就是Lua拨号规则语言。

7

拨号规则脚本语言

在上一个章节中，我们介绍了通过内置的XML IVR引擎创建基本的IVR应用。
XML IVR 引擎可以创建相对简单，相对静态的IVR应用场景。FreeSWITCH
同样支持另外一种搭建IVR场景的方式，这种方式比内置的IVR引擎更加强大，灵活。
这种方法就是通过调用各种脚本语言来配合FreeSWITCH 创建灵活的IVR应用模块。
FreeSWITCH 支持以下几种脚本语言来创建IVR:

- JavaScript
- Lua
- Perl

我们可以通过以上任何一种语言来进行IVR开发。在这个章节，我们重点技术Lua (www.lua.org),
它是一种轻量级的基本语言，可以嵌入到其他的项目中。最著名的例子就是魔兽世界。



每一种脚本语言都有他们各自的优势和劣势。因为Lua 执行比较快，稳定性好，而且容易学习，所以我们选择Lua 作为重点技术的基本语言。
如果一切条件不变，Lua相对来说是比较好的选择用来编写拨号规则脚本。.

在这个章节，我们将讨论以下内容:

- 使用Lua语言开发
- 创建语音应用
- 高级IVR
- 脚本使用提示

开发语音应用中使用了Lua脚本语言，在我们的实例中，我们将更多使用自定义的phrase macros。

使用Lua语言开发

当我们使用实例配置文件时，Lua 已经被默认加载。为了确认用户已经安装运行了Lua，打开fs_cli，然后执行命令lua。用户应该看到类似的结果:

```
freeswitch@internal> lua
-ERR no reply
```

如果看到错误信息，这个错误信息提示没有发现此命令，那么用户需要安装加载mod_lua 到系统中。就像我们编译加载mod_flite那样安装此模块。参考第二章，编译和安装的详细信息。

从拨号规则中运行Lua脚本

从<action> 标签中调用lua拨号应用，可以使用以下类似语法:

```
<action application="lua"
  data="my_script.lua arg1 arg2 arg3"/>
```

通过空格界定传递到脚本的参数。包含一个带空格的传递参数，请使用单引号来界定参数:

```
<action application="lua"
  data="my_script.lua 'arg 1' 'arg 2' 'arg 3'"/>
```

如果用户的脚本安装在了默认FreeSWITCH的scripts 子目录，用户没有必要指定脚本的完整路径。如果需要那样设置，用户可以使用绝对路径。例如，在Linux/Unix环境下，可以这样设置:

```
<action application="lua" data="/full/path/to/my_script.lua"/>
```

在Windows环境下:

```
<action application="lua"
  data="C:\full\path\to\my_script.lua"/>
```

开始编写脚本之前，让我们简单了解一下Lua语言的语法。

Lua基本语法

Lua 的语法比较容易学习，容易阅读。以上是简单的脚本:

```
-- This is a sample Lua script
-- Single line comments begin with two dashes
--[[
    This is a multi-line comment.
    Everything between the double square brackets
      is part of the comment block.
]]
-- Lua is loosely typed
var = 1-- This is a comment
var ="alpha"-- Another comment
var ="A1"-- You get the idea...
--[[
    When the Lua script is called from the dialplan
    you have a few magic objects. A handy one is
    the 'freeswitch' object which lets you do things
    like this:
    freeswitch.consoleLog("INFO","This is a log line\n")




    Another important one is the 'session' object which
    Lets you manipulate the call:
    session:answer()
    session:hangup()

]]
-- Lua makes extensive use of tables
-- Tables are a hybrid of arrays and associative arrays
val1 = 1
val2 = 2
my_table = {
        key1 = val1,
        key2 = val2,
       "index 1",
       "index 2"
}
```

```
freeswitch.consoleLog("INFO","my_table key1 is '" .. my_table["key1"]
.."'\n")
freeswitch.consoleLog("INFO","my_table index 1 is '" .. my_table[1]
.."'\n")
-- Access arguments passed in
arg1 = argv[1]-- First argument
arg2 = argv[2]-- Second argument
-- Simple if/then
if ( var =="A1" ) then
   freeswitch.consoleLog("INFO","var is 'A1'\n")
end
-- Simple if/then/else
if ( var =="A1" ) then
   freeswitch.consoleLog("INFO","var is 'A1'\n")
else
   freeswitch.consoleLog("INFO","var is not 'A1'!\n")
end
-- String concatenation uses ..
var ="This" .." and " .. "that"
freeswitch.consoleLog("INFO","var contains '" .. var .."'\n")
-- The end
```

每个从别后规则中执行的Lua脚本会受到一个session 对象，这个会话结束一个被处理的呼叫leg。session对象的基本原理就是控制呼叫，并且会大量使用在Lua 脚本中。

构建语音应用

现在我们已经了解了Lua的基本语法，让我们一个简单的Lua脚本和拨号规则中相应的入口。首先，需要创建一个拨号规则的extension ，当用户拨打9910时用来执行Lua:

1. 打开已经在第五章创建的文件01_custom.xml，添加以下新的extension:

```
<extension name="Simple Lua Test">
   <condition field="destination_number" expression="^(9910)$">
      <action application="lua" data="test1.lua"/>
   </condition>
</extension>
```

2. 保存此文件，启动fs_cli, 执行命令 reloadxml, 或者按F6键:

拨号规则已经可以呼叫 test1.lua脚本. 创建一个新的脚本如下.

3. 使用编辑器，在freeswitch/ scripts/目录下创建一个文件test1.lua, 添加以下代码:

```
-- test1.lua
-- Answer call, play a prompt, hang up
-- Set the path separator
pathsep = '/'
-- Windows users do this instead:
-- pathsep = '\'
-- Answer the call
session:answer()
-- Create a string with path and filename of a sound file
prompt ="ivr" .. pathsep .."ivr-welcome_to_freeswitch.wav"
-- Print a log message
freeswitch.consoleLog("INFO","Prompt file is '" .. prompt .."'\n")
-- Play the prompt
session:streamFile(prompt)
-- Hangup
session:hangup()
```

4. 保存文件.

上面的脚本例子已经可以执行，准备测试。使用任何在FreeSWITCH环境下注册的分机号码拨打9910，用户可以听到播放的提示音，然后系统自动挂机。



编辑保存Lua脚本以后，无需重新执行reloadxml.
只要脚本文件已经保存，拨号规则将自动呼叫被更新的基本文件.

让我们看看这几行代码，了解一下他们的作用.

```
pathsep = '/'
```

上面这行代码声明了一个变量pathsep， 这是一个正斜杆字符。 在Linux/Unix环境下，用来作为一个文件路径的分割标志。当然，Windows用户需要使用反斜杆作为一个路径的分割标志。



使用路径分割标志会使得脚本支持跨平台使用。

```
session:answer()
```

以上代码执行应答呼叫. 大部分的基本使用应答来作为第一个action.

```
prompt ="ivr".. pathsep .. "ivr-welcome_to_freeswitch.wav"
```

以上代码声明了一个变量prompt，这个变量包含语音文件的相对路径.

```
freeswitch.consoleLog("INFO","Prompt file is '" .. prompt .. "'\n")
```

以上代码在FreeSWITCH 控制台输出打印信息。
这样非常方便排查和debug 脚本执行情况。在拨打号码9910时，用户可以观察FreeSWITCH 控制台，用户可以看到以下输出结果:

```
2012-10-11 16:46:50.770343 [INFO] switch_cpp.cpp:1227 Prompt file is
'ivr/ivr-welcome_to_freeswitch.wav'
```



当使用FreeSWITCH consoleLog 时，确认包含换行字符 (\n)。.

```
session:streamFile(prompt)
```

以上代码调用session 对象的streamFile 函数对呼叫方播放语音文件。一定要记得，当我们在FreeSWITCH中指定了一个相对文件路径时，系统将匹配呼叫的样本采样率。在很多环境下，它的采样率是8000，因为采样率8000Hz（8kHz）是典型的电话呼叫的采样率。在这个实例中，实际Linux 路径是以下路径:
/usr/local/freeswitch/sounds/en/us/callie/ivr/8000/ivr-welcome_to_freeswitch.wav.



完整的英文语音文件列表可以和相关的文档形式的内容可以在FreeSWITCH代码路径下的docs/phrase/phrase_en.xml找到。

最后一行代码是挂断此通话，对呼叫方执行挂机指令。

```
session:hangup()
```

创建一个简单的Lua脚本不是一个复杂的工作。现在让我们写一个Lua脚本来和呼叫方进行一些基本的互动。

简单IVR – 如何和呼叫方互动

大部分的IVR应用场景需要呼叫方根据提示音执行输入。例如，比较常见的IVR场景是提示用户输入验证密码或用户帐号，然后执行相应的拨号指令。让我们看一个简单的脚本来演示一个使用场景-呼叫方根据提示输入数字，然后系统对呼叫方读出输入的数字，使用两种播报方式来演示这个场景：

1. 打开文件01_custom.xml 文件，添加一个新的extension:

```
<extension name="Read Back Entered Digits">
  <condition field="destination_number" expression="^(9911)$">
    <action application="lua" data="read_back_digits.lua"/>
  </condition>
</extension>
```

2. 保存文件，启动fs_cli, 执行命令reloadxml, 或按F6键.

拨号规则已经完成了对read_back_digits.lua的呼叫设置.
创建一个脚本:

1. 在the freeswitch/scripts/目录下，创建一个脚本read_back_digits.lua 文件，添加:

```
-- read_back_digits.lua
--Answer the call
session:answer()
-- Set the path separator
pathsep = '/'
-- Windows users do this instead:
-- pathsep = '\'
-- Set a variable that contains the sound prompt to play
prompt = "ivr" .. pathsep ..
"ivr-please_enter_extension_followed_by_pound.wav"
-- Set a variable that contains the invalid message to play
invalid = "ivr" .. pathsep ..
"ivr-that_was_an_invalid_entry.wav"
-- Play file and collect digits
-- Variable 'digits' will contain the digits collected
-- Valid input is 3 digits min, 5 digits max
-- Caller presses # (pound or hash) to finish
digits = session:playAndGetDigits(3, 5, 3, 7000, "#", prompt, invalid,
"\\d+")
-- Read back digits iterated, then pause
-- "one two three four five"
session:execute("say","en number iterated " .. digits)
session:sleep(1000)
```

```
-- Read back digits pronounced, then pause
-- "twelve thousand, three hundred forty-five"
session:execute("say","en number pronounced " .. digits)
session:sleep(1000)
-- Politely hang up
thankyou = "ivr" .. pathsep .. "ivr-Thank_you.wav"
goodbye = "voicemail" .. pathsep .. "vm-goodbye.wav"
session:streamFile(thankyou)
session:sleep(250)
session:streamFile(goodbye)
-- Hangup
session:hangup()
```

2. 保存文件。

这个脚本现在已经可以测试. 拨打9911，听到提示音，要求呼叫方输入一个分机号。输入分机号以后，按#号键作为结束符。系统将通过两种方式对呼叫方播报输入的分机号码，这两种方式分别为消耗播放的方式 (1-2-3-4) 和发音的方式 (一千，二百三十四), 最后在挂机之前播放, "谢谢，再见！"。playAndGetDigits 函数处理输入数值的有效性。用户可以测试输入两个数字，系统将对呼叫方播放无效提示的语音信息。如果呼叫方输入三次失败，playAndGetDigits将挂机.

条件和循环

以上实例演示了一个和呼叫方进行脚本互动场景。让我们介绍一个新的脚本，看看这个脚本是使用条件和循环语句。我们将应用我们在第六章学习的 Macros 来创建一个新的Phrase Macro，通过它来把几个独立的个体语音文件组合成一个比较大的语音提示场景。

首先，让我们创建一个Phrase Macro. 我们需要一个Phrase Macro，通过它可以把独立的语音文件组合成一个整体的语音提示结构体，对用户可以播放, "继续，请按1；退出请按2". 我们创建一个新的文件custom-phrases.xml，然后添加新的macro.

1. 打开文件conf/lang/en/demo/custom-phrases.xml. 添加以下代码:

```
<macro name="read_digits2_phrase" pause="100">
   <input pattern="(.*)">
      <match>
         <action function="play-file" data="voicemail/vm-continue.
wav"/>
         <action function="play-file" data="voicemail/vm-press.wav"/>
         <action function="play-file" data="digits/1.wav"/>
```

```
        <action      function="execute" data="sleep(250)"/>
        <action      function="play-file" data="voicemail/vm-to_exit.
        wav"/>
        <action
        <action      function="play-file" data="voicemail/vm-press.wav"/>
        <action      function="play-file" data="digits/2.wav"/>
      </match>       function="execute" data="sleep(250)"/>
    </input>
</macro>
```

2. 保存文件，启动fs_cli, 执行命令reloadxml, 或按F6键.

Phrase Macro read_digits2_phrase 完成设置，可以使用.



记住，用户在任何时候编辑了XML 配置文件，都需要执行命令reloadxml 重新加载，或者在FreeSWITCH命令行按F6更新. 这是一种非常良好的效果，无论什么时候修改了XML配置文件，用户及时重新加载配置文件，可以非常轻松对说做的修改进行排查定位。

3. 打开文件01_custom.xml，添加一个新的extension:

```
<extension name="Read Back Entered Digits #2">
  <condition field="destination_number" expression="^(9912)$">
    <action application="lua" data="read_back_digits2.lua"/>
  </condition>
</extension>
```

4. 保存文件，启动fs_cli, 执行命令reloadxml, 或按F6.

拨号规则已经可以呼叫read_back_digits2.lua 这个脚本.

创建一个新的脚本:

1. 使用编辑器在freeswitch/scripts/目录下，创建一个脚本read_back_digit2.lua:

```
-- read_back_digits2.lua
-- Demonstrates while loop and session:ready()
--Answer the call
session:answer()
-- Set the path separator
pathsep = '/'
-- Windows users do this instead:
-- pathsep = '\'
-- Set a variable that contains the sound prompt to play
prompt = "ivr" .. pathsep ..
```

```
"ivr-please_enter_extension_followed_by_pound.wav"
-- Set a variable that contains the invalid message to play
invalid = "ivr" .. pathsep ..
"ivr-that_was_an_invalid_entry.wav"
-- Set a flag for continuing or exiting
continue = true
-- Initiate while loop
-- Loop continues until caller hangs up or chooses to exit
while(session:ready() == true and continue == true) do
	-- Play file and collect digits
	-- Variable 'digits' will contain the digits collected
	-- Valid input is 3 digits min, 5 digits max
	-- Caller presses # (pound or hash) to finish
	digits = session:playAndGetDigits(3, 5, 3, 7000, "#", prompt,
invalid, "\\d+")
	-- Read back digits iterated, then pause
	-- "one two three four five"
	session:execute("say","en number iterated " .. digits)
	session:sleep(1000)
	-- Read back digits pronounced, then pause
	-- "twelve thousand, three hundred forty-five"
	session:execute("say","en number pronounced " .. digits)
	session:sleep(1000)
	-- Ask caller to continue or exit
	digits = session:playAndGetDigits(1, 1, 2, 4000, "#",
"phrase:read_digits2_phrase", invalid, "\\d{1}")
	freeswitch.consoleLog("INFO","digits is '" .. digits .. "'\n")
  if (digits == "2") then
		continue = false
		freeswitch.consoleLog("INFO","Preparing to exit...\n")
	end
end
-- Politely hang up
thankyou = "ivr" .. pathsep .. "ivr-Thank_you.wav"
goodbye = "voicemail" .. pathsep .. "vm-goodbye.wav"
session:sleep(250)
session:streamFile(thankyou)
session:sleep(250)
session:streamFile(goodbye)
-- Hangup
session:hangup()
```

2. 保存文件。

现在我们准备测试. 在FreeSWITCH 命令行下，执行命令 /log 6 ，系统将屏蔽debug 信息。当拨打分机时，观察控制台。拨打9912 然后输入数字，这些数字会自动回放给用户。数字回放以后，系统会出现第二个提示语音，要求用户按1 继续，或按2 退出。(技术上来说，除了输入2以外，用户输入任何数字都将继续这个脚本.) 测试这两种选项并且观察控制台输出。用户将看到按键以后的log 日志信息。

让我们看看脚本中这两行加深的代码:

```
while(session:ready() == true and continue == true) do
```

这行代码将启动while 循环语句. 注意，这里的两个条件必须同时满足While循环语句才退出执行，换言之session:ready() 必须是true 和变量continue 必须也是true。session:ready() 函数是一种简单的方法来判断呼叫方是否挂机。当呼叫方挂机，session:ready() 函数返回一个不为true的值。另外一个条件是测试变量continue, 这是我们创建的一个简单标志，设置为true。这个标志会一直为true，直到系统提示呼叫方是否继续或者退出时，呼叫方摁按键2，这时 continue 设置为false.

```
if (digits == "2") then
```

以上代码执行退出时检测用户是否摁按键2. 如果数字包含2，则脚本设置continue的值为false, 最后从while 循环语句退出。

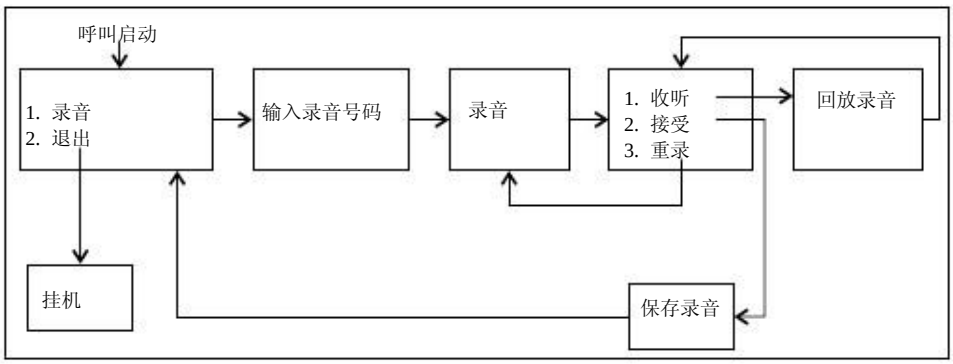


注意，从play_and_get_digits 接收到的值是字符串形式，不是一个整数类型。呼叫方拨打的数字号码中可以包含* 和# 键。

这个简单的例子演示了一种如何使用条件语句的方法。session:ready() 函数是一个非常重要的工具，如果用户挂机以后，利用这个函数来断开主条件控制语句的循环语句。

更多条件和循环语句

让我们回头看看我们学习的技术，创建一个简单的基本工具来支持呼叫方录音。这个脚本提示用户输入一系列的数字，这些数字作为文件名称，允许用户启动录音，然后给用户提供一个选择，接收录音或重新录音，最后，让用户选择录制另外一个提示音或退出这个脚本执行。同时，也介绍一个功能概念和使用setInputCallback 来处理某些按键。最后，我们将创建两个新的 Phrase Macros和一个重新利用的Phrase Macro. 基本的呼叫流程类似于这个流程图:



在拨号规则中添加，如下所示:

3. 打开01_custom.xml 文件，添加一个新的extension:

```
<extension name="Record Sound Files Utility">
  <condition field="destination_number" expression="^(9913)$">
    <action application="lua" data="record_sound_files.lua"/>
  </condition>
</extension>
```

4. 保存文件。启动fs_cli，执行命令reloadxml, 或按F6.

现在拨号规则已经完成，可以测试呼叫Lua 脚本文件record_sound_files.lua. 下一步，我们创建几个Phrase Macros. 第一个Phrase Macro 通知呼叫方, "按1录制问候语；按2退出". 注意，这个Phrase Macro几乎和上一个实例中的read_digits2_phrase macro 完全相同。另外一个Phrase Macro 提示用户输入录音号码(录音文件名称) 和按#号键. 添加两个新的Phrase Macros如下代码:

5. 打开文件conf/lang/en/demo/custom-phrases.xml，然后添加以下代码:

```
<macro name="record_greeting_or_exit" pause="100">
  <input pattern="(.*)">
    <match>
      <action function="play-file"
          data="voicemail/vm-to_record_greeting.wav"/>
      <action function="play-file"
          data="voicemail/vm-press.wav"/>
      <action function="play-file" data="digits/1.wav"/>
      <action function="execute" data="sleep(250)"/>
      <action function="play-file"
          data="voicemail/vm-to_exit.wav"/>
      <action function="play-file"
       data="voicemail/vm-press.wav"/>
      <action function="play-file" data="digits/2.wav"/>
      <action function="execute" data="sleep(250)"/>
    </match>
  </input>
</macro>
<macro name="enter_message_number" pause="100">
  <input pattern="(.*)">
    <match>
      <action function="play-file"
          data="ivr/ivr-please_enter_the.wav"/>
      <action function="play-file" data="ivr/ivr-file.wav"/>
      <action function="play-file" data="ivr/ivr-number.wav"/>
      <action function="execute" data="sleep(250)"/>
      <action function="play-file" data="currency/and.wav"/>
      <action function="play-file"
          data="voicemail/vm-press.wav"/>
      <action function="play-file" data="digits/pound.wav"/>
      <action function="execute" data="sleep(250)"/>
    </match>
  </input>
</macro>
```

6. 保存文件，启动fs_cli, 执行命令reloadxml, 或按F6.

Phrase Macro record_greeting_or_exit 和 enter_message_number现在可以调用。最后创建一个新的脚本，如下举例:

1. 使用编辑器，在freeswitch/ scripts/目录下创建文件record_sound_files.lua:

```
-- record_sound_files.lua
-- Lets user record one or more sound files
-- Sounds are stored in ${sounds_dir}
-- Input Callback to handle digits dialed during the recording
function onInput (s, type, obj)
   if ( type == 'dtmf' ) then
         return "break" -- This ends the recording
   end
end
-- Answer the call
session:answer()
session:sleep(500)
-- Set the path separator
pathsep = '/'
-- Windows users do this instead:
-- pathsep = '\'
-- Set a variable that contains the sound prompt to play
prompt = "ivr" .. pathsep ..
"ivr-please_enter_extension_followed_by_pound.wav"
-- Set a variable that contains the invalid message to play
invalid ="ivr" .. pathsep ..
"ivr-that_was_an_invalid_entry.wav"
-- Set a flag for continuing or exiting
continue = true
-- Specify action when digits are dialed during the recording
session:setInputCallback("onInput","")
-- Initiate while loop
-- Loop continues until caller hangs up or chooses to exit
while(session:ready() and continue) do
   -- First menu:
   -- 1 = Record
   -- 2 = Exit
   digits = session:playAndGetDigits(1, 1, 3, 7000, "#",
"phrase:record_greeting_or_exit", invalid, "\\d{1}")
   if (digits == "2") then
      continue = false
      freeswitch.consoleLog("INFO","Preparing to exit...\n")
   else
      -- Collect message number from caller
```

```
        -- Variable 'digits' will contain the digits collected
        -- Valid input is 3 digits min, 5 digits max
        -- Caller presses # (pound or hash) to finish
        msgnum = session:playAndGetDigits(3, 5, 3, 7000, "#",
"phrase:enter_message_number", invalid, "\\d+")
        -- Read back the message number
        session:execute("say","en number iterated " .. msgnum)
        session:sleep(1000)
        -- New loop: accepted or not
        accepted = false
        while (session:ready() and not accepted) do
           -- Record ile
           session:streamFile("phrase:voicemail_record_message")
           -- Play a "bong" tone prior to recording
           session:streamFile("tone_stream://v=-
7;%(100,0,941.0,1477.0);v=-7;>=2;+=.1;%(1000, 0, 640)")
           filename = session:getVariable('sounds_dir') .. pathsep ..
msgnum .. ".wav"
           session:recordFile(filename,300,100,10)
           -- New loop: Ask caller to listen, accept, or re-record
           listen = true
           while ( session:ready() and listen ) do
              session:streamFile(filename)
              -- Use handy record_file_check macro courtesy of the
voicemail module
              local digits = session:playAndGetDigits(1, 1, 2, 4000,
"#", "phrase:voicemail_record_file_check:1:2:3", invalid,
"\\d{1}")
              if (digits == "1") then
                 listen = true
                 accepted = false
                 session:execute("sleep","500")
              elseif (digits == "2") then
                 listen = false
                 accepted = true
                 -- Let the caller know that the message is saved
                 -- NOTE: you could put these into a Phrase Macro as well
                 session:streamFile("voicemail/vm-message.wav")
                 session:execute("sleep","100")
                 session:execute("say","en number iterated " .. msgnum)
                 session:execute("sleep","100")
                 session:streamFile("voicemail/vm-saved.wav")
                 session:execute("sleep","1500")
              elseif ( digits =="3" ) then
                 listen = false
```

```
                accepted = false
                session:execute("sleep","500")
            end -- if ( digits == "1" )
        end -- while ( listen )
    end -- while ( not accepted )
end -- if ( digits == "2" )
end -- while ( session:ready() )
-- Let's be polite
thankyou = "ivr" .. pathsep .. "ivr-Thank_you.wav"
goodbye = "voicemail" .. pathsep .. "vm-goodbye.wav"
session:sleep(250)
session:streamFile(thankyou)
session:sleep(250)
session:streamFile(goodbye)
-- Hangup
session:hangup()
```

2. 保存文件.

通过拨打号码9913测试这个脚本. 第一个菜单简单播放语音, "To record a greeting, press one; to exit, press two". 按1. 系统将要求用户输入录音信息的号码，然后输入#号键。输入1234#. 下一步，系统提示用户开始录音。启动录音后，然后按任意数字键停止录音。系统将回放这个录音文件，在最后的语音菜单提示中，提供三个选项：收听录音，接受录音和重录。测试每个选项，查看脚本是如何工作的。用户可以进行多次录音。



录音文件保存在默认的语音目录下。查看语音文件，访问fs_cli ，执行命令: eval ${sounds_dir}. 另外注意，recordFile函数在无告警提示的情况下，可以覆盖当前任何语音文件，请小心操作。

让我们检查一下脚本中几个关键的地方，从onInput 开始:

```
function onInput (s, type, obj)
    if ( type == 'dtmf' ) then
        return "break" -- This ends the recording
    end
end
```

这个函数只是简单测试输入的类型，如果用户输入的是DTMF 数字，则函数返回一个"break" 的值。什么时候这个函数会执行？，让我们看看以下代码:

```
session:setInputCallback("onInput","")
```

当呼叫方拨打一个数字时，setInputCallback method 指定一个函数来执行。(注意，这种方法不能应用在执行session:playAndGetDigits, 它有自己本身处理用户输入的处理方法.) 在我们的脚本中，当用户拨打了一个数字以后，函数onInput 将被调用。

这个函数会返回一个 "break"的结果, 这个结果会开始路由，和回放录音。所以，用户不仅仅可以根据用户输入的按键停止录音，而且用户可以跳过各种语音提示。再次拨打9913，用户将听到语音提示, "Record your message at the tone", 按任何数字键跳过这个流程。

注意，我们在主的while 循环语句中嵌入了一对while循环语句。主的while 循环语句会一直执行，直到呼叫方按2 按键以后，这个循环才会退出。中间的while loop 会一直执行，直到变量accepted为true. 里面的while 循环语句会一直执行，直到变量listen 为false. 里面的while 循环语句支持呼叫方收听他或她自己的录音文件，如果对录音不满意，用户可以删除，中间的while 循环语句，可以支持用户执行多次录音。外面一层的while 循环语句可以支持用户录制不同的语音文件。



用户可能注意到，在每一个while 循环语句中，有一个session:ready() 的检查. 这个过程是非常有必要的，当执行中间的while loop 循环语句时，负责处理用户挂机的事件。经验告诉我们，任何时候，如果在拨号规则脚本中使用了while loop 的循环语句，用户应该检查session:ready()的状态. 如果没有处理这个状态，会导致一个僵死的Lua 脚本，因为此时，这个通话实际上已经挂机，但是脚本会一直在等待从这个session 输入的结果。

通过脚本录音的文件格式都保存为 .wav 文件格式. 用户可以根据自己的需要修改成不同的文件格式，例如.ul 或.gsm. 但是，FreeSWITCH 开发人员建议使用.wav 格式，除非系统有什么非常特别的需求，必须修改到需要的文件格式。

还有一行非常重要的代码:

```
session:streamFile("tone_stream://v=-
7;%(100,0,941.0,1477.0);v=-7;>=2;+=.1;%(1000, 0, 640)")
```

以上这一行代码使用了内置的Tone Generation Markup Language (TGML) 创建了一个“bong”语音，开始录音之前，对呼叫方播放这个“bone”语音。FreeSWITCH同样支持用户创建一个回放的语音数组。参考http://wiki.freeswitch.org/wiki/TGML 获得更多信息。

用户可以通过组合播放语音文件，接受呼叫方输入的信息，对呼叫方录音等功能搭建一个自定义的语音系统。



更多关于Lua和session对象的介绍，请参考http://wiki.freeswitch.org/wiki/Mod_lua.

到目前为止，我们通过Lua完成了和XML IVR引擎类似的工作。让我们了解一下使用脚本语言开发的优势。

高级IVR概念

除了一些重要的语言结构例如条件语句和循环语句外，还有其他的一些工具可以利用脚本语言来支持更加高级的功能。一个非常有用的IVR高级功能就是集成第三方的数据库。很多环境下，这可能是一个简单的页面查询功能。在另外一些环境中，系统需要呼叫方的ID 号码，和认证密码等，然后推送这些信息到系统的数据库。让我们看看每个方法的实例。

通过LuaSQL连接数据库

LuaSQL在Lua和DBMS 之间调用了一个简单的接口。 (Kepler project提供的LuaSQL. 更多信息参考http://www.keplerproject.org/luasql/.)



这个部分的实例需要用户有一些数据库的知识背景，并且可以成功编译LuaSQL, 这样可以通过用户数据库连接LuaSQL。编译安装LuaSQL的讨论超过了本书讨论的范围。实例使用环境是在 Debian 6 32 位系统环境下安装的PostgreSQL 9.1.5.

通过如下方法配置数据库:

1. 创建数据库用户名称fsuser和密码 fspass.
2. 创建数据库fsbook.

3. 创建一个表名称为users:

```
CREATE TABLE users (
    name character varying(20),
    pin integer,
    acct integer,
    balance numeric(9,2),
    PRIMARY KEY(acct)
);
```

4. 添加以下数据:

```
INSERT INTO users(name, pin, acct, balance) VALUES('Anthony',
7654,9898, 123.45);
INSERT INTO users(name, pin, acct, balance) VALUES('Michael',
9642,1771, 0.00);
INSERT INTO users(name, pin, acct, balance) VALUES('Darren',
3756,2316, 15.75);
```

测试确认用户可以通过用户名访问数据库；否则Lua脚本将连接失败。

通过以下方法添加一个新的extension。

5. 打开文件01_custom.xml ，然后添加新extension:

```
<extension name="Simple db connection">
   <condition field="destination_number" expression="^(9914)$">
      <action application="lua" data="db_connect.lua"/>
   </condition>
</extension>
```

6. 保存文件，启动fs_cli, 然后执行命令reload_xml, 或者按F6.

现在拨号规则已经可以呼叫Lua脚本db_connect.lua. 这个脚本将演示基本的数据库连接，并且执行一个SQL 查询语句。我们将接受一个从呼叫方输入的帐号和认证密码，查询数据库对应的用户密码，如果验证成功，脚本将读取客户的账户余额。 让我们提供以下方式创建一个我们自己的脚本.

7. 使用编辑器在freeswitch/scripts/目录下创建一个基本db_conect.lua, 添加如下代码:

```
-- db_connect.lua
-- Connects to a database, checks PIN, reads balance

-- Load the LuaSQL
```

```
require "luasql.postgres"


-- A hangup function makes the code a bit cleaner
function hangup_call ()
   session:streamFile("ivr/ivr-Thank_you.wav")
   session:sleep(250)
   session:streamFile("voicemail/vm-goodbye.wav")
   session:hangup()
end


-- Clean up if necessary
function close_db_conn()
   cur:close()
   con:close()
   env:close()
end


-- Create database environment object
env = assert (luasql.postgres())

-- Create database connection object
con = assert (env:connect("fsbook","fsuser","fspass","localhost"))

-- Set invalid entry file
invalid = "ivr/ivr-that_was_an_invalid_entry.wav"

-- Greet caller
session:answer()
session:streamFile("ivr/ivr-hello.wav")


tries = 0
while (session:ready() == true and tries < 3) do
-- Collect account number
   acct = session:playAndGetDigits(3, 5, 3, 7000, "#",
"phrase:enter_message_number", invalid, ".+")


   -- Pull account from database
   cur = assert(con:execute("SELECT * FROM users WHERE acct = '" ..
acct .. "'"))


    -- Get the results, indexed alphanumerically by column names
    row = cur:fetch ({}, "a")


   -- Confirm that we received the record
   if (cur:numrows() == 1) then
      -- We have an account, now collect PIN and check
      tries = 0
      while (tries < 3) do
         pin = session:playAndGetDigits(3, 5, 3, 7000, "#", "ivr/ivr-
please_enter_pin_followed_by_pound.wav", invalid, "\\d+")
```

```
            if (pin == row.pin) then
               bal = row.balance
               user_repeat = true
               while(session:ready() == true and user_repeat == true) do
                  session:streamFile("voicemail/vm-you_have.wav")
                  session:execute("sleep",200)
                  session:execute("say", "en currency pronounced " .. bal)
                  session:execute("sleep",200)
                  digits = session:playAndGetDigits(1,1,3,7000,"#","ivr/
ivr-to_repeat_these_options.wav",invalid,"\\d+") -- repeat y/n
                  freeswitch.consoleLog("INFO","User entered '" .. digits
.. "'\n")
                  if (digits == "1") then
                     user_repeat = true
                  else
                     close_db_conn()
                     hangup_call()
                     break
                  end
               end
            else
               -- Caller entered wrong PIN
               session:streamFile("ivr/ivr-that_was_an_invalid_entry.
wav")
               tries = tries + 1;
            end
         end
         if (tries > 2) then
            -- Too many failed attempts to enter PIN
            session:streamFile("voicemail/vm-abort.wav")
            close_db_conn()
            hangup_call()
            break
         end
      else
         -- We did not find this account
         session:streamFile(invalid)
         tries = tries + 1;
      end
   end -- while (tries < 3)




   if (tries > 2) then
      session:streamFile("voicemail/vm-abort.wav")
      close_db_conn()
      hangup_call()
   end
```

8. 保存文件.

拨打9914测试这个新的extension. 输入四位数的帐号号码，然后按#号键。
在我们的举例中，用户可以输入1771. 输入相应的帐号验证密码，然后按#号.
系统将这些数据库查询，然后读回账户余额。测试不同的无效帐号和密码组合，
观察脚本是如何处理这些错误信息的。

让我们回顾一下脚本中这个重要功能。用户首先注意到，我们使用了一对
函数。这些不是必须要求的，但是它们使得代码的可读性增强。函数 hangup_call
只是执行结束通话。函数 close_db_conn 关闭我们打开的数据库连接。这些函数可能在
脚本的很多流程中执行，通过这样的方式，可以确保用户平滑退出脚本。

数据库连接是通过这样的方式实现的:

```
require "luasql.postgres"
```

这行代码简单加载了luasql.postgres模块. 根据用户使用的数据库不同，可能需要
加载不同的数据库模块，它们可能是luasql.mysql, luasql.odbc, 甚至于连接Oracle
数据库的 luasql.oci8。



当使用MySQL连接器连接LuaSQL时，一些用户可能会发生内存泄漏的情况。如果用户使用MySQL，我们强烈建议使用PDBC连接器来连接。

这行代码创建一个数据库环境连接对象:

```
env = assert (luasql.postgres())
```

这行代码做了更多的工作。它实际上创建了一个连接对象，允许我们和数据库
进行通信:

```
con = assert (env:connect("fsbook","fsuser","fspass", "localhost"))
```

连接数据库的参数如下:

- Database name
- Username
- Password
- 主机名称或IP地址

如果成功连接数据库，那么我们就可以通过创建的对象来执行数据库查询命令.

```
cur = assert(con:execute("SELECT * FROM users WHERE acct = '" .. acct
.. "'"))
```

这一行代码从users 表进行查询记录。查询后返回一个游标。游标代表SQL 查询语句返回的值。在我们的实例中，游标返回查询获得的一行数据:

```
row = cur:fetch ({}, "a")
```

fetch 函数传递了两个参数，一个是Lua 表名称 (可选), 另外一个是 "a" or "n" 表示每行对象的索引:

- "a": 代表字母形式索引。通过列的名称访问。
- "n" (default): 代表数字的索引；通过行中的数字索引访问列。

表名称是可选的一个参数，行数据将填充Lua 表。我们的例子中，我们传递一个空白的大括号表示我们不使用Lua 表。如果没有获得更多查询数据，fetch 函数将返回一行数据或者nil值。注意，一个SELECT 命令可以返回零个，或者1个，或者更多数据。Fetch 函数支持开发人员可以循环获取数据，每一次返回一行数据。在我们的实例中，我们基于acct 子段来查询数据，因为它是表中的主键，所以返回结果或者查询无数据或只有一个数据。现在我们再检查一下我们的返回结果:

```
if (cur:numrows() == 1) then
```

PostgreSQL, MySQL, 和Oracle LuaSQL 驱动都支持numrows() 函数。在我们的举例中，我们想获得仅仅一行数据，这个数据就是通过呼叫方输入查询以后的相应的结果。我们设想一下，如果查询语句没有返回确切的结果，那么可以确定，呼叫方输入了无效的帐号。如果确认输入了正确的账户号码， 我们进一步检查用户是否已输入了正确的验证密码:

```
if (pin == row.pin) then
```

这个检查就是确保用户输入了正确的验证密码。如果验证密码输入错误，系统将对呼叫方提示再次输入密码。如果输入三次错误，脚本将自动退出。



更多数据库连接和游标对象的讨论，可以访问以下网站:
http://www.keplerproject.org/luasql/manual.html

通过LuaSQL连接数据库是相对简单直接的方式。让我们看看使用另外一种连接方法来连接外部数据资源。

通过curl发起页面呼叫

有时，我们需要通过脚本语言发起一个页面呼叫。有很多应用模块的开发可以通过网页应用来实现，例如检查最新新闻，天气的动态信息推送，语音应用等。在我们的实例中，我们开发一个简单的页面呼叫实例，这个实例是从美国海军官方网站获得UTC时间，通过简单解析的方式，返回获得的时间结果。同时我们还要介绍几个新的概念，包括使用freeswitch.API，对Phrase Macro传递参数和使用Lua 字符串修改函数模式匹配和数据提取。

首先，我们必须安装mod_curl， 可以参考第二章安装mod_flite的方式来编译安装。执行以下几步:

1. 在FreeSWITCH 源代码路径下，打开文件modules.conf，找到以下这行:

   ```
   #applications/mod_curl
   ```

   移除#符，然后保存文件.

2. 在conf/autoload_configs 路径下，打开文件modules.conf.xml，找到以下这行:

   ```
   <!-- <load module="mod_curl"/> -->
   ```

   移除 <!-- and --> 标签，保存文件.

3. 在FreeSWITCH 源代码路径下，编译mod_curl 模块:

   ```
   make mod_curl-install
   ```

4. 等待完成编译安装，然后重新启动FreeSWITCH. 启动fs_cli，输入help命令. 如果发现mod_curl 成功加载，用户可以看到curl, 使用语法是:

```
curl,curl url [headers|json] [get|head|post [url_encoded_
data]],curl API,mod_curl
```

下一步，在拨号规则中添加一个新的extension，如下:

1. 打开文件01_custom.xml，添加一个新的extension:

```
<extension name="Web Lookup">
  <condition field="destination_number" expression="^(9915)$">
    <action application="lua" data="web-lookup.lua"/>
  </condition>
</extension>
```

2. 保存文件，启动fs_cli, 执行命令reloadxml, 或者按F6.

现在，我们的拨号规则可以呼叫脚本web-lookup.lua. 让我们参加一个新的Phrase Macro，这个macro 作用是接受参数hh:mm:ss，然后读回这个时间。执行以下几步:

1. 打开文件conf/lang/en/demo/custom-phrases.xml. 添加以下几行代码:

```
<macro name="simple_time" pause="50">
  <input pattern="(\d\d):(\d\d):(\d\d)">
    <match>
      <action function="execute" data="sleep(250)"/>
      <action function="say" data="$1" method="pronounced"
      type="number"/>
      <action function="execute" data="sleep(50)"/>
      <action function="say" data="$2" method="pronounced"
      type="number"/>
      <action function="execute" data="sleep(50)"/>
      <action function="play-file" data="currency/and.wav"/>
      <action function="execute" data="sleep(50)"/>
      <action function="say" data="$3" method="pronounced"
      type="number"/>
      <action function="execute" data="sleep(50)"/>
      <action function="play-file" data="time/seconds.wav"/>
      <action function="execute" data="sleep(250)"/>
    </match>
  </input>
</macro>
```

2. 保存文件，启动fs_cli, 执行命令reloadxml, 或按F6.

这个Phrase Macro simple_time 可以正常工作了。它接受一个参数格式为hh:mm:ss 的时间数据，输入的模式是 (\d\d):(\d\d):(\d\d) ，接收变量值$1, $2, 和$3。这三个变量分别包含小时，分钟和秒。最后，通过以下几步创建一个新的Lua 脚本:

1. 使用编辑器，在freeswitch/scripts/ 目录下，创建一个脚本文件web-lookup.lua:

```
-- web-lookup.lua
-- Makes a curl call to http://tycho.usno.navy.mil/cgi-bin/timer.
pl
-- Extracts time information and reads back to caller

-- Set a variable with the target URL
web_url = "http://tycho.usno.navy.mil/cgi-bin/timer.pl"

-- Number of times we've read time to caller
num_reads = 0

-- Get a FreeSWITCH API object
api = freeswitch.API()

session:answer()

while(session:ready() == true and num_reads < 10) do
   freeswitch.consoleLog("INFO","URL: " .. web_url .. "\n")
   raw_data = api:execute("curl", web_url)
   freeswitch.consoleLog("INFO","Raw data:\n" .. raw_data ..
"\n\n")

   -- Look for line that matches <BR>MMM. dd, hh:mm:ss UTC
   date_time = string.match(raw_data,"<BR>.-UTC",1)
   if (date_time == nil) then
      freeswitch.consoleLog("INFO","UTC date and time not found\n")
   else
      freeswitch.consoleLog("INFO","UTC date and time is '" .. date_
time .. "'\n")

       -- Now parse out the individual elements into smaller strings
       time = string.gsub(date_time,".-(%d+:%d+:%d+).+","%1")
       freeswitch.consoleLog("INFO","Time is '" .. time .. "'\n\n")
       session:streamFile("phrase:simple_time:" .. time)
    end

   num_reads = num_reads + 1
   session:execute("sleep","1000")
end

session:hangup()
```

2. 保存文件.

拨打号码9915，收听语音. 这个脚本将使用FreeSWITCH的curl API来执行一个web 查找。如果呼叫成功，未处理的时间数据将通过simple_time 解析最后提取出来完整的小时，分钟和秒的时间数据，然后对呼叫方播报这个时间。循环执行十次以后，脚本退出。用户可以在任何时间挂机。



当然，必须确认，FreeSWITCH 服务器必须可以进行网络访问，否则，web-lookup.lua 脚本将执行失败.

让我们看看实例中的这个新概念. 注意和这两行代码相关:

```
api = freeswitch.API()

raw_data = api:execute("curl", web_url)
```

第一行代码创建一个FreeSWITCH API ，支持用户通过脚本来发送API命令。(记住，API命令是从FreeSWITCH 发送的命令) 第二行代码实际上执行curl 命令并且捕捉符合的结果。脚本打印出从curl 呼叫返回的数据。这些数据类似于:

```
<!DOCTYPE HTML PUBLIC"-//W3C//DTD HTML 3.2 Final"//EN>
<html>
  <body>
    <TITLE>What time is it"</TITLE>
    <H2> US Naval Observatory Master Clock Time</H2> <H3><PRE>
    <BR>Feb. 28, 06:39:03 UTC Universal Time
    <BR>Feb. 28, 01:39:03 AM EST Eastern Time
    <BR>Feb. 28, 12:39:03 AM CST Central Time
    <BR>Feb. 27, 11:39:03 PM MST Mountain Time
    <BR>Feb. 27, 10:39:03 PM PST Pacific Time
    <BR>Feb. 27, 09:39:03 PM AKST Alaska Time
    <BR>Feb. 27, 08:39:03 PM HAST Hawaii-Aleutian Time
    </PRE></H3><P><A HREF="http://www.usno.navy.mil"> US Naval
    Observatory</A>
  </body>
</html>
```

所有数据是呈现为一串文本字符。以下代码提取出UTC时间:

```
date_time = string.match(raw_data,"<BR>.-UTC",1)
```

date_time 变量现在包含
Feb. 28, 06:39:03 UTC. 这个字符串匹配了在raw_data 字符设置的模式匹配。匹配模式将匹配
.-UTC. 在这个匹配模式中，有两个重要的标签，他们是一个特定的标志和破折号. 其他的字符没有限定。用我们一般的语言描述，这个匹配模式可以读为, "匹配所有以 '
' 开头，任何中间的字符，直到匹配UTC". 破折号代表, "尽可能匹配多个字符".



Lua字符串替换函数在线文档链接
http://www.lua.org/manual/5.1/manual.html#5.4.

现在我们有一个单行的文本, 我们需要提取出小时，分钟和秒:

```
time = string.gsub(date_time,".-(%d+:%d+:%d+).+","%1")
```

这行代码使用string.gsub 函数来执行"匹配和修改" 对指定的字符串进行处理。string.gsub 传递的参数是匹配的字符串，匹配模式和替换的值。在我们的实例中，我们希望从字符串中提取出小时:分钟:秒. string.gsub 函数通过匹配部分或全部输入值，获得结果，然后返一个替换的值。这个方法和其他的语言中的字符串处理机制和匹配模式有所不同。但是这是一个比较有效率的方式。在我们的实例中，我们使用的模式如下:

```
.-(%d+:%d+:%d+).+
```

这个匹配模式从字符串的起始部分开始，匹配尽可能多的字符，直到匹配时间的值。它会“捕捉”时间 (hh:mm:ss), 然后继续执行匹配，直到字符串结束。hh:mm:ss 值为%1. 设置 %1 作为替换的参数，这样string.gsub 函数就会仅返回捕捉的数据，这样的准确信息就是我们想获得的时间。现在我们已经有了时间的信息，通过以下方式，我们可以传递这个时间信息到我们的Phrase Macro中:

```
session:streamFile("phrase:simple_time:" .. time)
```

这个以hh:mm:ss 格式的时间值被传递到Phrase Macro, 在这里匹配输入模式:

```
<input pattern="(\d\d):(\d\d):(\d\d)">
```

小时，分钟，秒的值将捕捉到值$1, $2, 和$3 ，并且对呼叫方播报这些值。除非用户先挂机，否则，这个脚本重复9次，然后挂机。

在一些环境下，非常必要对字符串数据进行编码或解码。作为一个参考，用户在页面呼叫中使用以下方式对URL-encode和URL-decode 字符串进行处理:

```
function urldecode (s)
 return (string.gsub (string.gsub (s, "+",""),
           "%%(%x%x)",
           function (str)
            return string.char (tonumber (str, 16))
            end ))
end



function urlencode (s)
 return (string.gsub (s,"%W",
          function (str)
            return string.format ("%%%02X", string.byte (str))
          end))
end
```

现在看来，比较直接的方法获得数据就是通过数据库连接或页面查找的方式。真正的目的是对异常状态进行处理和基于接收的时间对呼叫发出指令。

Lua模式相对于正则表达式

从技术的角度来说，Lua本身不支持正则表达式。但是Lua 模式匹配语法非常接近于传统的Perl Compatible Regular Expressions (PCRE)。以下列表显示了Lua模式和PCRE等同语法:

Lua Metacharacter	PCRE Metacharacter
.	.
+	+
-	*?
*	*
%	\

以下列表显示Lua 字符集合和PCRE 字符集合等同语法:

Lua Character Class	PCRE Character Class
%d	\d
%w	\w
%s	\s

完整的Lua 模式语法文档，可通过网站此网站获得：
http://www.lua.org/manual/5.1/manual.html#5.4.1.

如果可以结合一些实际经验，熟悉正则表达式的用户将更加快速写出
Lua模式。

脚本开发提示

当在拨号规则中呼叫脚本语言时，有几个地方应该记得:

- 当Lua 脚本完成以后，呼叫自动结束。如果用户希望拨号规则进行
  执行，确认执行session:setAutoHangup(false). 让我们看看以下这段代码:

```
<condition>
      <action application="lua" data="my_script.lua"/>
      <!--the following is not executed unless setAutoHangup is
         false -->
      <action application="transfer" data="$1 XML default"/>
</condition>
```

- 运行命令结束是一个恰当的方法来保证脚本退出。
  就像在以前出版的书中讲述的，没有一个非常明确的命令来退出脚本运行；
  但是用户可以使用 函数error() 来强制脚本结束运行。
  如果已经设置了session:setAutoHangup(false)，拨号规则将会继续执行。

- 记住，和上一个提示一样，注意调用session:bridge()或者 session:transfer()，
  他们可能不能完全按照用户希望的那样去运行。直到脚本退出以后，bridge
  或transfer的指令才可能执行。让我们看看以下这段代码:

```
freeswitch.consoleLog("INFO","Before transfer...\n")
session:transfer("9664 XML default")
freeswitch.consoleLog("INFO","After transfer...\n")
```

transfer 不会执行转接命令，直到第二个consoleLog call 和剩余的代码执行以后，转接从开始执行。如果用户计划使用bridge 或者transfer 指令，确认脚本逻辑结束后执行这些指令。

- 不要使用Lua (或其他脚本语言) 来替代拨号规则. XML拨号规则可以非常有效率地路由呼叫，其他的脚本语言是无法和它相比的。Lua的用处在于，使用Lua语言和呼叫方互动或处理一些函数调用，通常这些函数在拨号规则中处理比较复杂。黄金法则是，用户如果可以在拨号规则中完成这个任务，那么用户就应该使用拨号规则。

- 不要在拨号规则中过度使用脚本。如果用户开发一个复杂脚本控制呼叫，处理，内联计费，第三方呼叫控制等等。这样的场景需要使用事件套接字来实现。我们将在第十章, 通过外部控制机制控制FreeSWITCH，更多详细介绍使用神奇的事件套接字来实现控制机制。

总结

Lua是一种比较好选择开发简洁的语音系统，支持和呼叫方进行互动。它是轻量级的，而且具有良好的拓展性。Lua 语法简单易学，社区提供了很多在线文档。

在本章节，我们取得了几个目标:

- 熟悉了Lua基本语法和控制结构
- 写了几个脚本演示如何和呼叫方进行互动，包括应答，挂机，播放语音和播放Phrase Macros, 和接受呼叫方的输入

- 学习了如何通过freeswitch 对象发送日志信息到控制台，如何执行API命令

- 安装了LuaSQL，演示了如何通过Lua脚本连接PostgreSQL 数据库

- 编译安装了mod_curl，设置为默认加载模块
- 演示了使用curl请求，通过Lua脚本执行页面呼叫

- 了解了Lua匹配模式语法

到现在为止，我们具备了基本的知识可以编写一个脚本和呼叫方进行互动，同样也是一个重新回味拨号规则的时间了。接下来的章节我们要回顾几个第五章 理解XML拨号规则中介绍过的概念，把用户对拨号规则的理解提升到一个全新的高度。

8

高级拨号规则概念

在上面的章节中，我们已经学习了FreeSWITCH XML配置文件的强大。并且用户学习了拨号规则的入口和使用XML来设置基本的配置。在这一章中，我们继续进一步研究拨号规则的基本结构，XML拨号规则的功能，如何通过基本的功能实现复杂的结果。

一些词条可能在这一章中重复出现，但是我们还要介绍一些已经在以前章节涉及的一下基本内容，包括基本的变化规则功能，并且我们还要解释拨号规则系统如何工作，为什么这样工作。通常情况下，很多用户使用了FreeSWITCH的XML 拨号规则，但是没有真正了解其中的原理，导致系统不能拓展或无法排查复杂的问题。这一章节的目的是让用户成为一个FreeSWITCH专家，能够了解他们开发的拨号规则是如何工作的，为什么这样工作。

在这个章节中，我们假设用户已经有了FreeSWITCH的基本知识，对路由和处理呼叫和XML配置文件有了基本的了解。如果用户安装配置了FreeSWITCH的演示系统，并且测试了一些基本的呼叫，那么对用户在这一章节的学习会非常有帮助。

在这一章节，我们将讨论以下内容:

- 拨号规则总览
- 基本拨号规则概念
- 解析和执行
- XML 拨号规则模块
- XML 拨号规则预处理
- 使用变量
- 通过正则表达式测试变量
  传递变量到其他的legs

- 拨号规则中的Macros
- 避免陷阱
- 多分机设置
- XML extensions 特别属性
- XML的可选方法

拨号规则总览

FreeSWITCH的拨号引擎是一款相当灵活的软件. 如果用户有使用软交换的背景，用户可能了解一些拨号规则的概念。用户通过系统本身的语言预设了这些流程，规定了一些静态的逻辑语句来执行一些指令 (他们是 应答呼叫，播放文件，采集拨号数字和抓接电话）。如果任何功能不能通过软交换预设的命令或有效逻辑语句来实现，那么就没有其他的办法可以实现这个任务。

在FreeSWITCH环境中, 拨号规则的处理是通过可加载的模块来完成。当处理呼叫时，模块的逻辑可以被调用来完成这些任务，同时用户可以根据自己的业务需求，加载多个模块来支持不同的处理方式。这是FreeSWITCH区别于其他软交换的地方，也是经常被忽略的地方。通过拨号规则处理的模块化设计，创建了一种非常自由灵活的录音呼叫方式。用户可以开发自己的模块或调用可选的模块来处理用户自己的拨号规则，并且为用户自己的拨号规则创建一套新的命令集。和其他的软交换系统相比，FreeSWITCH可以支持用户通过外部的脚本语言来处理自己的拨号规则。FreeSWITCH所有模块都是使用C语言编写，所以集成方便，并且允许用户使用内部的API 接口和链接库（如有必要）无需调用外部的脚本语言。这样的话，在处理呼叫时，FreeSWITCH将占用非常少的系统资源。

为什么拨号规则处理实行模块化设计呢? 首先需要理解的是为什么我们需要一个拨号规则。

让我们现在忘记程序语言，回顾一下软交换的原理。如果我们分解一下大部分的语音系统，我们会发现每个通话都按照一个逻辑流程图来执行。事实上，如果我们问客户关于电话业务的需求，他们经常也会回答是或否，或者指定一个相应的执行指令。客户的业务需求就可以转换成一个基本的呼叫流程图。无论用户正在做什么，如果用户通过图例的形式来表达整个业务需求的过程时，其实用户已经开始设计拨号规则了。本质上说，用户已经在构想一个拨号规则的模块来满足业务需求。



让我们拿一个普通的呼叫流程作为举例，然后分拆成更小的部分. 进一步观察这个举例，我们可以看到在这个流程图中，我们设置了很多逻辑假设，拨号规则必须处理这些假设的逻辑语句。例如，用户为了对是否是工作时间这个假设做一个判断，用户必须对比工作时间或下班时间。为了判断用户是否输入了按键1，拨号规则处理机制必须支持解析按键式电话机输入。基于条件对比，用户可以让系统执行相应的命令-转接电话，播放语音，挂机，等等。 所有这些要求会组成一些逻辑命令，拨号规则会使用这些命令执行电话流程。在很多系统环境中，通过编写一些晦涩难以理解的配置代码来设置一个逻辑判断，这样的方式具有很多局限性，也可能让用户发疯。在FreeSWITCH环境下，用户可以通过自己熟悉的语言来编写这些逻辑。

回顾一下语音系统的历史，没有拨号规则命令解析的系统已经完美的系统。从一代又一代的语音系统开发中发现，在呼叫处理的方式上，人们都有各自不同的命令结构。前瞻性的FreeSWITCH开发人员了解了这个趋势，所以决定通过模块化的设计来管理拨号规则处理。

从技术的角度来说，可加载的拨号规则模块设计有很多的优势。
首先，交换系统可以通过链接支持不同的库(例如SQL libraries, YAML libraries, CURL HTTP libraries)，软交换可以获得相应的返回结果，FreeSWITCH可以根据此返回结果来处理呼叫。其次，拨号规则的处理模块可以依赖于第三方事件驱动的系统，例如可以支持用户从远端的web服务器加载呼叫命令。

更加突出的优势是用户可以修改FreeSWITCH的C 语言语句，满足自己的业务要求，在加载的模块中自定义逻辑流程，非常清晰地路由呼叫。这样做的好处是用户可以设计自己的呼叫处理路由，而不依赖于FreeSWITCH内置的拨号规则。用户可以通过FreeSWITCH开放的API接口来开发自己的路由规则。例如，用户可以通过数据库查询呼叫用户状态，如果呼叫用户需要一个代理服务，就直接触发一个FreeSWITCH API启用代理，这个流程修改几行代码就可以实现。这样的方式具有相当大的优势，用户可以拥有更大的灵活性。用户无需再通过第三方处理机制或者在拨号规则中启动一个线程来协助系统工作(例如调用脚本语言或PHP脚本从数据库查询一个简单的true/false结果)。通过这样灵活的处理机制，FreeSWITCH 可以处理更多的呼叫路数.



一个通常被误解的说法是，FreeSWITCH拨号规则是基于XML，要求XML。这样说并不对。如果用户愿意使用一般文件，用户可以使用它来保存拨号规则配置。如果用户愿意使用YAML文件，当然用户也可以使用YAML。用户需要加载自定义的C语言模块，并且模块可以正确解析相应配置文件中定义的逻辑，确保FreeSWITCH调用。

另外，在FreeSWITCH中，最常用的（目前最健壮的）拨号规则处理机制仍然是基于XML拨号规则模块。大部分默认安装的实例或者网上传播的实例都是XML文件。因此，在这一章节仍然需要讨论这些XML文件。创建自定义的C语言模块已经超出了本书讨论的范围，但是对用户了解模块功能是非常重要的。当用户使用了更多的FreeSWITCH 高级功能以后，用户可能发现内置的XML拨号规则处理机制不能完全满足用户的所有需求，用户应该清楚，用户需求不仅仅局限于使用XML来完成! 有很多途径可以实现呼叫路由功能。

在我们深入探讨XML 拨号规则之前，让我们从总体上来回顾一下基本的拨号规则概念，并且进一步拓展这些基本的概念。

拨号规则基本概念

首先，让我们简单回顾一下第五章学习的内容，理解XML拨号规则。通常说，拨号规则就是帮助系统生成一系列的命令，呼叫方可以呼叫的人或者一个群组。拨号规则模块的强大之处在于支持了决策处理机制。拨号规则模块可以支持任何路由流程。每个呼叫都包含以下三个同样的问题:

- Contexts: 哪里查询到允许当前呼叫接通的一系列目的地或者 (功能)?
- Conditions: 呼叫方正在接通的对象是什么?
- Actions: 采取什么样的命令接通这个对象?

通过分拆路由决策基本上可以回答以上三个问题，路由决策可以通过三个概念来做进一步解释。它们是：呼叫方 context, 条件匹配，和指令. 这些概念对FreeSWITCH来说并不是非常不是独特的。我们将在本部分的内容中对每个概念做逐一解释。

拨号规则决策的最后结果都是一系列的指令命令。每个拨号规则模块必须符合一系列的执行结果，依次执行相关的模块，直到呼叫方A成功接通呼叫方B。

Contexts

当我们谈到context，实际上我们谈论的是呼叫方接通的一个单个目的地的列表。这些目的地或者部门组织不一定是个体的用户；他们可能是一个组或其他可互动功能（例如，语音邮箱功能），但是最终，呼叫方肯定是接通某一个电话或者执行其他的功能。一个 context 就是一些规则的集合，这些规则来决定呼叫方应该接通哪个分机，是否允许接通目的地号码。我们称这些“规则”为extensions。

用户可以认为呼叫方的context 是一组逻辑描述，这些逻辑描述用来定义被呼叫的目的地集合。在实例中，被经常使用的contexts 是"internal" 和"public" contexts. "internal" context 通常是指发起呼叫的是内部用户，或者在软交换（一个公司内部的呼叫）这些用户可以拨打一个四位数的号码来相互呼叫，或拨打带9的前缀呼出到外部号码。"public" context 通常代表外部的用户呼入到软交换内部，然后接通系统的仅小部分目的地号码，例如公司内部的员工桌面电话，但是不能直接接通酒店大堂电话。

为什么设置一个通话组？为什么不仅仅设置内部号码和外部号码? 大部分公司需要更加灵活的业务流程。例如，让我们看一个普通酒店的电话系统流程。我们我们从酒店整个用户用例来看，我们需要分解contexts—内部员工，客人，和外部用户三个部分。为了讨论方便，在用例中各自取名为 "staff","guests", 和 "external".

对于外部用户 "external", 把从外部呼入的用户设置这个组的理由非常简单-我们想让外部呼入的电话接入的前台接线员号码，其中一个主要的号码可以呼叫酒店餐厅。而且，我们不希望外部的呼叫可以直接呼入到酒店的客人房间，或者使用酒店电话系统的内部功能例如，酒店对客人提供的叫醒服务。因此，我们设置外部呼入的号码为context "external" ，并且路由外部呼入到这个context 做进一步处理。

另一方面，对酒店的客人来说，他们可以呼叫另外房间的号码，使用酒店的叫醒服务，呼叫前台接线员。因此，这一组用户的功能区别于外部 "external" 呼叫用户—所以他们设置的context 有自己的目的地。

最后，酒店员工有自己特别的功能要求，例如可以标注某位客人的号码为 "checked out" 已结帐客人，当此房间为空房间的时候，可以限制呼入到此房间。(为了安全，防止盗打电话). 这些功能房间客人和外部用户是不能使用的，只有酒店员工可以使用这些功能，所以员工必须设置自己的context。

可以看到，我们创建的酒店电话系统的逻辑应该是这样的:

		Caller (context)		
		Internal Guest	Internal Staff	External Caller
Calling To	Front Desk	√	√	√
(Destination)	Restaurant	√	√	√
	Hotel Rooms	√	√	X
	Wake-up Call	√	X	X
	Set checked-out flag	X	√	X

现在我们看一下对contexts的需求。上面的列表显示每个类型的用户都限制了呼叫目的地的类型，对应支持的呼叫。

特别注意，context的概念不是为了呼叫一个指定的目标用户，而是为了划归一个号码组，用户拨打这个号码组来接通目标用户。我们实际上接通的目标和指定的号码是由condition来负责处理。我们将在接下来的部分直接介绍condition。

Conditions

一旦系统已经决定了列表中允许被呼叫的用户，系统必须清楚知道哪个用户号码被拨打，如何接通他们之间的通话。这个过程是通过conditions来完成的.

Conditions是一个或多个逻辑描述，来决定这个呼叫应该路由到什么地方。通常通过触发一个呼叫信息对比（例如，拨打的号码，或者来电显示），配合一些设定的规则。这些信息通过拨号规则的变量获得，匹配正则表达式，对字符串或者其他变量做匹配检测。

Conditions 是最常用的一种方式来匹配用户拨打的号码和指定的目的地号码，这个号码映射到一个指定的电话号码。例如，我们可能测试一下，看看一个人拨打了号码，415-886-7949. 拨打号码以后，系统提供一系列的指令连接用户分机7949。

有时候目的地匹配检测来判断一些字段，而不是拨打的号码。例如，用户可能检查呼叫方的号码，如果是电话营销的来电，用户可能播放一个忙音信号。系统可以支持字段和其他值的组合来判断是否发送通话。甚至于，用户可以检查是否是技术设置或基于数据库驱动的设置，例如是否有NAT设置，或者存储在数据库表中的呼叫前转入口。

在拨号规则中，同一个呼叫可以通过多个conditions 匹配来触发多个命令指令。实际的例子中，当呼叫方呼叫某些用户时，这些用户挂机以后，系统根据设置的特定区号，对这些用户播放问卷调查。根据拨号规则处理机制的的不同，多Conditions 可以通过不同的方式执行。

通过我们对酒店电话系统的讨论，用户通过三个contexts，每个context 检查不同的condition来决定目的地号码，最后的设置如下：

"Internal Guest" Context	"Internal Staff" Context	"External" Context
Did they dial 0?	Did they dial 0?	
–Go to front desk	–Go to front desk	
Did they dial 2929?	Did they dial 2929?	Did they dial 646-222-2929?
–Go to restaurant	–Go to restaurant	– Go to restaurant
Did they dial 3000-3999?	Did they dial 3000-3999?	
–Ring room 3000-3999	–Ring room 3000-3999	
Did they dial *5?		
–Go to wake-up call		
	Did they dial *6?	
	–Set checked out flag	

默认环境下，大部分的用例中，拨号规则会一直处理，直到发现匹配结果。查看以下的Extensions 获得更多结果。

Actions

Actions 是当系统找到匹配的condition后，所采取的步骤. 在这里，拨号规则会生成一个命令列表，软交换根据这个列表执行任务，完成呼叫方和被呼叫方的连接。命令Actions 包括 "answering（应答）", "bridging（桥接）", "routing to Voicemail（路由到语音邮箱）", 等等。

这里非常重要的一点是，拨号规则模块只是创建一个需要执行的命令列表—实际上，它并不实时执行这些命令！这里包括了一些异常处理的规则，但是一般情况下，拨号规则的结构体会挂载一些临时性的模块，这些模块不会负担很大工作量，实际上也连接了这个呼叫，执行相应的actions.

为了进一步解释这些，用户可以把拨号规则看作是一个执行列表，这个列表根据需求来配合呼叫工作。这个列表有以下命令组成:

- Answer 应答呼叫
- Play welcome file 播放语音
- Transfer to user "John Doe" 转接电话
- Hang up 挂机

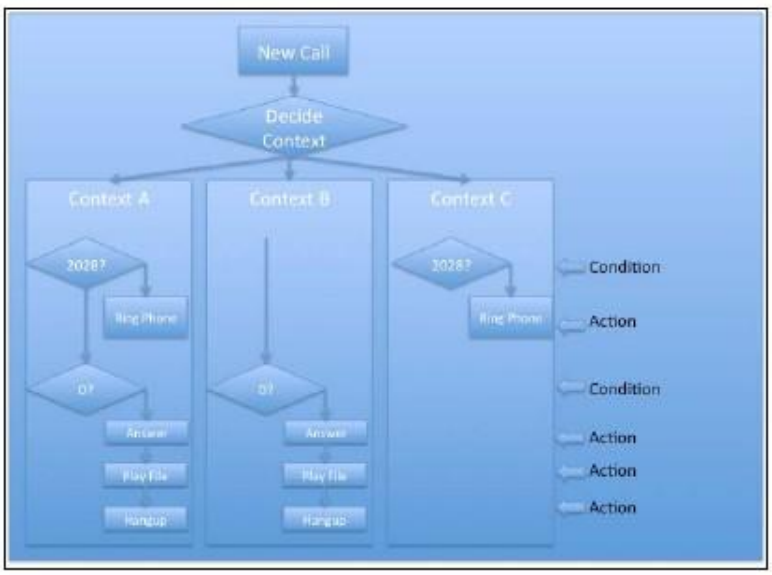
FreeSWITCH 将执行这些命令。

拨号规则的目的不是在实际的呼叫流程中和用户互动。如果用户需要在呼叫刚才中支持完整的互动交互，建议使用脚本语言连接FreeSWITCH。用户可以学习第七章， 拨号规则脚本语言Lua.

组成一套系统

FreeSWITCH 实际上处理所有的contexts, extensions, conditions, 和actions都是在路由阶段。每个呼叫都要经过ROUTE 状态。路由状态当FreeSWITCH 传递呼叫控制到在拨号规则中，通过以上四种概念来创建一个命令列表。

处理过程类似于这样的形式:



命令结果的列表最后返回到FreeSWITCH，例如:

- EXECUTE answer　执行应答
- EXECUTE and play the file.wav　执行播放语音
- EXECUTE hangup　执行挂机



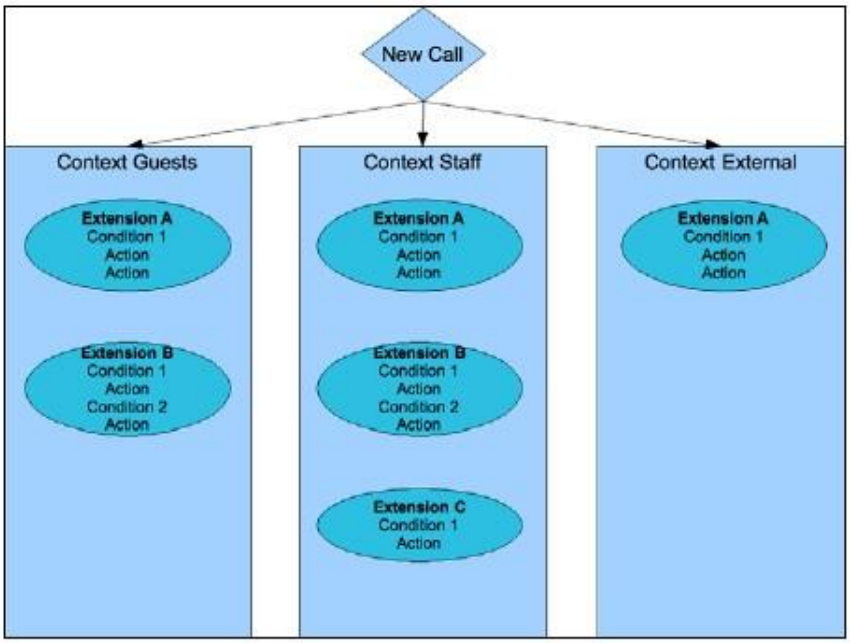
记住，在FreeSWITCH 社区通常说 Dialplan "phases"—其实引用了处理的两个阶段- ROUTE 阶段和EXECUTE阶段. 有时候人们说 ROUTE 阶段，事实上用"parsing" 来描述路由阶段任务不是一个非常专业的用词。我们也使用"hunting" 作为ROUTE 阶段的同义词。ROUTE, parsing, 和 hunting 在这里都有同样的意思

充分理解两阶段处理非常有助于掌握XML的工作流程。

XML拨号规则模块回顾

就像我们在第五章讨论 理解XML 拨号规则中讨论的一样，人们最喜欢使用的办法来设置FreeSWITCHXML就是拨号规则模块。到本书写作时，它仍然是最健壮的模块。它支持extensions列表，每个extension 包含一个或多个conditions, 并且每个condition 包含多个需要执行的命令指令。

让我们重新回顾一下这些概念，确保用户熟悉这些概念。
查询和处理拨号规则入口依赖于呼叫流程的布局设计，它的流程类似于一个多维树结构。



快速浏览了拨号规则的结构，和使用方法后，我们可以清楚看到为什么XML
适用于创建拨号规则。XML嵌套的属性可以完美支持上面提示的实例。
FreeSWITCH依赖于拨号规则中的配置树选项，XML是本质上说应该无限制的
树结构文件，可以在每一个树的叶子节点设置相应的值。另外，用户可以在任何
时候添加自定义的标签和属性—甚至于一些属性可能被FreeSWITCH忽略，但是当用户
软件学院读写XML文件时，可能这个属性会非常有用。

到目前为止，用户应该熟悉了contexts, extensions, conditions和actions. 让我们再详细
学习一下XML拨号规则来看看如何实现不同的拨号规则功能。

Extensions

Extensions是简单的XML容器，用来对conditions和actions 进行分组归类。
注意，名称"extensions" 可能会误导用户。大部分的用户使用 extension作为
一个分机号码，用户可以拨打某些分机号码来接通这些内部号码，例如2900 或3000,
但是在FreeSWITCH 环境中，extensions 的含义完全和你拨打的号码没有任何关联。
他们仅是拨号规则context的一个组成部分。例如，如果想接通某人，用户可以拨打
2900，实际上，用户可能呼叫了一个extension，用户名为darren, 类似于这样:

```
<extension name="darren">
  <condition field="destination_number" expression="^2900$">
    <action application="bridge" data="user/darren"/>
  </condition>
</extension>
```

在这个例子中，extension被定义为darren， 感谢condition的工作，拨打2900可以
接通用户darren。

通过修改设置，extensions 可以支持很多工作方式。默认环境下，extension 发现匹配的
Conditions时，FreeSWITCH 停止查询其他的extensions. 例如，如果有一个extension，这个
Extension 修改了用户的 Caller ID, 接下来执行了路由此呼叫， 默认环境下，FreeSWITCH
将不再查询第二个extension。 例如以下实例:

```
<extension name="set_callerid">
  <condition field="destination_number" expression="^2900$">
    <action application="set"
      data="effective_caller_id_number=4158867900"/>
  </condition>
</extension>
<extension name="darren">
  <condition field="destination_number" expression="^2900$">
    <action application="bridge" data="user/darren"/>
  </condition>
</extension>
```

在上面的例子中，第二个extension 从来不会执行。如果用户要求成功匹配以后，继续
执行后续的extensions，用户必须设置continue 标志。如果修改了上面的举例，拨号规
则就会安装用户要求的来执行:

```
<extension name="set_callerid" continue="true">
  <condition field="destination_number" expression="^2900$">
    <action application="set"
      data="effective_caller_id_number=4158867900"/>
  </condition>
</extension>
<extension name="darren">
  <condition field="destination_number" expression="^2900$">
    <action application="bridge" data="user/darren"/>
  </condition>
</extension>
```

注意，我们已经添加了continue="true"，表示如果第一个extension成功匹配，拨号规则将继续执行。我们没有对第二个extension 设置continue，因为我们不想在第二个匹配完成以后（第二个实际上接通了呼叫），继续执行。

Conditions

Conditions 通过正则表达式对变量进行匹配。Conditions 存在于extension 标签内。在这一章的后续章节，我们将讨论各种变量类型。

Conditions 可以测试一个或多个表达式。默认环境下和大部分基本的层面上，用户可以设置一个或多个condition，当所有condition被判断为true的情况下，一个组命令会执行。

让我们看看这段代码:

```
<extension name="test_two_things">
  <condition field="network_addr" expression="^192\.168\.1\.1$"/>
  <condition field="destination_number" expression="^2900$">
    <action application="playback" data="i-know-you.wav"/>
  </condition>
</extension>
```

在这个例子中，使用了两个 conditions 语句。一定要注意，第一个condition 以 /> 为结束标签. 在第一个condition中没有包含任何 actions 命令，但是它仍然需要判断，如果第一个condition 判断失败，将跳过整个extension。但是，如果通过了所有的conditions，代表用户正在拨打2900，用户IP地址是192.168.1.1， action(s) 就开始执行。这样，非常方便地在两个condition之间创建了一个AND 逻辑运算-必须两个条件必须同时匹配才能执行我们设置的playback action 指令。

这是如何工作的？秘密在于break 属性，也称之为break 标志符。每个condition 描述都带一个break标志符，当expression验证后，这个标志符来决定需要要执行的下一个指令。Break 标志符默认的值是on-false，表示只要在验证过程中出现一个false或者出现协商结果为负，我们将跳出执行流程，或停止处理，condition和extension。

break 参数支持以下四种设置:

- on-false: 如果第一个匹配失败，停止查询condition (默认设置)
- on-true: 如果第一个匹配成功，停止查询condition
- always: 无论是否匹配，在此condition停止执行
- never: 网络是否匹配，继续查询

用户可以使用condition创建一个伪 if/then/else 处理过程。让我们检查一下属性之一： never 标志。

现在我们创建一个extension，处理这个呼叫，仅允许这个呼叫来自于特定的IP地址，并且检查是否拨打的号码是*72 或*73。用户可以分别通过两个extensions来完成，例如：

```
<extension name="extension_72">
  <condition field="network_addr" expression="^192\.168\.1\.1$"/>
  <condition field="destination_number" expression="^\*72$">
    <action application="playback" data="forward-on.wav"/>
  </condition>
</extension>
<extension name="extension_73">
  <condition field="network_addr" expression="^192\.168\.1\.1$"/>
  <condition field="destination_number" expression="^\*73$">
    <action application="playback" data="forward-off.wav"/>
  </condition>
</extension>
```

用户也可以通过使用break标志来编写单个extension实现，例如:

```
<extension name="extension_72_or_73">
  <condition field="network_addr"
    expression="^192\.168\.1\.1$"/>
  <condition field="destination_number" expression="^\*72$"
    break="on-true">
    <action application="playback" data="forward-on.wav"/>
  </condition>
```

```
    <condition field="destination_number" expression="^\*73$">
      <action application="playback" data="forward-off.wav"/>
    </condition>
  </extension>
```

像用户看到的那样，在第一个condition 添加了break="on-true"，只有当Condition判断为True时，我们停止处理。否则，继续处理。可以把它看作一个 if/then/else-if 语句; 如果第一个condition匹配成功，运行这个语句，然后停止执行，如果第二个condition匹配，则执行第二个condition。

另外一个举例，考虑一下使用一个if/then condition 语句，然后再接一个if/then语句。用户可以使用break=“never”来模拟出这个逻辑。当失败condition中的action命令不能添加到执行列表中，后续的condition仍然继续执行。看看以下这个例子，我们检查同样extension中两个不同的网络参数:

```
<extension name="decide_routing">
  <condition field="network_addr"
    expression="^192\.168\.1\.1$"
    break="never">
    <action application="set" data="inhouse=true"/>
  </condition>
  <condition field="source" expression="PortAudio"/>
    <action application="set" data="portaudio=true"/>
  </condition>
</extension>
```

以上XML测试，如果呼入的用户的IP地址是192.168.1.1，则设置一个变量if true。XML将继续执行所有的环境要求，到第二个condition 时，测试用户是否使用了PortAudio，并且设置变量if true。这个过程代表了一个非常高效的if/then语句然后紧接着另外一个if/then语句。

每个condition必须在每个extension标签中，虽然用户想设置condition总是为true。以下这段代码是无效的:

```
<extension name="set_callerid">
  <action application="set"
    data="effective_callerid_number=4158867900"/>
</extension>
```

这里不存在condition标签，所以将忽略这个extension，相应的action指令从未执行。

特别condition变量

在大部分的conditions中，用户可能有时候使用一些变量和expressions，用户可以使用一些特别的变量来处理一些condition，使得这个过程更加简单和灵活。

以下变量可以作为XML condition标签的filed 属性:

- context: 当前拨号规则的context
- rdnis: 重定位号码。上一个呼叫的重转号码。
- destination_number: 拨打的号码；呼叫接通号码。
- dialplan: 被使用的拨号规则模块名称 (XML或YAML)
- caller_id_name: 呼叫方名称
- caller_id_number: 呼叫方号码
- ani: 被叫自动号码识别码
- aniii: 发起呼叫设备类型（例如，付费电话）
- uuid: 当前呼叫唯一识别码
- source: 被呼叫模块名称 (例如, PortAudio)
- chan_name: 当前通道名称 (例如, PortAudio/1234)
- network_addr: 信令源IP地址

简单实例如下:

```
<condition field="network_add" expression="^1\.2\.3\.4$">
<condition field="caller_id_number" expression="^1[01]\d\d"/>
```

以下值可以作为一个属性在condition 标签中直接使用:

- year: 表示日历年, 0-9999
- yday: 表示一年中的日, 1-366
- mon: 表示月, 1-12 (例如, January 等同于1)
- mday: 表示一个月的日, 1-31
- week: 表示一年中的星期, 1-53
- mweek: 表示一月中的星期, 1-6
- wday: 表示一周中的日, 1-7 (例如, Sunday 等同于 1，Monday 等同于2)

- hour: 表示小时, 0-23
- minute: 表示分钟, 0-59
- minute-of-day: 表示一天中的分钟, (1-1440) (例如, 午夜等同于, 1 A.M. 等同于60, 和中午等同于720)

这些condition 属性可以这样使用:

```
<extension name="holiday_example" continue="true">
  <condition mday="1" mon="1">
    <action application="set"
      data="open=false" inline="true"/>
  </condition>
</extension>
```

这个例子中，在新年那一天，设置了变量open 为false。注意，condition 这一行，同时使用了mday 和mon condition 变量。一个简单的天/时间检测例子如下:

```
<extension name="holiday_example" continue="true">
  <condition wday="2-6" hour="8-16">
    <action application="set" data="open=true" inline="true"/>
  </condition>
</extension>
```

这个例子中，如果在上午8到下午5点之间有电话呼入，通道变量open则为"true"。

Inline execution

在最初FreeSWITCH 设计中，FreeSWITCH 禁止在routing/Dialplan阶段产生Actions。这样做有其他方面的影响，可预知的副作用是，FreeSWITCH不鼓励用户在呼叫流程中修改或替换一些变量。相反，这些任务可以在嵌入的脚本语言中执行。道理相当简单—与其在拨号规则中设计一些可能隐秘的，有限的命令来验证变量和创建逻辑，不如让FreeSWITCH 连接脚本语言，例如Perl 或Lua来完成这些工作，因为这些脚本语言本身支持强大的的逻辑运算能力（并且文档丰富），没有必要再FreeSWITCH环境中设计另外一套这样的逻辑处理结构。这样的设计逻辑，不仅仅拓展了很多功能开发的可能性，同时避免再次增加对开发人员的培训成本，要求他们了解XML处理机制的来龙去脉。

但是，随着FreeSWITCH的发展，越来越多的用户希望在原生的XML 拨号规则语言中管理呼叫流程处理。一个最基本的，最为关注的功能就是在拨号规则路由阶段可以支持设置一个变量，然后后续的执行中测试这个设置的值（或者覆盖）。对核心开发人员提出了很多的需求，在XML 拨号规则处理中添加了inline标志。

inline 内联标志可以支持在拨号规则阶段，执行一些命令 (不是所有命令)来结束以前声明的许多规则。不建议使用它，内联有时候可以帮助开发人员实现一些额外的功能或只是当阅读代码时更为自然一些，增加可阅读性。

以下是一段代码实例，来我们看看为什么使用inline execution:

```
<extension name="check_for_user" continue="true">
  <condition field="${callerid}" expression="2035551212">
    <action application="set" data="user=yes"/>
  </condition>
</extension>
<extension name="route_users_only">
  <condition field="${user}" expression="yes">
    <action application="answer"/>
    <action application="playback" data="tada.wav" />
    <action application="hangup"/>
  </condition>
</extension>
```

以上的代码定义了两个extensions，这两个extension互相存在依赖关系
—第一个extension 设置变量user 为"yes"，如果有人从 203-555-1212呼叫，
第二段代码应该路由所有的呼叫，并且设置用户变量来播放tada.wav语音文件.
这段代码不能完全那样工作。让我们看看为什么不能那样工作。

回忆一下，condition 条件检测验证是在action 执行之前进行的。 在上面的举例中，set 是一个action 应用程序. 因此它不会执行，直到所有的conditions都被检测验证。这里表示，检查变量${user}的condition将总是失败，因为设置变量的代码已经执行，因此，这里需要添加一个inline 内联处理。

如果XML action 修改成这样去读:

```
<action application="set" data="user=yes" inline="true"/>
```

只要遇到set，XML Dialplan处理器将执行set 应用模块。现在这个XML代码就可以按照我们的思路工作了。

为了避免滥用这个功能和其他技术方面的原因，inline 内联标志仅使用在一些快速设置一些变量，获得变量的应用中。同时Inline函数确保不能访问或者修改当前会话状态。

以下是inline 内联处理机制支持的action列表，包括:

- check_acl: 检查访问控制列表
- eval: 执行内部API 或对控制台打印文本日志
- event: 发起任意事件
- export: 对通道设置一个变量，用来支持 B legs/transfer

- log: 手动创建一个日志入口
- presence: 发送PRESENCE_IN和PRESENCE_OUT事件
- set: 对通道设置一个变量
- set_global: 设置一个全局变量
- set_profile_var: 设置一个将要使用的用户 profile
- set_user: 设置当前用户，并且添加用户通道变量到活动的leg

- sleep: 暂停处理
- unset: 删除（恢复）变量设置
- verbose_events: 在发送的事件中打印信息
- cidlookup: 通过CNAM 查找设置caller_id_name 字段
- curl: 启动一个HTTP请求，接收回复
- easyroute: 路由到database-driven DID 路由引擎
- enum: 执行一个enum 查找或相关服务
- lcr: 设置最少资费路由
- nibblebill: 对话务账单以每分钟基础计费
- odbc_query: 手动执行ODBC 查询

以上命令的详细文档可在FreeSWITCH wiki获得.

Actions和anti-actions

另外一种控制呼叫的机制是使用action 和 anti-action 标签。不像其他的conditions, 这些标签支持一个失效转移的机制，如果一个condition 匹配失败，可以重新设置到另外的actions。这是另外一种版本的if/then/else condition，但是相对来说，容易阅读和管理，特别是在同一个condition 环境下，可以支持依次执行多个actions。

让我们看看以下例子，它是如何工作的:

```
<condition field="${inhouse}"
 expression="true" break="never">
   <action application="log" data="This is an in-house call"/>
   <anti-action application="log" data="Not in-house call"/>
</condition>
```

上面例子中，如果变量${inhouse} 为true，日志打印结果输出是 "This is an in-house call" ，否则它将输出在log中输出"Not in-house call"。这样的方式比较容易阅读，不像 if/then/else 逻辑运算那样分成两个condition 语句来实现以上结果。

regex 操作符

有时候需要创建更为复杂的逻辑结构。例如，可以简单测试 destination_number 表达式是 1000 或1001:

```
<condition field="destination_number" expression="^100[01]$">
```

从测试效果来看，这个正则表达式表示 "匹配目的地号码是1000或1001"。如果在两个不同的字段中使用逻辑 OR，应该怎么使用？ regex 操作符可以实现。

基本的regex 语法是:

```
<condition regex="all|any|xor">
   <regex field="some_field" expression="Some Value"/>
   <regex field="another_field"
      expression="^Another\s*Value$"/>
   <action(s) ...>
   <anti-action(s)...>
</condition>
```

用户可以在condition中设置多个regex标签。regex 操作字符可以支持三种不同的值:

- all: 它等同于逻辑 AND 操作。在这个condition中包含的expressions 必须都为true，那么执行actions。
- any: 它等同于逻辑 OR 操作。 在condition中的任何expressions为true 那么执行actions。
- xor: 它等同于逻辑 XOR 操作。只要其中一个expressions为true，那么执行 Actions。

执行复杂一点的逻辑语句，例如在两个字段执行逻辑OR，可以通过这样的方式：

```
<extension name="Regex OR example" continue="true">
  <condition regex="any">
    <!—- If either is true then perform actions–>
    <regex field="caller_id_name" expression="Some User"/>
    <regex field="caller_id_number" expression="^1001$"/>
    <action application="log"
      data="INFO At least one matched!"/>
    <!-- If *none* are true then do anti-actions -->
    <anti-action application="log"
      data="WARNING None of the conditions matched!"/>
  </condition>
</extension>
```

在上面的例子中，如果表达式 caller_id_name 是"Some User" 或 caller_id_number 表达式结果是"1001"，那么action标签执行. 如果任何一个 regex 返回结果不为true，将执行anti-action 标签。

用户可以开发更加复杂的正则表达式。例如，如果 "Some User" 有多个 Extension，可以这样使用regex:

```
<regex field="caller_id_number" expression="^1001|1005$"/>
```

执行复杂的AND 操作，用户可以使用一个extension，例如:

```
<extension name="Regex AND example" continue="true">
  <condition regex="all">
    <!—- If both are true then perform actions-->
    <regex field="caller_id_name" expression="Some User"/>
    <regex field="caller_id_number" expression="^1001$"/>
    <action application="log"
      data="INFO Both matched!"/>
    <!-- If *any* are false then then do anti-actions -->
    <anti-action application="log"
      data="WARNING At least one failed!"/>
  </condition>
</extension>
```

用户可以按照业务需求，在condition中堆叠多个<regex>标签。所有判断必须都为true，才可以执行action标签中的应用模块。如果任何一个regex 操作判断失败，将执行任何一个在Anti-action标签中的apps。

在一些极端的情况下，用户需要执行 XOR或独享的OR逻辑操作；那就是，仅仅只有一个condition可以为true。 如果没有一个为true，或如果测试结果多个为true，多于一个为true，那么整个逻辑操作验证为false。这里是一个实例：

```
<extension name="Regex XOR example" continue="true">
      <condition regex="xor">
        <!-- If only one is true then perform actions -->
        <regex field="caller_id_name" expression="Some User"/>
        <regex field="caller_id_number" expression="^1001$"/>
        <action application="log"
          data="INFO exactly one matched!"/>
        <!-- If none or 2+ are true then the XOR fails -->
        <anti-action application="log"
          data="WARNING XOR evaluated false!"/>
      </condition>
</extension>
```

内嵌的conditions

从FreeSWITCH 版本1.2.6 开始，用户可以内嵌一个 <condition> 标签来处理各种环境。通过一个逻辑结构可以解释这样的condition:

```
IF condition1 TRUE THEN

   IF condition2 TRUE THEN
      DO actions
   ELSE
      DO other actions
   ENDIF


   IF condition3 TRUE THEN
      DO actions
   ELSE
      DO other actions
   ENDIF


   IF condition4 TRUE THEN
      DO actions
   ELSE
      DO other actions
   ENDIF


ELSE
      DO other actions
ENDIF
```

通过内嵌conditions，系统会更加容易完成这个任务。以下这个摘要解释了一个实例，在这个实例中，destination_number 等同于我们逻辑结构中的condition1。我们有其他的字段，这些字段等同于condition2, condition3, 和condition4. 为了内容简洁，我们省略了指定的action和anti-action 标签:

```
<condition field="destination_number"
   expression="^1000$"
   require_nested="false">
   <action…>
   <anti-action…>
   <condition field="caller_id_number" expression="^1001$">
      <action…>
      <anti-action…>
   </condition>


    <condition field="caller_id_number" expression="1?408\d+">
       <action…>
       <anti-action…>
    </condition>


    <condition field="caller_id_number" expression="^1001$">
       <action…>
       <anti-action…>
    </condition>


   <action…>
   <anti-action…>
</condition>
```

一定要记住，内嵌的conditions首先获得处理，然后处理父级condition。表示的意思是，内嵌condition中的的action/anti-action执行以后，然后执行父级condition中的action和anti-action表达式，换句话说，当拨号规则解析器在解析阶段查询actions(或anti-actions)时，解析器在添加父级condition中的Action/anti-action之前，首先添加内嵌condition中的actions/anti-actions。

在上面的举例中，在处理父级condition（destination_number 字段）中的action/anti-action之前，首先判断三个内嵌的condition(判断caller_id_number字段)，并且处理它们的actions/anti-actions。简单来说，子级condition 优先判断匹配，然后处理父级condition。

以下部分解释了关于拨号规则的两个处理阶段，包括了新用户经常犯的错误。

避免陷阱

在FreeSWITCH 环境中，对于新用户来说，拨号规则的两个地方需要注意，那里让新用户感觉非常迷惑- 特别是一些有asterisk 技术背景的人需要更加注意。首先，容易迷惑的地方是在condition处理过程中的变量处理，另外，是在解析日志过程中。

切记，拨号规则是一个两阶段的处理，第一个阶段称为ROUTE(也称"hunting"或"parsing")阶段。这个阶段优先于其他任何命令。让我们看看这个实例，这个实例仅仅举例，还不能工作。这里，我们做一些对XML的debug 排查介绍，我们添加 info 模块到XML文件，这样就可以通过控制台打印出通道设置的变量信息。

```
<extension name="check_for_user" continue="true">
  <condition field="${callerid}" expression="2035551212">
    <action application="set" data="user=yes"/>
     <action application="info"/>
  </condition>
</extension>
<extension name="route_users_only">
  <condition field="${user}" expression="yes">
    <action application="answer"/>
    <action application="playback" data="tada.wav"/>
    <action application="hangup"/>
  </condition>
</extension>
```

当运行这段代码时，info 应用在屏幕上输出通道的所有变量，用户可以看到“user”设置为“yes”。还有一个condition 判断的变量user, 这个condition不运行。许多用户可能认为FreeSWITCH将停止，实际上，他们误读了输出结果。经过仔细检查，用户可以发现，在验证了所有的Condition后，在用户的logs中执行了set，然后执行了 info 应用模块，显示这些变量以及被设置，但是在执行info 之前，condition 判断过程已经完全结束。这样的话，如果用户不细心阅读输出log，用户可能要在这里花费几个小时来理解输出结果。

日志将显示是否匹配了个体的拨号规则入口，这样可能让很多用户困惑，他们的拨号规则是否真正执行。事实上，用户必须仔细查找日志，检查是否执行了设置的actions。通常的办法是查找EXECUTE 日志描述，仔细检查这里到底执行了什么命令。如果用户设置没有执行，可能用户的condition设置的不对。

XML拨号规则应用模块

我们应该把所有的拨号规则命令都查阅一遍，这个列表远远超出了本章节的内容空间。因此，我们仅仅把我们的讨论限定在拨号规则的三个方面—拨号规则工具，Sofia连接和基本API命令。mod_dptools, mod_sofia, 和mod_commands提供对以上三个方面的支持。他们是目前最经常使用的，也是最收欢迎的命令。

mod_dptools

mod_dptools 命令是拨号规则的管理工具集. 许多应用模块都支持拨号规则。用户已经学习了基本的命令，例如answer, hangup, bridge, 和set。让我们了解一下其中的一些高级命令。

- bind_meta_app: 对指定的呼叫leg（s）绑定一个应用。
  在一个桥接的呼叫中，在绑定的呼叫leg 的双音多频连续号码会触发应用模块的执行。没有绑定的呼叫leg 将不能听到拨打的双音多频连续号码。用户仅可以绑定单个数字，并且绑定时带有一个前缀字符* 号键。例如，用户希望按键*2 开始一个通话录音。

  当呼叫方摁 按键*2, 开始录音。在这个实例中，用户使用了以下拨号规则代码。注意，bind_meta_app被加深:

```
<action application="bind_meta_app" data="2 a s
   record_session::recording.wav"/>
<action application="bridge"
   data="sofia/sipprovider/+14158867900">
```

  这个action 允许在这个通道的A-leg 摁按键*2 来触发对同一leg录音(第三个参数s 表示同样的leg)。

  注意，特别指定的除外，bind_meta_app 将使用* 作为一个″meta key″. 通过设置bind_meta_key 通道变量问不同的值，可以修改行为方式。例如，使用# 替代* ，用户可以这样设置:

```
<action application="set" data="bind_meta_key=#"/>
```

  注意bind_meta_app参数的格式:

```
<action application="bind_meta_app"
   data="KEY LISTEN_TO RESPOND_ON
   APPLICATION[::PARAMETERS]"/>
```

以下列表解释了上面的参数含义:

- KEY: 定义一个监听DTMF按键。
- LISTEN_TO: 指定呼叫leg来监听DTMF按键.
  支持的参数是a, b, 或者ab。
- RESPOND_ON: 当DTMF 按键输入以后，响应的呼叫leg。
  例如，当播放一个语音作为一个响应命令时，相应的呼叫leg
  听到这个回放。支持的参数 s 表示同样的leg；就是说，输入DTFM
  按键的leg方或者相对的leg方。
- APPLICATION: 指定执行一个应用模块。
- PARAMETERS: 指定一个参数，这个参数将彻底到应用模块中。
  注意，可使用双冒号分开应用模块和参数(APPLICATION::PARAMS)。



一旦绑定一个呼叫的leg，这个应用绑定会一直存在，直到这个呼叫Leg的生命周期结束。

获得更多高级的双音多频DTMF 键绑定处理机制，查看bind_digit_action。

- bind_digit_action: 这个命令支持了比bind_meta_app 更加高级，简洁的按键映射机制。它可以捕获任何按键的输入组合，而不需要输入时以* 开始。

bind_digit_action 测试格式为:

```
<action application="bind_digit_action"
   data="REALM,KEY|REGEX,COMMAND,COMMAND_ARGUMENTS,LISTEN_
TO,RESPOND_ON"/>
```

以下列表解释了以上参数用途:

- REALM: 指定用户Michael。
- KEY|REGEX: 指定的监听的输入按键，或监听regex包含的组合按键。
- COMMAND: 指定一个可执行的应用模块。它可以是拨号规则命令(带前缀exec:) 或一个APL命令(带前缀api:).
- COMMAND_ARGUMENTS: 为命令指定的运行参数。

- LISTEN_TO: 指定监听输入DTMF 按键的call leg。可支持的参数为 a, b, 或ab。
- RESPOND_ON: 指定响应的call leg。例如，当播放一个语音文件时，哪个call leg 应该听到这个回放的语音文件。参数s 表示同一leg; 可以是输入按键音的call leg，或者对方leg。

这里有一个举例。如果拨打一个密码9348234，以下命令将对系统的所有呼叫执行挂机命令。它运行了一个API hupall 命令。

```
<action application="bind_digit_action"
  data="my_digits,9348234,api:hupall"/>
```

注意，bind_digit_action会导致bind_digit_action自己“吃掉”输入的DTMF数值，因此这些DTMF输入结果不会传递到其他的应用模块。

- eavesdrop: 这个命令可以用户侦听其他通道。在这个例子中，以下拨号规则将支持用户$1 中的UUID

```
<action application="eavesdrop" data="$1"/>
```

用户可以使用其他的UUID替换$1，或者从数据库提取出UUID，例如:

```
<action application="eavesdrop"
  data="${db(select/spymap/${extension})}"/>
```

在这个实例中，使用了变量extension，在拨号规则中设置成高的优先级，可以根据这个值来查询数据库，获得一个和UUID关联的extension。在这个流程中，如果在表spymap中添加了extension number 和呼叫的UUID，用户可以提取这些信息，可以配合eavesdropping 来使用。

execute_extension: 用户可以在拨号规则一个extension中执行另外extension。这样做的目的是执行一个临时路由到另外一个extension，然后返回到原来离开时的节点位置。这个方法类似于在其他switch中的loopback函数。execute_extension 执行一个macro 类似的extension，然后返回。它的使用方法不同于transfer，transfer会立即路由到一个新的extension，但是没有返回。execute_extension 命令始终在一个限定的范围内执行，并且创建一个单次extension，执行这个extension，然后按照原路返回到当初调用的那个逻辑点。

```
<action application="execute_extension"
  data="destination_number [Dialplan] [context]"/>
```

如果用户没有指定拨号规则Dialplan和context，则默认使用当前的Dialplan和context。

使用execute_extension 的环境是，当用户需要执行一个命令，然后返回到拨号规则处理的那个离开时的节点。如果用户不需要处理其他的任务时，使用 transfer 应用模块. 如果用户是一位程序员，一个类似的比喻是: execute_extension 相当于gosub语句，transfer 相当于goto语句.

- send_display: 用户可以发送一个自定义的 SIP INFO 消息到一个电话终端，这个电话终端（一些话机支持），终端显示发送的消息。

  实例:

  <action application="send_display" data="Support Call"/>

  此方法将在电话终端显示一个部门或者信息，提示呼叫对象或被呼叫的部门。

  FreeSWITCH wiki 有列出了很多支持的命令，可访问此链接获得帮助：http://wiki.freeswitch.org/wiki/Mod_dptools.

mod_sofia

mod_sofia 命令负责处理和SIP相关的业务。它包括两个部分的任务，作为一个终端endpoint来发送或接收SIP呼叫，也负责管理SIP支持和contact 信息。在mod_sofia 支持了各种命令，不仅仅是管理发起呼叫和接收呼叫，也管理和拆分endpoint信息，例如设备注册的IP地址，设备是否在NAT环境。

Sofia本身不提供任何可以在拨号规则中支持用户直接调用的应用模块，但是在很多命令所支持的参数中使用，这些参数在这里显得特别重要的。

通常来说，当桥接一个SIP呼叫时，需要访问Sofia。桥接指的是一个A leg连接一个刚初始化的B leg。当桥接一个呼叫时，一般的格式如下：

```
<action application="bridge"
        data="sofia/profile/endpoint[@domain]"/>
```

让我们分析一下data 命令部分。

因为bridge 命令是一个基本命令，不是专门去桥接一个SIP呼叫，用户必须首先指定sofia/ 参数来说明用户现在执行一个SIP呼叫。紧接着sofia/ 参数，用户必须设置语音网关或SIP profile 来连接一个呼叫。如果用户指定了一个已经设置好的网关，已经获得了接收方服务器的域名，无需在拨号字符串添加（可以不用@domain)。如果你设置的profile名称作为第二个参数，那说明用户正在告知Sofia使用IP 地址，端口和设置的参数来建立呼叫。在这种情况下，用户在Sofia bridge 中指定域名。最后，终端 endpoint 参数来设置用户名，发送到远端系统。这个用户名通常是一个DID 或者分机号码，以便让远端服务器知道如何连接到相关的部门。

这里有几种不同的方法桥接一个呼叫。在下面的举例中，我们假设有一个运营商网关，名称为supersip 和一个Sofia profile，名称为external.

```
<action application="bridge"
  data="sofia/supersip/+14158867900"/>
```

通过supersip运营商网关桥接呼叫到电话 (415) 886-7900。

```
<action application="bridge"
  data="sofia/external/+14158867900@sip.supersip.com"/>
```

通过这样的方式，将经过运营商网关supersip呼叫到(415) 886-7900，另外在这个例子中，我们目前指定我们希望通过profile external（这个profile包含了IP地址和需要连接的端口）呼出，并且正在通过 sip.supersip.com服务器路由呼叫。

```
<action application="bridge"
  data="sofia/external/someuser@otheroffice.com:5080"/>
```

这样，系统将桥接呼叫方到someuser，someuser是服务器otheroffice.com用户，通过端口5080连接。这样做的好处是，通过这样的方式可以在FreeSWITCH和asterisk，或者第三方其他的内网服务器之间桥接呼叫，没有任何人可以阻止。当采用这种方式来桥接呼叫时，用户通过命令完全控制了用户名，服务器，端口和呼叫路由。

现在让我们看一个复杂的实例。Sofia 可以支持通过拨号字符设置一些高级选项来控制呼叫执行。假如，我们准备通过TCP发送呼叫，而不是使用默认的UDP端口协议。 在拨号字符串后面添加一个分号，允许用户追加一个Sofia 选项。在这个实例中，我们在拨号字符串中添加了transport=tcp。它是这样实现的:

```
<action application="bridge" data=
"sofia/external/+14158867900@sip.supersip.com;transport=tcp"/>
```

重温这些实例的目的和重要性是对用户展示Sofia系统的强大之处，让用户了解如何通过拨号规则来访问Sofia 系统。这些应用不仅仅局限于基于已定义的基本的桥接选项—用户可以按照自己的需求，通过各种变量的灵活使用，选项和拨号规则功能来连接呼叫。

mod_commands

mod_commands模块对系统管理员提供了一些管理命令，可以通过CLI 执行。有时候CLI也可以在FreeSWITCH 呼叫处理过程中起到一定的作用。CLI 命令不同于拨号规则中的其他应用模块，用户可以通过检测字符串的形式，清晰地运行各种CLI命令。

在实例中，CLI 命令hupall(NORMAL_CLEARING) 通常重置所有活动的呼叫，然后终止这些呼叫，发送挂机提示符NORMAL_CLEARING。这个命令通常仅在命令行环境下运行。当拨打999时，用户需要支持这样的挂机方式，可以定义一个类似的extension:

```
<extension name="Make API call from Dialplan">
  <condition field="destination_number" expression="^(999)$">
    <action application="set"
            data="api_result=${hupall(normal_clearing)}"/>
  </condition>
</extension>
```

注意，加深的一行。在以上的extension中，通过表达式 ${hupall(normal_clearing)} 我们重新设置了CLI命令hupall。而且，这个CLI命令的结果会赋值到变量api_result，所以我们必须把这个表达式设置在 set 命令中。

当然，这个举例显然不是很实用，这里我们演示的重点是如何在拨号规则中使用CLI命令和如何使用命令结果。如果用户希望了解完整的CLI命令列表，可以访问：http://wiki.freeswitch.org/wiki/Mod_commands.

使用变量

到目前为止，我们已经学习了基本的通道变量使用方法。FreeSWITCH 环境下支持很多高级的变量使用方法，包括全局变量。让我们通过研究这些变量的使用，来完整理解这些变量。

通过正则表达式测试变量

我们已经讨论了condition XML标签的目的。现在我们将更进一步讨论使用不同的要素来帮助我们测试，并且对呼叫路由做出最后决定。

FreeSWITCH提供三中类别的变量来帮助用户测试– 呼叫方profile字段，通道变量和全局变量。另外，用户可以使用 macros 和API 函数，并且可以在用户的condition中使用这些结果。我们将仔细回顾每一个变量。

呼叫方profile字段

Caller profile 字段是变量，当呼叫方被认证后获得的变量。这些变量设置于目录下，可以包括呼叫方电话区号，使用的语音编码。当处理这个拨号规则时，用户可以在condition中使用caller profile字段，例如:

```
<condition field="${caller_profile_field}">
```

用户目录下变量设置，例如:

```
<user id="bob">
  <variables>
    <variable name="caller_profile_field" value="1234"/>
  </variables>
</user>
```

在这个例子中，当 bob 认证后，代表他被设置为当前的caller profile.
这样做最后的结果是，他的profile中包含的变量都可以被访问，就像其他通道的变量一样。

关于用户目录的内容已经在第四章 SIP和用户目录中讨论。

通道变量

在FreeSWITCH 环境中，每个通道有很多变量，关联的变量可以用来跟踪状态，设置和关于呼叫的其他信息。我们可以通过以下的格式来调用通道变量:

```
${variable}
```

通道变量可以在拨号规则，应用模块，或者目录下设置。它们可以影响一个呼叫处理或者呼叫设置。他们可以使用在任何可以触发变量的地方，例如拨号规则的condition，应用命令和类似的地方。

```
<condition field="${channel_variable}">
```

在FreeSWITCH ，在处理呼叫时，使用最频繁，最重要的处理呼叫的地方就是通道变量。任何一个呼叫可以支持多个通道变量，而且可以对呼叫进行操作设置。用户可以参考完整的在线变量列表获得更多信息：
http://wiki.freeswitch.org/wiki/Channel_Variables.

通道变量和呼叫设置

当设置呼叫或指定呼叫legs，例如发起新呼叫，通过bridge 命令桥接A leg到B leg时，用户可以使用通道变量。在这些场景下，有两种方法来设置通道的变量—大括号和中括号。两种方法工作的方式是不一样的，当同时对多方桥接或发起呼叫，使用这两种方法时需要特别小心。

在呼叫过程中，当使用大括号时，代表"全局"使用。看一下以下的举例，我们桥接一个通话到用户Darren's 移动电话203-555-1212。我们仅支持对用户振铃20秒，避免进入到语音邮箱。

```
<application action="bridge"
  data="{call_timeout=20}sofia/gateway/my_gw/2035551212"/>
```

在设置新通道时，使用了大括号中的变量，通道为sofia/gateway/my_gw/2035551212。现在，我们添加Darren办公室号码。我们设置办公室电话振铃30秒，但是支持移动电话振铃为20 秒。我们可以通过中括号的方式来实现，例如:

```
<application action="bridge"
 data="[call_timeout=20]sofia/gateway/my_gw/2035551212
,[call_timeout=30]sofia/gateway/my_gw/4158867901"/>
```

通过使用中括号设置变量，可以支持指定每一个呼叫的leg。



大括号仅仅对每个拨号字符串起始阶段有效。对新的拨号字符串无效—用户的桥接字符应该为一行，括号和sofia部分中间不能有空格。

用户可以使用大括号设置变量，接下来使用中括号修改变量。用户必须使用一个标志名称local_var_clobber来支持工作。我们可以重新创建一个完全一样的举例，仅对所有的legs指定一个默认的超时 设置为30秒，如果对移动电话呼出时，重新覆盖时间设置，设置为20秒。如下:

```
<application action="bridge"
data="{local_var_clobber=true,call_timeout=30}
[call_timeout=20]sofia/gateway/my_gw/2035551212
,sofia/gateway/my_gw/4158867901"/>
```

可以通过逗号来完成多个变量的设置。例如，用户可以这样指定：

```
{call_timeout=20,sip_secure_media=true}
```

以上这段代码对所有通道指定的两个变量，或者:

```
[call_timeout=20,sip_secure_media=true]
```

以上这段代码对个体的通道指定的变量。

FreeSWITCH有一个特别的标志来支持一个多次发起呼叫机制。我们使用小于号和大于号来替换中括号（每个通道）或大于号（每个发起的呼叫）。请看下面的例子:

```
<action application="bridge"
data="<ignore_early_media=true>{var1=val1}
sofia/gateway/my_gw/${dest1}:_:
{var1=val2}sofia/gateway/my_gw/${dest2}"/>
```

注意，在拨号字符串起始段使用了〈ignore_early_media=true〉。这样做的结果是对全部创建的“originates“，变量ignore_early_media 都将设置为true。

使用多次发起呼叫机制的讨论在这个章节的XML Dialplan cookbook 找到.

全局变量

当FreeSWITCH 第一次启动时，FreeSWITCH所有完整的XML配置文件都将加载到内存中。在这个过程中，FreeSWITCH 寻找以下代码:

```
<X-PRE-PROCESS cmd="set" data="domain=127.0.0.1"/>
```

这段代码定义了全局变量。

当FreeSWITCH启动时，全局变量可以在初始化过程中调整。X-PRE-PROCESS 标签设计了一个命令可以支持实际的XML加载。当用户在这个阶段设置了一个变量时，这个变量就被自动设置为全局变量，可以通过应用模块接口，作为$$(variable)被访问.

注意，当用户在XML中使用$${variable} 时，这个变量可以在XML加载时间被一个变量替换，这个替换的变量是在 X-PRE-PROCESS标签处理过程中设置的。

```
<X-PRE-PROCESS cmd="set" data="domain=127.0.0.1"/>
<param name="domain" value="$${domain}"/>
```

例如，这个上面的XML代码真正编译以后，从FreeSWITCH 角度来看，可以是一行代码:

```
<param name="domain" value="127.0.0.1"/>
```

这样的功能是XML解析器的功能，不是FreeSWITCH 本身支持的功能。全局变量的预处理会优先执行，然后FreeSWITCH的事件对XML文件的调用。

FreeSWITCH 输出编译后的XML文件，保存到硬盘中。用户可以查看预处理器命令和全局变量声明等记录信息。这个文件通常保存在：/usr/local/freeswitch/log/freeswitch.xml.fsxml。

用户可以在用户的condition，用户变量，参数声明中使用这些全局变量。例如:

```
<condition field="$${global_variable}">
```

Dialplan函数

Dialplan函数支持拨号规则condition处理的实时运行。当用户写自己的condition时语句时，使用这些函数获得更多的控制和灵活性。

Dialplan函数可以在任何地方使用，不仅仅局限于于拨号规则中。它们和XML没有联系—可以使用在任何FreeSWITCH 字符处理器触发的地方。可能是一的地方包括外部的脚本语言，用来执行设置变量，桥接和转接应用，等等地方。

基本的拨号规则函数格式是:

```
${api_func(api_args ${var_name})}
```

这里，api_func 是一个拨号规则函数名称，api_args 是一个需要传递到函数的参数名称，${var_name}是一个可选的变量名称。api_args 格式和要求的变量名称完全依赖于使用的函数。我们在下面的章节中，详细介绍了每个函数的使用。



实际上，可以从fs_cli 执行的API，同样也可以在拨号规则中执行，如果在拨号规则中使用，附带一个${api(args)}标志。

实时的condition判断

用户可以使用cond 函数在condition表达式中执行conditional判断。

Condition函数的基本格式为:

```
${cond(<expr> ? <true val> : <false val>)}
```

使用condition函数举例:

```
${cond(5 > 3 ? true : false)}
```

这个表达式将返回结果true。支持的对比运算字符是:

- ==表示相等
- != 表示不相等
- > 表示大于
- >= 表示大于或者等于
- < 表示小于
- <= 表示小于或者等于

注意，用户可以进行字符与字符对比，号码与号码对比，但是如果进行字符和号码对比，字符将以strlen(string) 和号码对比。

字符条件

用户可以使用变量替换${var:offset:length}标签来获得变量中的部分数值 (就像许多语言中的一个substr函数) 。支持的参数是:

- var: 字符变量，可以是字符或变量，例如${caller_id}。
- offset: 起始拷贝数据的位置。0代表第一个字母。
- length: 表示查找的字符数量。它是可选的，如果忽略此选项，剩余部分的字符将被拷贝。

参数使用举例如下:

```
var = 1234567890
${var:offset:length}
${var:0:1} // 1
${var:1}// 234567890
${var:-4}// 7890
${var:-4:2} // 78
${var:4:2} // 56
```

以下是一个使用API捕获美国区号前三位号码的举例，通过以上拨号规则，捕获呼出时，呼叫方的Caller ID，赋值到变量${callerid}中:

```
<application name="set" data="areacode=${callerid:0:3}"/>
```

使用任何小于或者等于0的参数作为一个截取长度，将返回一个从指定位置到字符结束位置的结果。

数据库查询

用户可以执行FreeSWITCH内部数据库相关命令，例如插入，删除，选择，和来更新任何数据。

数据库命令格式为:

```
${db(command/realm/key/value)}
```

数据库命令可以是insert, select, 或delete, 紧接这个表或者realm, 然后一个输入键和一对值。

这里的举例中，我们可以创建一个基于呼叫方CallerID的语音等待，通过查询数据库获得相应的结果：

```
<action application="playback" data="${db(select/music/${caller_id_
number})}"/>
```

在另外一个例子中，我们将在数据库中插入数据。在这个例子中，我们将在表spymap中插入当前呼叫方的UUID，把呼叫方的Caller ID作为记录标志。某人可以通过Caller ID搜索最新的UUID。

```
<action application="db" data="insert/spymap/${caller_id_
number}/${uuid}"/>
```

当写入FreeSWITCH 数据库时，用户使用了sqlite 或者ODBC数据库配置。通过这样的方式用户可以永久保存数据。有时候，实际环境中，用户可能不想那样操作，只是想在内存中保存临时数据。在这种情况下，用户可以使用我们以前讨论的模式，通过哈希表替换数据库。

例如，我们从哈希表中提取数据:

```
<action application="playback" data="${hash(select/music/${caller_id_
number})}"/>
```

或者把数据保存在哈希表中:

```
<action application="hash" data="insert/spymap/${caller_id_
number}/${uuid}"/>
```

Hash 山一种简单的以内存作为存储介质的表，来保存一对key/value。如果用户重新启动FreeSWITCH，哈希表中的数据将丢失。

SIP contact参数

用户可以通过sofia_contact 命令来提取已注册的Sofia contact 参数信息（或替换它们）。命令的基本格式为:

```
${sofia_contact(profile/foo@bar.com)}
```

这个命令非常有用。最简单的用法是提取字符串来检测一些参数，例如是否注册的用户在NAT背后，或是否完全注册。更为复杂的举例，用户可以使用这个功能截取部分contact字符，用户可以在将来使用或者替换这些字符。

在注册时，以下这段XML代码从contact 字符中查询用户名为foo@domain.com ，并且获得用户域名或IP地址和contact 字段参数。在用户contact 记录中提取出用户名。

```
<condition
   field="\${sofia_contact($user_id@domain.com)}"
   expression="^[^@]+@(.*)">
      <action application="set" data="to_domain=$1"/>
</condition>
```

使用用户真正查询的域名替换domain.com。注意，正则表达式的模式为:

```
^[^@]+@(.*)
```

这种模式以字符起始开始匹配，直到发现 @ 标志符，然后捕捉@标志符以后的字符，保存这些字符到$1中。

用户名被提取以后，可以使用其他的新用户名称来替换这个用户名称。
经常是通过这样的方式实现的，路由DID到一个客户的PBX时—用户可以通过被呼叫的DID号码替换接线员用户名。类似于这样的方法:

```
<condition
   field="\${sofia_contact($user_id@domain.com})}"
   expression="^[^\@]+(.*)">
      <action application="bridge"
         data="sofia/external/${DID_number}@$1"/>
</condition>
```

在这个例子中，如果在${DID_number} 字段设置了一个变量，那么这个变量就是用户IP地址和contact 路由信息组合而成。因此，如果用户注册为frank@72.44.12.28，则可以使用2035551212@72.44.12.28来替换。



如果用户有多个SIP profiles 注册多个设备时，当使用sofia_contacts时，有时候可能会收到ERR/USER_NOT_REGISTERED 错误，甚至于已经处于注册的状态。 改正的办法是，当呼叫sofia_contact 命令时，使用* 作为Profile名称:

```
sofia_contact(*/user@domain)
```

* 将通知Sofia对这个用户搜索所有的SIP profiles。

Set, export, and legs

当通过桥接来了解两个不同的呼叫legs时，用户可能发现，在有一些应用中，希望A leg的通道变量可以支持B leg的通道变量。有时，用户希望这个值可以出现在一个leg或者其他的leg中。这个部分的一些技术讨论将解释如何完成这些任务。

Set versus export

有两种基本的拨号规则应用来设置和修改呼叫信息或呼叫处理转换。
它们是set和export。

set 应用在通道生命周期中对通道设置通道变量。这些变量可以通过应用模块（CDR）或拨号规则的condition进行测试。用户已经看到过几次使用set 的实例。

export 应用在set 应用的基础上更进一步。它可以对当前的通道设置变量，也可以保存这些变量为了将来的其他通道使用。换句话说，export 设置的通道变量不仅仅支持呼叫的A leg， 而且支持未来B leg也可以访问。

两者之间有着细微的区别，直到用户需要访问 B leg的信息（已转接的呼叫），才发现两者的区别。当发生在多呼叫多个leg的环境下，用户需要访问这些变量时，变量的一致性是非常重要的，Export 在这样的环境下变得非常有用，因为它可以确保这些变量的一致性。

看一下以下这些例子:

```
<!--Variable "foo" is set on                both legs -->
<action application="export"                data="foo=bar"/>
<action application="bridge"                data="/user/1001"/>
<!--Variable "foo" is set on                b leg only -->
<action application="export"                data="nolocal:foo=bar"/>
<action application="bridge"                data="/user/1001"/>
```

在一些使用环境下，用户需要在所有的呼叫legs设置变量 foo。在另外的一些环境下，例如，处理CDR时，当用户需要一个特别的值仅出现在B leg。加深的一行代码说明一个例子如何使用nolocal:，这里对在仅leg B的bar设置通道变量foo，但是不在A leg设置通道变量foo。

通过呼叫的头域来传递变量

有时候，在呼出时，用户需要自定义头域。SIP 协议栈是最常用的地方。

用户可以通过set 或 export 命令对呼出的SIP呼叫添加自定义的头域信息，就像以下代码演示，但是需要在变量前添加一个前缀字符名称sip_h_。例如，我们对这个呼叫添加了一个头域信息CallerLikesTacos=1，用户可以在桥接呼叫之前来添加一个set 应用，例如:

```
<action application="set" data="sip_h_X-CallerLikesTacos=1"/>
<action application="bridge"
  data="sofia/mydomain.com/1000@example.com"/>
```

如果用户需要对BYE 请求添加一个头域，则需要在通道变量中使用前缀sip_bye_h_。



虽然这不是一个要求，用户应该添加前缀X-来避免和其他的SIP协议栈出现兼容性问题。X- 头域通常被看作是自定义的头域，如果没有被识别，在SIP世界是被忽略的。

XML Dialplan cookbook

我们罗列几个应用场景，因为这些场景是相对常用的场景，用户可以经常参考这些场景。本部分列出的举例实际上对读者来说，完全的私房菜的秘集。用户可以根据拨号规则的需求来修改这些例子。

根据IP地址和呼叫一个号码来匹配

在下面的举例中，选择了一个特别的extension，仅匹配被呼叫方终端的IP地址192.168.1.1．第二个condition中，提取到拨打号码，然后保存到变量$1中，然后通过桥接sofia 网关呼出。

```
<extension name="Test1">
  <condition field="network_addr"
    expression="^192\.168\.1\.1$"/>
  <condition field="destination_number" expression="^(\d+)$">
    <action application="bridge"
      data="sofia/profilename/$1@192.168.2.2"/>
  </condition>
</extension>
```

第一个condition 字段以一个斜杆符结束。最后一个condition 字段包含一个action 标签，以正常的</condition> 标签结束。也请大家注意，以上的例子和这个例子不同:

```
<extension name="Test1Wrong">
  <condition field="destination_number" expression="^(\d+)$"/>
  <condition field="network_addr"
    expression="^192\.168\.1\.1$">
    <action application="bridge"
      data="sofia/profilename/$1@192.168.2.2"/>
  </condition>
</extension>
```

Test1是一个错误的例子，它不能正确地路由呼叫，因为变量 $1没有任何值，因此目的地号码通过不同的condition，字段进行了匹配。

用户可以设置第一个condition中的变量来解决这个问题。这个变量同样可以在第二个condition中的action使用:

```
<extension name="Test1.2">
  <condition field="destination_number" expression="^(\d+)$">
    <action application="set" data="dialed_number=$1"/>
  </condition>
  <condition field="network_addr"
    expression="^192\.168\.1\.1$">
    <action application="bridge"
      data="sofia/profile/${dialed_number}@192.168.2.2"/>
  </condition>
</extension>
```

当执行一个extension判断时，action被呼叫，用户不能使用在这个extension设置的变量作为后续conditions/matches变量。

如果用户在一个extension中，基于一个设置的变量来支持不同的actions，或者使用对设置的变量执行execute_extension转接这个呼叫，或使用内联处理的方式（参考本章节内联执行的部分）。

根据IP地址和Caller ID匹配

在这个例子中，我们需要同时匹配一个以1为前缀的号码和一个IP地址。

```
<extension name="Test2">
  <condition field="network_addr"
    expression="^192\.168\.1\.1$"/>
  <condition field="destination_number" expression="^1(\d+)$">
    <action application="bridge"
      data="sofia/profilename/$0@192.168.2.2"/>
  </condition>
</extension>
```

这里，我们通过表达式规则 ^1(\d+)$来匹配，我们不使用变量$1，这个表达式包含一个拨打号码，这个号码包含一个前缀号码1，我们把前缀号码1删除。我们将使用变量$0替代，变量$0 包含一个原始目的地号码。

根据号码和截取的数字匹配

在这个例子中，我们需要匹配一个以00为前缀的呼叫号码，而且需要截取起始的数字。假设，FreeSWITCH 收到一个号码是00123456789，我们需要截取00这两个数字，我们可以使用以下这个例子:

```
<extension name="Test3.1">
  <condition field="destination_number"
    expression="^00(\d+)$">
    <action application="bridge"
      data="sofia/profilename/$1@192.168.2.2"/>
  </condition>
</extension>
```

另一方面，如果用户预想到有可能收到无数字的号码，或计划匹配的号码不仅仅是数字，可以使用.+ 替换\d+，因为\d+仅仅匹配号码数字，而一个.+将匹配从当前位置到字符结束的所有字符或号码数字:

```
<extension name="Test3.2">
  <condition field="destination_number" expression="^00(.+)$">
    <action application="bridge"
      data="sofia/profilename/$1@192.168.2.2"/>
  </condition>
</extension>
```



技术上说，我们不是真正想剔除那些我们不想要的数据，事实上，我们需要截取我们需要的数字。记住，在第一个参数集合中匹配的结果保存到了$1。简言之，我们仅仅需要最后的结果，这些数据可以支持我们其他的需求。

匹配号码，截取数字和添加前缀号码

在这个例子中，我们需要删除起始的数字，同时需要在呼叫号码前添加一个新的前缀号码。假设FreeSWITCH 收到一个号码为00123456789，我们需要替换前缀00为011，我们可以使用以下例子:

```
<extension name="Test4">
  <condition field="destination_number"
    expression="^00(\d+)$">
    <action application="bridge"
      data="sofia/profilename/011$1@x.x.x.x"/>
  </condition>
</extension>
```

呼叫一个已注册设备

这个例子演示了如何桥接一个已在FreeSWITCH系统注册的设备。在这个例子中，我们假设用户已经配置了一个Sofia profile 文件local_profile ，并且电话通过域名example.Com注册。注意，在拨号字符串中% 替代了@ :

```
<extension name="internal">
  <condition field="source" expression="mod_sofia"/>
  <condition field="destination_number" expression="^(4\d+)$">
    <action application="bridge"
      data="sofia/local_profile/$0%example.com"/>
  </condition>
</extension>
```

在FreeSWITCH中，使用 % 替换@ 有一个特别的功能。使用这个格式user%domain将告诉FreeSWITCH，这个用户已经通过这个域名注册，这个域名将添加到FreeSWITCH目录下，获得FreeSWITCH支持。

Try party A, then party B

以下举例演示了这样一个场景，如果第一个action 失败以后，呼叫另外一个action。

如果第一个action 成功，则桥接到1111@example1.company.com，
直到对方挂机以后退出action。这个处理过程完成以后，没有继续处理，
因为呼叫方的通道已经关闭。(换句话说，没有继续呼叫
1111@example2.company.com。)

如果初始呼叫1111@example1.company.com 没有成功的话，通道不会关闭，则执行第二个Action。

```
<extension name="find_me">
  <condition field="destination_number" expression="^1111$">
    <action application="set"
      data="hangup_after_bridge=true"/>
    <action application="set" data="continue_on_fail=true"/>
    <action application="bridge"
        data="sofia/local_profile/1111@example1.company.com"/>
    <action application="bridge"
        data="sofia/local_profile/1111@example2.company.com"/>
  </condition>
</extension>
```

路由DIDs到内部分机

如果用户需要路由呼入带DID号码的呼叫，例如从public context 到
另外一个context inhouse，可以通过这样的方式实现:

```
<context name="public">
   <extension name="test_did">
     <condition field="destination_number"
       expression="^\d{6}(\d{4})$">
       <action application="transfer" data="$1 XML inhouse"/>
     </condition>
   </extension>
</context>
```

这样的方式仅截取出十位数号码的最后四位数，然后通过inhouse 的context 转接
呼叫方呼叫到此号码。注意，\d{4} 代表仅截取最后四位数。

可选呼出网关

在这个例子中，我们从office A发送一个呼叫（十位数的号码）到gateway 1，从 Office B通过 gateway 2 呼出. 我们假设OfficeA和OfficeB 使用同一个FreeSWITCH ，但是通过不同的路由呼出。假设两个办公室的分机号码都是四位数号码，Office A的分机号码以2开头，Office B的分机以3 开头。

```
<extension name="officeA_outbound">
  <condition field="caller_id_number"
             expression="^2\d{3}$"/>
  <condition field="destination_number"
             expression="^(\d{10})$">
    <action application="set"
      data="effective_caller_id_number=8001231234"/>
    <action application="set"
      data="effective_caller_id_name=Office A"/>
    <action application="bridge"
              data="sofia/gateway/myswitch.com/$1"/>
  </condition>
</extension>
<extension name="officeB_outbound">
  <condition field="caller_id_number"
             expression="^3\d{3}$"/>
  <condition field="destination_number"
             expression="^(\d{10})$">
    <action application="set"
      data="effective_caller_id_number=8001231235"/>
    <action application="set"
      data="effective_caller_id_name=Office B"/>
    <action application="bridge"
      data="sofia/gateway/otherswitch.com/$1"/>
  </condition>
</extension>
```

Multiple endpoints with enterprise originate

考虑一下这个例子: 客户可能需要了解一个环境，他们是否可以路由一个呼叫到不同的最终用户。第一个人，希望桌面电话首先振铃，然后移动电话振铃。第二个人，希望桌面电话振铃的同时，移动电话也振铃。无论Alice或Bob谁先应答这个呼叫—其他所有的呼叫都停止振铃。

这个复杂的流程要求使用的FreeSWITCH的enterprise originate功能。
enterprise originate的基本思路是多个单个发起的呼叫连接成一个大型的"enterprise" 呼叫。
在我们的举例中，Alice的电话可以通过这样的方式被呼叫:

```
<action application="bridge"
data="[leg_timeout=10]user/Alice|
[leg_timeout=20]sofia/gateway/my_gw/${alice_mobile}"/>
```

Bob的电话可以通过这样的方式:

```
<action application="bridge"
data="[leg_timeout=10]user/Bob|
[leg_timeout=20]sofia/gateway/my_gw/${bob_mobile}"/>
```

每个独立的个体都桥接呼叫Alice 或Bob, 但是都不在同一时间呼叫。完成这个呼叫，
可以把它们合并成单一的bridge action， 通过特别的字符:_:把两者分离开。这是一个例子:

```
<action application="bridge"
data="<ignore_early_media=true>[leg_timeout=10]user/Alice|
[leg_timeout=20]sofia/gateway/my_gw/${alice_mobile}:_:
[leg_timeout=10]user/Bob|
[leg_timeout=20]sofia/gateway/my_gw/${bob_mobile}"/>
```

在这个例子中，当呼入流程进入到bridge 应用时，FreeSWITCH 将发起两个分离的
"originates"——一个进入Alice，另外一个进入Bob。 这样做的效果是，FreeSWITCH 尝试
进入到Alice的呼叫流程， 呼叫她的桌面电话和她的移动电话，同时呼叫Bob的桌面电话和
移动电话。如果有人应答了呼叫，那么整个“enterprise”将停止，呼叫连接到应答的终端。

这里注意，因为我们正在创建多个呼叫的legs，所以我们强制使用了ignore_early_media=true。
没有办法来接听和使用呼叫源的早期媒体（振铃）。如果用户需要对呼叫方提供振铃信号，
确认设置ringback 或 transfer_ringback 变量。

总结

在这一章节，我们详细探讨了FreeSWITCH 拨号规则的使用操作。在第五章的基础上， 理解XML 拨号规则，我们讨论了许多高级拨号规则概念:

- 拨号规则解析如何工作
- 使用全局变量和通道变量
- 正则表达式的高级用法
- 各种高级路由概念

拨号规则系统是FreeSWITCH 最为重要的概念。FreeSWITCH威力在拨号规则系统中没有完全显现出来，理解FreeSWITCH各种函数的复杂性是非常关键的一步，它可以确保FreeSWITCH真正执行了用户的要求。

在下一个章节，我们进一步夯实基础创建更为强大的FreeSWITCH配置，而且这些配置文件不单单依赖于静态的XML文件。

9

探讨XML高级配置

到目前为止，我们一直在使用FreeSWITCH默认安装的静态XML实例配置文件。在这个章节，我们将进一步演示如何在使用最少静态XML的情况下，来灵活配置FreeSWITCH。FreeSWITCH 提供了多种方法让用户可以对FreeSWITCH实行动态控制。每一种方法关注的领域不同，一些函数的功能可能有所重叠。例如，mod_xml_curl 和语言绑定都支持用户创建动态的配置文件。在这一章节，我们将涵盖以下几种方法：

- mod_xml_curl: 这个模块支持从web服务器提取一个配置文件。配置文件包括拨号规则，用户目录，和基本配置文件。在服务器出现问题时，它也可以通过一个静态的回退形式来支持一个动态的配置。

- Language bindings: 工作方法类似于mod_xml_curl, 用户可以使用系统支持的脚本语言(Lua, Perl, Java, Python, 和其他语言)来生成一个动态配置。

- originate API: originate API工作的方式比较独特，它可以在系统中创建一个新的电话呼叫，因此取名为originate. 我们将通过fs_cli 简单演示如何通过originate API创建一个呼叫。

- Event Socket/ESL: Event Socket和ESL (Event Socket Library)提供了一个强大的机制，可以对FreeSWITCH进行控制。在这一章节，我们将简要介绍Event Socket，在第十章，通过外部手段控制FreeSWITCH中，我们将做详细讨论。

mod_xml_curl基础

mod_xml_curl模块借用了著名的 cURL库 (curl.haxx.se) 从web 服务器提取XML配置文件。FreeSWITCH 可以及时解析这些文件，作为静态XML配置文件使用。因为用户可以控制web服务器，用户可以从请求中的获得XML，然后修改这些XML文件。当用户需要从同一台web 服务器配置多台FreeSWITCH时，这种方式对用户来说非常有帮助。这样设置的好处是可以通过修改一个地方来控制整个服务器集群。

所有在这一章节的mod_xml_curl举例以前web 服务器可以运行PHP脚本。这些举例已经在Apache2 和mod_php测试，因此可以在Linux/UNIX 和Windows运行。

为了让mod_xml_curl 在FreeSWITCH 启动时成功加载，需要完成以下几步:

1. 使用编辑器，打开conf/autoload_configs/modules.conf.xml 文件，添加mod_ xml_curl 模块:

```
<load module="mod_xml_curl"/>
```

2. 保存文件。



mod_xml_curl已经在Windows 环境下默认安装，但是没有在Linux/Unix安装。在FreeSWITCH 源代码目录下，检查module.conf文件, 确认启用mod_xml_cur，然后执行编译安装命令:
make mod_xml_curl-install.

就像用户想象的那样，我们必须对FreeSWITCH 提供一个URL，确保可以通过这个URL 提取需要的配置文件。我们可以编辑文件xml_curl.conf.xml 来实现，执行以下几步:

3. 使用编辑器，打开文件conf/autoload_configs/xml_curl.conf.xml，添加下面的配置:

```
<configuration name="xml_curl.conf" description="curl
conf">
<bindings>
  <binding name="arbitrary_name">
    <param name="gateway-url"
    value="http://localhost/xml_curl/index.php"
    bindings="configuration|dialplan|directory"/>
  </binding>
</bindings>
</configuration>
```

4. 保存文件。

5. 重新启动FreeSWITCH。

用户可以自定义binding name 属性，建议这个属性具有可读性。在binding中的param 支持一个URL值，这个URL获得名次请求的配置文件内容。在我们的例子中，使用了本地的PHP脚本，但是，用户可以使用一个公网的web服务器。就像用户看到的一样，我们选择了FreeSWITCH 环境的configuration，dialplan和directory 作为web服务器的一个页面请求。

非常关键的地方是，当FreeSWITCH 从用户的基本获得一个请求时，这个脚本可能不知道如何做出回复。使用以下的回复来告诉FreeSWITCH ，脚本不知道如何处理这个请求，用户可以通过在本地配置文件设置:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<document type="freeswitch/xml">
  <section name="result">
    <result status="not found"/>
  </section>
</document>
```

这样的方式非常有助于我们问题的处理，例如，用户想在拨号规则中处理几个Extensions或者contexts，或用户仅想对几个模块支持一些特别的选项。在接下来的部分，我们将演示一些完全可以按照我们要求来执行的例子。首先，我们将介绍index.php文件，这个文件负责对所有请求的驱动。把以下文件保存为index.php，保存在服务器上一个正确的路径:

```
<?php
  function not_found( $msg = '' ) {
    print "<document type=\"freeswitch/xml\">\n";
    print " <section name=\"result\">\n";
    print "<result status=\"not found\"/>\n";
    print " </section>\n";
    print " <!-- $msg -->\n";
    print "</document>\n";
    exit;
  }
  header( "Content-Type: text/xml" );
  print "<?xml version=\"1.0\" encoding=\"UTF-8\"
  standalone=\"no\"?>\n";
  if ( !array_key_exists('section', $_REQUEST ) ) {
    not_found( 'no section passed' );
  }
```

```
if ( !preg_match( '/^(directory|dialplan|configuration)$/',
$_REQUEST['section'] ) ) {
   not_found( 'section not valid' );
}
$gen_file = "${_REQUEST['section']}.php";
if ( file_exists( $gen_file )  && is_readable($gen_file)) {
   include $gen_file;
} else {
   not_found( '$gen_file not found' );
}
```

not_found() 函数打印没有找到的XML配置。这个例子中，我们使用静态的XML来处理这个请求，或者请求错误状态，最后我们响应一个XML 提示信息。

接下来的几行代码 (以header和print开始的) 将确保浏览器或者FreeSWITCH 识别XML响应并且可以正确处理这些响应。

接下来的三行代码的逻辑块确保请求包含一个section，以便让我们知道服务器端回复给FreeSWITCH的数据。如果没有传递 section，我们将返回一个 not found XML提示。

接下来将检查section 是 dialplan, directory, 或者configuration。如果不是以上三个sections其中之一，这个请求将收到一个 not found XML 响应消息，表示FreeSWITCH应该在本地查询这些正在被检测的配置。

最后的if 代码块确保，系统中有一个文件来处理请求中的section，并且这个文件是可读文件。

在以下部分，我们将看到如何通过dialplan.php 来处理拨号规则请求，这个文件和index.php文件在同样的文件夹。

mod_xml_curl拨号规则

在这个部分，我们将演示如何通过mod_xml_curl 来处理book_test 拨号规则context 和让其他的contexts 回退到静态的XML拨号规则。添加dialplan.php 到同样的index.php 文件路径:

```
<?php
  if ( !array_key_exists('Hunt-Context',$_REQUEST ) ||
  $_REQUEST['Hunt-Context'] != 'book_test' ) {
     not_found('not our context');
  }
  print "<document type=\"freeswitch/xml\">\n";
  print " <section name=\"dialplan\">\n";
  print "<context name=\"${_REQUEST['Hunt-Context']}\">\n";
  print "<extension name=\"no_name\">\n";
  print "<condition>\n";
  print "<action application=\"info\">\n";
  print "</condition>\n";
  print "</extension>\n";
  print "</context>\n";
  print " </section>\n";
  print "</document>\n";
```

FreeSWITCH将对我们的web服务器发送多个POST参数的请求。
使用频繁的参数是Hunt-Context和Hunt-Destination-Number，它们对应的是拨号规则中的context 和destination_number，这些参数通常设置在静态的XML拨号规则)。
Dialplan.php 前三行代码确保请求中的Hunt-Context是存在的，Hunt-Context是book_test，否则返回 not found XML 提示信息，通知FreeSWITCH 继续在其他地方查找context。其他的脚本打印出XML，这里包含一个extension。

非常有用的工具来测试web服务器的命令是curl。在Linux/UNIX系统中，执行这个命令：

```
curl -D- http://localhost/xml_curl/index.php -d
'section=dialplan&Hunt-Context=book_test'
```

输出结果类似于:

```
HTTP/1.1 200 OK
...Content-Type: text/xml
...
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<document type="freeswitch/xml">
  <section name="dialplan">
    <context name="book_test">
      <extension name="no_name">
        <condition>
          <action application="info"/>
        </condition>
      </extension>
    </context>
  </section>
</document>
```

从命令行执行cURL可以看到许多举例。我们通常执行-D 选项 转储响应的头字段信息。在我们的例子中，-D- 命令cURL转储头字段信息在显示器显示，不保存在文件中，所以我们可以立即看到这些信息。

注意，<condition>标签中没有任何属性。这样做的原因是，我们可以在外面的脚本中实时做出一个路由决定，仅返回一个XML结果。但是，用户仍然可以选择打印conditions的字段和expression属性，来支持FreeSWITCH 进行逻辑判断做出决定。大部分实现的结果是返回一个单一的extension，但是还有一些返回完整的context，并且使用mod_xml_curl在多服务器环境来，以便保持配置文件的一致性。



在这个章节的例子中，我们在linux环境下使用了cURL命令。但是，curl.exe也同样支持32-bit和64-bit Windows版本。参考: http://curl.haxx.se/dlwiz/?type=bin 下载可执行文件。

mod_xml_curl文件夹

在这个部分，我们将演示如何FreeSWITCH对文件夹请求做出响应。这个例子中允许任何使用密码1234的用户注册。这个例子表示仅仅演示一个FreeSWITCH返回有效XML的举例，它不能部署在实际的生产环境因为这里牵扯一个安全问题。

添加文件directory.php，文件保存路径和index.php一样。

以下是directory.php 文件内容:

```
<?php
  if ( !array_key_exists( 'domain', $_REQUEST ) ||
  !array_key_exists( 'user', $_REQUEST ) ) {
     not_found( 'missing domain or user' );
  }
  print "<document type=\"freeswitch/xml\">\n";
  print " <section name=\"directory\">\n";
  print "<domain name=\"${_REQUEST['domain']}\">\n";
  print "<groups>\n";
  print "<group name=\"default\">\n";
  print "<users>\n";
  print "<user id=\"${_REQUEST['user']}\">\n";
  print "<params>\n";
  print "<param name=\"password\"
  value=\"1234\">\n";
  print "</params>\n";
  print "<variables>\n";
  print "<variable name=\"user_context\"
  value=\"default\">\n";
  print "</variables>\n";
  print "</user>\n";
  print "</users>\n";
  print "</group>\n";
  print "</groups>\n";
  print "</domain>\n";
  print " </section>\n";
  print "</document>\n";
```

和dialplan请求一样，directory 请求通过POST参数发送一个号码，这个号码帮助用户来决定如何响应这个请求。大部分经常使用的参数是domain和user。在脚本的前三行，在执行之前，我们检查这些参数是否存在。就像以前的例子，如果请求的参数不存在，我们将返回一个not found XML错误提示信息。

其他的脚本将打印XML结果，这些结果是FreeSWITCH需要处理的。注意，在<domain> 和<user>中，我们仅简单回应从我们这里发出去的请求。这里将返回一个有效的XML响应消息和密码为1234的注册用户。再次说明，这样做不是一个好的方法，不能使用在生产系统，这样会有很多人使用认证的帐号，从受保护的contexts 呼出。这样做是非常危险的。这个例子仅演示一个如何对directory 请求返回有效的响应。在生产环境中，大部分情况下，我们需要将系统用户信息保存在数据库，脚本来查询需要的用户信息。

在系统中，执行以下命令:

```
curl -D- http://localhost/xml_curl/index.php -d
'section=directory&domain=example.com&user=1000'
```

输出结果类似于:

```
HTTP/1.1 200 OK
...
Content-Type: text/xml
...
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<document type="freeswitch/xml">
  <section name="directory">
    <domain name="example.com">
      <groups>
        <group name="default">
          <users>
            <user id="1000">
              <params>
                <param name="password" value="1234"/>
              </params>
              <variables>
                <variable name="user_context" value="default"/>
              </variables>
            </user>
          </users>
        </group>
      <groups>
    </domain>
  </section>
</document>
```

请注意一下<domain>标签中的name属性，和<user> 标签中的id 属性，如果用户通过curl请求发送不同的参数时，观察一下这些属性是如何变化的。牢记，在一个<domain>标签下的组或者用户都属于同一个domain。同样，在<group> 标签中的用户属于同一组。

mod_xml_curl 配置

用户可以通过使用mod_xml_curl和configuration binding，对FreeSWITCH提供多个配置文件。在这个部分，我们将动态生成一个sofia.conf 文件，并且构建一个框架以便让其他的配置变得相对轻松。我们将在web服务器端添加几个新文件和子文件夹。

添加以下文件configuration.php，保存在和index.php 一样的路径：

```
<?php
  if ( array_key_exists( 'key_value', $_REQUEST ) ) {
    $conf = $_REQUEST['key_value'];
    if ( is_file( "configuration/$conf.php" ) ) {
      include_once( "configuration/$conf.php" );
    } else {
      not_found( "unable to find config script ($conf.php)" );
    }
  }
```

注意，configuration.php将通过子文件夹configuration查找特定的配置。在包含文件index.php和configuration.php的路径下，创建一个子文件夹configuration。

在configuration文件夹下，创建一个文件sofia.conf.php：

```
<?php
  print "<document type=\"freeswitch/xml\">\n";
  print " <section name=\"configuration\">\n";
  print "<configuration name=\"sofia.conf\">\n";
  print "<profiles>\n";
  print "<profile name=\"internal\">\n";
  print "<settings>\n";
  print "<param name=\"sip-ip\"
  value=\"${_SERVER['REMOTE_ADDR']}\"/>\n";
  print "<param name=\"rtp-ip\"
  value=\"${_SERVER['REMOTE_ADDR']}\"/>\n";
  print "<param name=\"sip-port\" value=\"5060\"/>\n";
  print "</settings>\n";
  print "</profile>\n";
  print "</profiles>\n";
  print "</configuration>\n";
  print " </section>\n";
  print "</document>\n";
```

除了打印出FreeSWITCH XML的响应消息，文件sofia.conf.php 文件没有什么有用的内容。
这里我们假设，因为configuration.php 成功发现了这个文件，加载了这个文件，所以合理的结果只能是这些XML信息。唯一值得做的一件事就是，sip-ip和rtp-ip 设置了一个IP地址，这个地址是发送请求的地址，在我们的例子中，我们使用127.0.0.1。

让我们测试一下配置。从系统环境下执行命令:

```
curl -D- http://localhost/xml_curl/index.php -d
'section=configuration&key_value=sofia.conf'
```

类似的结果是:

```
HTTP/1.1 200 OK
...
Content-Type: text/xml
...
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<document type="freeswitch/xml">
  <section name="configuration">
    <configuration name="sofia.conf">
      <profiles>
        <profile name="internal">
          <settings>
            <param name="sip-ip" value="127.0.0.1"/>
            <param name="rtp-ip" value="127.0.0.1"/>
            <param name="sip-port" value="5060"/>
          </settings>
        </profile>
      </profiles>
    </configuration>
  </section>
</document>
```

就像以前提到的那样，rtp-ip和sip-ip 这两个地址是发送请求的地址。在这个场景中，我们使用了127.0.0.1，这个地址不是一个非常有用的地址。想象一下，如果我们有一个集群的解决方案，有很多个FreeSWITCH在运行，除了绑定的IP地址不同，都有同样的配置文件。在一个实际的环境中，突然，这个实例开始显示有用的标识。

另外一件事情应该注意，当用户开始检测这些请求时，其中一些对配置文件的请求带有一个post_load_ prefix。这些文件要求在加载mod_xml_curl之前，用户必须在这些配置文件中提供一个设置(例如，modules.conf.xml和switch.conf.xml)。在post_load_ files 文件中的内容允许和正常的静态文件一样。例如，用户可以有一个modules.conf.xml 仅加载mod_xml_curl，剩余的post_load_modules.conf要求的模块可以通过mod_xml_curl响应回复。

mod_xml_curl总结

我们这里演示的大部分的mod_xml_curl 举例都是每次打印同样的XML信息，
因此，它们本质上说是静态的，同时在一些地方可以动态配置。用户可能也注意
或可能我们仅使用打印语句打印了XML文件。在PHP语言中，或者使用其他的语言，
有很多XML生成库\类帮助通过不同的字符编码生成有效，完整的XML。
例如，PHP 有SimpleXML拓展(http://php.net/manual/en/book.simplexml.php)被系统
使用。作为一个基本的规则，如果用户对一个项目实施非常认真的话，用户应
该使用这样的库。

通过语言绑定生成动态配置文件

如果用户没有设置web 服务器来处理脚本，FreeSWITCH 提供了一个可选的方法，
它可以使用内置的脚本语言来处理同样的绑定请求。所有脚本语言模块支持用户
设置一些参数，所以用户可以用脚本语言来处理请求，工作方式类似于
mod_xml_curl 脚本。使用最普遍的语言是:

- Lua 和 mod_lua
- Perl 和 mod_perl
- Python 和 mod_python



FreeSWITCH 也支持Microsoft.NET 语言，通过mod_managed实现。
但是使用方法和其他语言Lua, Perl, 和Python有所不同。更多信息，请
访问http://wiki.freeswitch.org/wiki/Mod_managed.

查找conf/autoload_configs/文件夹，可以看到相应的配置文件:

- lua.conf.xml
- perl.conf.xml
- python.conf.xml

打开其中一个文件，用户可以看到这些参数:

```
<param name="xml-handler-bindings"
value="dialplan|directory|configuration"/>
<param name="xml-handler-script"
value="/path/to/script.ext"/>
```

和你猜到的一样，xml-handler-bindings 是一个配置部分，脚本负责来处理这个部分。因此，xml-handler-script 是脚本语言的执行路径，用来处理请求。



在这些例子中，我们使用mod_lua，同样的原理也可以用在mod_perl 和 mod_python上。

mod_xml_curl和mod_lua 之间主要的区别就是mod_lua是嵌入在FreeSWITCH的模块，无需设置Content-Type ，打印 XML 结果来支持FreeSWITCH 读取。相反，用户仅需要设置一个特别的变量XML_STRING，这个变量包含XML content来支持FreeSWITCH解析。用户可以看一下以下例子:

```
local xml_header = [[<?xml version="1.0" encoding="UTF-8"
standalone="no"?>
<document type="freeswitch/xml">
]]
local xml_body
if XML_REQUEST['section'] == 'configuration' and
XML_REQUEST['key_value'] == 'sofia.conf' then
local ip_v4 = params:getHeader( 'FreeSWITCH-IPv4' )
xml_body = string.format([[
    <section name="configuration">
      <configuration name="sofia.conf">
        <profiles>
          <profile name="internal">
            <settings>
              <param name="sip-ip" value="%s"/>
              <param name="rtp-ip" value="%s"/>
              <param name="sip-port" value="5060"/>
            </settings>
          </profile>
        </profiles>
      </configuration>
    </section>]], ip_v4, ip_v4)
else
```

```
    xml_body = [[
        <section name="result">
          <result status="not found"/>
        </section>
    ]]
end
local xml_footer = [[</document>]]
XML_STRING = xml_header .. xml_body .. xml_footer
```

这个例子应该看起来比较熟悉。这个例子本质上和我们这个章节介绍的
mod_xml_curl 配置上一样的。我们生成的XML打开和关闭的部分永远是一样的。
如果这个section 是configuration，key_value 是sofia.conf，我们将生成一个最小的
sofia.conf。如果要求有其他的部分和其他的配置，我们仅返回同样的错误提示
信息 not_found XML，这里的信息提示和mod_xml_curl是一样的。

咋看上去，在集群环境下，让这些脚本保持同步是一件非常麻烦的事情。
但是，如果用户考虑使用Box，Dropbox等其他第三方的解决办法的可能
性，通过这些解决方案来支持跨平台文件的同步，那么，用户可以考虑
使用mod_xml_curl来完美解决这些问题，因为mod_xml_curl 不要求使用web
服务器端。

从命令接口发起呼叫

用户可以在系统无用户的情况下发起呼叫。例如，我们假设用户有一个
终端，用户可以对这个终端发起一个未验证的呼叫。这个终端可能是一个
物理的IP电话机，软电话和甚至于另外一个FreeSWITCH的注册用户。
唯一的要求是呼叫的URL可以让一个电话振铃，用户可以应答这个电话。
在我们的例子中，我们使用了my.open.endpoint.example.com 作为一个目的地域名。
确认，在用户的环境中，使用合适的域名或IP地址。

打开fs_cli，执行这个命令：

```
originate sofia/internal/1234@my.open.endpoint.example.com &echo()
```

当然，在真实的环境中，这样的呼叫方式没有什么用处。通过这样的测试方式,
自己对自己说 "Hello, testing one, two, three…"，可以进行双向语音测试，但是
我们可以再添加一些内容，可以让这个实例变得更可用.

在下面的举例中，我们对终端发起一个呼叫，然后桥接到FreeSWITCH 会议室，
这个会议室的拨号规则已经在FreeSWITCH默认环境安装。

```
originate sofia/internal/1234@my.open.endpoint.example.com
&transfer(9888 XML default)
```

在上面的例子中，我们首先拨打终端。当接听电话以后，我们转接这个通话
到默认的XML拨号规则context，9888 将匹配默认context 中的
freeswitch_public_conf_via_sip extension，然后桥接这个通话。

在下一个例子中，我们通过SIP 桥接到FreeSWITCH 会议，但是这里不需要
拨号规则，因为我们从originate 命令桥接这个呼叫:

```
originate sofia/internal/1234@my.open.endpoint.example.com
&bridge(sofia/internal/888@conference.freeswitch.org)
```

可能大家对以上的两个例子有一些争论，这些例子没有实际用处，当然，那是
东部时间的星期三下午1点。在那个例子中，用户可能发现只有自己在那个
FreeSWITCH 电话开发者会议室。

用户可以稍微修改一下这些例子，可以创建一个页面点击的呼叫方式来启动脚本。
用户可以创建一个页面表格的形式，用户可以通过页面输入被呼叫的电话号码，然后
发生这个表格到一个处理流程，这个流程启用命令originate 连接到用户的IVR主菜单。

```
originate sofia/gateway/my_provider/12125551234 &ivr(main_menu)
```

originate 命令将通过网关my_provider 发起一个呼叫到用户12125551234，用户接听
以后，就可以引导用户进入 main_menu IVR 主菜单。用户可能觉得奇怪，怎么从页面输入的
号码可以连接到fs_cli。一种方法是通过页面表格处理流程启动系统命令，这个系统命令
包含fs_cli，例如:

```
fs_cli –x 'originate sofia/gateway/my_provider/12125551234
&ivr(main_menu)'
```

注意，使用fs_cli 必须带 - x（执行）参数。使用fs_cli - x 比较简洁明了。
但是，完全看用户的使用环境，这样的方式可能不是最具有拓展性和最有效的方式。
以下部分将介绍FreeSWITCH最为强大的方面，就是event socket 和ESL。

使用ESL执行命令

在上个举例中，我们从fs_cli 命令发起了呼叫。在这个部分，我们将演示一些代码实例，这些实例可以完成和脚本一样的工作。从举例中，我们将使用Lua，因为现在我们已经习惯了Lua语言。无论用户使用Python，Lua，PHP或者 Perl和其他的语言，ESL API 接口始终是一样的，因此，每一种语言都有风险，最好选择自己喜欢的语言。



ESL 脚本与内置语言

始终牢记，基于ESL的程序和内置语言是不一样的。
FreeSWITCH event socket 是通过TCP连接FreeSWITCH。
ESL是一个抽象库，可以支持多种语言，不像内置脚本仅仅支持很少的语言。在使用ESL前，用户必须在系统中首先安装Lua, Perl, Python, 或者PHP。关于使用Lua语言，请访问 http://www.lua.org 获得更多信息。

首先明确，因为ESL 模块不是默认安装的，我们需要编译这个模块，然后才能使用。在Linux/UNIX 环境下，用户可以执行以下几步:

1. 找到FreeSWITCH ESL 文件夹:

   ```
   cd ${FS_SRC}/libs/esl
   ```

2. 执行以下命令:

   ```
   make luamod
   ```

假设命令成功编译，用户有了Lua ESL模块，保存在路径 ${FS_SRC}/libs/esl/lua as ESL.so。



如果用户在安装Lua ESL模块时，有什么问题，请先安装Lua 的开发模块。

用户可以迁移ESL.so文件到任何路径，确认当用户运行脚本的时候，Lua模块可以查找，并且可以加载这个这个模块文件。最简单的办法就是从${FS_SRC}/libs/esl 文件夹路径来执行 ./lua/single_command.lua 来查找这些文件。执行命令以后会返回用户结果：

```
no file './ESL.so'
no file '/usr/local/lib/lua/5.1/ESL.so'
no file '/usr/lib/x86_64-linux-gnu/lua/5.1/ESL.so'
no file '/usr/lib/lua/5.1/ESL.so'
```

我们注意到，系统在当前目录下查找，然后返回几个内置的Lua 库的路径。因为，我们不想在每个脚本执行目录下保存一个ESL.so的备份文件。在这个例子中，用户系统可能有不同的安装执行路径。确认执行single_command.lua 命令，找出正确的执行路径。运行这些命令：

```
sudo mkdir -p /usr/local/lib/lua/5.1/
sudo cp lua/ESL.so /usr/local/lib/lua/5.1/
./lua/single_command.lua
```

注意，我们创建了文件夹，如果这个文件不存在就拷贝一份ESL.so 文件到此文件夹。最后，再次运行single_command.lua，确保可以找到 ESL.so文件。



Windows 用户必须使用Microsoft Visual Studio编译mod_esl (编译好的二进制文件不能满足需求)。在 mod_esl右点，然后点击编辑。系统将创建一个esl.dll文件。拷贝这个文件到Lua安装clib 的子文件夹。当从Windows环境下运行Lua 脚本时，使用系统提示符，然后输入lua脚本名称。例如：lua single_command.lua.

如果所有步骤成功执行，用户应该可以看到这样的错误:

```
/usr/bin/lua: Error in api (arg 2), expected 'char const *' got 'nil'
stack traceback:
        [C]: in function 'api'
        ./lua/single_command.lua:9: in main chunk
        [C]: ?
```

如果用户看到的也是那样的错误，说明一切正常。这个错误只是告诉用户我们没有对脚本single_command.lua提供命令输入。现在让我们开始测试。运行这个命令:

```
./lua/single_command.lua status
```

现在我们执行single_command.lua，并且带一个参数 status API命令，脚本应该会显示类似的结果如下：

```
UP 0 years, 0 days, 5 hours, 6 minutes, 27 seconds, 223 milliseconds, 190
microseconds

FreeSWITCH (Version 1.2.8) is ready

0 session(s) since startup

0 session(s) - 0 out of max 30 per sec

1000 session(s) max

min idle cpu 0.00/96.00

Current Stack Size/Max 240K/240K
```

如果用户可以看到我们需要的结果，那说明用户可以使用类似single_command.lua的脚本传递脚本任何API的命令参数。因此用户可以使用originate例子，从Lua中调用ESL。可以使用这样的方法尝试以下例子:

```
./lua/single_command.lua 'originate
sofia/gateway/my_provider/12125551234 &ivr(main_menu)'
```

还有一种方法比目前的办法更加有效。目前的办法是FreeSWITCH运行在这个安装脚本的系统中，更加有效的办法是在独立的服务器端运行这个脚本文件。高兴的是，用户还有一个选择。配置mod_event_socket可以支持用户通过外部命令控制FreeSWITCH。下一个章节将详细讨论这个话题，章节名称为通过外部命令控制FreeSWITCH。

总结

使用本章讨论的策略，用户可以开发一个FreeSWITCH集群解决方案，他们都配备最少的XML配置，并且通过页面服务器获得更多配置。这里任何集群的FreeSWITCH服务器都不需要用户目录或者拨号规则，当需要这些文件时，这些配置文件可以通过web服务器及时提取出来。通过脚本语言调用ESL可以发起呼叫，管理语音留言等等功能，现在用户可以创建一个集群的FreeSWITCH环境，可以不通过登录系统来对系统进行完全管理。

我们讨论过的这些方法仅是非常短小的实例，这些实例可以配置或控制FreeSWITCH，无需依赖任何单一的静态XML文件。在下一个章节，我们将学习更加有效的方法来控制FreeSWITCH，因为，现在我们深入了解了强大的FreeSWITCH 事件套接字。

10
外部命令FreeSWITCH

FreeSWITCH 事件系统是FreeSWITCH最有趣的模块之一。用户已经学习了如何使用各种静态配置维护和脚本语言来配合FreeSWITCH工作。事件系统实现了对FreeSWITCH实时动态的控制。使用了事件系统才使得FreeSWITCH变得更加有趣。

事件系统支持外部的程序作为一个监听者来监听系统正在执行的的任何任务。通过外部监听的方式可以支持一个实时的互动，通信软交换系统的电话可以通过外部的软件或者已经进行对接。在FreeSWITCH环境中，几乎所有发生的指令可以生成一系列的事件信息。外部实体可以检测到这些事件。类似于在平台信息队列软件使用的publish/subscribe(或 "pub-sub")系统，这些事件机制已经调整来满足FreeSWITCH 事件系统的需求。

事件系统是双向工作的：支持外部的程序来监听事件，然后外部程序可以对FreeSWITCH发送事件。用户可以通过自己的程序来实时地接收和发送事件。这样的组合方式可以实现大部分用户可以想象到的方式对FreeSWITCH进行外部控制。

在这一章节，我们将讨论以下内容：

- 事件系统的基本概要
- 事件系统的架构
- 访问事件套接字
- 事件套接字库
- 在PHP中使用ESL实例
- 使用事件系统创建一个会议管理员

概要

事件系统是FreeSWITCH的神经中枢，支持本地软件或外部软件订阅在软交换系统中正在发生的任务流。在FreeSWITCH中，执行任何的任务会生成一个事件。接收一个新的电话呼叫会导致一个事件。结束一个呼叫同样也导致一个事件发生。对硬盘写入系统的日志会导致一个事件。甚至于讲话或者静音也会导致一个事件。每个事件变成了事件流的一个部分，事件流可以触发一个事件类型，事件类别和各种事件信息。对端软件可以监听这些事件，通过事件执行相应的任务，例如，通过TCP 套接字连接发送文本流。

事件通过另外一种方式对FreeSWITCH提供一个功能扩展。事件不同于钩子或模块(在实时环境中，这些会影响处理和呼叫控制)。事件提供一个异步机制 (或非阻塞/队列) 来维持系统的任务跟踪。这些事件通过其他软件生成。在实际环境中，用户可以通过外部程序来控制系统中多个任务活动。

在这样的例子中，用户系统可能突然有一个呼叫高峰，这个环境高峰可能生成新的事件。用户可以通过web浏览器来处理这些事件，但是浏览器不能马上应对这个呼叫量。使用队列事件系统，用户可以等待浏览器提高处理能力，来应对此时的呼叫量，避免核心软交换引擎的阻塞（自己阻塞呼叫)。

在这个章节，我们将讨论事件系统几个不同的方面—从外部程序接收和处理事件和对FreeSWITCH发送事件。我们涵盖的模块包括，从外部启用事件系统，生成的事件类型和对事件类型调用的方法。最后，我们将了解一个实例使用场景和代码来帮助用户开启一个使用环境，通过自己的程序来控制FreeSWITCH。

事件系统架构

FreeSWITCH中的事件子系统支持以下两个方面的功能，一个是最大化的处理设计，另外一个是事件优先级设计，优先级取决于事件类型和事件系统加载结果。在FreeSWITCH的事件系统中包含两个层次的处理。第一层提供一个内部事件处理路由和一个吸纳FreeSWITCH自身内部事件的接口。第二层是一个模块架构，提供客户端对事件的访问。通过两个功能单元的分割独立，实现了publish/subscribe风格的事件系统。

在内部的事件层，FreeSWITCH 提供了核心功能来处理发生在系统级别和通道级别的事件。系统的任何部分包括模块可以发布或广播事件。目前有两个核心类型事件—系统事件和日志事件。核心子系统部分或模块可以生成系统事件。他们包括从系统内部定时器心跳到会议子系统事件，例如会议一方加入或者离开会议室。每次一个日志写入到FreeSWITCH日志文件时，系统会生成一个日志事件。这些子系统实际上由三个事件队列组成，每个队列有自己的线程和优先级。如果一个队列被填满以后，系统将自动跳到下一个队列，直到全部事件系统被填满。当呼叫或者系统功能处于处理状态时，后端的线程会生成事件，这些事件保存到内存中等待内部的订阅者来调用。一旦订阅的模块或子系统提取了信息，这个事件信息将被消灭。通过这样的方式，事件系统的拓展性得到了提升，已经使用的事件立即清除，这样当事件用户等待提取队列事件时不会导致呼叫被阻塞。

FreeSWITCH使用它的模块架构可以支持通过外部软件来处理事件。一个事件处理模块可以订阅内部事件信息，对其进行格式化，然后发送到外部程序。这样的模块称之为事件处理event handlers。FreeSWITCH没有绑定很多的event Handlers，但是这些event handlers功能非常丰富，处理能力强大。我们将再次回顾这些模块，学习如何使用这些模块。

基于事件的模块

有很多模块可以管理事件。目前，使用最普遍的是mod_event_socket。我们将重点讨论这个模块，然后简单介绍其他的结果模块。

mod_event_socket

在FreeSWITCH环境中，mod_event_socket 是使用最为普遍的模块，可以通过第三方来接收和发送事件。用户可以通过外部第三方的TCP 套接字连接这个模块。一旦认证成功，用户可以发送或接收普通的文本事件信息，这些信息可以容易理解和解析。它支持双向通信，用户可以轻松对FreeSWITCH发送或接收信息。

使用事件套接字是非常简单的。首先，用户需要从外部程序配置一个预设的套接字设置，这个设置需要支持mod_event_socket。如果用户成功通过认证，用户就可以对FreeSWITCH 发送事件信息。用户也可以发起一个请求来接收事件，这些事件是通过mod_event_socket挂载事件监听器到事件系统，对事件信息进行队列处理以后，发送这些事件到用户端，以便让用户可以尽可能快速处理这些信息。

mod_event_socket 模块开放一个接口，用户可以通过这个接口来请求接收一个普通文本信息，事件序列化拷贝处理，生成用户自己的事件。用户也可以选择请求接收XML格式的数据。这个模块包括一个事件过滤机制，支持用户订阅自己需要的事件类型。例如，用户程序可能专门设计仅处理会议功能；因此用户仅需要接收和会议相关的事件。这个模块最后负责捕捉来自于内部事件系统的事件，然后把这些回复这些事件，发送到每一个活动的TCP连接。每个TCP连接处理事件的数量周期不同，因此这个模块负责设置和维护每个队列对每个个体的TCP连接工作。队列本身就是FreeSWITCH核心操作的一个部分。

配置事件套接字设置

用户可以通过简单命令来启用加载事件套接字系统，在modules.conf.xml 文件中添加mod_event_socket来加载这个模块。加载模块以后，通过配置event_socket.conf.xml文件来配置mod_event_socket 模块。这个模块可以支持以下参数:

- listen-ip: 事件套接字监听的IP地址。外部程序将通过这个地址来建立一个连接。默认的监听连接地址是local host。用户可以指定一个其他的地址或设置为0.0.0.0来监听所有的内网IP地址。

  ```
  <param name="listen-ip" value="127.0.0.1"/>
  ```

- listen-port: 监听的端口。

  ```
  <param name="listen-port" value="8021"/>
  ```

- password: 连接端口要求的认证密码。

  ```
  <param name="password" value="ClueCon"/>
  ```

- apply-inbound-acl: Access Control List (ACL) 用来对这个端口的连接进行控制。通过这样的方式，用户可以严密控制任何已设置的IP:port 组合连接。用户可以使用已知的访问控制列表名称(设置在conf/autoload_configs/acl.conf.xml)或用户可以使用一个实际IP地址范围。

  ```
  <param name="apply-inbound-acl" value="<acl_list|cidr>"/>
  ```

  使用的apply-inbound-acl实例:

  ```
  <param name="apply-inbound-acl" value="known_machines"/>
  <param name="apply-inbound-acl" value="10.20.0.0/16"/>
  ```

  注意，多个apply-inbound-acl 参数将不能工作。

读取事件

当从mod_event_socket模块中读取事件时，数据的格式为名称和值的形式，中间以逗号分开。事件信息是以连续两个换行符为结束标识。FreeSWITCH使用传统的DOS/Windows回车换行符。用户的外部程序连接事件套接字，读取事件信息字符，直到发现这个换行符。以下是一个单行的一对key/value 格式举例:

```
Event-Name: CHANNEL_EVENT
```

有一些key/value 这样的值中包含多行间断的值。在这样的环境下，FreeSWITCH 仍然认定这些值为一个单行的值。为了解决这个问题，FreeSWITCH 使用了URL编码来处理这些数据，这些数据将显示为一行数据。以下是一个多行信息的回复：

```
variable_switch_r_sdp: v%3D0%0D%0Ao%3DUAC%206407%206867%20IN%20IP4%20
192.168.27.72%0D%0As%3DSIP%20Media%20Capabilities%0D%0Ac%3DIN%20
IP4%2061.231.8.102%0D%0At%3D0%200%0D%0Am%3Daudio%2012916%20RTP/AVP%20
0%2018%20101%0D%0Aa%3Drtpmap%3A0%20PCMU/8000%0D%0Aa%3Drtpmap%3A18%20
G729/8000%0D%0Aa%3Dfmtp%3A18%20annexb%3Dno%0D%0Aa%3Drtpmap%3A101%20
telephone-event/8000%0D%0Aa%3Dfmtp%3A101%200-15%0D%0Aa%3Dmaxptime%3A2
0%0D%0A
```

上面的例子是FreeSWITCH 呼叫过程中经过URL编码处理的SDP结构体。信息本身的原貌应该是这样的:

```
variable_switch_r_sdp: v=0
o=UAC 6407 6867 IN IP4 192.168.27.72
s=SIP Media Capabilities
c=IN IP4 61.231.8.102
t=0 0
m=audio 12916 RTP/AVP 0 18 101
a=rtpmap:0 PCMU/8000
a=rtpmap:18 G729/8000
a=fmtp:18 annexb=no
a=rtpmap:101 telephone-event/8000
a=fmtp:101 0-15
a=maxptime:20
```

如果任何一个name/value 的值中包含一个Content-Length头域，用户程序需要从套接字中读取从起始头域信息到两个换行符之间的准确事件信息。一旦完成从content length读取的信息，将启动读取下一个数据包。当读取的事件中包含Content-Length头，说明在这个事件中生成了其他的信息，这些信息的格式不是key/value这样的格式，包含信息自己特有的格式。

以下举例是一个事件举例，这个事件是一个通道状态修改的通告事件：

```
Content-Length: 646
Content-Type: text/event-plain
Channel-State: CS_EXECUTE
Channel-State-Number: 4
Channel-Name: sofia/default/1006%4010.0.1.250%3A5060
Unique-ID: 74775b0d-b112-46e2-95af-c28258650b1b
Call-Direction: inbound
Answer-State: ringing
Event-Name: CHANNEL_STATE
Core-UUID: 2130a7d1-c1f7-44cd-8fae-8ed5946f3cec
FreeSWITCH-Hostname: localhost.localdomain
FreeSWITCH-IPv4: 10.0.1.250
FreeSWITCH-IPv6: 127.0.0.1
Event-Date-Local: 2012-12-16%2022%3A33%3A18
Event-Date-GMT: Mon,%2017%20Dec%202012%2004%3A33%3A18%20GMT
Event-Date-timestamp: 1197865998931097
Event-Calling-File: switch_channel.c
Event-Calling-Function: switch_channel_perform_set_running_state
Event-Calling-Line-Number: 620
```

注意，加黑的那一行表示Event-Name 是CHANNEL_STATE。这个事件和通道状态的修改相关。

最少事件信息

无论事件的类型是什么样的类型，每次通过mod_event_socket 模块从FreeSWITCH接收到事件都包含最少事件信息或基本事件信息。这些字段提供了事件的基本信息，这些信息不仅仅帮助用户了解事件类型，同时也可以了解事件发生的时间和这些事件是发生哪一个服务器。在多个服务器环境下，这些字段非常有用，Core-UUID 头可以帮助系统获知哪一个服务器生成的事件，而且时间戳可以确保以正确的顺序来管理事件结构重组和事件处理。

用户也可以接收其他的事件字段，例如:

```
Event-Name: CHANNEL_EVENT
Core-UUID: 689fd828-e85b-ca43-a219-39332bc55860
Event-Date-Local: 2012-05-09%2018%3A48%3A59
Event-Date-GMT: Wed,%2009%20May%202012%2016%3A48%3A59%20GMT
Event-Calling-File: switch_channel.c
Event-Calling-Function: switch_channel_set_caller_profile
Event-Calling-Line-Number: 840
```

无论接收任何事件，事件都包含以上这些基本的信息。说明触发任何一个事件，事件都包含以下字段:

- Event-Name: 事件名称，描述事件类型
- Core-UUID: 当前FreeSWITCH core实体的UUID
- Event-Date-Local: 根据系统时钟获得的事件日期时间
- Event-Date-GMT: GMT时间的事件日期时间
- Event-Calling-File: 发起事件的C 源代码文件
- Event-Calling-Function: 事件发起的函数名称
- Event-Calling-Line-Number: 发起事件的C源代码文件确切行号

最后这三个头字段对于测试和排查问题非常有帮助。根据发送到事件类型的不同，在上面基本信息之后，可能有一些特别的事件信息。在event key/value 对值和event-specific key/value 对值之间没有行断开和空格。

发送事件

用户可以使用同样的TCP连接（连接是双向的），通过via mod_event_socket 对FreeSWITCH 发送事件。命令格式为命令名称和参数。一些命令要求额外的字段来支持。这些额外字段的格式类似于接收额外字段的格式。用户对 FreeSWITCH 发送一个对值列表，指定事件名称和定义的事件相关标志符，FreeSWITCH 将注入这些信息到事件子系统，以便模块或者FreeSWITCH core 来处理。

An一个基本的命令实例：

```
api sleep 5000
```

以上命令会运行API命令sleep，并且传递参数5000，执行的结果会让系统休眠5秒钟。

比较复杂的实例可以直接在FreeSWITCH 事件队列系统中注入信息。用户可以通过sendevent 命令对FreeSWITCH系统注入事件，并且携带关联的参数。

以下是一个使用sendevent 命令的举例：

```
sendevent NOTIFY
profile: internal
content-type: application/simple-message-summary
event-string: check-sync
user: 1005
host: 192.168.10.4
content-length: 5
hello
```

以上举例发送了一个NOTIFY 事件和相关的信息。在这个例子中，我们要求NOTIFY消息发送到用户Sofia's internal profile 的 1005@192.168.10.4。如果mod_sofia已经加载，正在监听它们的信息类型，mod_sofia将对用户1005的NOTIFY信息生成SIP数据包，并且在SIP消息content中包含content-type和event-string头，这个例子中的content是hello。

注意，所有发送到FreeSWITCH的事件信息必须以两个连续的回车换行符为结束符。

用户可以使用事件命令对FreeSWITCH发送事件，完整的发送事件命令列表在本章的FreeSWITCH事件系统命令部分做更多介绍。

从拨号规则调用事件

mod_event_socket提供了一个拨号规则应用socket，支持通过IP地址和端口外连TCP连接，对端程序执行命令对FreeSWITCH进行控制。这样的工作方式类似于Asterisk环境中的FAGI，但是它可以在全异步模式下工作，支持命令执行和立即返回其他事件或回复结果。

当用户呼叫外连的套接字socket时，FreeSWITCH自动设置为驻留呼叫。
用户可以通过检测CHANNEL_PARK事件来查看这个在驻留状态的呼叫。

从拨号规则中呼叫 socket的语法:

```
<ip>:<port> [<keywords>]
```

以下是在拨号规则中使用实例:

```
<action    application="socket"          data="127.0.0.1:8084"/>
<action    application="socket"          data="127.0.0.1:8084 async"/>
<action    application="socket"          data="127.0.0.1:8084 full"/>
<action    application="socket"
           data="127.0.0.1:8084          async full"/>
```

可选关键词async和full可调整流程的执行，如下所示:

- async: async 关键词表示所有命令快速返回，命令栈执行的同时，对事件的套接字检测成为可能。

  如果缺省关键词async，每一个事件套接字命令不会立即返回，直到命令结束。

- full: 关键词full表示对端有事件套接字的完整命令集。它类似于内连带套接字连接的命令集，所以用户可以执行API命令，例如获得全局事件，等等。

  如果缺省关键词full，那么命令集和事件仅支持一个特别的呼叫。
  换句话说，如果没有设置full，通过这个套接字连接发送的命令仅对当前被处理的通道有影响。同样地，套接字连接仅接收和这个特别通道相关的事件。

mod_event_multicast

mod_event_multicast和mod_event_socket非常相似。它可以通过网络广播对第三方程序或其他使用普通文本事件的FreeSWITCH实体来接收和发送事件。
其他的主机可以通过配置来监听事件，然后进行事件解析，也可能通过这些主机触发事件。

除了所有原始头带有一个前缀'Orig-'，事件头和其他典型的事件头一样，这个事件是一个CUSTOM 类型事件，带有一个multicast::event的子类型。另外，添加了一个Multicast-Sender头。以下这个例子就是接收到数据包，这个数据包是通过mod_event_multicast发送:

```
Event-Name: CUSTOM
Core-UUID: 12938281-57ce-11de-9be6-99a22d850f40
FreeSWITCH-Hostname: SYS1
FreeSWITCH-IPv4: 192.168.1.12
FreeSWITCH-IPv6: %3A%3A1
Event-Date-Local: 2010-01-16%2018%3A15%3A10
Event-Date-GMT: Tue,%2016%20Jun%202009%2022%3A15%3A10%20GMT
Event-Date-Timestamp: 1245190510366825
Event-Calling-File: mod_event_multicast.c
Event-Calling-Function: mod_event_multicast_runtime
Event-Calling-Line-Number:313
Event-Subclass: multicast%3A%3Aevent
Multicast: yes
Multicast-Sender:5211c5b8-ac42-11e2-8176-d16e41886f24
Orig-Event-Name: CUSTOM
Orig-Core-UUID: 8784372-5ecc-4eaa-9002-9992b7ab7c4d
```

用户可以通过conf/autoload_configs/event_multicast.conf.xml来配置mod_event_multicast。这个文件可以支持四个参数设置。他们是:

- address: 发送事件的目的地IP地址。
- port: 发送事件的目的地TCP端口。
- bindings: 对多广播地址发送的绑定事件设置。以下格式类似于事件格式event <arg> <arg>，是本章早期设置的。
- ttl: 用户可以设置数据包支持的TTL(存活时间)，以便在限定的时间内及时丢弃。这个设置依赖于公司网络设备的要求来设置。这个值是支持的跳数数量。

以下是一个mod_event_multicast配置举例:

```
<param     name="address" value="225.1.1.1"/>
<param     name="port" value="4242"/>
<param     name="bindings" value="PRESENCE_IN CUSTOM sofia::register
CUSTOM multicast::event"/>
<param     name="ttl" value="1"/>
```

FreeSWITCH事件系统命令

以下是一个命令列表，用户可以使用任何基于事件的工具来连接FreeSWITCH。
用户也可以从ESL (FreeSWITCH 事件套接字库，本章稍晚讨论)使用这些命
令，通过mod_event_socket 和其他FreeSWITCH支持的标准接口来访问事件系统。
尽管每个模块在处理格式方面有所不同，语法和所有的访问函数的语法是一样的。

auth <password>

当用户第一次通过mod_event_socket 模块连接FreeSWITCH event system时，用户必须首先认证。
以下命令支持用户传递认证参数:

```
auth ClueCon
```

api

api命令执行一个API命令。任何支持FreeSWITCH命令行接口的命令都可以支持。这个命令执行一个在限制模式下的命令，意思是，直到这个命令完成之前，控制机制不会返回到开启的事件套接字，也不允许执行其他的命令，这个命令完成以后，从允许其他的操作执行。

语法: api <command> <arg>

举例:

```
api originate sofia/mydomain.com/4158867999@telco.com 1000
```

这个例子通过telco.com 发起一个呼叫到(415) 886-7999，并且最后呼叫分机1000.

```
api sleep 5000
```

这个命令会使得系统执行命令 sleep，休眠五秒钟 (5000 毫秒)。

bgapi

bgapi 命令将在后台执行一个任务，返回一个Job-UUID字段，这个字段带有一个引用值关联后台任务。当这个命令真正执行，并且完成以后，命令返回结果以事件的形式被发送，并且这个事件携带一个UUID来匹配起初设置的值。

bgapi 命令接受和上面api命令同样的参数。唯一不同的是服务器端会立即返回结果，来支持处理更多的
命令。

语法: bgapi <command> <arg>

举例:

```
bgapi originate sofia/example/300@foo.com 8600
Content-Type: command/reply
Reply-Text: +OK Job-UUID: c7709e9c-1517-11dc-842a-d3a3942d3d63
```

当命令执行完成以后，FreeSWITCH 将触发一个事件，这个事件携带了在Job-UUID字段
同样的UUID。回复信息事件类型是BACKGROUND_JOB，所以用户必须已订阅状态来接收那些
事件类型，以便查看回复消息。回复信息样本类似于如下结果：

```
Content-Length: 625
Content-Type: text/event-plain
Job-UUID: c7709e9c-1517-11dc-842a-d3a3942d3d63
Job-Command: originate
Job-Command-Arg: sofia/default/300%20foo.com
Event-Name: BACKGROUND_JOB
Core-UUID: 42bdf272-16e6-11dd-b7a0-db4edd065621
FreeSWITCH-Hostname: ser
FreeSWITCH-IPv4: 192.168.1.104
FreeSWITCH-IPv6: 127.0.0.1
Event-Date-Local: 2008-05-02%2007%3A37%3A03
Event-Date-GMT: Thu,%2001%20May%202008%2023%3A37%3A03%20GMT
Event-Date-timestamp: 1209685023894968
Event-Calling-File: mod_event_socket.c
Event-Calling-Function: api_exec
Event-Calling-Line-Number: 609
Content-Length: 41
+OK 7f4de4bc-17d7-11dd-b7a0-db4edd065621
```

注意，在后台任务回复信息中，原来的job ID 在Job-UUID列出，而使用originate 创建的呼叫UUID在其他的数据中，本例中是+OK 7f4de4bc-17d7-11dd-b7a0-db4edd065621。如果用户正开发一个应用模块需要和FreeSWITCH进行异步通信，那需要使用bgapi 提交命令，订阅BACKGROUND_JOB事件。使用 Job-UUID字段值匹配bgapi命令相应的BACKGROUND_JOB事件。通过bgapi 发送的命令将产生一个返回一个结果，在BACKGROUND_JOB事件将包含这个“最终结果”。

event

event 命令启动或停止事件流。通过发起执行命令的模块把事件转换成流数据(可能是，mod_event_socket TCP 连接，mod_erlang_event等等)。用户可以订阅特定的事件类别类型，或订阅所有的事件类型。

后续针对'event' 的呼叫将覆盖和关闭上一个事件设置的请求。

语法: event plain <list of event types|all>

举例:

```
event plain ALL
```

这个举例请求一个所有事件的拷贝。

```
event plain CUSTOM conference::maintenance
```

这个举例请求一个CUSTOM 事件的拷贝，特别是会议模块的维护事件。

```
event plain CHANNEL_CREATE CHANNEL_DESTROY CUSTOM
  conference::maintenance sofia::register sofia::expire
```

这个举例请求一个事件拷贝，事件包含几种对通道的创建和清除事件，和所有来自于会议模块维护系统的自定义事件，和来自于Sofia's register和expire system的自定义事件。

noevents

这个命令关闭所有上一个event命令开启的事件。

使用:

```
noevents
```

divert_events

divert_events 命令支持嵌入式的脚本事件获得输入结果然后发送到事件套接字。
外部程序通过事件套接字连接发送事件回复，在FreeSWITCH中运行的脚本可以接收
这个回复消息，这个回复信息作为脚本的输入请求。

input callback可以通过setInputCallback()注册到脚本语言中。（我们在第七章讨论的
例子中使用了setInputCallback()。设置divert_events 为on 状态支持聊天信息像
Gtalk 通道，自动语音识别(ASR) 事件，和其他功能。

语法: divert_events <on|off>

举例:

```
divert_events on
divert_events off
```

filter

指定监听的事件类型。一个套接字连接可以支持多个filters。
注意，这个命令实际上是筛选出事件，不是过滤掉这些事件。 (参考nixevent部分。)
设置多个filters可以更加精确地获得用户希望看到的事件类型。

语法: filter <EventHeader> <ValueToFilter>

举例:

以下举例是订阅了所有事件:

```
events plain all
Content-Type: command/reply
Reply-Text: +OK event listener enabled plain
```

订阅到所有事件。

```
filter Event-Name CHANNEL_EXECUTE
Content-Type: command/reply
Reply-Text: +OK filter added. [filter]=[Event-Name CHANNEL_EXECUTE]
filter Event-Name HEARTBEAT
Content-Type: command/reply
Reply-Text: +OK filter added. [Event-Name]=[HEARTBEAT]
```

这个例子中，Filters仅接收事件类型是CHANNEL_EXECUTE和HEARBEAT。

用户可以筛选任何事件头。如果用户希望使用uuid 筛选一个特别通道的，可以这样使用:

```
filter Unique-ID d29a070f-40ff-43d8-8b9d-d369b2389dfe
```

使用的filters的组合可以限定用户从套接字收到的事件。

filter 取消

指定需要取消的事件filters。如果用户收到太多的数据信息，可以通过取消filter来控制。

语法: filter delete <EventHeader> <ValueToFilter>

举例:

```
filter delete Event-Name HEARTBEAT
filter delete Unique-ID d29a070f-40ff-43d8-8b9d-d369b2389dfe
```

这个例子取消了对给定的Unique-ID筛选。通过这个设置，用户将不在接收任何来自于这个Unique-ID的事件。

```
filter delete Unique-ID
```

这个设置权限了所有基于Unique-ID的filters筛选。

nixevents

这个命令正好和filter相反，它禁止接收一个指定类型的事件。

使用:

```
nixevents <event types | ALL | CUSTOM custom event sub-class>
```

sendevent

对事件系统发送一个事件(支持多结果输入)。

语法: sendevent <event-name>

以上命令在内部FreeSWITCH 事件系统中生成一个事件。任何订阅了这个事件类型的模块或系统处理机制都将收到这个事件。

如果用户在使用sendevent时，没有指定事件类型，并且Event-Name header携带了一个事件名称，用户可以设置需要的事件类型。
例如:

```
sendevent SOME_NAME
Event-Name: CUSTOM
Event-Subclass: albs::Section-Alarm
Section: 33
Alarm-Type: PIR
State: ACTIVE
```

使用sendevent命令的举例:

```
sendevent NOTIFY
profile: internal
content-type: application/simple-message-summary
event-string: check-sync
user: 1005
host: 192.168.10.4
content-length: 5
hello
```

sendmsg <uuid>

对指定的呼叫UUID发送消息 (呼叫命令执行或挂机)。使用这个命令可以控制一个指定的，正在处理的呼叫流程。用户需要对这个呼叫提供一个UUID。

为了通过发送消息来控制呼叫，这些呼叫应该是在驻留状态。驻留的电话代表这个通道处于一种中间状态或不安定的状态，允许用户对目前的这个通道执行应用处理，而且不影响已正在运行的其他程序。(注意：驻留的电话将不接收任何的媒体，包括音乐等待。)。

用户可以通过&park() 语法，使用originate发起呼叫，直接驻留这个呼叫:

```
originate sofia/example/300@foo.com &park()
```

这里对一个通道执行两个类型的actions：execute和hangup。
这两个actions 将在后续的部分讨论。

execute

execute 命令执行一个拨号规则的应用。用户可以在这些都请求中设置应用名称和所需参数，多次循环执行这个应用。简单的举例就是播放一个.wav 格式的语音文件。

格式如下:

```
SendMsg <uuid>
call-command: execute
execute-app-name: <one of the applications>
execute-app-arg: <application data>
loops: <number of times to invoke the command, default: 1>
```

以下例子中，SendMsg 命令将对通道指定的<uuid>播放一个语音文件，文件名为 test.wav。

```
SendMsg <uuid>
call-command: execute
execute-app-name: playback
execute-app-arg: /tmp/test.wav
```

如果用户通过SendMsg 发送了数据超过了2048字符，这些数据需要被解析为传递的参数，当执行这个命令时，用户可以适当调整发送格式:

```
SendMsg <uuid>
call-command: execute
execute-app-name: <one of the applications>
loops: <number of times to invoke the command, default: 1>
content-type: text/plain
content-length: <content length>
<application data>
```

注意，这里加深的部分消息。用户可以指定文本长度来触发应用模块，保证数据以全长的格式对应用程序发送。

hangup

对活动的呼叫挂机。

格式:

```
SendMsg <uuid>
call-command: hangup
hangup-cause: <recognized hangup cause>
```

nomedia

用户可以通过命令nomedia 来控制是否让FreeSWITCH 处于实时的媒体路径。这个命令支持用户对指定的通道启用或关闭媒体处理。

使用:

```
SendMsg <uuid>
call-command: nomedia
nomedia-uuid: <noinfo>
```

log <level>

使用这个命令启用log输出。用户可以根据自己的需求设置系统的日志级别。 这个设置可以支持用户接收所有的日志事件。

使用:

```
log <level>
```

nolog

这个命令关闭以前已开启对日志输出。

使用:

```
nolog
```

linger

当一个通道挂机时，通知FreeSWITCH 不要关闭这个事件连接。相反，让系统一直保持事件连接为打开状态，直到事件的客户端接收了最后一个和这个通道相关的事件。

使用:

linger

nolinger

这个命令关闭上次开启的linger 命令。

使用:

nolinger

FreeSWITCH后台应用

大部分用户没有意识到，但是他们已经使用的FreeSWITCH 后台应用(fs_cli)，他们已经使用了FreeSWITCH的事件套接字子系统。fs_cli是一个基于C语言的应用来连接由mod_event_socket提供的FreeSWITCH 事件套接字。它支持所有的系统事件，不同颜色显示，并且提供一个接口来支持以事件信息的格式发送返回命令。fs_cli 重新创建了一个整个FreeSWITCH 后台应用。(用户可以查看libs/esl/fs_cli.c 的fs_cli 源代码。)

事件套接字库

FreeSWITCH Event Socket Library (ESL)是一个标准API接口集，这些接口是可加载的模块并且支持各种开发语言。通常来说，APIs提供了默认的函数来支持访问FreeSWITCH事件功能-无需设置TCP或者网络套接字或其他方式来连接FreeSWITCH。

支持库

FreeSWITCH 采用了SWIG (www.swig.org) 来创建标准的API接口集。SWIG 调用了定义的变量列表和功能函数，自动创建了支持库，这些库把core FreeSWITCH 代码和所支持的开发语言接口模块(可加载）连接起来。以下是默认支持的开发语言:

- Perl
- PHP
- LUA
- Python
- Ruby
- C
- TCL
- .NET

以下的对象和函数可以支持任何一种开发语言，使用这些语言可以创建 ESL extensions。一旦加载了开发语言相应的模块，用户可以使用任何一个标准的ESL对象，函数和变量。基本的函数在下面的部分列出。FreeSWITCH ESL 使用SWIG为用户管理大部分的类型变换，当使用这些命令时，尝试使用默认的类型转换和变量结构。

ESLObject

ESLObject 是一个核心的ESL object。用户可以根据自己的需求设置 loglevel 来接收从freeSWITCH发出的事件。

eslSetLogLevel($loglevel)

eslSetLogLevel($loglevel) 设置服务器的日志级别。$loglevel 是一个整数变量，从0到7. $loglevel的值代表的含义如下:

- 0 是 EMERG
- 1 是 ALERT
- 2 是 CRIT
- 3 是 ERROR
- 4 是 WARNING
- 5 是 NOTICE
- 6 是 INFO
- 7 是 DEBUG

ESLevent object

当接收一个事件时，用户将获得一个 ESLevent object。这个对象包含各种帮助函数变量来帮助解析和处理收到的事件。

serialize([$format])

serialize([$format]) 转换事件的格式为"name: value" 冒号分割的对值，类似于 SIP/e-mail 包格式。

setPriority([$number])

setPriority([$number]) 事件触发时，通过$number来设置一个事件优先级。

getHeader($header_name)

getHeader($header_name) 从事件对象中获得一个头，这个头携带$header_name key。

getBody()

getBody() 获得event object body。

getType()

getType() 获得一个event object的事件类型。

addBody($value)

addBody($value) 对event object body 添加变量$value 。它可以对同样的enent object多次调用。

addHeader($header_name, $value)

addHeader($header_name,$value) 在key是变量值$header_name，value是变量$value 的地方对这个事件对象添加一个头。对同样的event object，它可以多次调用。

delHeader($header_name)

delHeader($header_name) 从event object 删除包含key $header_name的头。

firstHeader()

firstHeader() 设置第一个头为event object 指针，返回它的key名称。它必须在nextHeader之前调用

nextHeader()

nextHeader() 在event object中移动指针到下一个头 ，然后返回它的key 名称。它必须在firstHeader之前调用。当这个函数被调用时，如果用户的指针已经处于最后一个头的位置，返回空值。

ESLconnection object

ESLconnection object 负责维持一个对FreeSWITCH 连接以便来处理事件。它维持一个对FreeSWITCH的连接，来负责发送和接收消息。

new($host, $port, $password)

这个命令创建一个ESLconnection 实体，连接到变量$host 主机和$port设置的端口，并且提供连接FreeSWITCH服务器密码 $password。

这样做到目的是仅在内连模式下创建一个事件套接字。当FreeSWITCH 最初没有绑定任何特别的呼叫或通道时，可以使用这个方法对FreeSWITCH建立连接。

new($fd)

这个命令发起一个新的ESLconnection实体，使用目前包含在$fd 中的号码。

用户可以使用这个方法，创建一个Event Socket外连连接。在内连模式，即使通过了内连套接字验证，它仍会出现失败连接。

socketDescriptor()

如果连接存在，这个命令对连接对象返回一个UNIX 文件描述符。当在外连模式环境下，这个文件描述符和传递到新的($fd)的描述符相同。

connected()

这个命令测试是否和这个连接对象成功连接。如果成功连接，返回1，否则返回0。

getInfo()

当FreeSWITCH 连接到一个"Event Socket Outbound" handler时，将发送CHANNEL_DATA
事件作为初始连接以后的第一个事件。 getInfo() 返回一个 ESLevent包含这个通道的数据。

当使用"Event Socket Inbound" 事件套接字内连时，getInfo() 将返回一个空值。

send($command)

对FreeSWITCH发送命令，无需等待回复。

用户可以在一个循环逻辑中呼叫recvEvent或者recvEventTimed 事件来接收一个回复。
回复事件中将包含一个头content-type，这个头有一个 api/response或command/reply的值。

自动等待回复事件，使用sendRecv() 来替代 send()。

sendRecv($command)

实际上，sendRecv($command) 呼叫send($command)，和recvEvent()，返回
一个ESL 事件实体。

recvEvent()被循环呼叫，直到接收到一个带content-type 头的事件，包含api/response
或command/reply的值，然后返回一个ESL event 实体。

在recvEvent() 接收到事件中，任何和这个交换处理业务无关的事件将添加到队列中，
这些事件将返回到用户程序中recvEvent()的后续呼叫中。

api($command[, $arguments])

对FreeSWITCH 服务器端发送一个API 命令。这个命令将阻止接下来的
命令执行，直到发送的命令已经完成执行。

api($command,$args) is identical to sendRecv("api $command $args").

bgapi($command[, $arguments])

对FreeSWITCH发送一个后台线程API命令，通知FreeSWITCH使用自己的线程执行
此命令，并且不阻止其他命令执行。

bgapi($command,$args) 现在等同于sendRecv("bgapi $command $args")。

sendEvent($send_me)

在FreeSWITCH事件系统中注入一个事件。这个方法支持用户对FreeSWITCH发送一个事件，事件用户可以处理和使用这个事件。

recvEvent()

这个命令从FreeSWITCH返回下一个事件，如果没有事件，这个呼叫将一直在这里设置一个阻塞选择，直到有事件进入。

如果在一个呼叫过程中，有相应的事件队列，将返回第一个事件，然后从队列中删除这个事件。否则，将从连接中读取下一个事件。

recvEventTimed($milliseconds)

这个命令和recvEvent()的用法类似，但是它可以设置一个阻塞时间，设置单位为$milliseconds。

对recvEventTimed(0)的呼叫将立即返回结果。这样的方式对事件轮询非常有用。

filter($header, $value)

参考事件套接字filter 命令。

events($event_type,$value)

$event_type 支持的值包括 plain, json, 或者xml。任何其他在$event_type设置的值可以通过Plain替换。

execute($app[, $arg][, $uuid])

执行拨号规则应用，等待从服务器端的响应。在套接字连接没有绑定通道时(通常情况下是内连事件套接字连接)，必须要求这三个参数。
$uuid 指定执行这个应用的相应通道。

execute() 返回一个ESLevent object，并且包含服务器端的响应消息。ESLevent object调用的getHeader("Reply-Text")函数返回服务器响应。服务器端响应消息包括成功消息+OK [Success Message] 或者失败消息-ERR [Error Message]。

executeAsync($app[, $arg][, $uuid])

这个命令等同于execute；但是它不会等待从服务器端返回的响应消息。
(根据开发语言的说法，executeAsync 是一个 "非阻塞"形式。)

这个命令引起后续的呼叫执行一个execute()，然后在发送到通道的消息中添加 async: true 头。

setAsyncExecute($value)

强制启用async 模式支持套接字连接。这个命令对在拨号规则中设置为async模式的外连套接字连接无任何影响，对内连带套接字连接也没有什么影响，因为这些连接已经设置为async模式。

$value如果为1强制使用async 模式，0为不强制。

具体说，通过接下来的呼叫执行一个execute()，在发送到通道的消息中包含async: true头，这样可以调用一个setAsyncExecute(1)操作流程。其他事件套接字库路由不会受这个呼叫的影响。

setEventLock($value)

强制使用sync 模式支持套接字连接。这个命令对在拨号规则中设置为async模式的外连套接字连接无任何影响，因为这些连接已经设置为async模式。

$value 如果为1，则强制使用sync模式，0为不强制。

具体的说，通过接下来的呼叫执行一个execute()，在发送到通道的消息中包含 event-lock: true 头，这样可以调用一个setEventLock(1)操作流程。其他事件套接字库路由不会受这个呼叫的影响。

disconnect()

关闭对FreeSWITCH 服务器的套接字连接。

Events实战练习

让我们看看几个具体的实例来演示事件的使用。

Event Socket Library举例–运行一个命令

以下这个PHP例子演示了用户通过简单的脚本来处理单行命令。使用FreeSWITCH Event Socket Library对FreeSWITCH发送命令，等待命令响应。

```
// Include FreeSWITCH ESL Library. Note that ESL.php comes
// with the FreeSWITCH PHP ESL module.
require_once('ESL.php');
if ($argc <= 1) {
   printf("ERROR: You Need To Pass A Command\nUsage:\n\t%s <command>",
$argv[0]);
   exit();
}
// Strip off the executable's name ($argv[0]) array_shift($argv);
$command = sprintf('%s', implode(' ', $argv));
printf("Command to run is: %s\n", $command);
// Connect to FreeSWITCH
$sock = new ESLconnection('localhost', '8021', 'ClueCon');
// Send the Command
$res = $sock->api($command);
// Print the response
printf("%s\n", $res->getBody());
```

对FreeSWITCH发送事件的举例

以下举例非常有用，用户可以演示如何让事件套接字工作，例如如何对FreeSWITCH 事件系统发送(或者“注入”) 事件。

点亮电话指示灯

许多SIP电话机可以通过SIP presence 在线状态消息来点亮或者关闭电话指示灯。用户可以使用事件套接字让这些指示灯自己开启或关闭。

点亮指示灯

用户可以对FreeSWITCH发送在线状态事件来点亮电话指示灯，系统将对这个电话机发送一个SIP presence 在线状态消息。连接FreeSWITCH 事件套接字，然后发送以下事件:

```
sendevent PRESENCE_IN
proto: sip
from: 1000@example.com
login: 1000@example.com
event_type: presence
alt_event_type: dialog
Presence-Call-Direction: outbound
answer-state: confirmed
```

使用1000@example.com 的终端用户可以看到电话机线路按钮的指示灯会被点亮。注意，answer-state头—确认是key/value队值。代表线路有一个活动的呼叫（或我们正在模拟一个呼叫），指示灯应该处于点亮状态。

关闭指示灯

用户可以通过对FreeSWITCH发送在线状态事件来关闭指示灯，就像点亮一样的操作。连接FreeSWITCH 事件套接字以后，发送以下事件:

```
sendevent PRESENCE_IN
proto: sip
from: 1000@example.com
login: 1000@example.com
event_type: presence
Presence-Call-Direction: outbound
alt_event_type: dialog
answer-state: terminated
```

注意，answer-state头的状态是terminated状态，代表这条线路目前没有呼叫(关闭指示灯)。

重新启动SIP话机

FreeSWITCH支持对SIP话机发送命令来重新启动话机。在自动配置电话阶段，用户可能修改了配置入口，希望电话机获得一个新的配置，大部分的电话机可以通过重新启动。

用户需要了解需要重新启动的注册话机的Call-Id 字段。可以通过命令 sofia status profile <profile_name> reg 命令找到相关的话机。用户可以通过执行命令 sofia profile <profile_name> check_sync <call_id> reboot。此话机将重新启动。

用户可以连接事件套接字来执行这些指令。命令需要添加api为前缀:

```
# Search for the Call-Id of interest from within your program
api sofia status profile <profile_name> reg
# Reboot the phone
api sofia profile <profile_name> check-sync <call_id> reboot
```

请求电话配置

一些电话机可以支持功能重新配置。通过设置的服务器端，重新读取电话机设置:

```
sendevent NOTIFY
profile: internal
event-string: check-sync;reboot=false
user: 1000
host: 192.168.10.4
content-type: application/simple-message-summary
```

自定义notify消息

用户可以通过发起一个事件来发送自定义的notify 消息，消息内容可以是任意信息。
这个举例说明用户已经发送了以下消息:

```
sendevent NOTIFY
profile: internal
content-type: application/simple-message-summary
event-string: check-sync
user: 1005
host: 99.157.44.194
content-length: 2
OK
```

生成的类似的数据包:

```
NOTIFY sip:1005@99.157.44.203 SIP/2.0
Via: SIP/2.0/UDP 99.157.44.194;rport;branch=z9hG4bKpH2DtBDcDtg0N
Max-Forwards: 70
From: <sip:1005@99.157.44.194>;tag=Dy3c6Q1y15v5S
To: <sip:1005@99.157.44.194>
Call-ID: 129d1446-0063-122c-15aa-001a923f6a0f
CSeq: 104766492 NOTIFY
Contact: <sip:mod_sofia@99.157.44.194:5060>
User-Agent: FreeSWITCH-mod_sofia/1.0.trunk-9578:9586
Allow: INVITE, ACK, BYE, CANCEL, OPTIONS, PRACK, MESSAGE, SUBSCRIBE,
    NOTIFY, REFER, UPDATE, REGISTER, INFO, PUBLISH
Supported: 100rel, timer, precondition, path, replaces
Event: check-sync
Allow-Events: talk, presence, dialog, call-info, sla, include-
    session-description, presence.winfo, message-summary
Subscription-State: terminated;timeout
Content-Type: application/simple-message-summary
Content-Length: 2
OK
```

注意，因为我们的请求，在SIP notify端自己生成一个消息，我们包括了这个特别的字段在我们的请求中，这个请求将直接被传递到SIP消息中。

总结

一旦用户了解了FreeSWITCH事件系统的丰富功能，用户通过亲身实践，开始意识到可以配合FreeSWITCH应用实现很多灵活的应用需求。不像以前讨论的题目仅仅是发起电话呼叫或配置模块，这是FreeSWITCH一个非常小的部分，可以让用户之间能够进行实时互动，用户可以通过任何方法来满足你认为可能的需求。事件引擎是非常强大和健壮的，他的相关应用是可以无限拓展的。

在下一个章节，我们将讨论一个轻量级的方法来控制FreeSWITCH。通过mod_httpapi模块来控制FreeSWITCH。

11

通过mod_httapi模块支持基于页面呼叫控制

相对比较新的一个模块mod_httapi 可以支持用户更加动态控制IVR和其他的应用程序。配合这个模块，通过用户输入的不同可以生成自定义的IVR。FreeSWITCH的mod_httapi采用了一个简单的HTTP POST操作对页面应用程序发送各种信息，通过RESTful的实现方式来控制FreeSWITCH 呼叫流程。在中国章节，我们将讨论以下内容:

- HTTAPI 标记语法
- HTTAPI 配置
- 基本HTTAPI 操作，包括httapi 拨号规则应用
- 一个PHP样本库，使得HTTAPI 应用开发更加简单

当用户阅读这个章节时，必须牢记mod_httapi 使用了一个循环呼叫处理的流程;一个单个的呼叫，对于web服务器来说，有多个重复的HTTP POST请求。这样给了应用开发人员更大的灵活性来设计他们的应用模块。没有必要在单个响应中生成所有可能的呼叫逻辑。一个被httapi应用控制的呼叫将执行在HTTP中指定的响应命令，然后对web服务器发送另外一个HTTP POST请求。实际上，这个httapi 应用获得从web 服务器端来的指令，处理这些指令，然后对web 服务器端说，"我已经完成了那些指令，下一个指令是什么?"。这个循环会一直进行，直到呼叫结束或这个呼叫被转接到其他流程，超出了httapi 拨号规则应用的控制。

HTTAPI语法

HTTAPI标志符和原始的XML语法没有什么不同。大部分情况下，HTTAPI的语法构成类似于这样的:

```
<document type="text/freeswitch-httpapi">
 <variables/>
 <params/>
 <work/>
</document>
```

Document是从web 服务器HTTP POST请求响应中返回的。

一个HTTAPI 响应必须有text/xml的content type。所有HTTAPI 响应必须包括 document 标签，这个标签的类型属性是text/freeswitch-httpapi。在响应的对端，指定的响应中，用户可以使用任意一个或者所有的子标签。支持的子标签如下：

- params: FreeSWITCH对每个web服务器请求发送的POST params (那是，"parameters") 。用户可以使用<params> 标签通知FreeSWITCH 传递自定义的POST params。
- variables: 这些变量是正在呼叫 httapi拨号规则应用的通道变量。〈variables〉标签支持用户通道变量，这些变量可以在FreeSWITCH 拨号规则中使用或读回到httapi 下一个请求中。(本章后续将详细讨论。)

- work: 很多有兴趣的事情在这里发生。有很多不同的action标签可以作为<work>标签的子标签使用，通过这些标签来控制FreeSWITCH正在进行的呼叫：在后台进行发送日志信息，播放语音文件，启动自动语音识别，采集用户输入的DTMF按键数据，和其他的控制。支持的action标签和属性响应的每一个action将在下一个部分进行详细讨论。

以下许多的actions 可以对用户的程序添加绑定，通过这样的方式来获得FreeSWITCH采集的信息，然后传递这些信息到用户的应用程序中。这样的处理方式非常类似于页面的HTML格式。每一个元素都有一个名称，任何相对这个元素采集的数据将作为同名称的POST param 返回到用户的页面程序中。这个绑定将支持一个正则表达式来匹配返回的结果，然后通过可选的数据前缀从输入的值中来提取出最后的结果。

Work actions

HTTAPI work actions 在这个部分进行了描述。在以下的定义中，
DATA 是标签tag 的content (格式为, <tag>*DATA*</tag>)。

所有的work actions 至少支持两个two标签:

- action: 修改新默认目标URL。
- temp-action: 修改URL提交下一个请求。后续的请求将使用默认的URL或者使用在action标签指定的URL。

以下是一个work actions列表和他们的描述:

playback

Playback 播放一个文件，标签支持可选采集用户输入。它具有以下属性:

- file: 播放语音文件的路径
- name: 指定的Param名称来保存结果
- error-file: 无效输入时，播放错误提示文件
- digit-timeout: 播放文件完成后的等待输入的时间 (输入绑定时)
- input-timeout: 多数字输入时，等待更多数字时间设置
- loops: 播放语音文件的最多次数 (输入绑定时)
- asr-engine: 启用自动语音识别(ASR)引擎
- asr-grammar: 使用的自动语音识别语法(ASR)
- terminators: 输入结束键，一旦系统收到这个按键消息，立即停止采集，马上处理采集的结果数据。

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <playback action="http://newurl/index.php"
      temp-action="http://newtempurl/index.php"
      name="playback_user_input"
      error-file="ivr/ivr-error.wav"
      file="ivr/ivr-welcome_to_freeswitch.wav"
      asr-engine="pocketsphinx"
      asr-grammar="my_default_asr_grammar"
      digit-timeout="5"
```

```
            input-timeout="10"
            loops="3"
            terminators="#">
            <bind strip="#">~\\d{3}</bind>
        </playback>
    </work>
</document>
```

这个playback action 类似于拨号规则中的应用playback。

vmname

vmname 播放一个语音邮箱名称，并且支持可选输入采集。它具有以下属性：

- id: 播放用户名称
- name: Param 名称来保存结果
- error-file: 无效输入时播放的错误语音提示文件
- digit-timeout: 播放文件以后等待数字输入的时间 (绑定输入时)
- input-timeout: 多数字输入时，更多数字输入等待时间
- loops: 最大播放文件次数 (绑定输入时)
- terminators: 输入结束键，系统接收到此键将马上停止采集输入，立即处理采集到的数据。

举例:

```
<document type="text/freeswitch-httapi">
    <work>
        <vmname action="http://newurl/index.php"
            temp-action="http://newtempurl/index.php"
            name="vmname_user_input"
            error-file="ivr/ivrerror.wav"
            id="1007@192.168.1.101"
            digit-timeout="5"
            input-timeout="10"
            loops="3"
            terminators="#">
            <bind strip="#">~\\d{3}</bind>
        </vmname>
    </work>
</document>
```

record

record 录制一个文件，并且支持可选输入采集，然后返回这个文件到目的地URL。它具有的属性:

- file: 录音文件路径
- name: Param名称保存结果（上传一个多部分的表格文件）
- error-file: 无效输入时播放的错误语音文件
- beep-file: 播放一个beep提示文件提示开始录音 (这是语音邮箱beep文件)
- digit-timeout: 语音文件播放以后的等待输入数字时间（输入绑定时）
- limit: 最长录音时间（秒）
- terminators: 输入结束键，系统接收此键后停止采集按键输入，立即处理采集到的数据信息。

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <record action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      name="playback_user_input"
      error-file="ivr/ivr-error.wav"
      beep-file="tone_stream://$${beep}"
      file="12345.wav"
      digit-timeout="5"
      limit="60"
      terminators="#">
      <bind strip="#">~\\d{3}</bind>
    </record>
  </work>
</document>
```

这个record action 和拨号规则中的record相似。

pause

pause 在指定的时间内等待用户输入。它具有以下属性:

- milliseconds: 暂停的毫米数
- name: Param 名称保存结果
- error-file: 无效输入时播放的错误提示语音文件

- digit-timeout: 语音播放后等待输入数字的时间 (绑定输入时)
- input-timeout: 多数字输入时，等待更多数字输入时间
- loops: 最多播放语音文件的次数（输入绑定时）
- terminators: 输入结束键，系统接收到此键将马上停止采集输入，立即处理采集到的数据。

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <pause action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      name="pause_user_input"
      error-file="ivr/it_was_that_bug.wav"
      digit-timeout="5"
      milliseconds="15000"
      terminators="#">
      <bind strip="#">~\\d{3}</bind>
    </pause>
  </work>
</document>
```

speak

speak 使用语音合成对呼叫方读出文本，并且支持可选采集输入。它具有以下属性:

- text: 对呼叫方读出的文本内容
- name: Param 名称保存结果
- error-file: 无效输入时播放的错误语音提示文件
- digit-timeout: 文件播放以后，等待数字输入时间（输入绑定）
- input-timeout: 多数字输入时，等待更多输入时间
- loops: 最多播放语音文件次数
- engine: 语音合成引擎
- voice: 使用的语音合成语音功能
- terminators: 输入结束键，一旦系统接收到此按键，马上停止接收输入，立即处理接收到的数据。

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <speak action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      name="speak_user_input"
      error-file="ivr/ivr-error.wav"
      digit-timeout="5"
      engine="flite"
      voice="slt"
      text="Hello from flite text to speech engine"
      terminators="#">
      <bind strip="#">~\\d{3}</bind>
    </speak>
  </work>
</document>
```

speak action 类似于拨号规则中应用speak。

say

使用FreeSWITCH中say引擎，可以重复一段语音来模仿人的讲话。它具有以下属性:

- text: 指定的文本内容，可以读出，拼写，发音等等。
- name: Param 名称来保存结果
- error-file: 无效输入时播放的错误语音提示文件
- digit-timeout: 语音播放以后，等待数字输入时的限定时间 (输入绑定时)
- input-timeout: 多数字输入时，等待更多输入的时间
- loops: 最多语音文件播放次数（输入绑定时）
- language: 使用的语言
- type: 类型 (say 接口参数)
- method: 函数 (say 接口参数)
- gender: 性别 (say 参数)
- terminators: 输入结束键，一旦系统接收到此键，马上停止接收输入，立即处理接收到的输入数据。

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <say action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      name="say_user_input"
      error-file="ivr/ivr-error.wav"
      digit-timeout="5"
      language="en"
      type="name_spelled"
      method="pronounced"
      text="This is what the caller will hear"
      terminators="#">
      <bind strip="#">~\\d{3}</bind>
    </say>
  </work>
</document>
```

这个say action 类似于拨号规则应用中的say。参考第五章内容。

execute

execute 执行一个FreeSWITCH 拨号规则应用。它具有以下属性：

- application: 拨号规则应用
- data: 可选应用数据源
- *DATA*: 应用程序数据

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <execute action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      application="log"
      data="INFO this is an info log message"/>
    </work>
  </document>
```

sms

sms发送SMS信息。它具有以下属性:

- to: 目的地号码
- *DATA*: 信息内容

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <sms action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      to="sip:1007@192.168.1.101">Message text here</sms>
  </work>
</document>
```

注意: 要求编译加载mod_sms模块。参考 http://wiki.freeswitch.org/wiki/Mod_sms 获得更多信息。

dial

dial 执行一个对外的呼叫或转接。它具有以下属性:

- context: Dialplan context
- Dialplan: Dialplan 类型 (通常是XML)
- caller-id-name: Caller ID 名称
- caller-id-number: Caller ID 号码
- *DATA*: 拨打的号码或发起呼叫字符

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <dial action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      caller-id-name="HTTAPI Test"
      caller-id-number="19193869900"
      context="default"
      Dialplan="XML">
        sip:2019@10.1.1.12
      </dial>
  </work>
</document>
```

dial action 将通过拨号规则发起一个呼叫，如果创建了一个新的呼叫leg，这个呼叫将由已连接的HTTAPI控制。

recordCall

recordCall 发起一个呼叫录音。呼叫结束以后，此文件将被发送。它具有以下属性：

- limit: 超时时间（秒）。
- name: 如果以http://开始，必须指定一个URL，FreeSWITCH将根据这个URL来PUT这个文件。为了采用这个方法来处理文件，用户web服务器必须配置来处理PUT请求。如果忽略了这个设置，FreeSWITCH 将创建一个临时的路径来保存录音文件。

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <recordCall action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      name="http://localhost/newfile.wav"
      limit="60"/>
  </work>
</document>
```

这个recordCall action 类似于拨号规则中的record。

conference

conference 启动一个电话会议。它具有以下属性:

- profile: 使用的Conference profile
- *DATA*: 呼叫接入的会议室名称。

例如:

```
<document type="text/freeswitch-httapi">
  <work>
    <conference action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      profile="my_new_profile">
        My_Conference
      </conference>
 </work>
</document>
```

conference action 类似于拨号规则中的conference。

hangup

hangup 来结束一个呼叫。它具有以下属性:

- cause: 需要发送挂机原因

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <hangup action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      cause="NORMAL_CLEARING"/>
  </work>
</document>
```

hangup action 类似于拨号规则中的hangup。

break

break 退出httapi 应用，继续在拨号规则中执行。

```
<document type="text/freeswitch-httapi">
  <work>
    <break/>
  </work>
</document>
```

log

log对fs_cli，后台或者日志文件写一行日志信息。

- level: 使用的日志级别
- clean: 如果设置为真值，这行日志信息将不打印日志前缀

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <log action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      level="info">this is a log message with a prefix</log>
   <log level="warning"
      clean="1">and this is one without</log>
  </work>
</document>
```

log action 类似于拨号规则中的log 。注意，拨号规则中的log 不支持clean选项 httapi log action 可以支持。

continue

continue执行一个无指定work actions，和继续执行一个no-op action。 这个方法相当有用，有时候用户需要根据getVar结果来请求不同的 action URL地址。

```
<document type="text/freeswitch-httapi">
  <work>
    <continue action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"/>
  </work>
</document>
```

getVar

getVar 获得通道变量的内容 (依赖于权限设置)。它具有以下属性：

- permanent: 当设置为真值，获得的通道变量发送到这个呼叫所有后续的 HTTAPI 请求中，否则它仅被发送到下一个请求中。
- name: 从通道读取的变量名称 (例如，caller_id_name)

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <getVariable name="caller_id_name"
      action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      permanent="1"/>
  </work>
</document>
```

voicemail

voicemail 呼叫voicemail 拨号规则无需要求一个执行权限。它具有以下属性：

- check: 当设置为真值，允许呼叫方检查留言信息；呼叫方是语音邮箱用户。如果忽略此设置，系统将提示呼叫方留言。
- auth-only: 仅认证，继续执行。在事件中，选择了这个模式以后，对这个成功验证的通道中设置了两个变量：
  - variable_user_pin_authenticated 设置为true
  - variable_user_pin_authenticated_user 是成功验证用户的用户名称
- profile: 所使用Voicemail profile的名称 (省略，则为默认)。
- domain: 所使用的Domain (省略则为全局domain变量)。
- id: 说使用的ID(省略则需要输入id)。

举例:

```
<document type="text/freeswitch-httapi">
  <work>
    <voicemail action="http://localhost/newurl.php"
      temp-action="http://localhost/newtempurl.php"
      auth-only="1"
      check="1"
      domain="192.168.1.101"
      id="1010"
      profile="default"/>
  </work>
</document>
```

voicemail action 类似于拨号规则中的voicemail。

mod_httapi 配置文件

mod_httapi 配置文件可以保存在conf/autoload_configs，名称为httapi.conf.xml。它包含几个设置参数和profiles 部分。这个实例配置文件包含一个默认的HTTAPI profile，用户可以创建自己的profiles.

在profile 标签中，用户应该注意到这里有多个param 入口。这些参数用来控制一些流程，例如work actions 的默认设置，权限控制和支持HTTP请求的默认URL设置。

用户可能想起来第九章的mod_xml_curl 配置文件中的gateway-url parameter。在mod_httapi中，有一个和gateway-url 参数类似的设置，FreeSWITCH使用它作为基本的URL，这个URL实际上就是web 服务器地址，可以对此地址进行信息推送和提取。httapi 是一个拨号规则应用，我们也可以指定不同的URL。当我们看到以下这两个举例时，我们注意到，我们在第一个举例中传递任何应用的数据。这样的方式会导致系统从配置文件中调用预设的gateway-url 参数。在第二个举例中，我们对httapi设置了不同的URL。

例1:

```
<action application="httapi "/>
```

例2:

```
<action application="httapi"
        data="http://localhost/httapi/index2.php"/>
```

在下面的几个例子中，我们将演示用户如何对用户的页面应用程序传递附加的POST参数。这些例子在功能上完全和例2相似。

以下的例子传递URL作为一个参数名称，这些参数被设定在大括号内，发送到页面请求之前，FreeSWITCH解析这个参数。url 参数比较特别，字面对意思是“使用这个URL来支持请求”。

```
<action application="httapi"
        data="{url=http://localhost/httapi/index2.php}">
```

在我们的下一个例子中，向以前的一样，传递了url 参数，并且还传递了一个新的参数来设置HTTP 请求方式(例如，GET, PUT, 或者POST)。

```
<action application="httapi"
 data="{method=POST,url=http://localhost/httapi/index2.php}">
```

权限控制

在httapi中用户可以控制一切，有时，我们需要设置一个阀门，这个阀门可以支持
对一些逻辑流程有一定的权限控制，例如变量不能修改或，用户可能不小心执行了
一些应用或者API函数。在这个 httapi.conf.xml 举例配置文件中，在默认的
profile中支持了一个<permissions> 标签。在permissions 标签中，用户可以发现有许多
不同的权限设置，根据需要，用户可以关闭或者启用这些权限，这样用户可以根据
某些环境的需求来非常慎密地管理控制权限。

set-params是一个权限设置，可以允许用户设置或限定一个设置，这个同样的设置可以通过
从拨号规则中呼叫httapi，通过在{}来实现，并且这个设置会贯穿呼叫的整个过程。

set-vars 支持用户设置或限定通道变量配置。这个权限设置比set-params 权限更进一步，
支持用户设置自己需要的变量，这个方式类似于acl.conf.xml 的访问控制方式。
注意，在实例配置中的以下这段代码可以支持默认的策略和功能允许:

```
<permission name="set-vars" value="true">
  <variable-list default="deny">
    <!-- Variables here may be changed -->
    <variable name="caller_id_name"/>
  </variable-list>
</permission>
```

在上面的代码中已经解释，实际上，设置变量的权限被关闭，除非用户在变量
列表中特别指定这些变量。

用户可能注意到，在<variable> 标签下没有任何属性，表明支持给变量的设置。
这里有一个有效的type 属性。工作方式和用户看到的ACL的type属性一样，
但是如果type属性被省略，默认的设置将是策略的相反方式。就是说，如果用户列表的
默认设置是deny 拒绝，如果省略了type属性，权限入口将对所有的访问实体打开。
相反的设置也为真:

```
<permission name="set-vars" value="true">
  <variable-list default="allow">
    <!-- Variables here may *not* be changed -->
    <variable name="caller_id_name"/>
  </variable-list>
</permission>
```



请更多留意用户的允许或拒绝入口设置，
否则，因为疏忽大意，可能导致其他程序访问敏感数据。

就像用户希望的那样，get-vars权限设置支持用户从呼叫中读取通道变量。这个权限设置和set-vars 选项一样，具有同样的控制功能。它支持一个独立的ACL-风格的控制列表来获得变量而不是来设置变量。这个功能存在的原因就在于有时候系统操作时对用户权限设置要求不同，一些用户可以从用户端的应用中设置变量，同时另外一些用户则只能读取变量，还有的用户可能完全不能访问这些变量。

extended-data 权限设置支持用户对用户的应用程序传递更多地信息。默认的机制是提交一个通道简单数据，支持用户通过HTTAPI 命令获得用户需要的数据信息，和后续的回呼信息。如果用户宁愿在初始请求阶段通过用户应用程序来设置每一个通道变量，用户仅需要开启这个选项。

如果用户设置了execute-apps 权限，用户可以通过httapi 页面程序来呼叫拨号规则应用。它可以支持用户使用一个带<execute>标签的应用，类似的方法我们已经讨论过。这个权限设置本身支持一些ACL-风格的控制列表，类似于我们已经看到的另外一些权限控制。用户可以控制对所有应用的权限访问，可以一个个关闭或者开启他们。在下面的举例代码中，用户可以看到，我们允许访问我们的应用程序，访问的权限设置为默认的deny策略来支持对info和hangup应用的访问：

```
<permission name="execute-apps" value="true">
  <application-list default="deny">
    <application name="info"/>
    <application name="hangup"/>
  </application-list>
</permission>
```

expand-vars 权限支持用户在用户应用程序中使用变量，使用方法类似于XML 拨号规则的方法。变量就像 ${caller_id_number}将是一个拓展的内联。表达式 ${caller_id_number} 将给出呼叫方号码。它同样支持通过用户页面程序使用API命令。让我们看看这个例子:

```
${sofia_contact(1010@192.168.1.100)}
```

这个命令将执行sofia_contact API 命令，并且带有一个给定的参数，在结果中插入数据。当用户读到这个的时候，用户可能担心从用户程序使用API的安全问题。无需担心！有ACL-格式的控制列表可以管理这些安全问题。用户可以根据自己的需求允许和禁止API命令或变量。让我们看看以下这段代码:

```
<permission name="expand-vars" value="true">
  <variable-list default="deny">
    <variable name="caller_id_name"/>
    <variable name="caller_id_number"/>
  </variable-list>
  <api-list default="deny">
    <api name="expr"/>
    <api name="lua"/>
    <api name="sofia_contact"/>
  </api-list>
</permission>
```

以上这个例子仅允许对caller_id_name 和caller_id_number 设置通道变量，不允许进行其他的设置。它仅允许执行expr，lua，和sofia_contact API 命令，不允许其他的命令。在这个例子中，无论是应用开发人员或FreeSWITCH系统管理员都可以通过各自的权限来运行一个HTTAPI 应用模块，权限控制管理非常严密。

dial 权限支持用户从应用程序拨打号码，拨打的号码进入到拨号规则，然后根据不同的路由做进一步处理。

dial-set-Dialplan 和dial-set-context 权限支持用户修改拨号规则，如果可执行，这个拨号规则的context可用来拨打一个号码。

dial-set-cid-name和dial-set-cid-number 权限可以支持用户呼叫时，设置 caller ID name 和caller ID number。

dial-full-originate 权限支持用户使用完整的endpoint/profile/number 语法来拨打电话。(例如: sofia/internal/1010@192.168.1.100)

如果开启来dial-set-context, dial-set-Dialplan, dial-set-cid-name, dial-set-cid-number, 或 dial-full-originate 中的其中任何一个权限设置，即使在这个配置文件中其他地方设置为false，dial也将被启用。

conference 权限允许用户呼叫会议室，而conference-set-profile权限允许用户修改在每个请求中的会议profile。如果conference-set-profile被启用，则会议被开启，即使在文件中的其他地方设置为false，会议也将被启用。

注意，任何ACL-格式的支持列表可以通过关闭<permission> 标签跳过列表。
这样的话，如果用户已经在创建的列表中设置了default="allow"，默认的支持用户
的设置。用户创建的列表。以下这两个例子实际上是同样的工作方法。

例1:

```
<permission name="set-vars" value="true"/>
```

例2:

```
<permission name="set-vars" value="true">
  <variable-list default="allow">
  </variable-list>
</permission>
```

Exiting

在系统用户挂机事件中，FreeSWITCH 将传递一个"exiting" 参数，通知用户呼叫已经
结束，并且可以拆除相关这些会话，这些会话可能是用户已经打开的会话和并且完成
相关的用户跟踪记录。用户也可以完全忽略这个请求，FreeSWITCH 将通过返回的
文本信息”OK” 来拆除这些会话。

在连续请求中保存数据

当用户开始考虑通过使用积累的知识开发一个新的应用程序时，用户可能
也猜想如何使用一种最好的办法来保存一个个请求中的数据信息。当然，用户
也可以使用设置通道变量和获得通道变量来获取这些信息，但是在多请求状态
下执行每个set/get操作是一个非常消耗资源的做法。如果用户开发的是一个网页
管理系统，用户登录以后，使用 一些业务(例如，银行在线系统)，可以在session中
保存用户登录信息以便后续的操作使用。非常幸运，httapi 支持同样的方式。每个请求
将包含一个session_id POST或GET参数。通过这个session_id 参数的值，用户可以创建一个
Session，使用这个参数的值作为一个唯一标识。每一种页面程序语言都支持这样
的控制级别。这是一个使用PHP 开发举例:

```
if ( array_key_exists( 'session_id', $_REQUEST ) ) {
 session_id( $_REQUEST['session_id'] );


}
session_start();
```

用户创建这个session以后，可以访问session 的变量或实体，这些变量
用来保存需要的信息，用户可以在接下来的请求中访问这些信息。

请求中丢失的参数

如果用户使用实例配置，用户可以从每个HTTP请求获得所有的信息。当用户
开始编辑实例配置文件时，应该注意，如果启用了extended-data时，传递到用户
程序数据可能丢失。原因是，默认的method 是一个GET请求。所有在extended-data
配置环境下传递的信息的长度经常会大于CGI 参数支持的长度，请求中多出部分的数据就
被从数据中删除。有几种方法来解决这个问题。最好的解决办法就是在httapi.conf.xml file
设置method参数，如下举例:

```
<param name="method" value="POST"/>
```

但是，如果用户仅希望在请求中使用POST，并且这些请求导致了一些
数据丢失的问题，那么也可以使用另外一种方法来设置每个请求的method。
用户可以在URL 字符串中设置method，如下举例:

```
<action application="httapi"
  data="{url=http://localhost/httapi/index.php,method=POST}"/>
```

让开发变得更加简单

就像在本章看到的例子，用户可能想到httapi 可以发挥更大的威力，用户
可能也不愿意学习另外一种XML格式来控制FreeSWITCH。另外，手动打印所
有的XML是一件非常困难的事情。我们完全认可这样的说法。这就是
为什么我们开发了httapi XML来支持用户开发语言，使得开发工作变得
简单。在freeswitch-contrib 数据仓库中已经包含了支持PHP和Python的库。



在FreeSWITCH Git server (git.freeswitch.org) 包含了几个数据仓库，其中之一是freeswitch-contrib。这个库中(or "repo")包含用户贡献的实例代码。

继续往下看，我们将对用户演示几个实例，使用PHP 版本来创建基本的，但是非常有用的IVR。通过目前流行的IDE开发工具，可以轻松导入这些库，可以支持函数和变量查找，并且可以创建自定义的呼叫流程，和用户手动创建的httapi XML文件一样。

使用HTTAPI创建demo IVR

在开始之前，用户必须确认已经安装了一个web服务器和PHP XML库已经安装。如何设置web 服务器超出了本书讨论范围，所以，如果用户没有安装设置web服务器，需要首先安装web 服务器。成功完成了安装配置以后，可以继续这一部分的配置。

一旦完成web 服务器配置，确认可以支持PHP 文件，下一步就是从freeswitch-contrib repo 下载PHTT API库。所有的类文件都被写在一个PHP文件中，这样方便用户安装使用。从以下链接获得此文件：
http://git.freeswitch.org/git/freeswitch-contrib/plain/intralanman/PHP/phttapi/phttapi.php。
保存到一个指定的路径，以便将来使用。用户可能愿意把这个文件保存在Web目录下，和用户开发或下载的IVR在同一目录。在下一个小节，我们将涵盖demo-ivr.php 基本的重点部分，这些代码包括在本章的代码实例中。



访问Packt Publishing 网站(www.packtpub.com)获得代码实例。

在用户web服务器web文件目录下，保存文件1004_11_01.php为demo-ivr.php。使用编辑器打开此文件，查阅每一行代码，看看这些代码做了什么。

```
if ( array_key_exists('session_id', $_REQUEST ) ) {
    session_id( $_REQUEST['session_id'] );
}
session_start();
```

以上这段代码启动了一个session，并且使用了我们在本章早期讨论的session_id。

```
if ( array_key_exists( 'exiting', $_REQUEST ) ) {
    session_destroy();
    header( 'Content-Type: text/plain' );
    print "OK";
    exit();
}
```

如果我们发现了exiting 参数，我们将删除PHP session，并且通知FreeSWITCH
我们获悉此结果，然后退出脚本执行。

```
$demo = new phttapi();
```

这里，我们创建了新的httapi对象。$demo 将允许我们执行一些操作例如work actions。

```
$opt = array_key_exists( 'main_menu_option', $_REQUEST ) ?
        $_REQUEST['main_menu_option'] : '';
```

这是一个简单的if/then/else条件语句作为一个检测判断，甚至于main_menu_option为空，也
要确保$opt总是被设置。main_menu_option 选项将和呼叫方输入的按键匹配，我们将绑定到
这个选项。

```
if ( preg_match( '/^10[01][0-9]$/', $opt ) ) {
    $xfer = new phttapi_dial( $opt );
    $xfer->context( 'default' );
    $xfer->Dialplan( 'XML' );
    $demo->add_action( $xfer );
} else {
```

以上这段代码来测试选项是否匹配extension表达式。如果匹配，我们创建一个新的
phttapi_dial 对象 ($xfer)，设置目的地，然后在$demo 对象中添加一个action。如果不匹配，
我们将转到一个switch 语句来测试单按键输入选项。

```
    case '1':
        $conf = new phttapi_dial( '9888' );
        $conf->caller_id_name( 'another book reader' );
        $conf->context( 'default' );
        $conf->Dialplan( 'XML' );
        $demo->add_action( $conf );
        break;
```

如果用户输入了选项 1，我们创建了一个dial 选项，这个选项对应了我们本章讨论
过的dial标签。在这个标签中的每个属性对应一个phttapi_dial 类中的method。
例如，context method 设置了context的属性，Dialplan method 设置了Dialplan 属性，
等等。(Option 1 将发送呼叫方到公共FreeSWITCH 会议服务器。)

从Cases 2到5都是dial对象，他们具有同样的基本逻辑流程，具有不同的属性，通过
不同的属性获得不同的结果。

```
    case '6':
        if ( array_key_exists( 'sub_menu_option', $_REQUEST ) && $_
REQUEST['sub_menu_option'] == '*' ) {
```

```
            unset( $_SESSION['first_sub_play_done'] );
            $demo->add_action( $c = new phttapi_continue() );
            break;

    }
    $demo->start_variables();
    $demo->add_variable( 'main_menu_option', 6 );
    $demo->end_variables();


    $sub = new phttapi_playback();
    $sub->error_file( 'ivr/ivr-that_was_an_invalid_entry.wav' );
    $sub->loops( 3 );
    $sub->digit_timeout( '15000' );


    if ( !array_key_exists( 'first_sub_play_done', $_SESSION ) ) {
        $_SESSION['first_sub_play_done'] = TRUE;
        $sub->file( 'phrase:demo_ivr_sub_menu' );
    } else {
        $sub->file( 'phrase:demo_ivr_sub_menu_short' );
    }



    $star = new phttapi_action_binding( '*' );
    $sub->add_binding( $star );
    $sub->name( 'sub_menu_option' );


    $demo->add_action( $sub );
    break;
```

Option 6是有一点复杂，应该被拆分为自己的文件，技术上来说，它应该是一个分离的IVR文件。我们把这个文件保存为一个单个文件是为了用户能够简单安装和测试。（Option 6 演示了一个IVR 子菜单。）

```
case '9':
    $continue = new phttapi_continue();
    $demo->add_action( $continue );
    break;
```

在这些代码中，我们简单执行了一个continue语句，这个语句的作用是"重复这些选项"，这里没有任何绑定和没有任何办法传递main_menu_option 参数。

```
default:
    $intro = new phttapi_playback();
    $intro->error_file( 'ivr/ivr-that_was_an_invalid_entry.wav' );
    $intro->loops( 3 );
    $intro->digit_timeout( '2000' );
```

```
$intro->input_timeout( '10000' );
$intro->name( 'main_menu_option' );
$intro->terminators('#' );


if ( !array_key_exists( 'first_play_done', $_SESSION ) ) {
      $_SESSION['first_play_done'] = TRUE;
      $intro->file('phrase:demo_ivr_main_menu' );
} else {
      $intro->file('phrase:demo_ivr_main_menu_short' );
}
```

default 情况下的指令是播放一个intro 文件。为了模拟 ivr的应用场景，我们在这个session保存了一些信息，让我们知道是否我们已经播放了这个long intro 文件。(短和长问候语设置已经在第六章 IVR菜单定义中做了介绍。)

```
$b1 = newphttapi_action_binding( 1 );
...
$bext = newphttapi_action_binding( '~10[01][0-9]' );


$intro->add_binding( $b1 );
...
$intro->add_binding( $bext );
…
$demo->add_action( $intro );
```

在这个部分中，省略的地方表示没有列出其他的选项，这里是为了简洁。用户在这里可以看到，我们为每一个数字选项创建了一个绑定的对象，然后把每一个绑定添加到playback action。就像我们以前的例子一样，我们在$demo对象添加这个action。显然，我们可以通过单个绑定来支持更为复杂的regex来满足所有数字输入。但是，我们这样做的目的是为了支持用户可以通过单个数字and/or 表达式实现多个绑定来满足我们的设计开发需求。

```
header( 'Content-Type: text/xml' );
print $demo->output();
```

这里，我们设置了响应的content type为text/xml，并且打印出我们创建的对象结果。如果用户使用了其他的content-type ，而不是text/xml ，FreeSWITCH 将不能够理解来自用户的响应，因此必须确认设置了正确的content type。

总结

在这个章节，我们学习了新功能mod_httapi。通过web服务器端和FreeSWITCH的集成，完全可以使用简单HTT API 实现对呼叫控制。另外，我们讨论了一个PHP库(phttapi.php)，这个库提供了一个抽象层，使得通过web服务器端开发语音通信应用程序变得更加简单。使用mod_httapi，企业可以利用公司的web开发人员的特长来开发语音通信应用。此外，web 开发人员不需要学习FreeSWITCH系统管理员需要了解的知识。相反，web 开发人员仅需要学习HTTAPI，就可以开发一个功能丰富，支持web控制的语音通信应用。

在下一个章节，我们将讨论一个对VOIP管理员最为重要的话题：如何处理NAT问题。

12

NAT处理

在本书的开头我们讨论了传统。越来越多的事物发生了改变，而越来越多的人希望事物保持原貌。它仅是科技演变的一个部分。我们的汽车仍然需要一个里程表；我们喜爱的网站的图片看上去还是老式风格的按钮开关。我们作为社会的一员，座右铭: "如果东西没被损坏的话，那就不要修补它!" 伴随我们一生。同样的道理也可以应用在一些看起来相对现代的事情上，例如"我们如何获得网络"。我们大部分人还是对如何通过我们的一些小创新发明来访问网络这些细节毫不知情。在这个精彩的VOIP世界中，当我们面对令人担心的NAT问题（网络地址转换)来临时之前，我们可以冒险尝试一下这个离我们不远的挑战。

在这一章节，我们将讨论:

- NAT 简要介绍，包括一个历史简介
- 四个NAT陷阱
- 在FreeSWITCH 配置设置来帮助克服NAT
- 排查技巧

NAT简要介绍

给一个根本不关心技术术语的人介绍NAT问题，一个好的办法就是采用一个类似的方法来做介绍。想想一个巨大的报告大楼和和它的邮件收发室。一个在第十层工作的员工给你寄一个包裹，这个员工需要把包裹放到第一层的邮件收发室。这个包裹被投送到邮局，然后再转寄到你的住处。 在包裹上标明的寄回地址实际上是整栋办公大楼的地址，不是那个第十层的小办公间的地址。现在你计划把这个包裹再寄回去，你把这个包裹再投送到邮局，邮局再次把这个包裹投送到这个办公大楼，在邮件收发室的员工必须找出这个包裹需要分发的具体办公室地址，通过你的名字和在本大楼的办公室号码地址匹配查询，然后找到第十层的员工，最后把这个包裹发送到第十层的员工办公室。这里，邮件收发室就像一个NAT路由器，因为邮件收发室代理了邮局和办公大楼之间的邮件来往。办公室就像一个内网地址，因为它们不能直接获得或访问这些邮件。想象一下这样的场景，如果这个办公室投递的信息非常散乱，没有在包裹上标明办公室名称，当你寄回这个包裹时，邮件收发室的员工根本没有办法知道是哪个办公室寄出的包裹。这个就是一个NAT问题，包裹可能会丢失，就像你的呼叫一样。也许你可以收到这个包裹，同时也注意到包裹上没有标明办公室名称或号码，但是因为你正在等待这个包裹，可能你清楚是哪一个办公室寄出来的包裹，当你寄回到时候，你在包裹上做了一个办公室的标注。这个功能就是你创建的ANTI-NAT 功能。



NAT对PAT

网络地址转换（NAT）和端口地址转换（PAT）有所不同。在VOIP行业 (和其他地方) ，词条NAT 的使用非常自由。在这个章节我们延用这种说法，在NAT和PAT之间我们没有做更多详尽的区别。

当谈到网络时，NAT是一个技术话题，当整个内网(网络不能直接访问外网）连接到一个单独的设备，这个设备可以访问外网，并且有一个单独的公网IP地址，通过这样的方式对内网提供网络连接。因为我们目前的IP地址资源非常紧张，因此主要的目的是降低对公网IP数量的要求。到现在我们写这本书的时候，已经使用了四百万个IP地址。

把所有的内网设备都放置在NAT背后可以保护用户计算机设备和其他的设备不受攻击因为这些设备对于外网来说是不可见的。专家认为这不是最终的解决安全方案，因为仍然有一些方法可以威胁到NAT背后的设备安全，但是这种方法可以作为一个额外的保护，并且当配合其他的安全措施时，可以提高我们的对内网安全的保护。另外，用户将在第十三章VoIP安全中了解更多细节。

了解NAT的演变

自从网络技术普及以后，IPv4 地址的需求一直在增加。当需求增加时，地址池中的地址资源已经耗尽，出现了IPv4 地址的短缺。目前最受欢迎的解决地址短缺的两个主要解决办法是 - NAT和IPv6。

NAT 已经变成了一种非常受欢迎的方法，通过采用公网IP地址子集，利用他们来覆盖一个网内地大量设备，通过分配不同的内网地址来支持这些设备。NAT开始使用是在二十世纪90年代左右，当时是为了解决IP 地址的短缺问题直到现在IPv6已经开始使用，因为这种方式比较受欢迎，人们还是继续使用这种方式。与此同时，系统管理员还要被迫接受NAT的一些传统架构，并且因为它的流行我们还要接受这种方式来支持我们的软件和其他设备。

新标准IPv6可以解决IP地址短缺的问题，它添加了很多公网IP地址，我们有上万亿的IP地址，这些地址可以覆盖地球表面的每一个平方英寸的地方。我们可以设定一个IP地址段为我们目前所知道的整个网络世界，对每一个在这个星球的生命，即使这样设置地址资源也不会对整个IPv6 地址池有任何不利的影响。IPv6 规范在1998年发布，使用的势头真正慢慢开始增长。IPv6相对于已经从二十世纪70年代开始广泛使用的IPv4，它目前还落后于IPv4。很有可能，即使我们全部采用了IPv6，NAT问题的仍然会伴随我们很长一段时间。



也许如果用户一直考虑这个问题，FreeSWITCH已经支持了在IPv6 网络环境下的SIP和RTP。更多关于这个话题的讨论，请访问：http://wiki.freeswitch.org.

对于IP通信世界中的用户，NAT是一个尽人皆知的不雅的词汇(至少经常在一些语言交流中会伴随出现不雅的用词)。因为当用户遇到NAT问题时，常常被搞的抓耳挠腮。用户已经对IP网络环境有了一定的了解，并且用户可以轻松找到各种介绍关于NAT的文档，我们尽量省略了IP网络的细节。用户现在需要做到事情就是对NAT陷阱有足够的了解，这样可以为用户即使大量的时间，否则你可能真的要用头撞墙或让你愁的拽自己的头发了。本章的目标是对用户讲述如何通过使用FreeSWITCH的ANTI-NAT功能来成功地航行于NAT的危险水域。祝你好运！我们所有的人都在支持你!

处理NAT问题的关键是因为设备（电话）一般都在NAT背后，并且VOIP网络对于公共网络来说是不可见的，所以当用户呼叫这个设备时，非常困难连接到这个相应的设备。另外一个比较严重的问题是一些协议，例如SIP，当部署在NAT环境中时，协议可能被破坏。如果用户觉得整个事情让你彻底迷惑时，放松一下，事实上，我们已经为用户简化了NAT的设置。现在貌似看起来让人发疯，但是以前的环境可能更加糟糕。

老实说，FreeSWITCH 开发人员最初对NAT 其他的立场是,"不是我们的问题！"
在理想世界下，每个在NAT防火墙背后的设备都应该非常了解它们的环境，并且可以成功解决自己的问题。不幸的是，我们没有生活在那个理想的世界
(当然，如果我们生活在一个理想的世界，我们都没有工作，因为没有任何问题需要我们解决)。所以，我们决定,"好吧，我们试试!" 我们从用户中学习，他们使用的各种各样的设备来配合FreeSWITCH工作，但是这些设备完全没有任何办法来应对NAT问题。不久，我们开始了这个意义深远的任务，尽管这些设备本身还有很多缺陷，我们还是希望通过开发一种技术来支持这些设备。
NAT是一个非常难缠的对手，软弱的人是没有机会应对的。

NAT的四个陷阱

有四个基本的NAT陷阱，我们每个人都需要学习。了解了这些陷阱，用户学习到了处理NAT场景的技巧，就不会面对太多的疑问:

- NAT 可以在那些用户自己都不知道的地方。不一定非要涉及到网络（Internet）。
- 任何两种解决NAT的技术在一起使用，会导致互相之间冲突。
- 一些设备使用SIP ALG (Application Layer Gateway) 来战胜NAT问题。
- NAT纠正技术可以错误地定位一个环境，可能使得环境变得更加糟糕。

尽量熟悉这些陷阱。我们会经常讨论这些问题，这些问题会贯穿整个章节。

让我们现在详细讨论一下这些细节:

- NAT 可以在任何用户不知道的地方。不一定涉及网络。

  如果用户正在使用家庭网络服务，或商业级的服务，他们可能使用NAT来管理他们的客户，把客户端服务设置在一个分离的网络环境中，然后在进行一个网络地址转换，转换到其他的网络分段。在用户设备和目的地之间这样的情况不仅仅发生一次可能多次发生用户不能控制这样的事情发生。对很多使用VOIP的用户来说，这样的情况会引起很多问题。大部分的VoIP 协议仅支持基本的NAT处理功能而且经常出现问题。这就是很多家庭用户从家里面使用VOIP时首先面对的问题。NAT可以出现在一个内网地址访问多个其他内网地址无需连接外网网络服务的环境中。对于NAT，访问网络是最普遍的使用方法，但是可以在内部网络中划分一个独立的网络来获得网络访问。如果用户询问邻居VOIP专家在线问题，他可能判断是一个NAT问题，不一定是因为你使用没有访问网络，或可能用户自己也不清楚是否存在NAT问题。

- 任何两种解决NAT的技术在一起使用，会导致互相之间冲突。

  这是一个小技巧在VOIP用户遇到的非常普遍问题。最好的办法就是通过可见的方法来了解这些问题，可以画出一个奥赛罗棋来帮助用户更多了解这些问题。无论什么时候用户做一个移动的步骤来限制NAT，周围所有其他的环境都用翻转。如果用户做一个相反的移动步骤，周围其他的也要相应的翻转回去。这样的调整可能比用户所想的更加复杂(参考第一个陷阱)。只要出现奇怪的结果，用户就需要重新从没有问题的起始阶段入手，从最小修改的地方开始排查以免出现迷惑。如果用户话机支持NAT功能，用户可以启用这个功能，并且同时在FreeSWITCH端也同时启用，那么用户可能解决了单通或无语音的问题。更加让人不解的是有很多种方法取消NAT。 一些方法是要求仅在NAT背后的话机终端修改，但是另外的一些方法是要求仅在FreeSWITCH端进行修改，还有的方法是要求终端和FreeSWITCH都做修改。 现在你知道为什么我在开始的时候，对你说祝你好运？

- 一些设备使用SIP ALG (Application Layer Gateway)来战胜NAT问题。

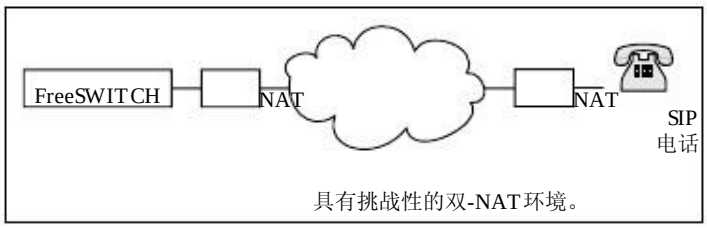
  我们在IRC中或者社区电话会议中无数次听到这样的说法
  "Arrrghh, curse you SIP ALG!"。使用ALGs的方式也可以，但是他们常常
  把一些事情搞定更糟。看起来这些问题都是陷阱1和陷阱2的组合，因为
  这些设备通常都部署在用户的运营商侧或用户路由器端默认的设置中。
  这些设备设置可能导致问题的解决变得更加困难，并且重新调整了所有的
  数据设置，因为当传递到用户路由器之前，这些数据已经通过ANTI-NAT功
  能对SIP数据包进行了修改。对用户来说，这样会导致产生错误命令和错误路
  由数据，最后完全是一个谜团。留意我说的话。如果用户发现自己正在读，
  "这绝对没有道理"，首先做到事情是用户应该检查在一个SIP ALG环境下。
  在很多情况下，简单关闭SIP ALG 可以解决NAT问题。

- NAT纠正技术可以错误地定位一个环境，可能使得环境变得更加糟糕。

  如果用户开启ANTI-NAT功能，至少可以帮助用户更多了解周围环境。一些
  SIP代理使用了SIP协议的一些非常特殊的设置，确实在数据包中的网络地址
  执行了很多意想不到的事情。熟悉SIP的用户也知道这些设置的结果，但是
  我们实现我们需要的环境设置。完整的数据包正在使用和和NAT相似的方法
  执行一些处理流程，并且触发一些功能，我们使用这些功能来检测NAT。
  因此必须小心，特别是在NAT后使用Cisco电话时，有时候情况会变得更加
  糟糕，并且同时取决于对NAT的错误检测。

揭秘FreeSWITCH中的NAT设置

现在我们回顾了经常遇到的NAT陷阱，我们可以继续了解一下用户遇到的
各种类型的NAT环境。不同的NAT部署方法有不同的技术特点，我们不会集中
讨论那些问题，否则用户可能会睡着，也会失去本章讨论的重点，这里我们需
要重点讨论FreeSWITCH NAT环境。基本上，用户可能在这样的一种环境，
用户电话或PBX在NAT后，另外一个通信的SIP电话终端则不在NAT后。
更加糟糕的环境是，用户可能处在比较担心的双-NAT环境，双方都在各自的
NAT环境下进行互连。双NAT环境类似于这样的一个图例：



具有挑战性的双-NAT环境。

让我们现在开始，目前的环境是，用户家里的电话机，不能支持NAT，但是用户需要注册到带有公网IP的FreeSWITCH服务器端。一个好的消息是FreeSWITCH举例配置文件已经涵盖了现在这个场景。核心FreeSWITCH支持一个ACL(Access Control Lists)功能。一个ACL可以支持用户创建一个网络地址列表，来通过ACL列表的设备发起地址控制访问。通过列表匹配定义的地址来决定是执行的结果。这个功能可以允许基于IP地址的设备认证或可以设置一个列表来过滤一些不安全的设备，任何在这个名单的设备都会被阻止。在这个举例中，我们将使用ACL来判断一个设备是否在NAT后，然后从那里决定应该执行什么命令。

一个NAT背后的设备带有一个特别的IP地址RFC-1918。针对RFC-1918的最早的解释是有一个特别的IP地址段，这部分的地址永远不能和外网接通，因为这段预留的地址段专门为内网的私有使用。内网地址段一般起始的是192.168.x.x，172.16.x.x 到172.31.x.x，或10.x.x.x。现在开始，我们称之为内网地址。



更多关于RFC-1918 IP 地址讨论，可参考：
http://en.wikipedia.org/wiki/Private_network.

因为他们都具有一个私有的地址，可以使用同样的IP地址，互相不连接，这样就可以组成一个无限广大的内网。现在当用户把这些网络连接到一个NAT路由器，网络中所有的将连接到FreeSWITCH 服务器。如果数据是从公网路由器进入，路由器将根据数据追踪这些内网地址，然后发送数据到对接公网的路由器。然后，公网的目的地地址对NAT路由器发送一个响应，路由器根据这个相应，通过地址的映射，发送数据返回到内网的发送方。在FreeSWITCH可以看到中，发送数据的源地址可能永远都不相同，这样的话，一个正常的呼入能够到达这个电话机就变得非常困难，这就是ACL可以做的事情。

FreeSWITCH对mod_sofia profiles 支持一个配置参数，这个参数是apply-nat-acl。这个参数可以在同样的profile，附带一个ACL列表名称多次使用。当mod_sofia 获得SIP REGISTER或INVITE 数据包时，apply-nat-acl会查找contact 地址，检查Contact 头中的IP地址来匹配设定的ACL。如果发现匹配结果，apply-nat-acl会推断出这个设备是一个在NAT后的设备。相当困难告诉我们这些IP地址代表的NAT后的设备，但是我们有了一点线索。还记得我们刚才提起的RFC-1918或内网地址？因为他们定义了一个地址段，我们可以推断出如果用户来自于这些地址的话，那说明用户正在从NAT后呼出。

小心，不要忘记第四个陷阱，这可不是百分之百安全。大部分情况下是这样的。一种情况是FreeSWITCH有一个内网地址，但是也在NAT后。当FreeSWITCH启动以后，我们创建一个指定的ACL nat.auto。这个特别的ACL已经包含了整个RFC-1918地址，而且检查机器的内网地址，排除那些地址空间，所以当FreeSWITCH有一个和自己同网的电话呼叫时，用户将很多正确的信息，那些“假”正确信息就被过滤掉了，从而做出有效判断。同时，这个设定的ACL可以检测来自于远端NAT后的电话终端。相对于公网的FreeSWITCH环境，FreeSWITCH预设了apply-nat-acl来配置nat.auto，并且可以修正大部分NAT后的典型设备。

如何解决这个问题？通常来说，当在NAT后的电话注册到服务器时，这个终端会被检测到，我们可以看到从哪里发起的注册，保存这个IP地址和端口到内部数据库中，数据库中同样给我们提供了一些已经试图注册，无法连接内网IP的未知的电话。当我们需要连接这个话机时，我们需要查询我们的数据库来决定外部的IP地址和端口，然后对其发送消息。SIP头仍然包含一个内网的IP:port 值，这是远端话机希望得到的。大部分的NAT路由器控制传输路径开启状态的时间周期非常短，我们可以通知这个电话频繁注册到服务器，服务器端也可以频繁保持这个映射，始终保持一个双方状态开启。这个技术针对第四个陷阱中的部分问题非常有效，因为我们从来不修改这些地址，不像ALG那样的方法。这表示这个电话终端将看到它们说希望看到的。

这里有一个FreeSWITCH CLI (Command Line Interface)输出结果。客户端是一个在NAT后的软电话，注册到一个带公网IP的FreeSWITCH。注意，Contact字段是使用IP 10.0.1.85 ，这是一个内网地址。状态显示已经检测到UDP-NAT，这是通过nat.auto ACL列表检测成功的。小窍门这个Contact结束的地方。额外的测试fs_nat和fs_path被追加到了电话注册的Contact地址中，所以我们可以搞明白如何解决NAT。看看以下的结果:

fs_nat=yes
fs_path=sip%3A1006%40206.22.109.244%3A43425%3Brinstance%3Db67dbafc
9baa9465%3Btransport%3Dudp.

fs_path 字段是一个SIP URI地址，这个地址将引导电话通过NAT设置。它是一个URL编码过的地址，因此在URL中包含一些特殊符号以避免和真实的Contact冲突。解码以后的地址是:

sip:1006@206.22.109.244:43425;rinstance=b67dbafc9baa9465;transport=udp

当我们呼叫这个电话时，我们将发送INVITE 包到206.22.109.244:43425，我们将保存这个地址，对应到这个地址10.0.1.85:5060。使用sofia status profile internal reg 命令可以看到完整的注册信息，例如:

freeswitch@myhost> sofia status profile internal reg

Registrations:

==========

Call-ID: ZWU1MjdiZTI2MTg2MmVhNTc5NTk3MDY5YjFmOTVkMTU.

User: 1006@myhost.freeswitch.org

Contact:"TEST" <sip:1006@10.0.1.85:10118;rinstance=b67dbafc9baa94
65;transport=udp;fs_nat=yes;fs_path=sip%3A1006%40184.58.189.244%3A43425%3
Brinstance%3Db67dbafc9baa9465%3Btransport%3Dudp>

Agent: eyeBeam release 1104g stamp 54685

Status: Registered(UDP-NAT)(unknown) EXP(2012-12-09 10:18:07)
EXPSECS(88)

Host: myhost

IP: 206.22.109.244

Port: 43425

Auth-User: 1006

Auth-Realm: myhost.freeswitch.org

MWI-Account: 1006@myhost.freeswitch.org

Total items returned: 1

==========

注意，Contact: header包含fs_nat和fs_path 参数。
任何从FreeSWITCH 发送到用户1006的SIP数据将使用这个在fs_path 参数中设置的URL。

处理媒体流

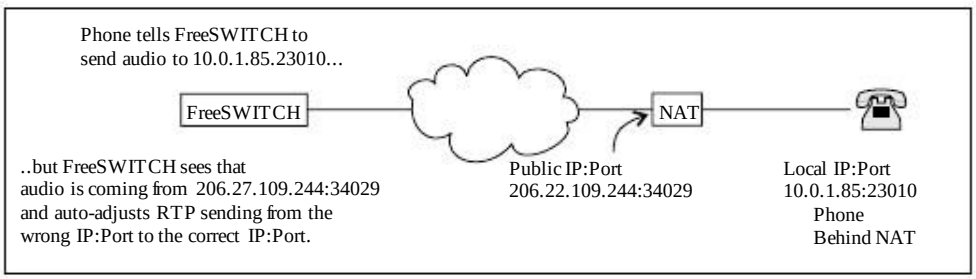
现在SIP消息可以正确地从FreeSWITCH传递到这个终端电话。语音媒体怎么办？如果用户互相听不到对方的语音，那就不是成功的呼叫。我们有很多问题通常就在这里，系统可以建立呼叫，但是当这个呼叫接通时，NAT将破坏了RTP包的传递，致使这个呼叫出现单通或者完全无语音的问题。因为第四种陷阱的问题，我们创建了一个独立的功能，这个功能大部分环境下可以支持RTP的处理，只有减少环境出现一些问题。这个功能为RTP auto-adjust。原因是很多时候在NAT后的电话呼叫我们时，通常情况下被FreeSWITCH认为是一个不可连通的内网地址，这样，语音媒体的发送就会出现很多问题。

我们猜测，因为设备在NAT后，我们确实应该对同样的地址发送语音流，这个地址是我们保存的SIP消息中的地址。但是，并不是总是这样，因为各种NAT类型有很多限定和端口映射，直到NAT后的设备已经发生了数据，这些端口映射有时候也并不存在。所以，在实际环境中，我们有时也不清楚如何才能保证媒体流能够成功到达这个电话终端。感谢auto-adjust这个功能，我们仍然还有尝试的机会。只要我们的这个电话终端有一个有效的地址，可以发送媒体给我们，我们可以等待这个媒体，直到这个电话对我们发送了语音数据包，我们就可以使用这个地址来返回语音数据。这不是百分之百保证可以工作，据我们了解，这个办法非常有效，特别是在一个绝望，没有任何办法来处理NAT的环境中。为了安全起见，我们仅允许在呼叫开始时做这个调整，否则可能导致一些坏人盗取用户的语音媒体流。

就像上面提到的，RTP auto-adjust 已经在默认配置中启用，会自动运行。通过查找log信息可以看到它是否正在工作。这个消息显示原始媒体目的地和检测到的新的媒体目的地。auto-adjust被启用以后，在呼叫开始日志中会显示出相关信息。

2012-05-09 10:37:48.183742 [INFO] switch_rtp.c:3607 Auto Changing port from 10.0.1.85:23010 to 206.22.109.244:34029

以下图例可以显示的更加清晰:



如果用户在一两个案例中，启用了这个功能来解决陷阱四的问题，可能出现了其他的一些问题时，用户可以添加这个参数在SIP profile来做测试:

```
<param name="disable-rtp-auto-adjust" value="true"/>
```

否则，在媒体开始之前，通过通道变量设置rtp_auto_adjust=false来关闭基于每个呼叫的设置。

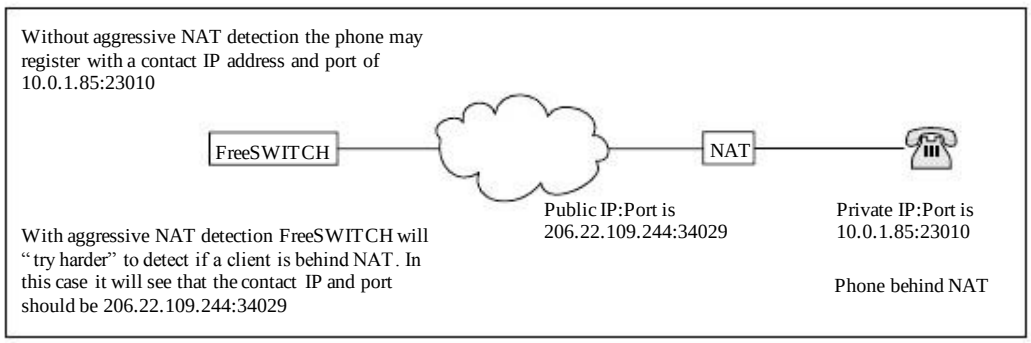
高级选项设置

现在，我们了解了它如何工作，我们可以看一下如何通过其他方法来触发NAT 检测。在很多环境下，ACL还是不够的，因为ALG可能已经破坏了这个数据包或这个数据已经经过了一个代理，或这个终端电话认为可以处理这个NAT环境，实际上可能出现了很多错误。

我们有另外一个选择来处理第四种陷阱，这个选择不是默认开启的，可以用来处理一些稍微特殊的NAT问题。启用这个参数相对比较危险，都是比较有用，如果用户实在没有其他办法的情况下，可以尝试这个参数。参数名称是aggressive-nat-detection，如果在SIP profile设置为true，将对所有的流量开启。通常，这个参数检查SIP数据包，如果看到在各种头域中看到各种IP地址变化，它会使用一个逻辑排查方法来确认哪一个是源地址。然后这个功能就执行和ACL 一样的工作，不过，它可能不一定总是那个写进数据库的源地址。

```
<param name="aggressive-nat-detection" value="true"/>
```

这个图例介绍了这样一个例子:



FreeSWITCH 有一个参数集，我们称为"No Device Left Behind" 或NDLB。这些参数代表一些环境，在这些环境中，我们完全模拟或在设备中接受一个漏洞，并且伪装所有的都是正常的流程或对设备做一个分组，这个设备仍然可以工作，虽然需要修改代码来实现我们的需求，但是使用另外一种完全违背事实的做法实现了这个需求。当对付NAT环境时，这个参数非常有效，这个参数是force-rport。在SIP中，rport属性的目的是通过在请求中附带一个；rport以最低成本来解决NAT问题。

当FreeSWITCH 看到这个属性时，它就回复一个信息，并且附带a rport=host:ip属性，因此，终端会意识到它在NAT后。比较有趣的事情是，当电话机在相应信息中看到rport，一些电话可以可以正确做出反应，但是出来没有请求rport.force-rport 参数可以导致FreeSWITCH 认为我们通话的设备都已经提供了rport 参数，所以我们按照他们的方式做了回复，从此解锁了这个其他方法无法获得的功能。

对于NDLB环境来说，这个参数可以通过默认设置开启。它也同样对于第四个陷阱打开了一个漏洞，因为许多设备可能被错误的设置。用户既可以设置为true，获得rport 或设置为safe仅支持一些我们需要这样设置的设备。根据呼叫方向，用户也可以设置为client-only或server-only，但是用户运气好，其他的方法可以实现，用户就不需要设置那些选项。



Polycom 电话机提供了一个经典的设置ndlb-force-rport为true或safe，因为这些话机不支持rport。大部分环境下，用户想使用safe 选项，因为大部分的型号不支持rport。如果因为其他的原因，用户需要使用ndlb-force-rport=true，那需要创建新文件SIP profile 时，添加这个参数。确认只有Polycom 话机可以使用这个SIP profile。

FreeSWITCH在客户侧

我们已经讨论了几个常见的环境，在NAT后的电话呼叫FreeSWITCH。现在我们继续讨论其他的环境，用户的FreeSWITCH在NAT后，和运营商或另外一个带公网IP的FreeSWITCH通信。幸运的是，我们已经准备了举例配置文件，有机会在这些NAT环境下工作。

我建议用户使用没有修改的举例配置文件来参数FreeSWITCH，发现用户丢失了默认的设置。FreeSWITCH 支持双客户侧NAT协议，包括NAT-PMP和UPnP。两种协议使用的方法稍微有所不同，但是基本原理是一样的。

这两种方法都使用网络协议来发现NAT路由器，然后和它通信，因此不是通过设置实时的NAT映射，而是要求路由器打开端口，获得端口映射的信息，当FreeSWITCH和另外一个服务器通信的时候，在双方SIP和媒体端数据中设置这些信息。比较酷！现在要意识到，现在我们正在对第二种陷阱，第三种陷阱和第四种陷阱打开了一扇门。



更多关于NAT-PMP和UPnP 信息，可以从这里获得：http://en.wikipedia.org/wiki/NAT_Port_Mapping_Protocol and http://en.wikipedia.org/wiki/Universal_Plug_and_Play, respectively.

现在我们掩饰了一个事实，我们在NAT后，另外远端检测不到我们。一个ALG设备可能隐藏在我们中间，可能把传输的数据损坏。另外一侧可能使用类似于主动式NAT检测的方法，并且可能跳出一个双anti-NAT陷阱。比较大的事情是，如果用户有一个限制非常严格的路由器或防火墙，这个功能不仅仅解决了地址映射的尴尬，而且还解锁被防火墙锁定的端口。等一下，还有更多的！如果用户在用户程序中需要类似的映射，用户还可以在事件中使用FSAPI 接口来映射和关闭映射端口。

在SIP profile 中有一个参数集，called ext-sip-ip和ext-rtp-ip。如果涉及到外网IP地址时，这些参数用来提供一些设置信息。默认的设置使用auto-nat来定义这些参数。这些参数配合NAT 路由器控制功能一起使用，可以帮助我们正确处理NAT问题。也有一些用户的路由器不支持NAT-PMP或者UPnP，或更加糟糕的是，声明可以支持本地或者远端，但是最后完全不能工作因为它本身已经损坏或因为几种陷阱致命的组合导致完全不能工作。

当freeSWITCH启动时，我们可以使用 -nonat 命令行选项来关闭这个功能，我们还有其他的锦囊妙计。如果你知道这个IP地址的话，用户可以设置一个IP地址，例如用户有一个静态的外部IP地址。更好的办法是，用户可以设置为autonat:x.x.x.x (这里x.x.x.x 替换成用户自己公网的IP) 因此，使用已知的外部IP地址仍然可以玩出一些小花样。

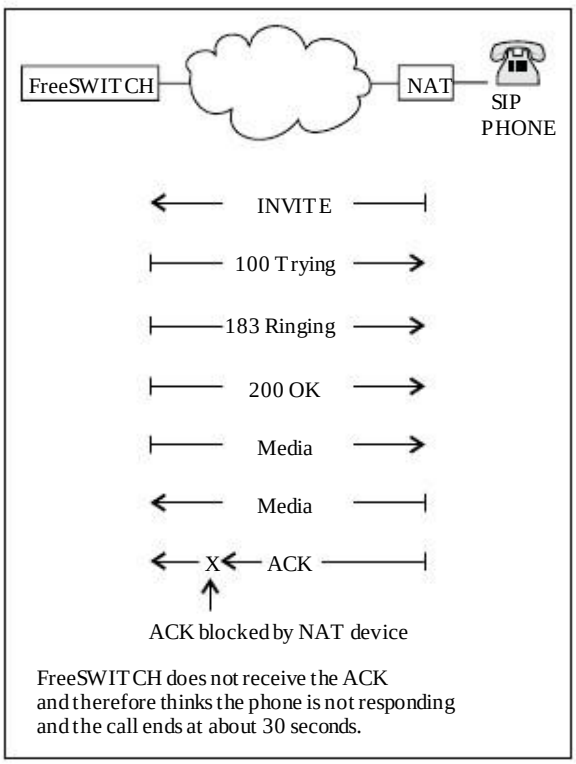
另外，如果需要，用户可以使用动态的DNS或STUN，设置主机名为: my.domain.com或stun:stun.myhost.com (这里my.domain.com或stun.myhost.com 是用户自己的动态域名或STUN服务器。). 这样有一个缺点，因为它可以使得一些处理变慢或停止工作。正常情况下，我们已经默认设置了最佳的选项，但是用户仍然应该了解其他的选项。如果用户可以控制自己的路由器，那么可以创建一个永久的NAT映射，正确地路由指定的流量到FreeSWITCH服务器或电话终端，ext-sip-ip和ext-rtp-ip 可以帮助用户解决类似的环境问题。同样，如果用户还有其他的手段，当双方用户都在NAT后时，假设用户可以控制双方路由器用户可以使用一个VPN来路由两个独立的网络。

NAT下一种创新的FreeSWITCH使用方法

使用FreeSWITCH可以克服两个设备中间的NAT问题。用户可以配置一个本地FreeSWITCH，所有终端电话都注册到这个服务器，然后注册这个FreeSWITCH的一个对象实体到运营商，这个实体来代表所有的终端电话，凿一个洞来实现NAT透传，这样所有需求都可以实现。同样，用户也可以配置一个带公网IP的FreeSWITCH，所有的终端电话或多个地址的本地的FreeSWITCH实体都注册到这个公共的服务器，甚至这些地址都在NAT后，他们之间仍然可以成功通话。

结论

我们希望用户有一个非常愉快的NAT世界之旅。在这个章节，我们证明当用户在ALG环境下跌跌撞撞时，做好充足的准备是非常有必要的。这样的问题有时候是非常讨厌的。如果你记住这个章节所有的细节，而且还有问题时，用户访问FreeSWITCH，获得社区成员的帮助，参考附录B，FreeSWITCH 在线社区。记住最后这句相当有智慧的话，可能让用户变得聪明一些，如果下一次用户听到有人抱怨他们之间的通话开始以后30秒钟就挂断，那就是NAT问题。以下图例显示了整个流程:



在很多情况下，FreeSWITCH不能解决这个问题，用户需要修改或者替换这个NAT设备。

总结

本章一次让我们吞下这么多的信息，我了解用户可能仍然比较困惑。
你能相信这实际上是一个已经淡化高水平的解释吗？幸运地是，FreeSWITCH
开发人员已经尽最大努力做了调整，默认环境就可以工作。因此，在本章中，
我们明确了NAT陷阱，可能一个读者免去了被首次遇到ALG或NAT路由
器时被“灼伤”的痛苦，我们也经历了一个非常有意义的奇怪旅程。

我们也涵盖了一些选项来解决NAT相关问题，用户冒险进入了让人困惑的
这个领域，这里没有人认为电话呼叫是可以实现的。在我们进入VOIP安全话题
讨论之前，我们给用户留了几个讨论的提示，当被NAT困扰时，希望用户可以
成功解决问题。 用户应该记住以下几个提示:

- 从那四个NAT陷阱中学习，始终张开你的眼睛来看着他们

  非常容易分心和被其中的一个陷阱迷惑。如果用户注意到花了太多的时间
  重新检查排查，确认用户没有发生任何错误，这些错误可能误导了用户。

- 尽量使用最少的修改来解决NAT问题

  修改太多的NAT设置，用户就会没有头绪，常见的是犯了一个小的错误，或者
  修改以后发生了不兼容的问题。容易犯的毛病就是这个问题解决了，但是引起了
  另外一个问题，然后花几个小时再返工。

- 如果用户可以访问自己的路由器，配置路由器尽量考虑NAT的支持

  最好的办法就是调整用户周围的环境，用户有一个理想的环境。
  选择基本的NAT设置，配置成默认的环境保证满足大部分环境
  需求。

下一个章节，我们将换一个讨论的话题，把我们讨论的重点从克服NAT问题转移到如何
提高VOIP安全。

13 VoIP安全

VoIP安全的话题是非常重要的一个话题，对于FreeSWITCH系统来说相当重要。安全策略包括主动防御策略和被动防守策略。在FreeSWITCH中，主动防御策略包括对SIP和RTP实行多种类型的加密技术来防止篡改和窃听通话。在FreeSWITCH中的被动防守策略通过结合其他开源的网络工具来防止从未知道源地址过来的恶意的数据传输，防止网络滥用和电话盗打。当用户的系统是生成系统时，非常必要使用一些开源的VOIP工具配合FreeSWITCH工作。

这一章节我们分成四个部分来逐一介绍:

- 网络的层次保护
- 拨号信令
- 保护语音
- 保护密码

网络的层次保护

大部分搞恶意攻击的家伙使用开放的网络端口侵入到内部的VOIP网络。他们查找比较弱的秘密来获得软件的bug，利用内部的设置来控制电话系统的配置和通话路由。最终目的是获得电话盗打，通话窃听，或窃取系统信息(例如邮箱的语音留言)。

因为网络是用户系统的入口，对用户网络的设置要格外小心，用户可以利用FreeSWITCH的功能来进一步保护系统。

分离接口和限制数据流量

在开放的网络中，SIP是经常遭受恶意攻击的一个技术。大部分情况下，那些攻击者通过对5060端口发送UDP包来扫描IP地址段，然后查看服务器的响应。一旦他们发现了服务器端的响应，攻击者将疯狂测试通常使用的密码然后呼出。在大部分环境中，攻击者通过虚假注册或其他数据包来攻击服务器，导致服务器不能正常工作。

保护FreeSWITCH最简单的办法是通过分离SIP接口，强制防火墙或IPTables路由表来支持不同的接口。

就像用户在上一个章节学习的，FreeSWITCH 支持用户在同一个系统中设置不同的Sofia SIP接口，通过不同的地址和端口来接收和发送SIP数据流量。通过这样的设置可以增加额外的一层对安全性和稳定性的保护。

从安全的角度来看，Sofia SIP profiles配置了默认的contexts支持呼入的呼叫。那些contexts 可以默认支持严格限定的拨号规则。如果用户把限定的contexts和保护工作和相关的SIP profile，用户将严格限制某些人发送SIP盗打信息进入到用户系统，即使用户偶然创建了错误的配置文件，也不会对系统安全造成很大影响。

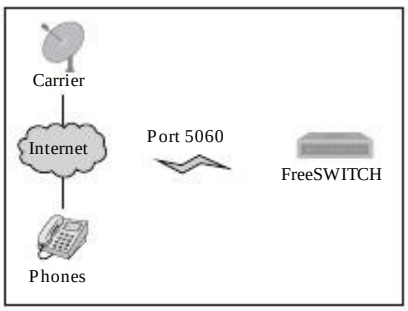
另外，每个Sofia SIP profile 可以支持一个不同的ACL列表。通过这样的方法，用户可以配置更加严格的限制措施应对IP地址和一些控制不是非常严格的内网地址。

从稳定性和性能来说，已知的事实是，在FreeSWITCH设计中，每个Sofia SIP接口是一个分离的线程。这表示，每个线程可以有各自的端口和IP地址，如果有人用户对系统干涉的话，独立的线程可以帮助用户把这个干扰降低到最少程度。但是，这不是一个万无一失的方法来保护用户系统，如果被攻击时，可能这样的方法对用户有所帮助。

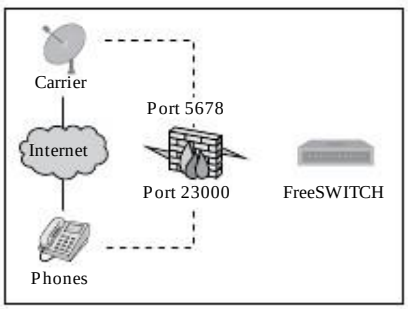
举例设置-简单方式

最简单的设置中，系统有一个接口，运营商可以通过这个接口来连接到用户端，对应另外一个接口，这个用户电话可以轻松使用。大部分恶意的攻击是通过端口扫描5060发现用户正在这个端口接收和响应SIP数据流量。在这里，攻击者会使用各种认证方式的组合来发现漏洞，直到发现一个生效的端口，或肆意不停测试那个端口。如果用户使用ACL限定了这个IP地址，修改端口号码为一个任意的端口，仅允许从运营商来到呼入，用户可立马防止攻击者访问系统，即使这个攻击者获得了正确的用户名和密码，也不会非常轻松地访问系统。

以下条例显示用户如何设置一个默认环境的FreeSWITCH系统:



另外一个办法是，用户使用特别的和非常不同的端口，这样攻击者可能非常困难对系统进行攻击。另外，用户可以使用防火墙来限定一个仅支持运营商的端口，开放其他的端口给终端电话。以下条例演示了我们的建议:



为了实现上面说的例子，对运营商开放5678端口，对终端电话开放23000端口，用户可以这样设置:

```
<profile name="incoming_from_pstn">
  <settings>
    <param name="auth-calls" value="true"/>
    <param name="apply-inbound-acl" value="my_carriers"/>
    <param name="context" value="inbound_call"/>
    <param name="sip-port" value="5678"/>
    ... other settings here ...
  </settings>
</profile>
```

在上面的例子中，从运营商来到呼叫必须进入到5678端口。当进入到这个端口时，使用了my_carriers ACL，确认只有运营商才有权通过。如果在my_carriers ACL有错误的话，也没有什么问题– 呼叫方进入的context是inbound_call，这里仅允许呼入不会允许呼出。所以这是一个比较严格的安全措施。

另外，因为用户知道，用户只有允许运营商来连接端口5678，用户可以修改防火墙或IP路由表的规则来支持这个端口的数据流量，并且仅允许来自于这个运营商的IP地址。对呼入访问来说，这是一个相对安全的的方法。

用户创建以下profile 支持系统用户的使用:

```
<profile name="customer_access">
  <settings>
    <param name="auth-calls" value="true"/>
    <param name="apply-inbound-acl" value="default_deny_list"/>
   <param name="context" value="customer_call"/>
    <param name="sip-port" value="23000"/>
    ... other settings here ...
  </settings>
</profile>
```

在这个Sofia SIP profile，从外部客户来的呼叫将进入到SIP端口23000。这个端口要求认证，使用ACL default_deny_list 默认拒绝了所有的呼叫。这样就强制进入的用户进行认证，表示用户提供有效的用户名称和密码通过系统认证，如果认证成功，则可以使用该系统。一旦用户提供了有效的信息，用户就被路由到customer_call context，在这里呼叫被进一步处理。

举例设置–复杂方式

一个复杂的举例说明来说明网络隔离，并且假设用户有一个网络路由装置这个装置可以支持用户通过物理方式或子网，或VLAN的形式来分割网络访问。

在这个复杂网络设置中，用户将添加额外使用不同的IP地址，在同一系统中绑定不同的网卡。另外，为了映射SIP端口到不同的接口，用户也要映射事件套接字和其他FreeSWITCH端口来管理这些接口。

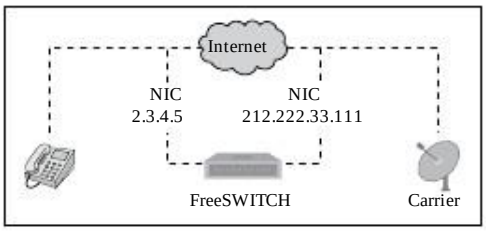
为了实现这个需求，用户可能需要创建两个Sofia SIP profiles，如下所示:

```
<profile name="incoming_from_pstn">
  <settings>
    <param name="auth-calls" value="true"/>
    <param name="apply-inbound-acl" value="my_carriers"/>
    <param name="context" value="inbound_call"/>
    <param name="sip-port" value="5678"/>
    <param name="sip-ip" value="2.3.4.5"/>
    ... other settings here ...
  </settings>
</profile>
<profile name="customer_access">
  <settings>
    <param name="auth-calls" value="true"/>
    <param name="apply-inbound-acl" value="default_deny_list"/>
    <param name="context" value="customer_call"/>
    <param name="sip-port" value="23000"/>
    <param name="sip-ip" value="212.222.33.111"/>
    ... other settings here ...
  </settings>
</profile>
```

在上一个场景中，留意sip-ip 设置支持不同的接口。一个接口是2.3.4.5，另外一个接口是212.222.33.111. FreeSWITCH将使用使用在Sofia SIP profile指定的物理接口地址。这样可以支持用户使用不同的防火墙和网络连接支持每一个接口，来提升FreeSWITCH 系统能力。

在这个场景中，2.3.4.5 应该是一个内网地址，这个地址不能路由到外网到国外网络，212.222.33.111 可以设置为一个公网IP地址，可实现路由。212.222.33.111 将仅对运营商开放，除非有用户的电话不在本地网络。作为一个可选的办法，如果允许公司员工使用软电话访问网络，可以允许员工通过VPN来访问公司网络。这样的方法可能更加安全。

上面这个场景可以通过以下图例来说明:



在上面这个例子中，终端电话将和2.3.4.5 的网卡通信，这个地址已经设置在FreeSWITCH 服务器端，而运营商将和FreeSWITCH绑定的另外一张网卡通信，这个地址是212.222.33.111。

VLANs

VLANs 是一种非常不错的办法，可以把终端电话从数据通信的内网环境中隔离出来，如果设置的正确，通话质量可以得到改善，并且防止恶意攻击的活动。

作为一个可选方法，VLANs经常被忽视，但是事实上，在同样的网络环境中，把终端电话看作类似计算机的设备，可能引起一些损失。在同样的网络中，通常非常容易来定位一个电话设备的IP地址，然后可以方便地登录这个设备。从那里可以轻易得到很多SIP电话的的用户名称和密码。例如，如果用户登录到一个Polycom 电话机时，用户可以导出电话机的配置文件，配置文件中包含了电话的安全信息，可以通过普通文本的格式看到这些安全信息。当电话终端在一个网络的私网网段下，相当比较困难获得这些信息。

另外对直接修改电话终端，VLANs 也防止工具从计算机运行发送伪装信息到语音网络中。这样的简单环境类似于无授权的软电话，使用软电话劫持分机来发送伪装BYE信息，故意引起呼叫挂机，实际上这个呼叫应该继续执行。

值得注意，在大部分的网络中，VLANs可以设置为基于端口的标签，指定网络交换机的物理端口作为虚拟LAN的一个部分或基于软件的标签，基于软件的标签是通过网卡或者操作系统对每个数据包加一个标签，并且在IP头中附带一个指定的虚拟LAN号码。

设置VLANs 超出了本书的讨论范围。但是用户应该注意这样的功能，很多流行的桌面终端话机和比较新的网络设备都支持VLANs，包括基于软件的或者基于端口的VLAN 标签。

入侵检测

入侵者试图访问我们的系统，一些非法用户创建一个拒绝服务攻击，类似这样对系统的干扰都是我们面对的一个非常大的挑战。而且，我们也要考虑什么类型的数据流量认为是异常的，同时如果设置了一个规则自动检测和防止攻击时，一些极限环境必须考虑进去。

注册监测

一些工具通过发送伪装的认证试图攻击VOIP系统，无需在SIP中响应认证请求。一种比较受欢迎的工具指的是相对友好的扫描工具或SIPvicious。这些类型的工具通过不停发送伪装请求信息，会让系统一直处在忙的状态，出现系统负载，使得系统不能接受真正的请求。另外，一些诡异的行为也可以被检测出来，一些人在很短时间周期内，不停重复呼叫长途电话或国际长途电话。

当攻击者在系统中使用了安全证书（识别或不能识别时），FreeSWITCH支持系统发出日志告警。第三方程序例如Fail2Ban可以用来监测这个产生日志信息的频率。如果这个频率到达了设置的阀值，那么认为现在的数据流量是非常异常，可能是可疑的攻击信息。一定时间内（或者永久），产生这个异常流量的IP地址就会被阻止。通常来说，如果在相对短的时间内，同样的IP地址产生了大量的认证请求，这个地址可以认为是一个恶意攻击地址。

当FreesSWITCH接收到了无效认证时，为了确保告警信息那个生成，用户可以修改SIP profiles，包括这个设置:

```
<param name="log-auth-failures" value="true"/>
```

这里，将生成一个认证失败的日志信息，信息如下:

[WARNING] SIP auth challenge (REGISTER) on sofia profile 'customer_access' for [user_rdkj7h@2600hz.com] from ip 184.106.157.100

这些信息将被自动记录，可以用来阻止来自于IP184. 106. 157. 100的认证。

Fail2Ban

Fail2Ban 是一个第三方程序，它可以在后台运行，并且监测日志信息。当特别的日志信息，例如认证信息多次出现时，根据这个出现的周期Fail2Ban将执行一个相应的命令，或采取其他措施。可以通过程序设置对用户发送邮件，或自动使用IPTables来阻止这个在一定的时间内多次进行无效注册的IP地址。

这本书不是一个使用Fail2Ban的完整指导文档，但是我们在本章节的后续部分给出一些举例脚本。

为了配置Fail2Ban，用户需要创建几个文件来命令Fail2Ban查找用户日志中的特别信息，当可以匹配这些信息时，应该采取的措施。

对于过滤来说，Fail2Ban已经默认支持一个文件，用户可以把过滤的信息保存到这里。这些过滤文件包含某些字符，这些字符可以匹配用户日志信息。用户可以设置多个过滤设置来过滤用户日志中不同的流量类型。当和配合FreeSWITCH的错误日志设置一起工作时，通过这些错误日志显示的无效登录测试记录，实现一个非常有用的过滤机制。

第二个文件是jail 配置文件，配置一个应用场景支持特别的规则例如我们可以支持的错误出现的频率，如果超出我们设置的阀值，系统一个采取相应的措施。jail 配置文件非常有效地设置相应机制(当过滤关键词匹配时)。

Filter 配置

让我们首先看看filter 配置文件。这个文件通常保存在 /etc/Fail2Ban/filter.d/ 和一个需要过滤的设置名称。在这个例子中，我们可以称这个文件为 freeswitch-auth.conf。这个文件包含了过滤设置来查找失败的认证测试。格式是一个标准的正则表达式。在这个例子中，如果注册失败或呼出时不能通过安全认证我们就认为是失败的。文件举例如下:

```
# freeswitch-auth.conf Fail2Ban filter configuration file
[Definition]
failregex = SIP auth failure \((?:[REGISTER|INVITE])\) on sofia
profile \'[^']+\' for \[.*\] from ip <HOST>
```

这个过滤脚本将匹配FreeSWITCH 日志中失败的REGISTER或INVITE消息。

Jail 配置

现在，通过使用jail 过滤入口阻止一个IP地址，因为在一定时间内我们从这个地址多次收到失败的INVITEs or REGISTERs消息。编辑文件 edit /etc/Fail2Ban/jail.conf ，然后添加以下入口:

```
[freeswitch-auth]
enabled = true
# place your custom port entries in here if needed (per the Sofia
settings above)
port = 5060
filter = freeswitch-auth
logpath = /var/log/freeswitch/freeswitch.log
maxretry = 50
findtime = 30
bantime = 6000
action = iptables-allports[name=freeswitch, protocol=all]
```

在设置中，我们使用了freeswitch-auth filter，在30秒的周期内，收到50次失败的INVITE 或REGISTER认证测试后 (最大测试次数)，我们将拒绝这个攻击者的IP地址。如果我们从这个地址在30秒钟内收到50次失败的INVITE或REGISTER认证攻击，这个IP地址就被拒绝6000秒。

其他应该考虑的问题

The Fail2Ban 基本必须是可调整的，有时候因为大型的网站非常繁忙，通过可调整的设置方法，这样大型的网站就可以不会意外瘫痪。例如，用户可以设置一个Fail2Ban的速率限定入口，如果在5秒钟内发送了50个认证请求时，用户不想设置Fail2Ban拒绝这些IP地址，因为有时候可能网站的50个电话机因为断电，在上电以后，可能这50个电话机同时重启注册，这样可能导致这些IP地址被拒绝。这不是我们想要的结果。

当设置Fail2Ban时，必须加倍小心，当然也要考虑到一些极端环境，例如就像我们刚才所讨论的电源断电。

加密

保持语音通信的安全对语音平台上非常必要的。特别是通过PSTN通信路由网络通信，通话中涉及安全的隐私或者这些一些财务交易时，终端用户通常认为线路是安全的。

VoIP 加密包括两个基本概念– 对信令加密和对媒体(语音/视频)。像其他的加密机制一样，VoIP加密使用了标准的密码库，并且引入了密匙交换和密码协商来传输和接收信息。在VOIP中，两个主要的加密方法非常类似于SSL对网页验证的机制，通过SSH使用密钥交换连接远端的服务器。主要目标是通过加密机制和一个仅双方都获知的普通加密密钥来建立一个安全连接，对实际的数据内容-电话呼叫实现加密和解密。

许多用户对这些技术TLS，SSL，和SRTP比较随意，没有完全理解他们的意思。用户应该理解这些技术以便完全保护通信系统，通常建议选择信令加密策略和语音加密策略来进行加密处理。

在以下部分，我们将详细讨论每一种加密策略。

保护SIP信令

SIP信令加密锁非常重要的。默认环境下，信令中包含文本形式的了电话呼叫的认证信息，和呼叫方被呼叫方的Caller ID Name和Number。因此非常容易获得和欺骗。加密以后，可以让信令消息的破解变得异常复杂，不容易被破解。另外，如果用户使用SRTP (安全RTP)，SIP信令包含加密密钥来保证语音的安全。如果有人获得了文本形式的密钥，可以非常容易攻击加密的媒体数据。

选择加密方法

FreeSWITCH支持很多种加密。用户可以对信令加密 (SIP消息加密)，对媒体加密(RTP语音流stream)，或两者都加密。Transport Layer Security (TLS) V1对通过TCP连接传输的都进行了加密；不过这里有一个缺点就是可能导致因为是TCP的原因，会出现网络抖动或迟延。而对于UDP连接传输RTP语音流使用TLSV1 会增加数据开销。ZRTP 基于一个优点，它可以支持端-对-端加密无需任何预交换密钥和安全证书，配置比较复杂，特别对一些客户环境。最后，还有一个是Secure Sockets Layer (SSL) v2/3；这个方法是在加密的TCP连接中使用SSL证书来加密SIP控制信道，默认情况下，不提供对任何RTP数据的加密。登录信息和呼叫元数据通过控制信道被传输，如果用户编辑关注这些数据的话，保护这些数据就足够了，无需对RTP数据添加任何开销。如果用户需要对语音数据本身加密 (可以保证语音数据不能被破解)，可以配合SRTP来一起使用。SRTP 支持对RTP加密，仅对每个RTP UDP数据包添加很少的开销。这样做有一个好处，呼叫数据加密，但是仍然通过UDP传输，无论使用或不使用SRTP，实际上看起来没有什么不同。加密密钥用来通过控制信道（SSLv2/3 加密的）执行SRTP交换，这样双方都获得了最好的结果。通常情况下，SSLv2/3 + SRTP 是对防火墙支持最友好(在目前的安装中，是修改最少的)。SSLv2/3 + SRTP 也是相对比较容易在FreeSWITCH配置的，支持了大部分客户端的加密和大部分的SIP终端，通常来说，他们可能需要加密呼叫数据。



ZRTP是一种协议，是和创建了PGP加密的同一团体联合开发的。更多信息访问http://zfone.com.

使用SSL加密

SSL加密工作是类似于一种协商机制，它通过网站或基于SSL服务端协商来实现加密。使用一个第三方来验证安全证书，然后在发送方和接收方之间使用这个安全证书来交换信息。理论上，基于公共哈希和私有哈希的安全证书应该加载到电话本身和服务器端。

FreeSWITCH支持SIP包SSL v2/3加密。默认环境下，开启了SSL，仅支持对SIP加密，不支持任何对媒体或RTP的加密。对媒体加密是通过加密的SSL控制信道来对RTP数据加密，而且SRTP也要开启。呼叫信息和呼叫元数据可以通过被保护的连接传输，如果用户比较关心数据保护的话，用户可以单独使用SSL加密，没有RTP加密或开销。例如，如果用户不想让探测到电话号码，SSLv2/3已经完全可以胜任这个工作。如果用户希望对语音数据本身加密 (语音可以不被破解)，可以配合SRTP一起使用(参考本章早期部分)。

配置SSLv2/3

为了使用SSL加密，用户必须在FreeSWITCH环境下，编译OpenSSL库。另外，用户需要生成和self-signSSL安全证书。这些证书工作方式完全和web服务器SSL证书一样。

为了编译支持OpenSSL库，确认OpenSSL开发包已经安装。然后执行配置命令，附加这个标签选项--with-openssl。注意，这个标签是默认设置的，很可能用户已经安装了这个库，但是为了保险起见，还是建议附加那个参数。

FreeSWITCH 包括了一个简单的脚本可以帮助用户生成self-signed 认证，可以在Internal SIP Profile中快速使用这些证书。如果服务器的内部主机名是pbx.freeswitch.org，可以这样运行:

```
bin/gentls_cert setup -cn pbx.freeswitch.org -alt
DNS:pbx.freeswitch.org -org freeswitch.org
bin/gentls_cert create_server -cn pbx.freeswitch.org -alt
DNS:pbx.freeswitch.org -org freeswitch.org
```

一些的电话机通过检查主机名来匹配证书，所以用户要修改这个命令来支持用户环境。如果必要，用户可以生成多个证书支持每一个主机名。这些基本生成的证书会自动保存到用户的conf/ssl/文件夹。

一旦生成了用户证书，用户需要让FreeSWITCH知道，SIP profiles使用了这些证书，并且开启了对这些profiles的SSL加密支持。通过以下方法进行配置，在用户Sofia SIP profile中设置变量:

```
<param name="tls" value="true"/>
<param name="tls-version" value="sslv23"/>
<param name="tls-cert-dir" value="/opt/freeswitch/conf/ssl"/>
```

如果用户生成的证书支持passphrase 本身的文件保护, 用户可以在这里输入密码:

```
<param name="tls-passphrase" value=""/>
```

为了增加安全性，如果用户需要的话，可以为不同的SIP profiles 生成不同的证书。

如果用户已经安装了证书，启动了FreeSWITCH的话，用户对终端开启了SSL加密支持，所有的SIP数据包被加密。

使用TLS方法加密

TLS是另外一种加密机制支持建立安全信令。它看起来是相对比较成熟的策略。TLS对所有TCP连接的数据加密，在对话过程中一直维持这个连接。

像SSL，TLS要求OpenSSL库的支持。在FreeSWITCH环境下编译OpenSSL，确保OpenSSL开发包已经安装。执行配置时，使用--with-openssl。注意，这个标签是默认设置的，很可能用户已经安装了这个库，但是为了保险起见，还是建议附加那个参数。
开启TLS加密方式对信令加密，在拨号规则中添加以下一行:

```
<param name="tls-version" value="tlsv1"/>
```

使用TLS是有一些应该注意到地方。TLS (比较早的SSL)运行在TCP，而不是UDP。这里有一个问题。当FreeSWITCH需要连接一个电话机时，如果这个终端电话在NAT后，或NAT转换机制后时，这个电话终端可能就不能连接到FreeSWITCH。用户必须确认所有的防火墙配置都支持TCP呼入的数据。确保终端侧的时间配置是正确的，如果时间太久可能用户会收到奇怪的坏证书错误信息，那么它们之间就的握手就会失败。

在上面的例子中使用SSL，用户也要指定一个认证存放的路径:

```
<param name="tls-cert-dir" value="/opt/freeswitch/conf/ssl"/>
```

如果用户生成的证书支持passphrase本身的文件保护, 用户可以在这里输入密码:

```
<param name="tls-passphrase" value=""/>
```

一旦完成证书安装，启动FreeSWITCH以后，用户就可以开启终端对TLS的支持并且所有的SIP数据包都会加密。

保护语音

语音内容(称之为RTP流) 也许是VoIP通话中最有价值的部分。它也成为VoIP安全作为重要的部分。对RTP流加密可以保证通话内容不被监听或录音或防止其他方法手段获得电话语音。有多种办法可以取得这样的安全保证。

在算法核心是，加密算法的内容要求涉及的双方都要支持一个加密方法，加密算法对来往的传输数据进行加密和解密。换句话说，用户不能使用另外的方法来加密或者解密数据，双方必须使用同样的方法，另外加密算法是基于密钥交换的方法，通常发生在呼叫开始阶段。双方密钥交换的方式类似于密码交换的方式，但是通常是以电子的或自动方式进行。

对语音和媒体流来说，有两种方式的加密方法。这两种方式是SRTP和ZRTP。

SRTP是在2004年发布，由思科和爱立信的IP协议加密技术专家团队开发。SRTP 定义了发送和接收RTP的方法，在这个方法中，使用认证信息和集成方法来保护RTP数据。它支持单播和多播程序。因为它是一个比较老的方式并且由一些主流IP通信厂家开发，逐渐成为大部分标准设备必须支持的加密方式。现在，绝大部分的设备都支持了SRTP。

ZRTP是在2006发布，由Phil Zimmermann (PGP创建者)开发。它是一个相对新的方法，使用密钥自动协商，极大简化了对RTP呼叫的加密设置和操作过程。它也添加了更多的优点，不在依赖于服务器端的加密。在服务器之间可以进行加密，并且双方服务器都不会感觉到正在对RTP流媒体的内容加密。因为极大降低了它们之间的依赖，这样的方式提升了服务器的运行速度。因此，大部分的硬件厂家将部署支持ZRTP 来帮助提升服务器性能。

FreeSWITCH支持SRTP和ZRTP技术，在本章中做更多介绍。

使用SRTP加密

SRTP是一种加密机制，通过SIP在呼叫创建过程中进行加密协商。SIP双方都必须同意支持RTP加密，通过交换密钥对SIP包加密。SRTP的密钥交换通过控制信道完成。通过这些信息对语音流加密。

SRTP启用RTP加密会对每个UDP包会增加少量开销。这样的好处是呼叫数据被加密，但是仍然通过UDP传输，减少了延迟和通常使用的未加密语音流的网络转换机制。

通常来说，SSLv2/3和SRTP 是对防火墙支持最友好的策略，因为实际的工作已经完成，RTP流可以成功通过网络传输。SSL和SRTP对FreeSWITCH来说，配置相对简单。

注意，除非用户开启了SIP数据的加密和明确SRTP的密钥已经启用。为了实现一个终端和FreeSWITCH完全的安全连接，用户应该使用SIP加密和SRTP加密。这样可以防止某些人探测或中间人攻击。如果仅开启了SRTP，仅对RTP包的净荷数据类型加密。

开启SRTP

用户可以从拨号规则中设置基于每个呼叫开启的SRTP加密，通过以下方式设置:

```
<action application="set" data="sip_secure_media=true"/>
```

这个需要双方的legs都支持，呼入和呼出环境都将有效。当然运营商可能不支持SRTP，所以用户仅需要从终端到FreeSWITCH之间的legs 启用它。

用户可以在拨号规则中通过检查变量${sip_secure_media_confirmed} 来确认是否设置了加密保护。如果检测到SIP媒体流被加密，以下这段代码播放一段提示语音:

```
<extension name="is_secure">
  <condition field="${sip_secure_media_confirmed}"
  expression="^true$">
    <action application="sleep" data="1000"/>
    <action application="gentones" data="${bong-ring}"/>
  </condition>
</extension>
```

当debug加密时，一个比较好的提示是查看SIP包中，检查终端电话双方正确请求了加密。如果用户在SIP设置时提供了RTP加密的话，用户将看到SIP包中包含了a=crypto信息。

使用ZRTP方式加密

ZRTP是一种基于SRTP的加密算法，它在交换密钥时和SRTP优势区别，可以使得加密更加安全，并且对服务器端是透明的。这样，ZRTP比SRTP更加灵活，可以对终端实现各个层次的完全控制，实现加密请求完全控制无需担心中间人攻击的风险。在媒体创建时，ZRTP也不要求密钥交换。密钥交换发生在RTP语音交流的初期部分。

当语音在初期不安全状态下，通过Diffie-Hellman密钥交换协议，ZRTP对RTP创建一个密钥。ZRTP协议是基于RFC6189设计。



了解加密

当第一次学习加密时，非常容易迷惑。因为有很多不熟悉的词汇例如Diffie-Hellman和密钥交换。这也就是我们所说的公共密钥体系，它是加密中一个比较大的领域。我们建议用户咨询一些好的学习资料以便更多了解这些领域。

ZRTP主要的功能就在于可以支持代理。在使用SRTP的典型环境中，每个通过加密媒体通信的点都需要清楚这个加密协议，这样才可能对语音流加密或解密。如果用户访问服务器时存在一个媒体转换，这样就可以探测加密的中间语音流。如果使用ZRTP，结果是相反的；服务器在通信流程的中间，它完全不需要清楚加密协议。它们相信互相传递标准的RTP包。因为服务器没有意识到RTP流包含的内容，仅双方终端都支持ZRTP就可以实现绝对安全的通话交流。代理不需要理解加密协议，它们仅传递加密信息。

还有一个在FreeSWITCH环境下默认支持的一个ZRTP优点。ZRTP协议本身在每个初始的RTP交流中插入了协商数据包，这个数据包等待一个回复。如果接收到回复信息，当其他终端请求这个加密时，ZRTP 加密将自动开启。

虽然ZRTP协议不是一个非常受欢迎的协议，但是还是有一些终端电话和软电话支持了ZRTP，还有代理也支持了ZRTP。这个代理软件仅用户安装ZRTP插件Zfone，RTP数据会被自动加密，即使软件本身默认不支持ZRTP。Zfone在后台运行，当执行密钥交换时，会弹出一个提示信息说明密钥交换执行。用户可以使用另外的SDK来开发支持ZRTP软件或者硬件。

用户可以使用拨号规则命令来开启或关闭ZRTP支持，命令如下:

```
<action application="set" data="zrtp_secure_media=[true|false]"/>
```

当ZRTP协商执行时，用户从FreeSWITCH后台可以看到ZRTP已经支持:

```
[DEBUG] switch_rtp.c:928 [ zrtp main]: START SESSION INITIALIZATION.
```

ZRTP开始对这个RTP流注入ZRTP协商数据包。在这个会话中，如果ZRTP成功启用，用户就可以看到一些相关ZRTP日志信息，还有一些确认信息，这样就说明这个通道是安全的。例如:

```
[zrtp protoco]:        Enter state SECURE (DH).
```

用户也可以看到被自动保存的缓存信息，以此和下一个呼叫做对比:

[zrtp cache] Storing ZRTP cache to </usr/local/freeswitch/db/zrtp.dat>...

用户必须确认当FreeSWITCH编译时，ZRTP必须同时被编译。如果用户不能确认的话，可以在配置程序执行时，附加--enable-zrtp 来强制编译ZRTP。

保护密码

在FreeSWITCH环境中，注册话机需要密码。当FreeSWITCH注册到一个外部网关时，或者管理员认证登录系统时都需要密码。这些地方使用的大部分密码都是以文本形式保存的，而且密码强度不够。

另外，许多用户使用简单易猜的密码组合来设置他们的电话终端。更糟糕的是，一些用户从来不修改或设置他们的语音邮箱密码，仅使用默认的密码。

这些密码都是被经常攻击的目标，一旦获得这些密码，系统就会暴露在攻击者面前，可能出现电话盗打。

有几个方法来缓解密码问题的发生。

注册密码

注册安全信息无需保存以普通文本的形式保存在硬盘中。当在用户文件中定义一个SIP安全信息时，不用包含以下设置:

```
<param name="password" value="samiam"/>
```

使用一个预先计算出的a1-hash密码来替换上面的密码，像这样:

```
<param name="a1-hash" value="c6440e5de50b403206989679159de89a"/>
```

为了生成a1-hash，首先获得一个字符串username:domain:password的md5计算结果，它把用户名称，系统域名，和密码绑定在一起，通过冒号互相隔离。以下是一个举例:

```
echo -n "darren:2600hz.com:pass1234" | md5sum
  b62d1e3e27773ffd173c87e342a6aace
```

用户可以在文件入口使用返回的哈希结果。这样的话，用户不需要在硬盘中保存实际的SIP注册信息，有人如果盗用了这个文件，也不会看到密码。



在Mac OSX，使用md5而不是使用md5sum。

一个完整的举例如下:

```
<user id="darren">
  <params>
    <param name="a1-hash"
      value="c6440e5de50b403206989679159de89a"/>
  </params>
</user>
```

Voicemail密码

过去因为很多语音出现了语音邮箱被攻击的记录。除了仅简单监听某些人的留言信息，语音邮箱通常被暴露还因为它们有回报和前转功能，这些功能可以通过远端来激活。最受关注的策略是破解了语音邮箱，前转个人通话，路由到电话资费比较贵的国际长途目的地，在很短时间内打出上千美金的话费。甚至于，今天，攻击语音邮箱是非常普遍的手段。

对语音邮箱密码的保护相对比较简单。FreeSWITCH在数据库中保存文本形式的邮箱密码，支持用户对强度比较弱的密码检测，例如1111或1234. 用户可以对系统用户实施检测，是否分机用户使用分机号码作为语音邮箱密码（不安全），这样的设置通常也是用户经常使用的密码设置方法。

为了支持检测弱密码设置，用户需要写一个脚本从语音邮箱配置数据库查找这些密码。假设，用户在FreeSWITCH使用默认的设置，语音邮箱数据库补充在FreeSWITCH DB文件夹中的一个SQLite文件。这个文件的保存路径可能不同，完全依赖于FreeSWITCH安装路径，但是大部分环境下，安装在/opt/freeswitch/db，/usr/local/freeswitch/db或/var/lib/freeswitch/db。

通过以下SQLite查询语句检查用户数据库的举例:

```
sqlite3 db/voicemail_default.db "select * from voicemail_prefs
where password=1234 or password=1111"
```

这个命令使用SQLite3 linux 客户端查询voicemail_prefs 表，查询任何使用密码1111或1234的邮箱。它将打印所有使用这些密码的用户名称和域名信息。用户可以通过强行重置密码的形式修改密码或通知邮箱用户修改他们的密码。

总结

这个章节我们简要介绍了今天最常用的VOIP安全技术。 现在有很多资源包括网站介绍这些技术，例如www.hackingvoip.com，VoIP攻击，RTP加密等等。

在本章中，我们采取了几个基本步骤，这些步骤已经完全可以防备今天通常使用的攻击手段，DoS攻击和恶意攻击。这些安全措施可以保证大部分中小型PBX用户和托管VoIP系统的需要，实现其安全性和可靠性的要求。

现在我们将接下来学习本书的最后一个章节，我们把一些难以归类的话题放到了最后这个章节。我们也将介绍更多关于SIP，VoIP和FreeSWITCH的学习资料。

14

高级功能和进一步阅读

两种类型的应用程序可以使用FreeSWITCH—一种是使用C语言开发的内置的模块，另外一种应用程序就是通过外部程序控制管理FreeSWITCH。两种程序都在这个章节做简单介绍。

FreeSWITCH包含各种应用模块，当呼叫发起后这些模块提供呼叫功能和路由转换。这些模块包括Caller ID 查找，实时计费模块，和多方会议模块。模块之间可以互相调用来丰富基本的呼叫应用设置功能，管理呼叫，或者提供其他的功能。

另外，整个FreeSWITCH开源社区的应用模块也在迅速增长，可以提供各种软件程序来支持FreeSWITCH工作。

在这个章节，我们假设用户已经对FreeSWITCH如何工作有了基本的了解。我们将回顾FreeSWITCH的各种应用程序和模块，了解这些模块的基本工作原理。我们也将讨论一些第三方的工具，通过第三方工具可以进一步拓展FreeSWITCH。

我们将讨论以下内容:

- 对方会议 (mod_conference)
- 实时计费(mod_nibblebill)
- 其他终端设备类型: Skype，GSM和TDM
- Web界面管理和其他项目

多方会议

FreeSWITCH包含一个强大的内置多方会议模块mod_conference支持对用户的语音通道混音。这个系统同样可以完全控制语音混音和呼叫方互动的功能，例如按键音检测，管理每个通道接收和发送路径，音量控制，增益控制和其他的功能。只要有空闲的系统资源（例如内存，CPU等等），用户可以创建多个会议室。

配置

在XML的conference 部分配置mod_conference。这个文件保存在autoload_configs/conferenceconf.xml 文件。这个配置文件提供一系列的profiles定义了会议如何工作。这些prifiles通过用户创建的拨号规则来设置会议。conference 配置文件分成几个部分，每个部分有自己的参数。这些关键部分将在本章讨论。

Conference profiles

Conference profiles是一个设置模板，可以应用在任何会议中。通过caller-controls，conference profiles支持用户对单个的会议进行完整的自定义设置。用户可以创建一个模板类型，在会议中使用这些模板，创建一个用户可以自定义的会议profile。将来用户可以使用这些profile。

Conference profiles在每个命名的profile要素中包含一个参数列表。基本结构是这样的:

```
<profiles>
  <profile name="default">
    <param name="paramName" value="paramValue"/>
  </profile>
</profiles>
```

用户可以有多个<profile> 标签，每个<profile>标签可以支持多个<param>标签。以下列表是支持的参数:

- rate: rate 参数定义会议桥使用的默认的采样率 (和最高)。
  所有进入会议通道的呼叫方，如果没有语音没有在默认的采样率转换，通道使用的采样率都将转换成这里设置的采样率。为了混音的要求，这里定义了最低的采样率—如果两个用户的电话终端都是支持的是高清语音，他们加入到了一个采样率为8000的会议，这些用户的语音的采样率将降低到相对低的采样率。

  - 语法: <param name="rate" value="8000"/>
  - 默认: 8000
  - 可支持选项: 8000, 12000, 16000, 24000, 32000, 和48000 (未来可能其他的选项)

- caller-controls: 这个参数指定一个caller-controls profile来支持会议桥。

  - Parameter syntax: <param name="caller-controls" value="default"/>

- auto-record: 这个参数支持是否对会议执行自动录音。
  两方或更多方用户加入以后开始录音。如果设置了这个选项，为了录音，必须设置一个录音文件路径。

  - 语法: <param name="auto-record" value="filename"/>
  - 默认: off
  - 样本文件: /usr/local/freeswitch/sounds/conferences/${conference_name}.wav

  样本文件名称是基于还有桥名称设置的。

- interval: 这个参数设置一个时间间隔，每帧混音需要的时间。这个设置类似于SIP ptime，但是不需要匹配呼叫方实际的ptime。设置数字高要求的CPU资源相对较少。
  使用时可能引起沟通质量问题，因此根据实际情况做适当的调整。默认设置应该可以工作。

  - 语法: <param name="interval" value="20"/>
  - 默认: 20

- energy-level: 这个参数指定一个语音强度级别 (强度/音量)，以此设置发送到其他用户的声音音量。强度级别是一个阀值，这个阀值用来控制语音强度级别，在这个强度下监控用户通话和接收到的背景噪音。这个功能可以帮助FreeSWITCH从混音后的会议中消除背景噪音和周围环境噪音。如果这个值设置的比较高，可能导致在用户通话开始和结束时发生卡顿。
  值是0将关闭检测，桥接所有数据即使可能它们都是背景噪音。
  - 语法: <param name="energy-level" value="20"/>
  - 默认: 20
  - 设置为0 完全关闭

- member-flags: 这个参数支持对单个会议成员设置指定的标志或参数。
  这些选项包括是否对没有发言的用户发送会议语音（对没有发言的用户发送语音），一个指定的成员是否是会议主持人（如果是，这些人离开会议时，会议就应该结束）。选项通过管道符号隔离。

  - 语法: <param name="member-flags"
    value="waste|endconf"/>
  - 支持member-flags 参数选项如下:
    - deaf: 默认环境下，防止成员窃听其他成员会议谈话 (这个设置会议开始以后通过事件可以修改)。
    - waste: 发送语音到通道，即使没有通话进行。
    - dist-dtmf: 发送DTMF信号到每个通道。
      当有人输入DTMF 按键音时，通常被FreeSWITCH处理掉。这个设置可以防止DTFM按键音被FreeSWITCH处理掉，而是对所有其他成员发送DTMF信号音。
    - endconf: 指定这个参数，当这个成员退出时，会议结束。

- conference-flags: 这个参数用来控制会议流程活动。
  目前可支持的选项仅是当会议开始前，强迫用户等待会议主持人。
  主持人通过拨号规则流程，当进入会议室时，传递一个额外的标志作为一个主持人标识。当等待主持人加入时，成员听到语音等待的语音。

  - 语法: <param name="conference-flags"
    value="wait-mod"/>

- tts-engine: 这个参数指定一个在会议桥中使用的语音合成引擎。

    - 语法: <param name="tts-engine"
      value="cepstral"/>

- tts-voice: 这个参数指定一个在会议桥中使用的语音合成引擎语音。

    - Parameter syntax: <param name="tts-voice" value="david"/>

- pin: 这个参数指定一个加入会议室的秘密，加入会议室之前必须正确输入会议室密码。

    - 语法: <param name="pin" value="12345"/>

- max-members: 会议室支持的最多成员数。
  如果已经达到这个最大会议成员数，其他将进入的成员会听到会议系统播放的max-members-sound提示音，加入者不能再加入会员。

    - 参数: <param name="max-members" value="20"/>

- caller-id-name: 当通过会议桥发起外呼时，这个参数用来设置外呼的caller ID name。

    - 语法: <param name="caller-id-name" value="John
      Doe"/>

- caller-id-number: 当通过会议桥发起外呼时，这个参数用来设置外呼的caller ID number。

    - 语法: <param name="caller-id-number"
      value="4158867900"/>

- comfort-noise: 这个参数支持对背景噪音的强度的处理。有时，如果语音音量相当低的话，一些成员可能认为他们用户不在会议室，或者线路断掉。舒适噪音设置对通话线路提供一个白噪音，这个噪音会调整语音环境，让呼叫者感觉到这个线路仍然是连接的。注意，设置在高的采样率，噪音可能变得相当糟糕，因此，如果采样率高于8000 HZ，用户需要做适当调整。

    - 语法: <param name="comfort-noise"
      value="1000"/>

- announce-count: 当新成员进入，当达到了这个设置的阀值时，这个参数会对新加入的成员播报目前会议室的总人数。它要求一个有效的语音合成引擎来支持播报。
    - 参数: <param name="announce-count" value="5"/>
- suppress-events: 这个参数使用在FreeSWITCH 事件系统中。这个特别的设置选项表示不能对其他监听会议事件的对象触发某些事件类型。
    - 语法: <param name="suppress-events" value="true"/>
- sound-prefix: 这个参数用来设置一个默认路径来接收会议语音文件。
    - 语法: <param name="sound-prefix" value="/usr/local/freeswitch/sounds/"/>

以下参数支持当某些活动启动后，客户可以在会议桥内自定义语音播放文件。所有语音文件可以支持对独立的呼叫通道播放，而不是对所有在会议室的对象，enter-sound和exit-sound特殊环境语音文件可以对所有成员播放。

所有文件格式为: <param name="sound-name" value="file.wav"/>

以下是自定义的语音文件:

- muted-sound: 当用户被静音，播放此文件。
- unmuted-sound: 当用户被解锁静音，播放此文件。
- alone-sound: 当仅留下本成员时，对剩余成员播放的语音。
- enter-sound: 当有新成员加入时，对其他成员播放此语音。
- exit-sound: 当一个成员离开会议室时，对所有成员播放此语音。
- kicked-sound: 当一个成员被踢出会议室时，播放此语音。
- locked-sound: 当用户试图加入一个已锁定的会议室时播放此语音文件。
- is-locked-sound: 当会议室被锁定时对会议成员播放此语音文件。

- is-unlocked-sound: 当会议室被解锁时，对会议成员播放此语音。

- pin-sound: 进入会议室时，播放要求密码输入的提示语音。
- bad-pin-sound: 当输入无效密码时，播放此错误语音提示。
- perpetual-sound: 一个特别设置—当对象在会议室时不停播放一个循环的语音。
- moh-sound: 文件和数据源来处理一个特别的music-on-hold 语音流，当会议室仅有一个成员时，第二个成员加入时，这个语音将停止播放除非，指定了mod-wait 设置 (前面提及)。

- max-members-sound: 如果有人加入会议室时，而此时会议室人数已经达到最多人数，则播放此文件。

Caller-controls

多方会议主持用户控制会议，用户通过按键输入来执行命令。命令包括修改会议的音量，静音/关闭静音，或更多高级选项，例如对一些人播放语音菜单，把会员从一个会议室移到另外的会议室。

呼叫方会议室控制是基于预设的模板来实现，当会议第一次启动以后，这些模板就被启用。例如，用户可以设置一个控制列表(按键0静音，按键1表示降低音量，按键3表示提高音量) ，这些控制方法可以应用在三个不同的会议室。这些设置在会议启动以后就可以使用，一直到会议结束。

注意，这里不能支持一个成员进入了会议室对会议设置了控制，而另外一个成员使用其他的控制方法来管理会议。



警告:不要default或none作为用户的caller-controls。
这些词是系统保留的关键词支持默认的按键输入映射或无按键输入映射。

以下配置是一个caller-controls 配置举例:

```
<caller-controls>
  <group name="standard-keys"
    <control action="vol talk
    <control action="vol talk
    <control action="vol talk
```

```
>
dn" digits="1"/>
zero" digits="2"/>
up" digits="3"/>
```

```
        <control action="transfer" digits="5"
          data="100 XML default"/>
        <control action="execute_application" digits="0"
          data="playback conf_help.wav"/>
        <control action="execute_application" digits="#"
          data="execute_dialplan conference-menu"/>
      </group>
    </caller-controls>
```

以上例子演示了如何创建一个caller-controls profile，命名为standard-keys。按键是1，2和3各自表示调低音量，正常音量和调高音量，按键5支持呼叫方输入按键5后转接此呼叫方到分机100，按键0和#分别执行一个指定的拨号规则应用，

Advertise

会议的advertise 配置文件部分支持用户通过FreeSWITCH 事件系统对服务和订阅对象生成一个呈现事件。这样的思路是创建一个永久的会议房间名称，这些房间名生成呈现事件，类似于终端电话和其他设备生成事件的方式。外部的程序可以检测这个会议室是否被占用。

Advertise设置在advertise标签中，每一个要素包含一个会议室名称，就像下面举例：

```
<advertise>
  <room name="888@$${domain}" status="FreeSWITCH"/>
</advertise>
```

发送和接收XMPP事件

会议模块允许XMPP服务器例如Gtalk通过Jabber/XMPP接收命令。这些命令包括踢出用户，转接电话和其他功能。配置举例如下:

```
<chat-permissions>
  <profile name="default">
    <user name="bob@somewhere.com" commands="all"/>
    <user name="harry@somewhere.com"
      commands="|deaf|dial|energy|kick
                |list|lock|mute|norecord
                |play|record|relate|say|saymember
                |stop|transfer|undeaf|unlock
                |unmute|volume_in|volume_out|"/>
  </profile>
</chat-permissions>
```

连接呼叫方到会议

呼叫方可以通过conference 应用来加入到会议中，这样的方式通常是通过XML拨号规则或是从事件套接字通过API调用呼叫来触发这个流程。连接呼叫方到会议的基本语法如下:

```
<action application="conference" data="confname@profilename"/>
```

confname 是会议室名称，profilename是一个从会议批准中调用的profile（在本章早期指定的）。

用户可以在conference profile名称后附加一个+flags标识，来传递可选的用户参数，代码如下:

```
<action application="conference"
  data="confname@profilename+ConfPIN+flags{
  mute|deaf|waste|moderator}
  "/>
```

当第一个会议成员进入到会议桥时，系统按需创建这个会议。在这个创建流程的基础上，和这个会议相关的活动的profile设置和设定的会议室密码将被保存在内存中。非常重要的一点是，因为修改了加载到内存的配置不会影响一个正在进行的会议。例如，一旦会议开始以后，后续加入会议的成员也将使用会议启动时创建的密码，会议所有成员必须使用同样的密码。

用户设置的profile名称必须匹配在conf/autoload_configs/conference.conf.xml 文件中的profile。

动态创建的会议室会一直保持活动状态，直到会议室成员人数降为0。

以下是一个桥接电话到会议室的举例，在这个例子中，我们设置了这样的配置:

Action data	Description
confname	Profile is "default", no flags or PIN
confname+1234	Profile is "default", PIN is 1234
confname@profilename+1234	Profile is "default", PIN is 1234, no flags
confname@profilename++flags{mute|waste}	Profile is "default", multiple flags, no PIN
confname+1234+flags{mute|waste}	Profile is "default", multiple flags with PIN

注意，一些选项是可选的，它们的顺序非常重要。

控制活动的会议

一些CLI和API命令可以控制存活动会议。大部分被使用的命令是用来踢出会议成员，调整讲话音量和发起呼叫(邀请成员加入一个会议)。这些设置超出了我们讨论的范围，用户可以查看FreeSWITCH wiki 获得更多利用CLI命令来控制会议桥。



更多关于会议的详细信息，可访问
http://wiki.freeswitch.org/wiki/Mod_conference.

Nibblebill计费

mod_nibblebill 是FreeSWITCH呼叫计费模块。最初这个模块是由Darren Schreiber开发，模块的目的是弥补专业级中继系统缺乏的一个实时环境下检测电话盗打的功能。

它的目的是支持在实时呼叫环境中，通话过程中，模块可以实时地从数据库中扣除这个实时呼叫产生的费。
Darren有以下几个目标:

- 从用户账户存款中中实时扣除通话话费
- 支持对单个呼叫实行不同的计费费率
- 如果用户存款过少，支持对呼叫用户发出告警提示 (通过通道中的语音)
- 当用户存款完全耗尽时，支持挂机或重新路由电话呼叫
- 可以对多并发呼叫使用以上计费功能

用例

mod_nibblebill 可以使用在多种不同的用例，我们列出其中一部分以便讨论。

计费(预付方式)

可以支持用户在账户中存入现金，然后根据通话时间逐渐从账户中扣除通话费用。另外，当用户使用完账户的金额时，对呼叫方播放一个告警提示信息（或执行其他流程）。

直到账户金额完全耗尽，呼叫方就被转接到一个extension入口，呼叫方可以
通过DTMF按键输入再次充值，或只是简单挂机。

计费(后付费方式)

如果用户数据库允许，可以添加一个阀值告警，如果余额为负数就发出告警。
呼叫方插入一个负数数值在数据库中，使用完成以后对呼叫方实施计费。
通过这样的方式，用户可以防止盗打电话，呼叫方仍然被挂断，主要通话话费
低于用户设置的阀值(负值）。

这是一种典型的做法来对落地通话计费，如果账户被超额使用，没有人付话费账单，
系统自动会切断通话或服务。

基于每个通话的计费服务

用户可以提供一些特别的计费服务模式，在确认一些事件后（例如，输入了信用卡，
然后确认了信用卡有效身份），既可以使用固定的费率或通过每分钟计费。

设置最大余额防止电话盗打

可以对系统用户设置一个余额值，类似于我们以前肯定的，但是不能告诉用户主观设置。
当用户消费了余额对天，星期，和月设置的限定时，用户将不能发起更多呼叫。
用户可以使用一个外部脚本程序在一定的预设时间内对账户充值。支持了类似这样的
功能，例如“一天可以打100分钟”或者其他的方式。

设计目标

如果用户计划使用mod_nibblebill 计费模块，请注意模块的设计目标:

- 并行设计: 支持多个实时通道对同一账户同一帐号码的管理。
- 拓展性: 支持不同的心跳周期 (或者同时呼叫时关闭管理功能)。
  根据系统负载支持管理员调整检查设置。
- 灵活性: 支持对告警级别和“余额不足”级别灵活设置，这个设置
  支持基于全局或每个用户，允许对呼叫方在处于余额不足状态做自定
  义设置。

- 可自定义：这些设置应该是可自定义的，包括当用户被挂断后或被警告后系统对他们所采取的流程。

按照和配置

mod_nibblebill 是FreeSWITCH主要代码树的一个部分。它要求database/ODBC支持。必须获得ODBC名称来配置mod_nibblebill，就像FreeSWITCH中其他的ODBC模块一样。默认环境下，这个模块没有安装，Linux/Unix必须编译mod_nibblebill。

启用mod_nibblebill方法类似于第二章介绍的如何安装mod_filte编译安装方法:

1. 在FreeSWITCH源代码路径中，找到modules.conf文件，删除注释，例如:

   #applications/mod_nibblebill

   移除# 并且保存文件。

2. 通过以下命令编译这个模块：

   make mod_nibblebill-install

3. 打开文件conf/autoload_configs/modules.conf.xml，添加此模块:

   <!-- <load module="mod_nibblebill"/> -->

   移除 <!—和-->标签，保存文件。

4. 在conf/autoload_configs/nibblebill.conf.xml中修改设置:

   <param name="odbc-dsn" value="database:user:password"/>

5. 保存文件，然后退出。

现在，当FreeSWITCH启动以后，mod_nibblebill 将自动加载。注意，用户也可以无需重新启动FreeSWITCH来加载或卸载mod_nibblebill 模块。通过配置nibblebill文件就可以无需关闭整个系统来管理这个模块。



更多关于FreeSWITCH中的Data Source Names (DSN)信息，可访问 http://wiki.freeswitch.org/wiki/Data_source_name 获得。

Database tables

对于配置文件来说 (我们以前从nibblebill.conf.xml 文件中看到的)，确保用户已经安装了ODBC database驱动和数据库，和正确的database，table和column名称。讨论ODBC配置超出了本书范围，一个基本的odbc.ini 配置文件类似于这样:

```
[NIBBLEBILL]
Description = Nibblebill
Driver= MySQL
SERVER= localhost
PORT = 3306
DATABASE = nibblebill
OPTION= 67108864
Socket = /var/lib/mysql/mysql.sock
User= db_user
Password = db_pass
```

样本范例:

```
mysql> use nibblebill;
mysql> select * from accounts;
+--------+-----------+--------+
| id     | name      | cash   |
+--------+-----------+--------+
|      1 | Darren    | 41.4161 |
|      2 | Joe       |     50 |
|      9 | tester9   |     50 |
|     10 | tester10  | 44.8213 |
| 837269 | My Company |     50 |
+--------+-----------+--------+
5 rows in set (0.00 sec)
```

在上面的例子中，在数据库nibblebill 中包含了account表。这个表中包含了id和cash，计费脚本使用id和cash。id列代表用户的account code，cash代表用户目前的账户余额。以上实例在nibblebill.conf.xml文件中相应的配置如下:

```
<param    name="db_dsn" value="nibblebill:user:password"/>
<param    name="db_table" value="accounts"/>
<param    name="db_column_cash" value="cash"/>
<param    name="db_column_account" value="id"/>
```

创建一个数据库表支持PostgreSQL

使用以下SQL命令创建一个Postgres表:

```
create table accounts (
  id bigserial not null,
  name varchar( 256 ),
  cash double precision not null
);
```

创建一个数据库表支持MySQL

使用以下命令创建一个MySQL表:

```
CREATE TABLE accounts
(
   id int NOT NULL PRIMARY KEY,
   name VARCHAR(255),
   cash double precision NOT NULL
);
```

注意，用户的业务逻辑可能需要在accounts 表中添加更多的列。
nibblebill 数据库仅使用了id, name, cash; 它忽略了其他的列。

对一个呼叫计费

有很多种方法支持计费呼叫。这个部分将讨论修改的方法和相应的选项。

nibble方法(默认)

默认的计费方法是基于FreeSWITCH心跳的概念。对某一个x 秒，我们从账户扣除y 的金额。

为了计费，用户必须在一个实时的通道最少设置两个变量。两个变量是nibble_rate和nibble_account。 作为一个简洁的计费功能，mod_nibblebill真正不关心用户从哪里设置的计费变量，只要在挂机动作发生之前，它们可以退出。这表示用户可以拨号规则中设置它们，这个拨号规则安装在Lua脚本中的任何地方，只要用户可以修改通道变量。

在这个最简单的表中，用户可以把这个表添加到用户目录入口:

```
<variable name="nibble_rate" value="0.03"/>
<variable name="nibble_account" value="18238"/>
```

现在，系统就可以对用户的每个呼出和呼入进行计费，费率是$0.03/分钟。
系统将针对账户18238计费。

默认环境下，心跳周期设置为60秒。这代表每60秒，系统将从账户中扣除$0.03。
注意，所有数学计算是通过FreeSWITCH的内部微秒计算器来运算。这表示:

1. 如果心跳不能被准确及时触发，用户将获得一个几厘钱的话费。
   用户应该确认数据库是否支持这样的数据结果。
2. 计算器必须正确计数钟摆之间的时间。不能有“消失”的话费。
3. 最低计费还不存在。

用户可以修改心跳周期支持全局配置，配置如下:

```
<param name="global_heartbeat" value="300">
```

这个设置将在每300秒钟或5分钟触发一次心跳。心跳可以设置到很低，直到
每秒钟一次 (但是这不是非常明智的做法，因为用户正在每秒钟，对每个通道
做一个数据库调用)。

Nibble计费的可选方法

我们可以使用这个计费模块无需心跳支持。这表示仅是在通话结束后计费。
用户可以像上面提到的例子一样设置同样的变量，但是用户同样也需要在
mod_nibblebill.conf.xml 文件中设置另外的变量:

```
<param name="global_heartbeat" value="off">
```

这样设置以后，计费将在通话结束以后进行。时间计算将从呼叫应答开始直到
呼叫结束。如果呼叫从未被接听，跳过计费。

当设置以下参数后，系统将使用以下计算公式来进行话费计算:

```
([time call ended] - [time call answered]) x [rate per minute] = total
charge
```



注意，这样的方式不会支持实时通话的监督管理，表示可能发生
电话盗打，一些用户通话时会超出所设的限额。

举例

以下举例演示了如何部署各种计费场景。

单用户不同费率

计费方法可以实现对每分钟，每一个用户设置不同的费率。

现在如果有两个用户—一个是以$0.05/分钟计费，另外一个以$0.10/分钟计费。
对呼叫800号码不计费。用户应该这样设置目录入口：

```
<user id="dschreiber">
  <params>
    <param name="password" value="1234"/>
  </params>
  <variables>
    <variable name="nibble_rate" value="0.05"/>
    <variable name="nibble_account" value="8182"/>
    <variable name="default_areacode" value="415"/>
    <variable name="toll_allow"
      value="domestic,international,local"/>
    <variable name="user_context" value="default"/>
  </variables>
</user>
<user id="expensive_guy">
  <params>
    <param name="password" value="1234"/>
  </params>
  <variables>
    <variable name="nibble_rate" value="0.10"/>
    <variable name="nibble_account" value="2932"/>
    <variable name="default_areacode" value="212"/>
    <variable name="toll_allow"
      value="domestic,international,local"/>
    <variable name="user_context" value="default"/>
  </variables>
</user>
```

在拨号规则中，仅覆盖800号码计费费率:

```
<extension name="tollfree800">
  <condition field="destination_number"
    expression="^1"(800\d{7})$">
    <action application="set" data="nibble_rate=0"/>
    <action application="bridge"
      data="sofia/gateway/flowroute/$1"/>
  </condition>
</extension>
```

所有不是800号码的通话通过不同的费率来计费，而免费800的呼叫则设置费率为0，(相当于不计费)。

对所有用户设置单一费率

在用户帐号输入以下代码:

```
<user id="mercutioviz">
  <params>
    <param name="password" value="1234"/>
  </params>
  <variables>
    <variable name="toll_allow"
      value="domestic,international,local"/>
    <variable name="user_context" value="default"/>
    <variable name="nibble_account" value="1"/>
  </variables>
</user>
```

然后添加这些代码:

```
<user id="dschreiber">
  <params>
    <param name="password" value="1234"/>
  </params>
  <variables>
    <variable name="toll_allow"
      value="domestic,international,local"/>
    <variable name="user_context" value=" default"/>
    <variable name="nibble_account" value="2"/>
  </variables>
</user>
```

在拨号规则中，添加一个extension的入口，这个入口是用户计费的入口:

```
<extension name="outbound">
  <condition field="destination_number"
    expression="^91(\d{10,})$">
    <action application="set" data="nibble_rate=0.05"/>
    <action application="set"
      data="nibble_account=${nibble_account}"/>
    <action application="bridge"
      data="sofia/gateway/flowroute/1$1"/>
  </condition>
</extension>
```

每个地区代码不同费率

这个例子对所有通话按照$0.05/分钟计费，除了地区代码为919的通话，这个地区代码的通话则按照$0.07/分钟计费，呼叫800号码的通话则免费。这些通话时根据用户目录下的account code来设置。

在下面的例子中，我们从拨号规则中设置了费率。小心！这个方式可以覆盖在用户/命令级别的变量设置:

```
<extension name="tollfree800">
  <condition field=" destination_number"
    expression="^1?(800\d{7})$">
    <action application="set"
      data="nibble_account=${accountcode}"/>
    <action application="set" data="nibble_rate=0"/>
    <action application="bridge"
      data="sofia/gateway/flowroute/1$1"/>
  </condition>
</extension>
<extension name="special919rate">
  <condition field="destination_number"
    expression="^1?(919\d{7})$">
    <action application="set"
      data="nibble_account=${accountcode}"/>
    <action application="set" data="nibble_rate=0.07"/>
    <action application="bridge"
              data="sofia/gateway/flowroute/1$1"/>
  </condition>
</extension>
<extension name="domestic">
  <condition field="destination_number"
```

```
        expression="^1?(\d{10})$">
      <action application="set"
        data="nibble_account=${accountcode}"/>
      <action application="set" data="nibble_rate=0.05"/>
      <action application="bridge"
        data="sofia/gateway/flowroute/1$1"/>
    </condition>
  </extension>
```

每一种服务不同的费率

当进行实时通话时计费，这种方法是围绕对nibble_rate对象修改。实际通话过程中根据服务级别的不同设置不同的费率。

这是一个基本思路: 用户首先呼入到呼叫的第一个流程中，现在对他们的计费费率是$1.00/分钟，允许通过tier1 来支持。如果他们需要tier2支持，费率就会调整到$5.00/分钟。当他们被转接以后，费率也做了相应的调整。如果用户在等待队列中的时候，用户呼叫费率可以设置为0. 在下面的例子中，分机2000 路由到第一个tier座席，在分机1000. 分机2001 路由到第二个座席tier 座席，在分机1001:

```
<extension name="tier1">
  <condition field="destination_number" expression="^2000$">
    <!-- Save anything billed at a previous rate -->
    <action application="nibblebill" data="flush"/>
    <!-- Change the rate -->
    <action application="set" data="nibble_rate=1.00"/>
    <!-- Transfer to Tier1 rep -->
    <action application="transfer" data="1000 XML default"/>
  </condition>
</extension>
<extension name="tier2">
  <condition field="destination_number" expression="^2001">
    <!-- Save anything billed at a previous rate -->
    <action application="nibblebill" data="flush"/>
    <!-- Change the rate -->
    <action application="set" data="nibble_rate=5.00"/>
    <!-- Transfer to Tier2 rep -->
    <action application="transfer" data="1001 XML default"/>
  </condition>
</extension>
```

另外一种使用场景是，当用户正在和公司技术支持通话时，当呼叫结束后，用户可能进入一个服务满意度调查流程，这个情况下，计费停止。这个例子和上面提到的一样，当然这个例子则路由到了extension 2002的满意度问卷调查流程：

```
<extension name="survey-after-call">
  <condition field="destination_number" expression="^2002">
    <!-- Handle support request here at $1.00/minute via
      extension 1001 -->
    <action application="set" data="nibble_rate=1.00"/>
    <action application="set"
      data="hangup_after_bridge=false"/>
    <action application="bridge"
      data="sofia/internal/1001@$${domain}"/>
    <action application="nibblebill" data="flush"/>
      <!-- Set rate to 0, then xfer caller to survey IVR -->
    <action application="set" data="nibble_rate=0.00"/>
    <action application="bridge"
      data="sofia/internal/1002@$${domain}"/>
  </condition>
</extension>
```



警告: 有一个“缓存的”方式. 在计费费率修改之前，用户应该在数据库中刷新当前呼叫的计费。因为查询命令可能按照原来的费率设置进行，这样可以保证数据库写入的完整性。参考本部分的/CLI/API 命令flush。

当余额不足是挂断通话

当帐号余额到了用户设置的限定时（设置在nobal_amt），通话被转接到一个用户指定的extension。一旦进入到这个指定的extension，可以播放对呼叫方播放一个语音提示，例如"因为账户余额不足，您的通话已挂断。”,我们已经把此通话转接到了一个extension, 此时暂停计费过程，系统非常灵活地支持了另外的流程，支持用户通过输入信用卡号码对账户充值。

在配置文件conf/autoload_configs/nibblebill.conf.xml file 添加类似的设置:

```
<param name="nobal_amt" value="0"/>
<param name="nobal_action" value="hangup XML default"/>
```

在这个例子中，注意，"hangup XML default"的参数nobal_action。
当账户余额达到了nobal_amt 设置的阀值时，这个设置通知mod_nibblebill
转接电话到到拨号规则的default context 中的extension hangup。
用户可以在拨号规则中添加这些设置:

```
<extension name="hangup">
  <condition field="destination_number"
    expression="^(hangup)$">
    <action application="playback" data="no_more_funds.wav"/>
    <action application="hangup"/>
  </condition>
</extension>
```

在这个例子中，当帐号余额到0时，呼叫被转接到这个hangup extension。
这个extension 将开始播放我们设置的余额不足的提示音 (假设用户已经录制了
这个语音文件no_more_funds.wav)，然后挂机。



注意，现在B leg 也被转接到同样的extension。换句话说，其他在这个通话中的用户也可以听到这个余额不足的播放信息。

Application/CLI/API命令

以下命令可以在拨号规则中使用CLI，或API。语法和应用中的基本类似，
在应用中使用的格式:

```
<action application="nibblebill" data="action [params]"/>
```

而CLI和API 命令中的格式是:

```
nibblebill <channel-uuid> <action> [params]
```

Check

在用户的应用程序中插入check 命令，或使用在CLI命令中则返回一个当前的计费余额。
它不包括任何没有被写入数据库的增量。

```
<action application="nibblebill" data="check"/>
```

Flush

在用户拨号规则中插入以下代码:

```
<action application="nibblebill" data="flush"/>
```

以上代码立即将等待的费用插入到数据库。计费将继续，经过一段时间后，所有需要结算的通话到达这个时间点时，经过计算后和被录入。这个不会影响计费暂停的时间。

Pause

在拨号规则中插入以下代码:

```
<action application="nibblebill" data="pause"/>
```

这段代码会设置一个标识暂停计费。如果通话结束时，计费暂停，则无计费，这个时间是呼叫暂停的时间，这个时间将被算作计费时间，因为计费启动的时间早于pause 仍然会被记录。用户可以在后续的通话过程中，通过命令resume手动恢复计费(查看以下部分).

注意，如果通话已经被暂停，用户使用pause 命令暂停通话，这个pause 命令将被忽略。

Resume

在用户拨号规则中插入以下代码:

```
<action application="nibblebill" data="resume"/>
```

这个命令恢复一个被暂停的呼叫。介于暂停和恢复之间的时间将不计费。
注意，用户可以多次暂停或恢复通话。介于每个暂停和恢复之间的时间将不计费。

Reset

在拨号规则中添加以下代码:

```
<action application="nibblebill" data="reset"/>
```

这个功能将重设当前时间为计费定时器。但是，注意，这里我们做的所有事情是重新设置所有的内部计数器，在当前时间跟踪呼叫过程，因此任何当前时间点为准的时间（没有被记录到硬盘）都将“消失”或者看作是“空闲的”。



对于具体的账户来说，任何已经从数据库扣除的金额都被当作是已记录的数据-一个完成的业务。这个命令不会对已经写入硬盘或者数据库的数据有影响。

添加和扣除金额

在拨号规则中插入一个adjust 命令:

```
<action application="nibblebill" data="adjust 5.00"/>
```

它可以从账户中添加或扣除一定的金额 (在这个举例中，我们添加了$5.00)。
注意，这个充值会立即执行，可以规避任何为数据库生效时所设置的保护措施。
当用户使用这个命令时，支持对接用户的功能数据库。

如果设置为负数，可以从账户中扣除金额。

开启一个会话心跳

在通话过程中开启一个会话heartbeat 是通过以下代码实现:

```
<action application="nibblebill" data="heartbeat 60"/>
```

这设置当前呼叫（仅）的心跳为60秒。用户可以根据每个呼叫设置不同的值。

仅对B Leg计费

如果用户仅对B Leg计费，必须对B leg通道开启enable_heartbeat_events变量。用户可以通过设置bridge命令中的心跳事件开启这些心跳。就像我们本书以前讨论的那样，在bridge命令中括号包含的变量将传递到B Leg。这里是一个举例:

```
<action application="bridge"
data="{enable_heartbeat_events=5,nibble_rate=1,nibble_
account=0838833133}sofia/external/$1@tel.co."/>
```

可选终端接入方式

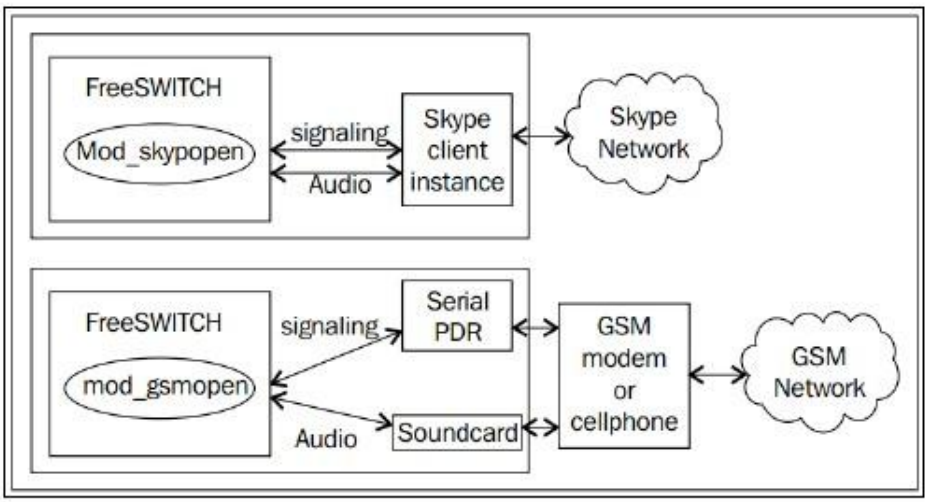
大部分FreeSWITCH 用户使用SIP (和mod_sofia)，还有其他的方法支持freeSWITCH和我们这个世界进行通信。这里，我们简要介绍三种方法，用户可能希望对这三种方法做进一步的研究。

Skype和GSM终端接入

不是每个人都有预算买硬件或服务连接Skype和GSM网络，特别是第一次听说这个新通信手段的用户。FreeSWITCH对那些用户提供了一个可选的方法，愿意稍微投入一点: 通过最经济的方法，只要一对终端(是通道驱动) 支持语音呼入和呼出呼叫和文本信息(聊天和短信)。

mod_skypopen 和mod_gsmopen 都完全支持FreeSWITCH功能，具有CLI命令来进行排查和控制，完整的事件互动，并且可以按照Sofia SIP 终端模块一样工作。

mod_skypopen和mod_gsmopen 模块具有同样的基本架构:
它们可以通过自有信令协议控制外部的逻辑实体（接口），并且可以通过这个接口经过FreeSWITCH转发媒体流到目的地的网络，也可以通过目的地网络，从FreeSWITCH转发媒体流到模块接口，实现双向通信。



对于mod_skypopen，这个“接口”是一个标准Skype客户端软件实例，它可以默认支持和Skype网络的交互，重定向媒体流到FreeSWITCH服务器。通过Skype API接口（信令）命令，mod_skypopen模块控制Skype客户实例。

对于mod_gsmopen，“接口”是一个GSM modem (或二手的手机) 可以实现和GSM网络的交换。当语音流通过声卡的时候，mod_gsmopen 通过串口来控制GSM接口，通常是标准的AT命令集(信令)。

mod_skypopen 和mod_gsmopen 都可以支持通过标准的FreeSWITCH API接口，拨号规则和事件完全支持语音呼叫和聊天功能。因此，如果用户的应用程序可以支持SIP或Jabber来传输语音和外部消息，那么可以无需修改，直接配合GSM 和Skype网络运行，当然也包括聊天和短信的双向通信。

通过mod_skypopen模块使用Skype

mod_skypopen 模块通过公共Skype API接口连接一个运行在FreeSWITCH的Skype 实体。Skype 不需要外部硬件来支持这个功能。mod_skypopen 这是简单使用一个合法的Skype API 接口。mod_skypopen 则根据Skype许可证规定完全使用了Skype API接口，它不是一个被Skype支持和经过认证的方式。

因为这些属性，mod_skypopen 要求运行一个Skype的拷贝实例，这个拷贝可以通过对Skype APIs的访问。运行Skype通常要求 X Windows，X Windows会消耗很多系统资源。这个过程相对简单，为了让mod_skypopen工作，需要创建一个虚拟的语音驱动程序，Skype 可以对其发送或接收语音流。这个虚拟的语音驱动程序实际上帮助我们仅实现对FreeSWITCH 进行双向数据传输，而不是使用真正的物理声卡的扬声器和麦克风来实现语音传输。

为了使用mod_skypopen，用户需要编译和加载mod_skypopen模块，并且至少在同一台FreeSWITCH服务器启动一个标准的Skype客户端的实例。

用户可以在Linux和Windows平台运行多个Skypopen呼叫，只要这台机器有足够的内存和强大的CPU资源支持这些呼叫。但是，到这本书写作时，Mac 环境下，仅支持一个Skype 实例。

在Linux和Windows平台，用户可以创建多个Skype实例使用同样Skype用户名称支持接收Skype的呼叫和聊天功能（可以对"mycompany_tech_support"支持多个并发呼入)。

可以在linux环境下，通过同样的Skype用户名称发起多个外呼和聊天(所有并发呼叫通过"mycompany_sales" 一个Skype用户名称呼出)，而在Windows环境下，每个呼出使用不同的不同的Skype 用户名称("mycompany_sales01", "mycompany_sales02", 和等等)。

对于Linux和Windows Skypopen，Skype客户端实例可以不需要桌面安装。

Skypopen支持Skype对Skype之间的呼叫接收和发送(通常说的"Skype-对-Skype" 呼叫)，也可以支持从Skype用户帐号呼出到PSTN和手机终端(用户必须从Skype充值实现通话) Skype 终端实例使用这个帐号呼出，同样Skype 用户也可以接收从PSTN的呼叫（用户必须从Skype购买服务），当然也支持Skype用户之间聊天。此书完成时，Skype还不支持短信服务。



详细的使用设置文档可以从以下链接获得：
http://wiki.freeswitch.org/wiki/Mod_skypopen.

通过mod_gsmopen模块使用GSM

用户需要编译和加载mod_gsmopen 模块，和满足以下要求:

- 几个GSM modems 或二手的手机("接口")
- 几个串口端口 (USB端口)
- 多个声卡(通常是便宜的USB)
- 串口和语音的连接线，用来连接"接口" 和串口端口或声卡

每个"接口"都是安装以上的列表配置。

大部分的手机终端都可以通过自带的数据线连接到串口，为了支持语音，用户需要一根语音线连接免提接口插座和声卡接口插座。

市场上有很多便宜的嵌入式设备支持了GSM modem，声卡和一个内部的USB接口。通过这些设备，用户仅需要一个标准的USB集线器支持FreeSWITCH到这个"黑盒子"的完整接口。

一个GSM modem 支持一个并发呼叫。如果添加更多的线路(号码)，仅需要了解添加一个设备连接到USB 集线器。参考Wiki链接可以获得更多信息，包括支持的手机和设备。如果使用这个功能仅作为SMS网关(无语音需求) 用户仅需要"接口" (无需声卡和语音线)。用户可以连接多台二手的手机到这个USB集线器（Linux支持多串口功能非常好）SMS的流量对CPU负载来说可以完全忽略。

每个接口都有自己的SIM卡，带有自己的移动号码。 理论上说，用户可以从运营商获得一个比较好的话费套餐。注意，大部分运营商提供运营商之间免费的呼叫，"SME 套餐"，"家庭套餐"，免费几分钟，或通过特别时间段的特别套餐服务。

用户可以通过不同运营商不同套餐服务混合匹配多个接口支持呼出和短信服务，类似的功能可以通过拨号规则的最少资费路由来实现。

在此书写作时，GSMOpen仅支持Linux平台，但是Windows平台的支持不久可以实现。



更多关于mod_gsmopen和支持的GSM dongles设备可以通过在线文档查阅
http://wiki.freeswitch.org/wiki/GSMopen.

通过FreeTDM模块实现TDM呼叫

FreeSWITCH完全兼容支持各种语音卡。Sangoma, Digium, OpenVox, Rhino Technologies，RedFone和其他厂家生产各种类型的语音接口卡。FreeSWITCH 开发人员起初发布了基于BSD-licensed 抽象层的库，称之为OpenZAP。这个抽象层支持FreeSWITCH到Sangoma和Digium-based卡之间通信，要求安装厂家的板卡驱动。FreeTDM 由Sangoma Technologies Corporation赞助开发维护，是一个新的抽象层，已经完全替代了OpenZAP。



更多关于模拟板卡和E1数字板卡设置文档(T1/E1 and PRI)可以访问
http://wiki.freeswitch.org/wiki/FreeTDM.

配置工具和相关项目

FreeSWITCH 社区在过去几年成长非常迅速，不同用户开发了很多不同的软件来满足不同的需求。这些需求从小型家庭PBX到大型电话系统公司。沿着这条路，很多用户发布了不同的软件帮助用户节省时间或支持FreeSWITCH以独到的方式来工作。贡献者代码从完整的开源GUI界面和架构到小的单一目的库。我们简要讨论一些项目。

界面管理

在今天的FreeSWITCH社区，有多种web 界面管理软件。一些是图形工具来生成XML
配置文件，另外一些采用完整的抽象层的做法提供简化的但是灵活的用户界面。
我们将讨论几个比较流行的管理界面。

注意，选择正确的用户界面确实是用户个人的决定或喜好。一些用户喜欢使用其他的语言。
一些用户则非常关心功能。一些用户可能更多感谢可拓展性。大部分的发布者使用了
容易安装的ISO发布，在做出决定之前，用户应该测试所有的界面系统，选择自己喜欢的
系统。

FusionPBX

FusionPBX 是PHP开发的开源图形界面。这个项目启动时间正好是FreeSWITCH包
支持pfSense 防火墙的时候。后来，这个项目重新命名为FusionPBX，支持多种平台
支持，包括Linux，BSD，Windows，Mac OS X，和其他的平台。它支持的数据库包括
PostgreSQL, MySQL,和SQLite。

功能包括无限制支持, IVR菜单, voicemail-to-e-mail, 振铃组，传真服务器，
可交互会议控制，查看进行的呼叫和extensions，队列，呼叫前转，页面点击呼叫
, DISA, 话机自动分配，多用户租用和更多。

FusionPBX 支持可定制开发配置。例如，一个用户被设定到"superadmin"
组，这个组可登录到页面执行许多管理员工作包括自定义菜单，内容和
主题，用户组，多用户租用配置和功能权限。

其他信息，文档和页面截图可以通过官方网站获得(http://www.fusionpbx.com)。
可以也可以使用irc.freenode.net的IRC channel #fusionpbx on 获得支持。

FreePyBX

FreePyBX 是基于Python的FreeSWITCH GUI ，由Noel Morgan开发。它支持
配置和基于界面的FreeSWITCH管理员。它支持丰富的功能包括多用户租用，呼叫
中心，内置的一个缺陷跟踪管理系统，更多信息访问 http://www.freepybx.org。

blue.box

blue.box 是一个开源的基于PHP/MySQL的FreeSWITCH图形管理界面，由2600hz 团队发布，支持通过界面来配置XML文件。 它的目的是通过使用仅仅使用内置的FreeSWITCH功能和XML配置提供对中小型的FreeSWITCH安装。它支持高度可定制化和模块化的功能，同时支持根据需要安装模块。它也支持多用户托管。

更多信息，访问http://www.2600hz.org/，也可以通过在irc.freenode.net的#2600hz 的IRC获得支持。

Kazoo

Kazoo 是一个开源的平台，由2600hz 团队开发，这个平台的目的是支持分布式的云通信服务。这个项目正在开发阶段，支持开源通信平台。它的设计目的是帮助运营商拓展到中型和大型部署，通过网络或者大型的私有WAN网支持网络网站冗余。整个架构的目的是以API的形式开放所有的功能，因此所有的功能都可以通过外部的软件来实现。界面，呼叫处理API，数据库存储引擎和短信引擎都被经过了抽象化处理，成为独立的模块，用户可以选择不同的模块来满足开发的需求。

如果用户对PBX托管和分布式通信有兴趣，可以访问Kazoo。用户可以获得更多的资料从这个链接http://2600hz.com/platform.html.

支持库

除了FreeSWITCH提供的Event Socket Library (ESL) 库，还有很多其他的第三方库，支持库是基于ESL拓展开发，并且对某些语言添加了特别的功能。

Liverpie (Ruby)

Liverpie (独立的IVR代理) 是一个免费的软件，使用Ruby开发，它可以在一端和FreeSWITCH通信，另外一端配合使用界面应用，无论什么语言，平台等等。它可以转换FreeSWITCH的mod_event_socket 对话成为HTTP 标记(在HTTP头中嵌入各种参数)，所以用户可以开发自己的HTTP-对话限定状态机，通过Livepie连接Livepie到FreeSWITCH端。注意，如果用户觉得使用Livepie转换非常满意的话，Liverpie希望获得在YAML中获得一个相应，这样用户可以减少痛苦，因为用户需要提供XML文件。

用户可以通过http://www.liverpie.com 获得更多信息。

FreeSWITCHeR (Ruby)

FreeSWITCHeR是一个基于EventMachine的Ruby库，支持和FreeSWITCH交互。通过mod_event_socket 实现FreeSWITCHeR 库的交互。它可以创建呼入呼出事件监听器，可以使得整个从Ruby库来的呼叫更加强大。大量的文档和实例代码可以帮助用户启动开发。

用户可以从https://github.com/bougyman/freeswitcher 获得更多了解。

Librevox (Ruby)

在重写FreeSWITCHeR的主要版本时，Librevox发布。Harry Vangberg 参与了原来FreeSWITCHeR 代码的开发，他决定重写FreeSWITCHeR和这个库。从网站上得知: "Librevox and Freeswitcher look much alike on the outside, but Librevox tries to take a simpler approach on the inside." 就像FreeSWITCHeR应用，大量的文档和举例代码可以帮助用户启动开发。

用户可以从这个链接获得更多介绍 https://github.com/vangberg/librevox.

EventSocket (Python/Twisted)

EventSocket 是一个Twisted protocol支持FreeSWITCH 事件套接字。它提供支持在事件字套接字方式的呼入和呼出，这个方式是在一个单文件类中。它可以广泛使用在不同的环境。它支持简单和可拓展方式，导出所有FreeSWITCH功能，导入到基于Twisted的应用程序中。更多的是，它完全是基于事件驱动，支持通过事件套接字，使得复杂应用的简化部署来控制FreeSWITCH。

代码是核心Nuswit Telephony API的一个部分(http://nuswit.com)，完整功能的基于页面的呼叫器正工作在巴西和哥伦比亚。

源代码和实例文档可以通过这个网站获得：http://github.com/fiorix/eventsocket。

FSSocket (Perl)

FSSocket Perl 库是基于Perl Object Environment (POE) 架构，支持从perl来实现FreeSWITCH 事件套接字的方便集成。它可以解析FreeSWITCH 事件变为哈希值。用户可以要求多个事件类型或所有事件类型。

源代码和文档可以从这里获得: http://search.cpan.org/~ptinsley/POE-Filter-FSSocket-0.07/.

Vestec Automatic Speech Recognition

FreeSWITCH可以支持PocketSphinx 项目，一些专业级的语音识别可以考虑Vestec。Vestec支持一个ASR平台，这个平台可以配合FreeSWITCH工作，适合真正的应用程序集成。Vestec是一个商业软件，但是这个公司提供免费无限制全功能的开发者许可证支持和FreeSWITCH集成。通过邮件联系Vestec, 联系方式是info@vestec.com 。支持的语言列表在：http://www.vestec.com/acoustic_models。

总结

FreeSWITCH提供了强大的工具集帮助用户开发功能丰富的应用程序。这些和FreeSWITCH绑定的模块是用户早期开发的实例，并且支持了强大的内部API接口。另外，活跃的FreeSWITCH爱好者们正在让软件开发提升到更高的水平。我们希望在未来几年，基于FreeSWITCH平台的托管型，基于云通信的和内建式的解决方案可以蓬勃发展，并且可以满足VOIP和通信领域不同需求。

A FreeSWITCH在线社区

对开源软件项目来说，一件比较好的事情是很多用户来自于世界各地，组成一个社区，很多热情的用户在贡献自己的时间。FreeSWITCH 就是其中之一。

在下面的附录中，我们将介绍在线社区的几种支持方式。他们是:

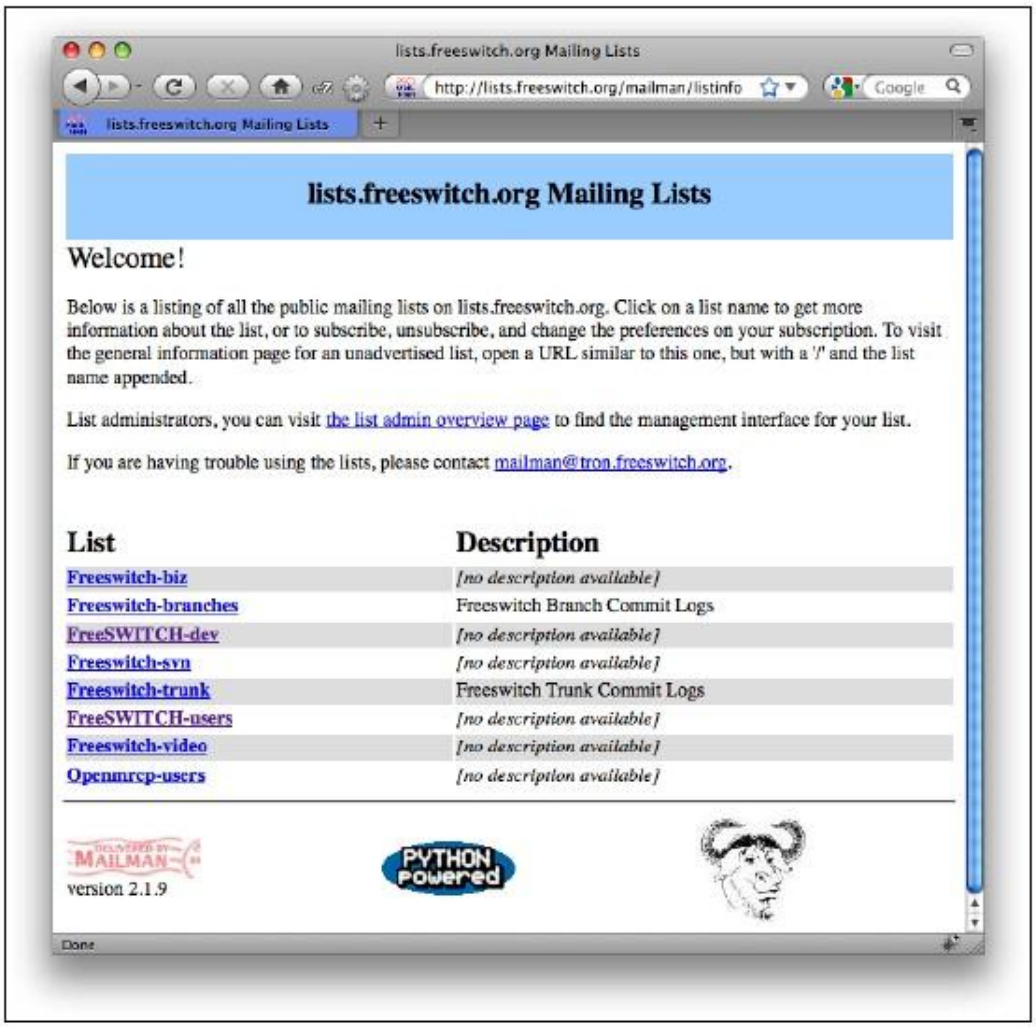
- FreeSWITCH 邮件列表
- 通过IRC进行实时交互
- 主要的FreeSWITCH 官方网站和wiki
- 每年一度在芝加哥举行的ClueCon通信会议

这个附录将帮助用户成为活跃的社区成员。

FreeSWITCH邮件列表

FreeSWITCH项目维护几个主要的邮件列表，通过这个网站访问：
http://lists.freeswitch.org。对于一般用户来说，主要使用的是freeswitch-users。
像其他的项目一样，这个列表通过GNU 邮件列表管理员，MailMan安装支持。

访问这个邮件列表，仅通过浏览器访问lists.freeswitch.org 点击列表名称，如下截图:



新用户应该加入到FreeSWITCH-users直到他们慢慢熟悉了这个项目。其他的列表讨论了很多关于相对高级的技术，除了Freeswitch-biz 这个列表，这是专门为用户讨论FreeSWITCH相关的商业方面的列表。

用户需要输入用户名和密码订阅这个列表。在手上保持这些信息以便未来相关需要。一个重要的功能是用户可以考虑是否接受"digest"形式的邮件。一个摘要形式的邮件是一个对多个邮件绑定以后以单个邮件发送到形式，一次发送过去的多份邮件而不会每天收到邮件。digest 方式发送对一些用户方便一点，因为同时获得很多邮件这种方式可能引起邮件内容被忽略。但是，如果用户需要和其他的用户互动，最好不使用Digest的方式因为这样的方式，用户感觉非常困难及时参与对某一个话题的讨论。

使用邮件列表时，注意以下几点:

- 使用邮件客户端来处理一些相关话题的讨论。
- 不要对一个主题不停发送一样的内容或刷屏！这样的情况发生经常，当有人回复当前存在的一个主题时，用户已经修改了邮件的主题，出现沟通不畅的问题。如果用户需要讨论另外一个新的话题时，牢记！重新开一个主题。
- 第一次加入时，不要接收太多的信息，这样自己难以承受。用户仅可以检查一定时间范围内的信息，留一点时间来检查这些信息。
- 使用网站归类邮件历史记录中的讨论查找自己需要的话题。例如，如果使用Google，从Google 查找site:lists.freeswitch.org 和 "early media" 关键词将可以看到所有列表中相关 "early media"的讨论.

邮件列表是一个非常好的资源和世界各地的其他用户进行互动。但是，有时用户需要一个实时的对话聊天来查询问题。也许，用户希望使用聊天工具和其他用户进行实时互动。

通过IRC进行实时交流

IRC或Internet Relay Chat 是一款非常优秀的工具支持用户之间的聊天互动。

在irc.freenode.net，FreeSWITCH有几个不同的聊天室，他们是:

- #freeswitch
- #freeswitch-dev
- #freeswitch-social
- #freetdm

还有来自世界不同地方，使用各种语言的的聊天室。其中的一些是:

- #freeswitch-de
- #freeswitch-es
- #freeswitch-fr

一旦用户知道了如何使用IRC，IRC相当易用。用户需要在电脑上安装一个IRC客户端。有很多客户端可以使用。以下是几个经常使用的:

- Chatzilla: A Firefox add-on
- IRSSI: A text-based IRC client
- Colloquy: An IRC client for Mac OS X
- mIRC: An IRC client for Windows

用户也可以使用FreeSWITCH官方网站的Java applet 加入到#freeswitch 。

使用IRC时，用户需要给自己取一个名字，"nick" 是一个临时性的名称。选择一个特别的名称比较容易记忆。访问 http://freenode.net/faq.shtml#userregistration 了解更多关于设置名称和注册的信息。

用户可能经常在IRC看到以下几个人:

- anthm: Anthony Minessale
- bkw_: Brian K West
- MikeJ: Michael Jerris
- mercutioviz: Michael S Collins
- pyite: Darren Schreiber
- intralanman: Raymond Chandler
- SwK: Ken Rice

他们是FreeSWITCH 社区的活跃用户。他们其中的有一些人会经常在线 (晚上也可能在线，取决于用户的时差)。

使用IRC时，请记住以下几点:

- 这是一个公共场合，用户来自于不同的背景和需要用户得体。
- 尽量礼貌，即使他人的出言不逊。
- 不要在聊天室不同刷屏或贴很多日志信息。如果用户有两行以上分享的信息，可以上传到：http://pastebin.freeswitch.org.
- 加入论坛室时，无需询问是否可以发问题，直接提出问题。例如，"I'm a new user tryingto set up a gateway. Why does FreeSWITCH say that username and password are REQUIRED parameters when my provider uses IP authentication?".
- 耐心! 通常几分钟内可能有人回答你的问题，但是记住，通常这些会议室的成员都在北美的时区的工作时间在线。
- 欢迎来自不同背景的用户。主 #freeswitch 是以英文交流，但是使用其他语言的用户也可能加入到了这里，包括西班牙，法国，德国，葡萄牙和中国的用户。(也可以参考我们刚才提到的IRC，支持不同的语言。
- 始终尊重用户c888!

免费加入FreeSWITCH IRC聊天室，看看有什么正在讨论的话题。

FreeSWITCH主要网站和技术wiki

FreeSWITCH 项目目前有两个主要的网站:

- www.freeswitch.org: 项目官方网站
- wiki.freeswitch.org: 公开的技术wiki页面

FreeSWITCH主页–www.freeswitch.org

FreeSWITCH主页是一个所有关于FreeSWITCH的起始页面。从这个页面用户可以获得以下内容:

- 阅读FreeSWITCH和VoIP新闻
- 下载和查阅源代码
- 提交bug和功能请求
- 阅读文档
- 通过Freenode Java applet 加入到#freeswitch IRC

每个星期会有新的信息被添加上去，经常登录网站检查。

FreeSWITCH技术wiki页面–wiki.freeswitch.org

FreeSWITCH wiki是主要的FreeSWITCH 文档源。Wiki是一个网页，支持用户添加，编辑和删除链接和其他内容。典型的例子就是Wikipedia 维基百科。FreeSWITCH wiki页面使用了MediaWiki (http://www.mediawiki.org)，和Wikipedia使用的一样的引擎。

FreeSWITCH wiki是一个社区学习资源。Michael S. Collins 是主要的wiki管理员，欢迎所有FreeSWITCH 用户提交或更新wiki的内容。像其他的wiki一样，wiki包含了很多的内容。有时候查找出自己需要的内容是非常困难的一件事。如果用户非常困难查找一个特别的话题。我们建议用户使用Google搜索引擎来搜索中心内容，格式为(site:wiki.freeswitch.org <查找的话题>)。 如果使用wiki一段时间以后，就可能发现比较容易找到一些信息的地址来源。

将来的wiki贡献者应该记得这些:

- 先搜索然后再添加内容—这些内容可能已经被添加到了wiki，可能仅仅限于简单更新。

- 确认添加到内容链接正确。
- 确认添加到内容在正确的内容类别下
- 不用担心错误！其他人愿意纠正用户的错误

开源软件的文档是一个非常大的挑战，如果用户觉得有能力协助写一些相关的文档，请联系Michael，邮箱是msc@freeswitch.org。社区一直需要有这样技巧的开源用户来检查资料的准确性，更新一些实例，测试配置和举例，翻译成其他的语言。

每年一次的ClueCon开源开发者大会

每年在芝加哥，我们会举办一个为期三天的会议，在这个会议中，一些开源通信的专家和爱好者汇集在这里讨论许多话题。会议举行的日期一般在八月份的第一个星期，这是一个全世界开源通信爱好者非常良好的互动机会。访问网站http://www.cluecon.com 获得更多详情和往届会议的演讲内容和视频记录。

尽管ClueCon 会议是"由开发者发起，面向开发者"，参加的人数也逐渐增多，一些非开发人员也加入了这个会议。大部分的演讲仍然是相关的技术讨论；但是，会议上还有一些非技术的通信产品介绍，例如演示新的产品。会议的目的是为了让用户，开发人员和厂商彼此联系。用户也非常感谢可以见到开发人员和厂家，厂家也非常乐意可以非常有针对性地和开发人员互动而不是仅仅呆在他们的展位。

ClueCon 邀请所有其他的开源项目来做专门的演讲。过去几年中，演讲包括了Asterisk, FreeSWITCH, Kamailio, 和OpenSIPS, 还有厂商例如Sangoma和Vestec，这两家公司对开源通信项目给予很多的支持。

我们鼓励所有的FreeSWITCH用户通过这些资源可以相互了解，彼此帮助。

B

从Asterisk切换到FreeSWITCH

特别感谢Stefan Wintermeyer 贡献这个附录。

这个附录是针对已经熟悉了Asterisk的管理员和开发人员，但是希望学习FreeSWITCH，了解如何切换目前的系统到FreeSWITCH。因为这本书是技术FreeSWITCH，不是技术Asterisk，我们不会介绍更多的asterisk实例。假设用户已经对Asterisk 分机，优先级和应用程序有所了解。

附录中的信息不是非常详细的参考资料。它是帮助用户在切换过程中了解基本的设置，这些设置是在FreeSWITCH平台如何实现的。讨论的话题包括:

- 停止和关闭Asterisk和FreeSWITCH
- 设置debug 排查级别
- 在每个软件中查阅基本的配置文件路径
- 在Asterisk和FreeSWITCH设置两个分机
- 针对每个软件设置简单的语音邮箱配置

用户可以找到其他的信息来充分利用目前的Asterisk系统，用户可以获得更多这方面的知识，咨询"Rosetta Stone" ，访问wiki链接:
http://wiki.freeswitch.org/wiki/Rosetta_stone.



切换到FreeSWITCH，比仅仅可以获得更加好的软件平台，同时也比较好的管理了拨号规则和配置。我们了解一些修改的功能，这些功能也得到了提升，无需优化整个系统。一个新的转换是一个机会，可以从一个新的起点开始。

开始准备切换

为了本书的写作，我们在Debian linu运行一个Asterisk和FreeSWITCH，IP地址是10.0.0.10.我可以运行同时运行两个服务，可以非常方便配置举例手动关闭asterisk或启动FreeSWITCH。我们都使用最新版本的软件，软件版本对了解两个软件系统的架构没有什么影响。SIP 电话使用IP地址10.0.0.20和10.0.0.21。查看第四章，SIP和用户目录来配置不同的SIP话机。



用户可以同时在同一台机器运行Asterisk和FreeSWITCH。但是，用户不能同时使用他们，除非使用不同的IP地址和端口。例如在 /etc/asterisk/sip.conf 修改SIP端口。只要两个服务器不同时使用端口5060，用户就可以同时运行Asterisk和FreeSWITCH。

启动和关闭Asterisk或FreeSWITCH

在默认安装环境下，asterisk可以通过命令asterisk启动。它将在后台启动Asterisk，进行一个后台的流程。关闭Asterisk，用户可以使用命令行 命令asterisk -r 进入到CLI后台，输入core stop now。一个典型的会话是这样的:

```
debian*CLI> core stop now
debian*CLI>
Disconnected from Asterisk server
```

```
Executing last minute cleanups
Asterisk cleanly ending (0).
root@debian:#
```

在典型的FreeSWITCH 安装环境，用户可以通过以下命令实现后台启动:

```
/usr/local/freeswitch/bin/freeswitch –nc
```

关闭FreeSWITCH 执行同样的命令，加一个参数–stop:

```
/usr/local/freeswitch/bin/freeswitch –stop
```

基本的排查命令

用户可能希望看到系统正在进行的流程。用户希望看到更多的debug信息。
我们建议用户尝试不同的日志级别，发现适合于自己的。

Asterisk

如果用户已经有一个正在运行的Asterisk，可以使用asterisk -r 打开一个命令行。
使用这个命令core set verbose 3 ，用户可以设置日志级别到3。不要使用
core set debug 3 因为可能完全让用户感觉困惑，这些日志信息可能对开发人员有帮助。
输入core set debug 0 重新设置日志级别为0。 一个典型的会话可能类似于这样:

```
debian*CLI> core set verbose 3
Set remote console verbosity to 3
debian*CLI>
```

FreeSWITCH

如果用户有一个已经运行的FreeSWITCH，用户可以使用fs_cli 命令
打开一个命令行环境，执行命令 /log info 用户可以设置日志级别为6。用户
可以使用这个命令实现同样的结果:

```
freeswitch@internal> /log info
+OK log level info [6]
```

现在我们设置了debug的变量信息，让我们简单介绍一下Asterisk和FreeSWITCH的
配置举例。

配置文件

Asterisk和FreeSWITCH包含了许多目录结构对应一些实例配置设置。查看Asterisk配置文件书，可以执行以下命令:

```
tree /etc/asterisk
```

在FreeSWITCH环境，执行:

```
tree /usr/local/freeswitch/conf
```

就像用户看到的一样，这两个项目都包含了大量的文件和目录结构支持相应的配置。和其他的复杂软件一样，非常重要，我们必须明确哪些文件是我们必须配置的。

最大不同是asterisk中的每一个文件都有特别的含义。例如，Asterisk将在extension.conf文件中查找拨号规则。FreeSWITCH则不是这样工作的。它会读取一个大的XML文件，这个文件被分割成一些更小的文件。如何命名这些文件完全在于用户自己。默认配置文件仅是一个例子。用户可以任意命名和管理配置文件，只要用户觉得设置是合理的。事实上，只有一个配置文件是绝对需要的，就是freeswitch.xml.

下面的部分帮助我们更加详细了解这些配置。



无论什么时候我们使用这个词 "替换" 我们确认意思是替换。不要尝试打开存在的文件添加这些例子。删除这个文件，创建一个新的文件。一旦用户了解了软件的基本逻辑，用户就可以查阅这些默认的设置。它们是一些举例，这些例子是非常好的学习资源，可以简单拷贝粘贴。但是它们不是理解基本概念的好的开始。

两个SIP电话

最小的PBX举例是两个SIP电话的设置。我们创建了分机2000和另外一个分机2001。每个分机可以通过拨打对方的分机号码实现通话。

第一个SIP帐号是2000，密码是1234。第二个分机号码是2001，密码是1234。请使用这些帐号来设置用户话机。

Asterisk配置

Asterisk在文件/etc/asterisk/sip.conf 保存分机信息，请使用一个新文件替换默认的sip.conf 文件，这个新文件包含以下帐号信息:

```
[general]
port=5060
bindaddr=0.0.0.0

[2000]
type=friend
secret=1234
context=default
host=dynamic

[2001]
type=friend
secret=1234
context=default
host=dynamic
```

拨号规则保存在/etc/asterisk/extensions.conf 文件。使用以下代码替换:

```
[default]
exten => _200[1-2],1,Dial(SIP/${EXTEN})
```

进入到Asterisk 后台(asterisk -c) 设置debug 级别3 (core set verbose 3) ，我们会看到电话机是如何注册的:

```
*CLI>          -- Registered SIP '2000' at 10.0.0.21:2048
    -- Registered SIP '2001' at 10.0.0.20:3072
    -- Unregistered SIP '2001'
    -- Registered SIP '2001' at 10.0.0.20:3072
```

现在，我们可以呼叫对方。在呼叫的过程中，命令 sip showchannels 执行以后会看到以下信息:

```
*CLI> sip showchannels
Peer              User/ANR         Call ID          Format           Hold
Last Message      Expiry     Peer
10.0.0.21         2000             150b1e3879a2bff  (ulaw)           No
Tx: ACK                      2000
10.0.0.20         2001             ea88263cebdd-la  (ulaw)           No
Tx: ACK                      2001
2 active SIP dialogs
```

现在，我们创建了Asterisk环境下的分机呼叫。让我们做同样的事情，继续在FreeSWITCH环境下进行测试。如果必要的话，先通过命令core stop now 关闭Asterisk。

FreeSWITCH配置

FreeSWITCH 没有一个固定的文件和目录结构。用户在/usr/local/freeswitch/conf 看到的是一个举例。用户可以保存所有的配置在一个XML文件，如果用户喜欢，也可以分割成独立的XML文件。
配置文件已经包含了我们需要测试的SIP帐号和extensions。我们重新创建一个Asterisk的例子。

创建文件/usr/local/freeswitch/conf/directory/default/2000.xml，包含以下SIP 帐号信息:

```
<include>
  <user id="2000">
    <params>
      <param name="password" value="1234"/>
    </params>
    <variables>
      <variable name="user_context" value="default"/>
    </variables>
  </user>
</include>
```

创建第二个文件/usr/local/freeswitch/conf/directory/default/2001.xml，包含第二个SIP帐号信息:

```
<include>
  <user id="2001">
    <params>
      <param name="password" value="1234"/>
    </params>
    <variables>
      <variable name="user_context" value="default"/>
    </variables>
  </user>
</include>
```

最后，我们创建一个拨号规则实现分机双方的互通。创建文件
/usr/local/freeswitch/conf/dialplan/default/01_New.xml, 包含以下拨号规则信息:

```
<?xml version="1.0" encoding="utf-8"?>
<include>
  <context name="default">
    <extension name="Local_Extension">
      <condition field="destination_number"
        expression="^(200[1-2])$">
        <action application="export"
          data="dialed_extension=$1"/>
        <action application="bridge"
          data="user/${dialed_extension}@${domain_name}"/>
      </condition>
    </extension>
  </context>
</include>
```

保存以上三个文件后，启动FreeSWITCH:

```
/usr/local/freeswitch/bin/freeswitch –nc
```

等一会FreeSWITCH启动以后，使用命令fs_cli连接:

```
/usr/local/freeswitch/bin/fs_cli
```

确认两个分机正在注册到FreeSWITCH服务器。(如果这两个分机已经注册在了Asterisk服务器上，用户需要重新启动它们或者重新注册一次。) 观察注册状态信息，执行命令:

```
sofia status profile internal reg
```

输出结果类似于:

freeswitch@internal> sofia status profile internal reg

Registrations:

Call-ID:	a270263caa23-uocan9j61z5y
User:	2000@127.0.0.1
Contact:	"2000" <sip:2000@10.0.0.20:3072;line=0tqusdnm>
Agent:	snom821/8.4.35

Status:Registered(UDP)(unknown) EXP(2013-01-13 06:46:31) EXPSECS(3529)

Host:	debian
IP:	10.0.0.20
Port:	3072
Auth-User:	2000
Auth-Realm:	10.0.0.10
MWI-Account:	2000@127.0.0.1

Call-ID:	3c26708e4d57-yzfzr61f7x4l
User:	2001@127.0.0.1
Contact:	"2001" <sip:2001@10.0.0.21:2048;line=9r6kyu0i>
Agent:	snom360/8.4.35

Status:Registered(UDP)(unknown) EXP(2013-01-13 06:46:45) EXPSECS(3543)

Host:	debian
IP:	10.0.0.21
Port:	2048
Auth-User:	2001
Auth-Realm:	10.0.0.10
MWI-Account:	2001@127.0.0.1

Total items returned: 2

freeswitch@internal>

现在用户可以实现从分机2000到2001的呼叫。

用户可以分析这些通道，使用命令showchannels。两个分机之间发起呼叫，可以看到以下信息:

```
freeswitch@internal> show channels

uuid,direction,created,created_epoch,name,state,cid_name,cid_num,ip_
addr,dest,application,application_data,dialplan,context,read_
codec,read_rate,read_bit_rate,write_codec,write_rate,write_bit_
rate,secure,hostname,presence_id,presence_data,callstate,callee_
name,callee_num,callee_direction,call_uuid,sent_callee_name,sent_callee_
num

af6dc664-5cb3-11e2-ae64-41a8c0d6e735,inbound,2013-01-12
13:29:18,1357993758,sofia/internal/2000@10.0.0.10,CS_EXECUTE,2000
,2000,10.0.0.20,2001,bridge,user/2001@127.0.0.1,XML,default,PCMU,8
000,64000,PCMU,8000,64000,,debian,2000@10.0.0.10,,ACTIVE,Outbound
Call,2001,SEND,af6dc664-5cb3-11e2-ae64-41a8c0d6e735,Outbound Call,2001

af861bd8-5cb3-11e2-ae6d-41a8c0d6e735,outbound,2013-01-12
13:29:18,1357993758,sofia/internal/sip:2001@10.0.0.21:2048,CS_EXCHANGE_ME
DIA,2000,2000,10.0.0.20,2001,,,XML,default,PCMU,8000,64000,PCMU,8000,640
00,,debian,2001@127.0.0.1,,ACTIVE,Outbound Call,2001,SEND,af6dc664-5cb3-
11e2-ae64-41a8c0d6e735,2000,2000

2 total.

freeswitch@internal>
```

用户现在已经在Asterisk和FreeSWITCH上设置了SIP帐号。



如果用户编辑了FreeSWITCH XML 配置文件，而FreeSWITCH 正在运行，执行命令Reloadxml，或者摁F6键。

分析

FreeSWITCH使用XML，Asterisk使用传统的"ini"文件。XML一个大的优势在于可以非常容易检查出配置文件的语法错误。Asterisk在这方面好像比较弱，有时候对系统管理员不能提供充足的回复信息。有时候，我们的拨号规则看起来没有错误，工作正常，但是有一些极端的例子中，也许就不能工作。很多时候就可能是拨号规则中的语法错误，asterisk自己可以发现这些错误。因此支持一个严格的XML配置文件是相对比较好的办法让系统顺利工作，但是用户可能需要花费一定的时间来实现XML配置方法。一个好的XML编辑器可以帮助用户。



当查看或编辑FreeSWITCH配置文件时，安装一个支持语法高亮的编辑器是非常好的办法。

在两个软件中对SIP帐号的定义也是非常不同。拨号规则也是非常不同。我们通过默认的方式在两个软件中定义了SIP帐号。每个软件在配置文件中查找default context。Asterisk使用配置文件extensions.conf 。在 [default] context中查找匹配分机。正则表达式_200[1-2]匹配一个以拨号吗，启动拨号规则应用程序，通过SIP协议，对分机${EXTEN}发起呼叫, 这个变量代表用户拨打的号码。

FreeSWITCH也查找一个默认的default context(因为SIP 帐号定义在这个context中)。它会在context中查找所有的定义的extensions，直到发现一个匹配的condition 字段。conditions 可以是任何定义，例如时间或者目的地号码，这里使用目的地号码匹配正则表达式^(200[1-2])$。一个condition 本身包含包含一段代码来触发执行流程如果条件匹配为真。在我们的举例中，代码为:

```
<action application="export" data="dialed_extension=$1"/>
<action application="bridge"
   data="user/${dialed_extension}@${domain_name}"/>
```

我们可以写成这样:

```
<action application="bridge" data="user/$1@${domain_name}"/>
```

设置一个dialed_extension 变量非常方便。我们可能想起Asterisk的中的${EXTEN}。
我们也可以设置FreeSWITCH的bridge ，它类似于Asterisk中的Dial命令。
在FreeSWITCH中，我们在bridge 参数中看到了一个值@${domain_name}。这个通道变量
已经在其他地方设置。

很多信息已经在第五章，理解XML拨号规则和第八章，高级拨号规则概念做了介绍。

Voicemail

我们现在把我们的注意力转移到两个系统都支持的一个功能: voicemail。

Asterisk

Asterisk 拨号规则的Dial 提供一个电话振铃的时间。它等待一定的时间(例如，10)
支持被呼叫方接听应答，然后转入到下一个优先级。在我们的这个例子中，我们有
一个VoiceMail 应用程序。请使用现在的内容替换掉 /etc/asterisk/extensions.conf:

```
[default]
exten => _200[1-2],1,Dial(SIP/${EXTEN},10)
exten => _200[1-2],n,VoiceMail(${EXTEN},u)
```

Asterisk需要在语音邮箱中添加一些额外的配置。请使用一下内容替换
/etc/asterisk/voicemail.conf :

```
[general]
format = wav
attach = yes
[default]
2000 => 1234,Mr. X
2001 => 1234,Mr. Y
```

现在用户可以测试这个呼叫，10秒钟振铃以后，Dial停止。Asterisk增加了一个优先级
，对${EXTEN}启动了VoiceMail，呼叫方开始留言。

FreeSWITCH

FreeSWITCH中的语音邮箱配置上完全不同的。请使用以下内容替换文件 / usr/local/freeswitch/conf/dialplan/default/01_New.xml :

```
<?xml version="1.0" encoding="utf-8"?>
<include>
  <context name="default">
    <extension name="Local_Extension">
      <condition field="destination_number"
       expression="^(200[1-2])$">
        <action application="export"
            data="dialed_extension=$1"/>
        <action application="set" data="call_timeout=10"/>
        <action application="set"
            data="hangup_after_bridge=true"/>
        <action application="set"
            data="continue_on_fail=true"/>
        <action application="bridge"
            data="user/${dialed_extension}@${domain_name}"/>
        <action application="answer"/>
        <action application="sleep" data="1000"/>
        <action application="bridge"
data="loopback/app=voicemail:default ${domain_name} ${dialed_
extension}"/>
      </condition>
    </extension>
  </context>
</include>
```

FreeSWITCH 拨号规则比较复杂，但是可以对语音邮箱实现更多的控制。使用call_timeout=10 ，用户可以设置最大桥接时间。hangup_after_bridge=true 通知FreeSWITCH桥接执行以后FreeSWITCH挂机。continue_on_fail=true 处理当被呼叫方参与忙音状态的场景。桥接以后，我们可以支持一个answer，表示FreeSWITCH自己应答率此通话。一秒钟的sleep以后，桥接替换到loopback 的目的地，这个目的地支持voicemail 应用。用户也可以使用voicemail 应用无需 loopback 通道。但是使用指定的语法，我们可以转接呼叫方到语音邮箱。

访问语音邮箱

我们现在已经设置了语音邮箱，保存了语音留言。让我们讨论一下如何访问这些留言。
在每个环境中，确认呼叫以后有用户留言，可以通过一定的指令访问这些留言。

我们创建拨号规则，支持用户拨打4000访问留言。

Asterisk

请使用以下内容替换/etc/asterisk/extensions.conf :

```
[default]
exten => _200[1-2],1,Dial(SIP/${EXTEN},10)
exten => _200[1-2],n,VoiceMail(${EXTEN},u)
exten => 4000,1,VoiceMailMain(${CALLERID(num)})
```

Asterisk的VoiceMailMain 提供了一个网关接口给呼叫方的个人邮箱。
${CALLERID(num)} 返回呼叫方号码。不要和${EXTEN}混淆，这个号码是
用户拨打的号码，在这个例子中就是4000。

FreeSWITCH

首先，我们需要设置一个密码来访问语音邮箱，例如简单的密码1234。

打开文件 /usr/local/freeswitch/conf/directory/default/2000.xml 找到以下一行:

```
<param name="password" value="1234"/>
```

添加新的一行:

```
<param name="password" value="1234"/>
```

保存文件。在文件/usr/local/freeswitch/conf/directory/default/2000.xml 重复这个设置。

到这里为止，用户可以拨打4000 来获取语音邮箱的留言，因为FeeSWITCH 拨号规则
已经定义了一个extension 4000来支持语音邮箱获取。用户同样可以拨打*98 访问语音邮箱。

注意，系统要求用户输入用户的"ID number" (2000或2001) 和密码。
一些用户希望系统假设已获知用户已经拨打了登录邮箱的号码。这个功能可以相当简单实现。打开文件 /usr/local/freeswitch/conf/dialplan/default.xml 找到这个extension:

```
<extension name="vmain">
  <condition field="destination_number" expression="^vma
  in$|^4000$|^\*98$">
    <action application="answer"/>
    <action application="sleep" data="1000"/>
    <action application="voicemail" data="check default ${domain_
    name}"/>
  </condition>
</extension>
```

注意voicemail 应用。我们现在修改一下语音邮箱参数，假设呼叫方的ID number是用户需要访问的号码。修改参数如下:

```
data="check default ${domain_name} ${caller_id_number}"
```

这里我们添加了${caller_id_number} 这样的参数。告诉语音邮箱应用程序我们假设caller ID number (2000或2001) 是呼叫方需要访问的语音邮箱号码。

保存文件，执行命令reloadxml 或从fs_cli 摁F6键。拨打4000，系统会要求输入密码。



FreeSWITCH 全局变量设置在/usr/local/freeswitch/conf/autoload_configs/voicemail.conf.xml.

总结

从Asterisk切换到FreeSWITCH可能是一个非常让人担心的任务。但是它可以实现。想一下我们第一次学习Asterisk，对比我们现在已经了解到，其实都是可以完成的事情。看起来花费了时间，但是用户学习了很多知识。学习FreeSWITCH同样需要相同的时间来学习。幸运的是，用户可以利用以前学习的Asterisk知识来帮助你快速了解FreeSWITCH。

C

FreeSWITCH历史

为了正确解释FreeSWITCH的起源，我们需要回到我们设想开发这个项目的时间。VoIP革命真正开始形成是在世纪转折时期，发明了Asterisk PBX和OpenH323。这两个先驱软件支持许多开发人员可以访问开源的VOIP资源，无需承担任何商业解决方案所需要的费用。这样出现了许多新的基于这些项目的发明创造，快速传播，目前大家看到的证据是很多真正的VoIP通信应用确实已经存在。

我第一次涉足这个行业是在2002年，我们公司那时做外包的技术支持，我们需要管理我们的电话，发送呼叫到一些外部的地址。我们那时正在使用商业的解决方案但是商业解决方案部署是根据每个座席来收费的。作为一个网站托管平台的架构师，我已经在开源的应用项目中例如Apache和MySQL做了很多工作，所以我决定对当时存在的一些开源通信应用程序做一些研究。这样就进入了Asterisk。

当我第一次下载了Asterisk时，我非常惊讶。我有一些模拟语音卡，在我家的Asterisk平台上安装了这些模拟语音卡， 使用带拨号音的电话插入到我的Linux PC上的语音板卡。相当不可思议。不久，我自己开始浸淫到代码中了，试图发现它们是如何工作的。我尽量快的通过动态加载模块来拓展这个软件，当然动态加载的方式也和Apache类似。我开始研究更深层次的代码，并且开发出来几个测试模块。比以前要好很多。我不仅仅可以使用PC和电话通话，我也可以拨打号码让自己的代码开始工作了。

我测试了几个自己的想法，让自己真正明白其中的逻辑。嘿！我确实喜欢Perl，这个电话的东西同样也确实很酷。如果我把两者组合起来是什么样的？在我查找了嵌入到C语言中的Perl程序，我获知我已经有一个app_perl.so，当呼叫被路由到我自己的模块时，这个可加载的模块支持用户执行Perl代码。它不是非常完美，我开始了解嵌入perl到多线程的程序中，但是至少这是一个不错的想法，几天以后完成了这些修修补补。

随着时间的流逝，我逐渐加入到了Asterisk在线社区。经过几个星期的代码测试，我开始使用Asterisk作为一个通信引擎开发一些呼叫中心，以一些页面作为前端管理。沿着这条路继续的同时，我发现了一些Asterisk bugs，我向社区提交了这些bugs。回来，这样的过程出现的更多，我越来越深地参与了到了这个项目，我开始创建一个新的经过优化的软件，零星修补一些bugs。到2004年时，我已经修补了很多其他人提交的bug和自己提交的bugs。最后，我发现我为什么不能创建一个免费的解决方案解决我所有的问题。

推进到更高水平

当我测试我自己的应用程序时，我对系统发出了很多呼叫，观察页面更新，控制和队列，和呼叫统计。但是一件事我没有花时间测试，那就是并发呼叫和呼叫量本身。我仅同时呼叫一路或者两路，我也没有真正对我的测试软件做完整测试。当我第一次安装在一个生产环境时，我也是第一次看到以前没有看到的事情，在一个锁竞争环境下，当多线程软件出现不可解决的冲突，就是常说的死锁。我测试我们自己开发的模块时，对出现段错误和内存分配的错误比较熟悉，但是让我惊讶的是这些令人费解的错误出现的次数不断在增加，而且发生的时间仅在某些时刻。

segmentation fault是通常是违规操作引起的，软件没有正确访问内存地址或访问越界地址。用户运行很多的C程序，用户可以对操作系统进行低级别的访问，这里没有对软件做任何保护，除非用户软件的软件规范了这个保护。我没有轻易放弃，这可以认为对我的一种历练的话，当我面对这些问题时，我准备从底层彻底排查这些问题。我花费了无数时间使用GNU debugger工具，通过模拟流量来发现我的问题。几个我的试错发排查，我们最后成功了。我想办法在我的实验室通过负载工具重现崩溃信息。我尝试找到问题，并且解决了这些问题！

我想办法慢慢从我的软件中退出，这些软件增加了死锁和内存越界定问题，但是我仍然不能完全杜绝这些问题。我最后发现，我的很多问题是由app_queue模块引起的，对我的呼叫中心解决方案来说，这不是一个好的消息。一些相关的代码不能打包到主要的代码发布中，我最后不在使用我自己的代码，而是继续更新Asterisk的部分模块，这样保持Asterisk的稳定，另一方面，寻找足够稳定的其他解决方案。

到这时，我已经写了Asterisk很多的功能，正在想重新写一个Asterisk功能。我创建了一个新的概念是“function variables"，支持模块开放一个接口，这个接口可以通过拨号规则来拓展(如果用户阅读了本书的其他章节，可能有所了解这个想法)。 我仍然和队列问题纠缠，我和另外一个Asterisk社区的成员一起进行了一次头脑风暴，创建了一个Asterisk新的ACD队列，称为mod_icd。

ICD代表智能呼叫分配，ACD的缩写代表是自动呼叫分配。 我们已经确定了所有app_queue 模块和功能的缺点，我们都有一个共同点兴趣，开发一个稳定的模块，这个模块不会导致无数的崩溃和死锁问题。我们已经有了一个可工作的雏形。我们使用状态机和支持data pools的高级内存管理抽象层，并且引入了几个以前标准的Asterisk缺乏的功能概念。问题是，我们自己过度设计了太多的模块，而且我们正在替代整个Asterisk核心，这样的做法当然不能完全让我们的模块成为核心代码中的一个可加载的模块。

我们从来没有完成mod_icd。那是在2004晚些时候，我有一个机会参与一个呼叫中心的优化项目中，清除了大量的segmentation faults 和死锁问题。我们开始专门针对通信服务，但是没有引入队列。我开发了一个新功能支持免费呼叫落地和传真邮件服务。使用了几个我添加的新的功能，这些功能已经成为了Asterisk主流版本的功能，一些还没有被认证通过对其他模块，我开发了一个七个Asterisk服务器的集群，然后连接让他们和一个大型的电信电路对接。这个部署不是完全没有问题但是是一个比较聪明的做法，如果有的机器崩溃，其他的机器可以启动来接手它们的工作。

新想法和新项目

到这个时候，我已经累积了几个想法:一些是经过测试的，一些没有，还有一些想法可能需要对Asterisk做很大的改动。我的团队—Brian West, Michael Jerris, 和我贡献了大量的时间在Asterisk 项目。我们帮助维护issue tracker系统。我们修正了很多的bugs，通过举行每周的开发人员电话会议来帮助其他人。甚至于，我们还要在我们网站托管一部分次要的代码。我们非常积极地参与，并且提出了我的想法，结果在Asterisk社区引发了一些不安宁，因为在社区中出现了一些开发人员无必要的竞争和一些矛盾。每个代码贡献者必须和Asterisk 签订一个表格，提交的代码可能被包含在Asterisk代码库，签订了这个表格，用户将承认Digium的免版权协议，可以左右处理用户的代码。这样，他们可以销售一个无限制的许可证，用户可能需要更高的价格购买。是不是真正的开源精神，但是是另外一种故事了。我认为这种隔阂引起了志愿的开发人员之间的争斗，就像我自己和一些没有被Digium雇用，但是仍然为Asterisk工作的人。

带着这种不安，我们专门启动了这个项目，并且希望看到它能够成功。我们有一个每周的电话会议，他们开始互相鼓励，互相帮助。我们认为我们应该召开一个面对面交流的会议，花费几天的时间，我们可以互相分享通信知识，我们称这个会议为ClueCon。有一个线索表示用户知道你们正在做的事情，因此ClueCon是一个会议帮助每个人"获得线索"。我承认，我说的，我们也不清楚我们正在做什么，因此这也是一个非常有意思的事情，没有线索的人开始一个线索讨论会议。但是，事实证明好运多于问题，这个会议是讨论关于通信的，不单是运营一个会议。

因此，直到第一次2005年的ClueCon，我们已经有几次电话会议，开始深入讨论Asterisk的缺点。这不是一个平常的事情，因为主要目的是定位这些问题并且转换成一个解决方案。这个时期，很多人遇到了这些Asterisk问题。很多人加入了这个电话会议，说服我们希望我们查看他们的问题。我想了很多这方面的问题，比较大的任务看起来是分拆这些烦恼的大的结构性的问题。许多概念看起来实际上都是一个整体问题，不能被拓展。许多功能依赖于其他的功能，如果修改这些功能实现我们的提升目标，会导致其他功能的问题。一些问题可以通过一个大的修改来解决，找出一些旧的代码，重新修改这些代码。这样看起来是不可行的，可能使得Asterisk几个月或者一年内不能使用。那个时候，我有一个新的想法：让我们开发一个2.0 版本！

这不是最糟糕的想法；我知道这对我说一个挑战，但是，我想到我们已经有一些旧的代码，我们可以分离出这些引起问题的部分代码，替换这些代码，我们仍然维护这些可工作的代码。我那时非常兴奋，当项目经理听到我的想法是，他的反应让我非常震惊，我也同样对他的反应相当震惊，他认为我怎么会有这样的想法，一句话，我们没有做Asterisk 2.0. 那时，我有很多的想法和清晰的思路，考虑做我们真正需要做的事情，而且不像Asterisk那样的方式。

我在空白的目录下，凝视着空白的文本编辑器长达一个小时。我知道我应该做什么，但是非常困难把我的想法变成文字。直到添加了几个古怪的标点符号，我都没有找到任何的词语。它们不是大家生活中使用的词语；它们是一些标志名称和变量声明。我写了C代码，简单开发了一个基本的应用程序的初稿，尝试拼凑了几个我以前喜欢的工具。我有Apache Portable Runtime或APR库，Perl语言和其他的包。我编译了一个简单的core，和可加载的模块结构和几个辅助功能来使用内存池，我还有一个简单的命令行提示程序支持用户输入help，和一个退出命令来关闭应用程序。我开发了一个实例模块来支持用户使用telnet访问一个指定的TCP端口，回复用户输入的所有信息，我还有一个非常基本的状态机。我称为合唱团。我想我的创意如果可以一起工作的话就可以组成一个统一的语音像一个合唱团。开始开发以后，我暂停了一段时间。ClueCon 就要到来了我没有着急做这个事情，我仍然认为这里还有很多问题需要考虑。

第一次ClueCon

2005年八月份，我们召开了第一次ClueCon 会议。这次会议邀请了几个VoIP开源社区的负责人，包括Craig Southeren，OpenH323开发者之一，和Mark Spencer，Asterisk创建者，他当时也不赞同我的Asterisk 2.0想法。但是，非常让人高兴的是，这些大佬们可以坐在同一个房间。我们每天有几个演讲和讨论，我们得到了每个人的想法。那是一个非常大的成功！离开会议以后，我非常兴奋地投入了合唱团代码的开发。但是，我没有那样做。相反，在后来的几个月中，我仍然在电话会议中讨论这些问题，仍然在我的Asterisk平台代码中挣扎。秋天来了，Asterisk社区中的这些焦虑最后爆发，最后演变成了反抗。部分开源社区成员发布了一个Asterisk分支，一个新的应用程序称之为OpenPBX。

我完全理解为什么他们那样做，我也尽最大努力帮助他们，我还给他们贡献了一些代码。如果有一有机会我尽量帮助他们，但是没有完全参与他们的项目，因为我也有同样的问题—我有这样一个需求，确实应该从基础开始另外一个新项目的负责人应该是最感兴趣修复特定的紧迫的问题的人，这些问题可能是Asterisk核心团队没有及时发现的问题。我们仍然有一个电话会议，但是没有人提及Asterisk项目因为他们对于这种不稳定的状态非常不高兴。我对他们道歉，因为我没有尝试解决OpenPBX的问题，OpenPBX没有重新设计也没有重写新的core。那时有人问我, "你认为多久可以写出你的代码来测试呼叫?" 像来自于Tootsie Pop commercials 的Mr. Owl，我也没有任何的想法，所以我决定开始。也许需要花费一个星期，两个星期或者三个星期。

第一个可以发出声音的模块是mod_woomera；它是由Craig Southeren 开发的一个终端模块，使用了Woomera 协议。我为Asterisk开发了一个相似的模块无需添加任何其他的代码。想法是把H323复杂的部分拿出去，支持用户通过简单的协议来集成VoIP 应用程序，这看起来是一个好的起点。当我开始工作时，我意识到在我的基本的core程序中需要一些其他的要素，所以我慢慢把这些代码集成在一起，我可以呼叫Woomera-powered H323 监听流程，并且可以在我的Pandora 代码中获得一个活动。是的，我重新命名它为"Pandora" 因为没有人喜欢这个名词Choir。我第一次非常高兴地从我的扬声器中听到Sirius 播放的Alan Parsons Project，是通过我的程序播放的！这一次经历比我第一次玩Asterisk的经历更加兴奋，因为是我自己从头开始写的代码，它可以工作了。

现在我了解了一些东西；我知道如何让两个通道桥接起来，如何支持中心协议并且可以做一些基本的操作不再是那些帮助信息和退出路径。这个想法来自于对OpenPBX 2代码的研究，最后我已经有了足够的名称参数，我决定让我所做的开发程序呼叫一个程序: FreeSWITCH。最后我决定一直坚持使用这个名词，在这个项目上继续做开发，这也是我所追求的一个理想。我低下头开始写我的代码。当然还是有很多工作要做。这些已经完全超出我的想象。我不停地开发，到2006一月份，我可以有足够的代码和大家公开分享。我们创建了一个代码仓库，支持开发人员注册访问这些代码。我们也有人检查代码仓库，提供一些回复信息，我们真正开始了一个我们自己的项目。

我们有一个模块可以桥接呼叫，播放声音和一些代码，还有一些拨号规则模块的实例当然还有一点其他的东西。哦，我有没有提到它可以在Windows工作？

最初的网站仍然在那里保留着，尽管没有任何链接是活动的:
http://www.freeswitch.org/old_index.html

介绍FreeSWITCH

按照我们计划的思路，我们实际上已经开发了一些代码，这些代码可以运行在Windows平台也支持Linux和Mac OSX。我最早的团队成员Michael和Brian和Mike，他们有很多Windows 开发的经验，可以在MSVC编译安装。第一次安装的时候，我们有经历了一些痛苦的过程，但是我们必须解决这些不同环境出现的问题，我也第一次学习如何开发支持不同平台的软件。时间很快就过去了，我知道又是ClueCon会议的时间了。那一年，我第一次做了关于FreeSWITCH的演讲，演示了core设计和本书涉及到底一些基本概念。我们看到了一些非常精彩的模块，例如终端模块，它可以和Google Talk通信。我的演讲的功能是通过我们的mod_exosip SIP模块实时演示一个上千呼叫的创建和拆除。它是一个相当不错的demo，但是我们仍然对这个演示不满意。

Exosip是一个SIP库，对于Osip来说也仅是一个辅助库，这是一个开源的SIP库，提供了大部分的SIP功能。Exosip可以轻松支持终端的工作，所以我们决定使用这个库，但是使用时，我们也遇到了一些以外，我开始感觉有一点沮丧，这和我当初尝试Asterisk完全一样的感觉，所以我们决定寻找一个替代的方案。另外，还有一个潜在的许可证问题，因为它本身是以GPL许可证发布，尽管它的起源库是LGPL许可证(以我的观点，这也是一个比较合理的许可证形式)。因为我们选择了MPL发布我们的项目，GPL许可证禁止GPL形式发布的代码包含在MPL的程序中。许可证的争议非常有意思，是一种好的办法让人们兴奋，但那不是一个好的时间。

我们广泛查询了开源的很多资源，为了找到支持FreeSWITCH新的SIP协议栈和RTP栈因为那是用户已经对SIP功能的要求很高了。我们查阅了几个库。最后确认了至少五个不同的协议栈。我没有发现一个是我满意的，因此我自己写了一个RTP协议栈。我还没有那么愚蠢，再次在SIP上犯同样的错误。看到上次搞Asterisk时的失控状态和Exosip失败的错误信息窗口，我也不会再次重新写一个SIP的协议栈。我继续寻找SIP协议栈，直到发现了Sofia-SIP，这个协议栈上有诺基亚开发的。我们搭建了功能模块mod_sofia 来测试实际效果，它给我们留下非常深刻的印象。我们一直在精心打磨这个模块，直到我们觉得可以放弃mod_exosip 开始使用mod_sofia，后来，它成为我们基本的SIP 终端模块。这仅仅是一个开始，到目前为止，我仍然不停地在mod_sofia模块上添加代码。SIP是非常复杂协议，很多人对SIP也非常害怕，因为它给我们带来很多很多不愉快的想法，例如它的名字，但是现在不是交流这个话题的时间。

我在2007年的ClueCon大会上做了另外一个关于FreeSWITCH的演讲，这次我使用了一个新的SIP模块。那时，我们也发布了一个OpenZAP，是一个TDM库，用来连接FreeSWITCH和语音板卡。OpenZAP后来被FreeTDM替代，目前有Sangoma Corporation维护。我非常高兴以前使用同样的语音卡可以支持Asterisk，现在也支持了FreeSWITCH。我宣布不久将发布FreeSWITCH 1.0 版本。有人如果阅读最早的网站主页，仍然可以看到我们的那个声明。那个声明是在2006年一月份发布的，我们是在铤而走险。我们确实希望集中精力发布一个稳定的版本，而不是其他什么版本，我们也一直在努力那样做，但是直到那个时间，我们的稳定版本1.0还是没有发布。

截至到2008年四月份，我们有了一个稳定的SIP，我们支持了事件套接字通过远端控制FreeSWITCH，我们有了API接口和HTTP通信，我们也发布了XML curl和其他功能模块。最后，我们决定那是一个正确的时间发布稳定的版本，因此，我们咬紧牙关发布了FreeSWITCH 1.0 Phoenix。我使用Phoenix这个名称作为一个名字是我们感觉到我们所有的努力是来自于我们经历的很多失败，凤凰涅磐终于发布了我们的一个稳定版本，这个版本已经被很多用户使用，也包括了NASA，那个时间正好是NASA发射Phoenix到火星的时间，我认为那是一个合适的名称。

ClueCon 2008会议上终于声明这个稳定的1.0在2008年五月份发布。会议上有几个和FreeSWITCH相关的演讲，还有其他的开源项目，例如Asterisk，在那一年也发布了Asterisk-1.6版本。我们花了整年的时间针对宽带语音的支持和其他的高级功能支持例如，对一些特别语音流的实时采样率支持。我们添加了许多新的SIP功能，例如presence指示和其他有趣的功能，不仅仅是简单呼叫的创建，还有一个1.0.1版本的发布。

2009年我们发布了1.0.2和1.0.4版本，在第五届ClueCon大会。一些以前的创意变成了现实，我们演示了Polycom电话机高清语音和Skype模块。FreeSWITCH介绍了一个总览，这个总览包括了所有的事情。如果用户尝试从四维角度来考虑问题，用户可能没有意识到自己也可以做，就像Dr.Emmett Brown (回到未来)和我一样都喜欢那样尝试。在某些流程上，我们和Asterisk非常相似，但是我们是完全不同的方式，我们打开这个门，用户只要一台PC和电话机就可以做任何有趣的事情。

2010年，我们在Trump Tower再次举行了ClueCon MMX，这是一次非常有纪念意义的会议，本书的第一版发行了。会议上作为奖品派送。我们做了一个对FreeSWITCH和性能方面的非常详细的演讲。我们发布了1.0.5和1.0.6版本。那一年几乎就是一个Erlang年。每个人都仿佛卷入了一个浪潮，开发事件驱动的架构，所以那次会议确实是在正确的地方在正确的时间。

2011年对于整个项目是一个转变的一年。因为被我们演讲的激励，之前我们在Sofia SIP模块方面的修改，通过这次修改可以支持分布式应用，在这个应用中，信息到达应用程序时，我们可以把每个呼叫的信息处理推送到它们自己的线程。通过这样的修改，支持了更多的平行处理，如果一个通道出现问题时，降低了整个SIP协议栈被阻塞的几率。 那一年的会议，我们在欧洲举行。我们演示了很多新的功能来帮助开发人员的开发，例如数组变量的概念，限域变量，用户可以设置一个通道变量仅对一个特定应用的运行支持。

2012年早期，我们宣布创建一个FreeSWITCH稳定版本分支的代码仓库。这是一个非常艰巨的任务，因为必须从新代码中分离出成熟的代码，还要检查这些分支，确认所有的代码可以平稳工作。我们在这方面做了很多工作，重写这本书的时候，我们标记了FreeSWITCH 1.2稳定版本的起始分支。在Hyatt，我们有一个非常精彩的ClueCon会议，和一些新的功能介绍，例如faxmodem传真仿真可以配合Hylafax工作而且还有一个mod_htttapi 模块，这个模块在本书前面的章节有所介绍。

在我写这本书的时候，2013年已经过去大半。幸运的是，我们从玛雅人的预言中活了下来，世界末日没有到来。这一年，我们努力填补电话通信和HTML5之间的缺口，因为HTML5和WebRTC是目前最流行的技术之一，同时发布了第一个FreeSWITCH 1.4 alpha版本。ClueCon将再次在Hyatt举行，这些更新的功能也让我们更加兴奋。我们希望到时在那里看到大家，希望大家已经学习了更多的FreeSWITCH知识，而且我当初电脑面前输入的几行代码已经逐渐发展壮大，在FreeSWITCH Core中，现在已经有接近五十万行的代码。

索引

Symbols

403-Forbidden 57
404-Not Found 57
408-Timeout 57
0911, extension 64
0912, extension 64
0913, extension 64
1000 - 1019, extension 64
2000, extension 64
2001, extension 64
2002, extension 64
3000 - 3399, extension 64
4000 or *98, extension 64
5000, extension 64
5900, extension 64
5901, extension 64
6000, extension 64
6001-6099, extension 64
7243, extension 64
9178, extension 64
9179, extension 64
9180, extension 64
9181, extension 64
9182, extension 64
9183, extension 64
9184, extension 64
9191, extension 64
9192, extension 64
9195, extension 64
9196, extension 64
9197, extension 64
9198, extension 64
9664, extension 64
Aleg 93
character 31
@domain parameter 112
** + extension number 64
<groups> elements 67
/log command 52
<match> node 131

A

Aastra phones 58, 59
Access Control List. See ACL
accountcode variable 71
ACD 77
ACL 237, 293
actions 98, 174-176
addBody($value) 253
addHeader($header_name, $value) 253
admin name 57
admin password 57
advanced routing 132, 133
A leg 46
ALG (Application Layer Gateway) 290
alone-sound 328
analog telephone adapter. See ATA
announce-count parameter 328
answer application 109
anti-actions 98
Apache Portable Runtime. See APR
API 9, 243
api($command[, $arguments]) 255
api command 243
application module 9
APPLICATION parameter 192
Application Programming Interface.
See API
APR 32
asr-engine 265

asr-grammar 265
Asterisk
  configuration files 366
  debugging 365
  migrating from, to FreeSWITCH 363, 364
  SIP phones 366
  starting 364
  stopping 364
  voicemail 373
async keyword 241
AT A 23
attribute-value pairs. See AVPs
audio content encryption
  about 316
  SRTP, enabling 317
  SRTP, encryption with 317
  ZRTP, encryption with 318-320
authentication 69
authorization 69
auth <password> 243
Automatic Call Distribution. See ACD
Automatic Speech Recognition (ASR) 9, 13
auto-record parameter 325
AVPs 14

B

B2BUA 84
back-to-back user agent. See B2BUA
bad-pin-sound 329
beep-file 267
bgapi($command[, $arguments]) 255
bgapi command 244
bind_digit_action command 192
bind_meta_app command 191
B leg 93 46
blue.box 351
break 273
break flag 180
break parameter
  always 180
  never 180
  on-false 180
  on-true 180
bridge application 46, 106

C

Call Detail Record (CDR) data 49
caller-controls parameter 325, 329
caller-id-name parameter 327
caller-id-number parameter 327
caller profile variables 48
callers
  connecting, to conference 331
callgroup variable 72
call legs 46, 92, 93
calls testing, for multiple phones
  about 63
  another telephone, calling 63
  call, parking 63
  conference, calling 63
calls testing, for single phone
  ClueCon 62
  demonstration IVR menu 62
  echo test 62
  FreeSWITCH public conference 62
  information application 63
  music on hold test 62
  sample sub-menu 62
  Tetris extension 61
channel variables
  about 48, 93, 95, 197
  accessing 94, 95
  and call setup 198, 199
chat module 9
chat rooms
  in other languages 358
  on irc.freenode.net 357
Chatzilla 358
check_acl 185
check command 343
cidlookup 185
ClueCon 62, 382-386
ClueCon open source developer conference
  about 361
  URL 361
code
  building 30
codec module 9
Colloquy 358
comfort-noise parameter 327

command line interface
used, for call making 227, 228
Command Line Interface (CLI)
about 41, 50, 294
FreeSWITCH, controlling through 50, 51
commands
executing, ESL used 229-231
conditions 173-180
condition tag 48
condition variables
about 182
ani 182
aniii 182
caller_id_name 182
caller_id_number 182
chan_name 182
context 182
destination_number 182
dialplan 182
network_addr 182
rdnis 182
source 182
uuid 182
conference
DATA 272
active conferences, controlling 332
profile 272
URL 332
conference-flags parameter 326
conference permission 279
conference profiles
about 324
alone-sound 328
announce-count parameter 328
auto-record parameter 325
bad-pin-sound 329
caller-controls parameter 325
caller-id-name parameter 327
caller-id-number parameter 327
comfort-noise parameter 327
conference-flags parameter 326
custom sounds 328
energy-level parameter 326
enter-sound 328
exit-sound 328
interval parameter 325
is-locked-sound 328
is-unlocked-sound 329
kicked-sound 328
locked-sound 328
max-members parameter 327
max-members-sound 329
member-flags parameter 326
moh-sound 329
muted-sound 328
perpetual-sound 329
pin parameter 327
pin-sound 329
rate parameter 325
sound-prefix parameter 328
suppress-events parameter 328
tts-engine parameter 327
tts-voice parameter 327
unmuted-sound 328
configuration files
Asterisk 366
FreeSWITCH 366
confirm-key attribute 122
confirm-macro attribute 122
connected() 254
contexts
about 47, 88, 172
default 89
features 89
internal context 172
public 89
public context 172
continue 274
curl 185
custom events 19

D

daemon 42
database queries
about 202, 203
database table
about 335
creating, for MySQL 336
creating, for PostgreSQL 336
deaf 326
debugging
Asterisk 365
FreeSWITCH 365

delHeader($header_name) 253
demo IVR
  in HTTAPI 282-285
demonstration configuration 22, 23
demonstration IVR menu 62
design, FreeSWITCH 8, 9
development branch 29
dial
  DATA 271
  caller-id-name 271
  caller-id-number 271
  context 271
  Dialplan 271
Dialplan
  about 12, 87
  actions 174, 175
  concepts 171
  conditions 173, 174
  contexts 172, 173
  overview 168-171
Dialplan applications
  about 105
  answer 109
  bridge 106
  hangup 110
  ivr 109
  play_and_get_digits 108
  playback 106
  pre_answer 110
  say 106, 107
  set 110
  sleep 109
  transfer 110
Dialplan elements, FreeSWITCH 88
Dialplan functions
  about 200
  database queries 202
  real-time condition evaluation 201
  SIP contact parameters 203, 204
  string, conditioning 202
dialplan module 9
Dialplan processing
  working 98-102
dial-set-context permission 279
dial-set-Dialplan permission 279
Dialstring formats 111, 113
DID 89
digest authentication 54, 69
digit-len attribute 121
digit-timeout 265, 267
directory module 9, 70
disconnect() 257
dist-dtmf 326
divert_events 246
domain 70

E

easyroute 185
eavesdrop command 193
echo test 62
effective_caller_id_name variable 71
effective_caller_id_number variable 71
encryption, VoIP
  about 312
  media 312
  signalling 312
endconf 326
endpoint module 9, 11
endpoints
  about 345
  GSM endpoint 346, 347
  mod_gsmopen, GSM with 348
  mod_skypopen module,
    Skype with 347, 348
  Skype endpoint 346, 347
  TDM, with FreeTDM 349
energy-level parameter 326
enter-sound 328
enum 185
error-file 265
error() function 164
ESL
  about 215, 251, 351
  ESLevent object 253
  ESL object 252
  supported libraries 252
ESLconnection object
  about 254
  bgapi($command[, $arguments]) 255
  connected() 255
  disconnect() 257
  events($event_type,$value) 256
  execute($app[, $arg][, $uuid]) 256

executeAsync($app[, $arg][, $uuid]) 257
filter($header, $value) 256
new($fd) 254
new($host, $port, $password) 254
recvEvent() 256
recvEventTimed($milliseconds) 256
send($command) 255
sendEvent($send_me) 256
sendRecv($command) 255
setAsyncExecute($value) 257
setEventLock($value) 257
socketDescriptor() 254
ESLevent object
about 253
addBody($value) 253
addHeader($header_name, $value) 253
delHeader($header_name) 253
firstHeader() 254
getBody() 253
getHeader($header_name) 253
getType() 253
nextHeader() 254
serialize([$format]) 253
setPriority([$number]) 253
ESL object
about 252
ESLconnection object 254
eslSetLogLevel($loglevel) 252
ESL scripts
versus built-in languages 229
eslSetLogLevel($loglevel) 252
eval 185
event-based modules
about 235
mod_event_socket 235, 236
event category 234
event command 245
event handlers module 9, 235
event message 233
events
about 185
from Dialplan 241
mod_event_multicast 242
sending 240
events($event_type,$value) 256
event sending, examples
custom notify messages 261
phone lights, sending 258
phone lights, setting 259
phone, rebooting 260
phone reconfiguration, requesting 260
Event Socket Library. See ESL
Event Socket Library example 258
EventSocket (Python/Twisted)
about 353
URL 353
event system architecture 16, 233, 234, 235
event system commands, FreeSWITCH
api command 243
auth <password> 243
bgapi command 244
divert_events command 246
event command 245
execute command 249
filter command 246
filter delete command 247
hangup command 250
linger command 251
log <level> command 250
noevents command 246
nolinger command 251
nolog command 250
nomedia command 250
sendevent 248
sendmsg <uuid> 248
event type 234
example config 45
example configuration. See example config
example, Dialplan
calls, testing for multiple phones 63
calls, testing for single phone 61
extension number, functions 64
testing 61
execute
DATA 270
application 270
data 270
execute($app[, $arg][, $uuid]) 256
executeAsync($app[, $arg][, $uuid]) 257
execute command 249
exit-sound attribute 119, 328
Exosip 384
export 185
versus set 205

exposed functions 8
extended-data permission 278
Extensible 69
extension number
  functions 64
extensions
  about 12, 47, 89, 178, 179
  conditions 90, 91
external profile 84

F

Fail2Ban
  about 310
  configurations, filtering 311
  Jail configurations 311
  other considerations 312
features context 89
Festival Lite text-to-speech. See TTS
file module 9
filter 246
filter($header, $value) 256
filter delete 247
firstHeader() 254
flush command 344
formats module 9
Freenode
  URL 358
FreePyBX
  about 350
  URL 350
FreeSWITCH
  about 7, 46, 383, 384
  compiling, for Linux 30
  compiling, for Windows 35
  configuration files 366
  console application 251
  controlling, with CLI 50-53
  debugging 365
  Dialplan elements 88
  event sending, examples 258
  event system 233
  event system commands 243
  general overview 234
  history 377, 378
  launching 41
  mailing lists 355-357
  migration from Asterisk 363
  module types 9
  on client side 299, 300
  passwords 320
  running, in background 42, 43
  starting 365
  stopping 365
  URL 289
  uses, in NAT 300, 301
  voicemail 374
  web page 360
  web page, URL 359
  wiki page 360
  wiki page, URL 359
  wiki, URL 364
FreeSWITCH API. See FSAPI
FreeSWITCH, compiling for Linux
  about 31
  configure script, running 31, 32
  edit modules.conf 31
  Makefile, running 32, 34
  modules.conf.xml, editing 34
  music files, installing 35
  sound 35
  sound files, installing 35
FreeSWITCH, compiling for MacOX. See
    FreeSWITCH, compiling for Linux
FreeSWITCH, compiling for Unix. See
    FreeSWITCH, compiling for Linux
FreeSWITCH, compiling for Windows
  about 35
  solution, MSVC/MSVCEE used 36-41
  user considerations 36
FreeSWITCH environment
  operating system 26
  setting up 26
FreeSWITCHeR (Ruby)
  about 352
  URL 352
FreeSWITCH public conference 62
FreeSWITCH user directory
  about 67-69
  working with 70
FreeTDM
  with TDM 349
FSAPI 19, 20
fs_path field 295

FSSocket (Perl)
about 353
URL 353
full keyword 241
FusionPBX
about 350
URL 350

G

gateway
about 79
calls, making 82, 83
calls, receiving 82
setting up 79-81
getBody() 253
getHeader($header_name) 253
getInfo() 255
getType() 253
getVar
name 274
permanent 274
global variables 50, 199, 200
Graphical User Interface. See GUI
greet-long attribute 118
greet-short attribute 119
GSM
endpoint 346, 347
with mod_gsmopen module 348, 349
GUI 28

H

hangup
about 250
cause 273
hangup application 110
hard phone
Aastra phones 58, 59
about 53, 57
admin name 57
admin password 57
IP address 57
Polycom phones 59, 60
Snom phones 60, 61
help command 53
HTT API
about 264
child tags 264
demo IVR 282-285
params 264
syntax 264
variables 264
work 264
work actions 265
hunting phrase 190

I

IDE 29
information application 63
Inline execution
about 183
check_acl 185
cidlookup 185
curl 185
easyroute 185
enum 185
eval 185
event 185
export 185
lcr 185
nibblebill 185
odbc_query 185
presence 185
set 185
set_global 185
set_profile_var 185
set_user 185
sleep 185
unset 185
verbose_events 185
input-timeout 265
Integrated Development Environment. See IDE
Interactive Voice Response. See IVR
inter-digit-timeout attribute 120
internal context 172
internal profile 84
Internet Relay Chat. See IRC
Internet Telephone Service Providers (ITSPs) 305
interval parameter 325
intruders
detecting 309

invalid-sound attribute 119
IP address 57
IP authorization 69
IPv6 289
IRC
  about 357
  Chatzilla 358
  clients 358
  Colloquy 358
  IRSSI 358
  mIRC 358
  pointers 359
  using 358
IRSSI 358
is-locked-sound 328
is-unlocked-sound 329
IVR 47, 115, 135
ivr application 109
IVR concepts, advanced
  about 152
  database connections, LuaSQL
    used 152-157
  Lua patterns 163, 164
  regular expressions 163, 164
  web call. making 158-163
IVR engine
  calls, routing to 125
  menu definitions 118
  menu destinations 122
  nesting 125
  overview 116
IVR XML configuration file 116

K

Kazoo
  about 351
  URL 351
KEY parameter 192
kicked-sound 328

L

language bindings
  about 215, 225
  configurations, generating
    dynamically 225, 227
languages module 9, 22
latest source project 29
lcr 185
leg 46
length 202
libraries
  about 351
  EventSocket (Python/Twisted) 353
  FreeSWITCHeR (Ruby) 352
  FSSocket (Perl) 353
  Librevox (Ruby) 352
  Liverpie (Ruby) 352
  Vestec Automatic Speech Recognition 353
Librevox (Ruby)
  about 352
  URL 352
linger 251
Linux/Unix 25
LISTEN_TO parameter 192
Liverpie (Ruby)
  about 352
  URL 352
locked-sound 328
log 185
  clean 273
  level 273
loggers module 9
log <level> 250
loops 265
Lua
  about 135
  running, from Dialplan 136
  syntax, basic 137, 138
  starting with 136

M

Mac OS X 27
mailing lists
  about 355, 357
  URL 355
main registry 50
Makefile 32
max-failures attribute 120
max-members parameter 327
max-members-sound 329
max-timeouts attribute 121
member-flags parameter 326

menu-back action 124
menu definitions, IVR
  about 118
  confirm-key attribute 122
  confirm-macro attribute 122, 123
  digit-len attribute 121
  exit-sound attribute 119
  greet-long attribute 118
  greet-short attribute 119
  inter-digit-timeout attribute 120
  invalid-sound attribute 119
  max-failures attribute 120
  max-timeouts attribute 121
  timeout attribute 120
  tts-engine attribute 121
  tts-voice attribute 121
menu destinations, IVR
  menu-back 124
  menu-exec-app 124
  menu-play-sound 124
  menu-top 124
menu-exec-app action 124
menu-play-sound action 124
menu-top action 124
Microsoft Visual C++. See MSVC
milliseconds 267
mIRC 358
mod_commands command 196
mod_conference configuration
  about 324
  active conferences, controlling 332
  advertise section 330
  caller controls 329, 330
  callers, connecting to 331
  conference profiles 324-329
  XMPP events, receiving 330
  XMPP events, sending 330
mod_conference module 18, 19
mod_dptools command
  about 191-194
  bind_digit_action command 192
  bind_meta_app command 191
  eavesdrop command 193
  send_display command 194
mod_event_multicast 242
mod_event_socket
  about 235, 236
  event socket settings, configuring 236, 237
  events, reading 237, 238
  mnimum event information 239
mod_event_socket module 236
mod_exosip SIP module 383
mod_gsmopen module
  about 346
  GSM with 348, 349
  Skype 347, 348
mod_httapi configuration file
  about 275, 276
  data, storing across successive requests 280
  exiting 280
  missed parameters 281
  permissions 277-280
mod_httapi module 263
mod_nibblebill
  about 332
  billing, default method 336, 337
  billing (post-pay) 333
  billing (pre-pay) 332, 333
  call, billing 336
  database tables 335
  database tables, creating for MySQL 336
  database tables, creating for PostgreSQL 336
  design goals 333, 334
  enabling 334
  example 338
  fraud, preventing 333
  nibble billings, alternative for 337
  pay-per-call service billing 333
  use cases 332
mod_nibblebill, examples
  API command 343
  Application command 343
  bill, B leg based 345
  call hanging up, on balance depletion 342
  check command 343
  CLI command 343
  flush command 344
  funds, adding 345
  funds, deducting 345
  pause command 344
  rates per area code, different 340
  rates per service delivery, different 341, 342
  rates per user, different 338, 339

rates per user, single 339, 340
reset command 344
resume command 344
session heartbeat, enabling 345
mod_skypopen module
about 346
URL 348
mod_sofia command 194, 196
module, types
application module 9
Application Programming Interface (API) 9
Automated Speech Recognition (ASR) 9
chat module 9
codec module 9
dialplan module 9
directory module 9
endpoint module 9
event handlers module 9
file module 9
formats module 9
languages module 9
loggers module 9
say module 9
Text-To-Speech (TTS) 9
timers module 9
XML interfaces 9
mod_xml_curl
basics 216, 217, 218
summary 225
mod_xml_curl configuration 222-224
mod_xml_curl Dialplan 219, 220
mod_xml_curl folder 220-222
mod_xml_curl method 215
moh-sound 329
MSVC 28
MSVCEE 28
music on hold test 62
muted-sound 328
my_status variable 20

N

NAT
about 288
advanced options 297, 298
evolution 289
FreeSWITCH, uses 300
pitfalls 290-292
settings 297, 298
settings, demystifying 292-295
versus PAT 288
NAT-PMP
URL 299
Network Address Translation. See NAT
network level protection
about 303
Fail2Ban 310
interfaces, separating 304
intruders, detecting 309
registration, monitoring 309, 310
setup sample, complex 307, 308
setup sample, simple 305, 306
traffic, restricting 304
VLANs 308
new($fd) 254
new($host, $port, $password) 254
new extension
creating 103-105
nextHeader() 254
Nibblebill. See mod_nibblebill
nibble method 336, 337
nixevents 248
noevents 246
nolinger 251
nolog 250
nomedia 250
NOTIFY event 240

O

odbc_query 185
offset 202
onInput function 150
OpenZAP 384
operating system
about 26
prerequisites 27
operating system, prerequisites
Linux/Unix 27
Mac OS X 27
Windows 28
originate API 215
originate command 113
outbound_caller_id_name variable 72
outbound_caller_id_number variable 72

P

PARAMETERS parameter 192
params 264
parsing
  about 98
  phrase 190
passwords
  about 320
  registration 320
  voicemail passwords 321
PAT
  versus NAT 288
pause
  digit-timeout 268
  error-file 267
  input-timeout 268
  loops 268
  milliseconds 267
  name 267
  terminators 268
pause command 344
PCRE 32, 95, 163
Perl Compatible Regular Expressions. See PCRE
Perl Object Environment (POE) 353
perpetual-sound 329
Phrase Macro 17, 118
phrases
  Phrase Macros, calling 126
  using, with IVRs 126
  voicemail system 127-131
pin parameter 327
pin-sound 329
PKI (Public Key Infrastructure) 318
Plain Old Telephone Service. See POTS lines
play_and_get_digits application 108
playback
  about 265
  asr-engine 265
  asr-grammar 265
  digit-timeout 265
  error-file 265
  file 265
  input-timeout 265
  loops 265
  name 265
  terminators 265
playback application 106
polycom phones 59, 60, 298
Port Address Translation. See PAT
POTS lines 8
pre_answer application 110
pre-processor 21
presence 185
proxied 170
ptime 325
public context 172

R

rate parameter 325
record
  beep-file 267
  digit-timeout 267
  error-file 267
  file 267
  limit 267
  name 267
  terminators 267
recordCall
  limit 272
  name 272
recvEvent() 256
recvEventTimed($milliseconds) 256
regex command 97
regex operator
  all value 186
  any value 186
  values 186
  xor value 186
regular expression pattern-matching 12
regular expressions 46, 96, 97
reloadxml command 74, 79
reset command 344
RESPOND_ON parameter 192
resume command 344
RFC-1918 IP addresses
  URL 293
RFC (request for comment) 11
ROUTING 48
RTP stream. See audio content encryption

S

sample sub-menu 62
say
  application 106
  digit-timeout 269
  error-file 269
  gender 269
  input-timeout 269
  language 269
  loops 269
  method 269
  name 269
  terminators 269
  text 269
  type 269
say module 9
scripting tips 164, 165
Secure Sockets Layer. See SSL
send($command) 255
send_display command 194
sendevent 248
sendEvent($send_me) 256
sendevent command 240
sendmsg <uuid> 249
sendRecv($command) 255
serialize([$format]) 253
service 42
session_id parameter 280
Session Initiation Protocol. See SIP
session:ready() method 145
set 185
  versus export 205
set application 110
setAsyncExecute($value) 257
setEventLock($value) 257
set_global 185
setInputCallback method 151
set-params permission 277
setPriority([$number]) 253
set_profile_var 185
set_user 185
set-vars permission 277
show calls command 53
show channels command 53
SIP
  about 11, 67
  contact parameters 203, 204
  digest authentication 54
  modifying 310
  settings 53
SIP Endpoint module 48
SIP phone
  analysis 372
  Asterisk configuration 367
  configuring 53
  FreeSWITCH configuration 368-371
SIP profiles 84
SIP registrar 54
SIP signalling
  about 313
  encryption options, selecting 313
  SSL, encryption with 314
  SSLV2/3, setting up 314
  TLS, encryption with 315
SIP telephone 23
Skype
  with mod_gsmopen module 347, 348
Skype endpoint 346, 347
slash command 51
sleep application 109, 185
sms
  DATA 271
  to 271
Snom phones 60, 61
socketDescriptor() 254
sofia_contact API command 279
Sofia-SIP
  URL 11
Sofia SIP module 69
soft phone
  about 53
  X-Lite soft phone 54-56
software-based SIP softphone 23
sound-prefix parameter 328
source
  downloading 29
speak
  about 268
  digit-timeout 268
  engine 268
  error-file 268
  input-timeout 268
  loops 268

name 268
terminators 268
text 268
voice 268
SRTP
about 316
enabling 318
encryption with 317
SSL
encryption with 314
SSLV2/3 encryption
about 314
setting up 314, 315
stable source project 29
status command 20, 53
string conditioning 202
string.gsub function 162
subclass 19
suppress-events parameter 328
SWIG 252

T

TDM
with FreeTDM 349
telephony revolution 7, 8
temp-action 265
terminators 265
Tetris extension 61
text-only editors
about 29
Emacs 28
Notepad++ 28
Vi/Vim 28
Text-to-Speech (TTS) 13
TGML 152
three-way call 46
timeout attribute 13, 120
timers module 9
TLS
encryption with 315
toll_allow variable 71
Tone Generation Markup Language.
See TGML
tone stream 49
transfer application 110
Transport Layer Security. See TLS
TTS 31
tts-engine attribute 121
tts-engine parameter 327
tts-voice attribute 121
tts-voice parameter 327

U

Unix-like 25
unloop 99
unmuted-sound 328
unset 185
unstable branch 29
UPnP
URL 299
URI (Uniform Resource Identifier) 295
url parameter 276
user
adding 72-75
user agents 84
user_context variable 71
user features 70, 71
user groups 77, 78

V

var 202
variables
about 264
caller profile fields 197
channel variables 197
global variables 199, 200
passing, via call headers 206
testing, with regular expressions 197
utilizing 197
verbose_events 185
version command 53
Vestec Automatic Speech Recognition
about 353
URL 353
Visual C++ Express Edition. See MSVCEE
VLANs 308
vmname
digit-timeout 266
error-file 266
id 266
input-timeout 266
loops 266

name 266
terminators 266
voice applications
building 138, 139, 140
conditions 142-152
looping 142-151
simple IVR 141, 142
voicemail
accessing 375
auth-only 275
check 275
domain 275
id 275
in Asterisk 373
in FreeSWITCH 374
profile 275
testing 75, 76
voicemail, accessing
Asterisk 375
FreeSWITCH 375, 376
Voicemail application 15-18
voicemail_message_count macro 130
voicemail passwords 321
Voice over Internet Protocol (VoIP) 7
VoIP
encryption 312
security 303
VoIP Security
audio, protecting 316
network level protection 303
passwords, protecting 320
SIP signalling 313

W

waste 326
Web Graphical User Interfaces.
See Web GUIs
Web GUIs
about 350
blue.box 351
FreePyBX 350
FusionPBX 350
Kazoo 351
web page 360
wiki page 360
Windows 28
work 264
work actions, HTTAPI
about 265
action 265
break 273
conference 272
continue 274
dial 271
execute 270
getVar 274
hangup 273
log 273
pause 267, 268
playback 265
record 267
recordCall 272
say 269, 270
sms 271
speak 268
temp-action 265
vmname 266
voicemail 275

X

X-Lite soft phone 54-56
XML Dialplan applications
mod_commands command 196
mod_dptools command 191-194
mod_sofia command 194, 196
XML Dialplan cookbook
about 206
alternate outbound gateways 211
caller ID, matching by 208
DIDs, routing to extensions 210
endpoints, with enterprise originate 211, 212
IP address, matching by 206-208
number and strip digits, matching 208
number, calling 207
number call, matching by 206
registered device, calling 209
XML Dialplan module
about 176, 177
actions 185, 186
anti-actions 185, 186
conditions 179-181

condition variables 182, 183
extensions 178, 179
inline execution 183-185
nested conditions 188, 189
regex operator 186-188
XML interfaces 9
XML registry 21
XMPP events
receiving 330
sending 330
xor value 186

Z

ZRTP
about 317
advantages 319
encryption with 318-320
URL 313

为开源代言

靠谱的开源SS7/ISDN解决方案

